

Universiteti i Europës Juglindore
Fakulteti i Shkencave dhe i Teknologjive Bashkëkohore

Agni Dika

**Programimi i Orientuar në Objekte
në C++**

2008

U lejua për botim nga Komisioni për botime pranë Universitetit të
Europës Juglindore në Tetovë.

© Ndalohet ribotimi dhe kopjimi i librit, ose edhe i pjesëve të tij, pa pëlqimin
e autorit.

Parathënie

Libri që e keni në dorë u dedikohet studentëve të Fakultetit të Shkencave dhe Teknologjive Bashkëkohore në Universitetin e Europës Juglindore në Tetovë. Por, sigurisht, ai mund të shfrytëzohet edhe nga studentë të universiteteve të tjera, ose edhe nga të gjithë ata që dëshirojnë ta përvetësojnë teknikën e programimit, e cila është e orientuar në shfrytëzimin e objekteve.

Materiali që përfshihet në libër është i nivelit të tillë që për mësimin e tij nevojitet një dije bazike mbi principet e programimit në gjuhën C++.

Lexuesi mund komunikojë me autorin përmes adresave elektronike **agnidika@yahoo.com** dhe **a.dika@see-university.edu.mk**, ose edhe përmes lidhjes që ofrohet në faqen e internetit **www.agnidika.net**.

Hyrje

Programimi i zakonshëm, i cili mbështetet në shfrytëzimin e komandave të gjuhës programuese C++, përfshirë edhe thirrjen direkte të funksioneve të ndryshme, njihet si *programim i strukturuar* (ang. structured programming). Gjatë programimit të tillë, pjesë të ndryshme funksionale bashkohen me qëllim të krijimit të programeve komplete. Por, kjo mënyrë e programimit nuk lejon fleksibilitet të madh gjatë ndryshimit të programeve, gjë që vjen në shprehje sidomos kur kemi të bëjmë me programe komplekse.

Aktualisht, gjatë shkruarjes së programeve shfrytëzohen gjuhë programuese të cilat e përmbajnë *disajnimin e orientuar në objekte* (ang. object-oriented design, OOD). Te këto gjuhë, për zgjidhje të problemeve me kompjuter shfrytëzohen *strukturat* dhe *klasat*, përkatësisht komponentet që quhen *objekte*. Prandaj, programimi përmes tyre njihet edhe si *programim i orientuar në objekte* (ang. object-oriented programming, OOP). Por, gjatë përcaktimit të strukturave dhe klasave, si dhe operimit me objekte, përkatësisht me komponentet e tyre, përsëri shfrytëzohet programimi i strukturuar. Gjuha programuese C++, meqë e përmban këtë lloj programimi, është *gjuhë e orientuar në objekte* (ang. object-oriented language, OOL).

Gjatë programimit me objekte, strukturat dhe klasat brenda komponenteve të tyre i përfshijnë të dhënat dhe funksionet që shfrytëzohen për manipulim me ato të dhëna, të cilat mund të deklarohen si *private* dhe si *publike*. Kurse objektet përkatëse karakterizohen me *gjendjet e brendshme* (ang. internal state) dhe *gjendjet e jashtme* (ang. external state) të tyre. Gjendjet e brendshme përcaktohen nga *anëtarët privatë* të objekteve dhe nuk mund të ndryshohen nga jashtë. Kurse, në përcaktimin e gjendjeve të jashtme të objekteve shfrytëzohen *anëtarët publikë* të tyre.

Programi i orientuar në objekte paraqet një formë natyrore të veprimeve që ndërmerren gjatë zgjidhjes së problemeve të ndryshme me kompjuter. Shfrytëzuesi i një objekti nuk ka nevojë ta dijë përbërjen e brendshme të tij. Kjo i ngjanë, p.sh., shfrytëzimit të orës, ose ngarjes së veturës, pa i ditur pjesët përbërëse dhe mënyrën e funksionimit të tyre.

Në kapitullin e parë të librit lexuesi do të njihet me grupimin e të dhënave përmes *numërimit* (ang. enumeration). Pastaj, kapitulli i dytë dhe i tretë

2 Programimi i orientuar në objekte

merret me *strukturat* dhe *klasat*, si dhe me *objektet* që deklarohen përmes tyre. Në kapitujt vijues jepen njohuri të nevojshme për punë me *pointer* (kapitulli i katërt) dhe *referencat* (kapitulli i pestë). Përmes tyre lehtësohet operimi me të dhëna të zakonshme, veçanërisht gjatë shfrytëzimit të funksioneve, si dhe operimi me objekte. Në kapitullin e fundit, në mënyrë të detajizuar shpjegohet shfrytëzimi i *fajllave* (ang. file), përkatësisht hapja e fajllave, mbushja e tyre me të dhëna, leximi i të dhënave nga fajllat dhe mbyllja e fajllave, krejt kjo përmes shembujve të ndryshëm.

Përmbajtja

Hyrje

1. Numërimet

Përcaktimi i grupit	4
Shfrytëzimi i grupit	6
Degëzimi përmes vlerave të numëruara	7
Degëzimi me komandën if	7
Degëzimi me komandën switch	9
Disa variabla të numëruara të tipit të njëjtë	12
Përcaktimi dhe deklarimi njëkohësisht	13
Shoqërimi direkt i vlerave	16
Operimi me variabla të numëruara	19
Barazimi i variablave të numëruara	20
Llogaritja me vlera të numëruara	21
Variablat e numëruara në unaza	23
Leximi i të dhënave të numëruara	27
Definimi i disa grupeve njëkohësisht	28
Të dhënat e numëruara në nënprograme	31

2. Strukturat

Definimi i strukturave të zakonshme	34
Deklarimi i variablave të tipit të strukturës	36
Qasja te komponentet e strukturës	37
Deklarimi direkt i variablës së strukturës	39
Inicializimi direkt i variablave	44
Llogaritje me variablat e strukturës	46
Ruajtja e rezultateve në strukturë	49
Disa variabla të një strukture	51
Deklarimi pasi është definuar struktura	51
Deklarimi gjatë definimit të strukturës	55
Përdorimi i operacionit të shoqërimit	60
Përdorimi i operatorëve relacionalë	62
Disa struktura njëkohësisht	65

Strukturat e ndërthurura	70
Strukturat si parametra të funksioneve	73
Funksione me parametra të përzier	78
Thirrja e përsëritur e funksionit	82
Disa struktura si parametra të funksioneve	85
Disa nënprograme njëkohësisht	88
Funksionet në komponentet e strukturave	90
Funksionet pa parametra formalë	91
<i>Definimi brenda strukturës</i>	91
<i>Definimi jashtë strukturës</i>	100
Funksionet me parametra formalë	104
<i>Definimi brenda strukturës</i>	104
<i>Definimi jashtë strukturës</i>	108
Shfrytëzimi në strukturë i funksioneve të tjera	110
Disa funksione brenda strukturës	115
Disa struktura brenda funksioneve	119
Fushat në struktura	124
Fushat e strukturave	128

3. Klasat

Definimi i klasave të zakonshme	134
Deklarimi i objekteve	136
Qasja te komponentet e klasës	137
Forma e përgjithshme e klasave	139
Definimi i funksioneve jashtë klasës	143
Forma të tjera të inicializimit të variablave	145
Inicializimi përmes leximit	145
Inicializimi gjatë deklarimit të objekteve	147
Shfrytëzimi i vlerave të variablave private	149
Funksionet pa parametra formalë	149
Funksionet me parametra referentë	151
Llogaritjet me variablat e klasës	153
Llogaritjet në program	153
Llogaritjet përmes funksioneve të klasës	157
<i>Rezultatet te variablat e programit</i>	157
<i>Rezultatet te variablat e klasës</i>	161
Shfrytëzimi i funksioneve brenda klasës	164
Funksionet në komponentet publike	164
Funksionet në komponentet private	168
Konstruktorët	172
Konstruktorët pa parametra formalë	172
Konstruktorët me parametra formalë	176
Llogaritjet brenda konstruktorëve	179
Disa konstruktorë njëkohësisht	182
<i>Thirrja në bazë të numrit të parametrave</i>	183

<i>Thirrja në bazë të tipit të parametrave</i>	185
Destruktorët	188
Trashëgimia	191
Definimi i funksioneve jashtë klasave	195
Shfrytëzimi i anëtarëve të mbrojtur	196
Shfrytëzimi i anëtarëve gjatë trashëgimisë	198
Ridefinimi i funksioneve të klasës bazë	199
Trashëgimia e shumëfishtë	202
Operatori i shoqërimit tek objektet	204
Krahasimi i variablave të klasës	206
Fushat brenda klasave	212
Fushat e objekteve	214
Dukshmëria e klasave dhe e objekteve	217

4. Pointerët

Deklarimi i pointerëve	218
Adresat e variablave	220
Vlera në adresën e variablës	222
Shoqërimi i vlerave	224
Shoqërimi i vlerave të konstanteve	224
Shoqërimi i vlerave të variablave	225
Operatorët inverzë	227
Llogaritjet përmes pointerëve	231
Operimi me vlerat e pointerëve	239
Rritja dhe zvogëlimi i vlerave	239
Shoqërimi dhe krahasimi i vlerave	241
Pointerët gjatë operimit me fusha	242
Operimi me anëtarët e vektorëve	242
<i>Pointerët në anëtarët e vektorëve</i>	242
<i>Pointeri në anëtarin e parë të vektorit</i>	244
<i>Indekset si pointerë</i>	248
<i>Pointerët pa i deklaruar</i>	251
Operimi me anëtarë të matricave	254
<i>Shtypja e anëtarëve të matricave</i>	254
<i>Gjetja e anëtarit të caktuar në matricë</i>	258
<i>Formimi i vektorit nga anëtarët e matricës</i>	260
<i>Shuma e anëtarëve të matricës</i>	262
Pointerët në stringje	264
Fusha pointerësh	265
Pointerët si parametra të funksioneve	267
Mundësitë themelore	268
Mundësi të tjera	273
Vektorët si pointerë	276
Pointerët në funksione	279
Fusha pointerësh në funksione	282

- Pointerët në struktura 286
- Pointerët në objekte 289
 - Qasja te variablat e klasës 289
 - Qasja te funksionet e klasave 291

5. Referencat

- Referencat e zakonshme 296
 - Konstantet referente 300
- Parametrat formalë referentë 302
 - Parametrat referentë si rezultate 305
- Fushat referente 306
 - Vektorët referentë 306
 - Matricat referente 307
 - Kufizime për variablat referente 309
- Parametrat referentë brenda strukturave 309
- Variablat referente brenda klasave 311
- Objektet referente 312

6. Fajllat

- Fajllat me qasje sekuenciale 317
 - Shkruarja në fajll 318
 - Memoria ndërmjetësuese 320
 - Leximi nga fajlli 321
 - Kontrollimi i hapjes së fajllave 323
 - Deklarimi i objektit para hapjes 325
- Qasja te fajllat në unazë 328
 - Shkruarja në unazë 328
 - Shfrytëzimi i manipulatorëve 329*
 - Leximi në unazë 331
 - Flamujt e statusit 334*
 - Shkruarja dhe leximi në një program 337
 - Shfrytëzimi i dy objekteve të veçanta 337*
 - Shfrytëzimi i një objekti të vetëm 339*
- Tekstet në fajlla 341
- Tekstet dhe numrat në fajlla 343
- Shkruarja dhe leximi i karaktereve 347
 - Forma të tjera të unazave për lexim 350
 - Shfrytëzimi i pointerëve 351
- Leximi i rreshtave 352
 - Leximi i komplet rreshtave 352
 - Leximi deri në simbolin e caktuar 353
- Mode të hapjes së fajllave 356
 - Kombinimet e modeve të hapjes 356
 - Hapja e fajllave në modin binar 359*

Pozita në fajll	360
Leximi i pozitës aktuale në fajll	363
Qasja dhe lëvizja e lirë brenda fajllave	365
Qasja relative në fajll	366
Fajllat me qasje direkte	372
Shkruarja në fajlla	373
Leximi nga fajllat	374
Vlerat e disa variablave në fajlla	376
Vlerat e fushave në fajlla	377
Vlerat e vektorëve	378
Vlerat e matricave	381
Vlerat e llogaritura në fajlla	382
Tekstet në fajlla	384
Tekstet dhe vlerat numerike në fajlla	386
Qasja direkte në të dhënat e fajllave	387
Shfrytëzimi i të dhënave nga fajllat	389
Objektet në fajlla	391
Objektet te fajllat me qasje sekuenciale	391
Objektet te fajllat me qasje direkte	394
Përmes variablave të komponenteve me të dhëna	394
Përmes blloqeve me të dhëna	396
Objektet me funksione	399
Disa fajlla të hapur njëkohësisht	404

Literatura

	1	
Numërimet Përcaktimi i grupit 4 Shfrytëzimi i grupit 6 Degëzimi përmes vlerave të numëruara 7 Disa variabla të numëruara të tipit të njëjtë 12 Përcaktimi dhe deklarimi njëkohësisht 13 Shoqërimi direkt i vlerave 16 Operimi me variabla të numëruara 19 Leximi i të dhënave të numëruara 27 Definimi i disa grupeve njëkohësisht 28 Të dhënat e numëruara në nënprograme 31		

Në gjuhën programuese C++ të dhënat e caktuara mund të grupohen, siç thuhet përmes *numërimit* (ang. enumeration) të tyre. Kështu, p.sh., mund të grupohen ditët e javës, muajt e vitit, ngjyrat, vlerat logjike etj. Në këtë mënyrë krijohen tipe të reja të të dhënave, të cilat njihen si *tipe të numëruara* (ang. enumerated type). Pastaj, këto të dhëna mund të shfrytëzohen për deklarimin e variablave të tipeve përkatëse, me qëllim të shfrytëzimit të tyre gjatë shkruarjes së programeve të ndryshme.

Përcaktimi i grupit

Grupi i të dhënave përcaktohet duke e shfrytëzuar komandën **enum**, e cila në formë të përgjithshme shkruhet:

```
enum e
{
    a0,
    a1,
    ...
    an
};
```

ku janë:

e - emri i grupit.

a0, a1, ..., an - anëtarët e grupit.

Anëtarët e grupit quhen edhe *numërues* (ang. enumerator) dhe në fakt paraqesin *konstante të emëruara*. Gjatë grupimit, çdo anëtar i grupit kompjuteri automatikisht i shoqëron një *vlerë të nënkuptuar* (ang. default value) në formë të numrit, duke filluar me vlerën **0**, e cila pastaj për çdo anëtar vijues rritet për **1**. Emri i grupit (**e**) si dhe emrat e anëtarëve që përfshihen në grup (**a0, a1, ..., an**) formohen në bazë të rregullave që vlejnë për identifikatorët.

Shembull

Programi **enum1a**, në të cilin definohet grupi **java**, i përbërë prej ditëve të javës.

```
// Programi enum1a
#include <iostream>
using namespace std;
enum java
{
    hene,
    marte,
    merkure,
    enjte,
    premte,
    shtune,
    diel
};
int main()
{
}
```

Me komandën **enum** këtu definohet grupi **java**, në të cilin përfshihen identifikatorë që kanë të bëjnë me ditët e javës. Gjatë përcaktimit të emrave të ditëve të javës, nuk janë shfrytëzuar shkronjat e alfabetit shqip, meqë rregullat për krijimin e identifikatorëve e ndalojnë.

Çdo identifikatori që përfshihet në grup, ashtu siç u tha edhe më sipër, kompjuteri i shoqëron një vlerë numerike. Kështu, identifikatorit **hene** ia shoqëronë vlerën **0**, identifikatorit **marte** - vlerën **1** dhe kështu me radhë. Për këtë arsye, *anëtarët e grupit paraqesin konstante*, dhe, kur flitet për grupe përmes komandës **enum**, duhet nënkuptuar *grupimin e konstanteve*.

Këtu, pjesa e programit është e zbrazët dhe grupi i përcaktuar nuk është përdorur për asgjë.

Grupi mund të përcaktohet duke e vendosur edhe brenda programit. Gjatë kësaj, programi i dhënë më sipër do të duket si në vijim.

```
// Programi enum1b
#include <iostream>
using namespace std;
int main()
{
    enum java
    {
        hene,
        marte,
        merkure,
        enjte,
        premte,
        shtune,
        diel
    };
}
```

Në këtë rast, grupi ka karakter lokal dhe mund të shfrytëzohet vetëm brenda programit, por jo edhe jashtë tij.

Shfrytëzimi i grupit

Pasi të jetë përcaktuar grupi, përmes emrit të tij mund të deklarohen *variabla të tipit të grupit*. Forma e deklarimit të variablave është plotësisht e njëjtë me deklarimin e variablave të tipeve standarde dhe në rast të përgjithshëm duket:

```
e v;
```

ku janë:

e - emri i grupit.

v - variabla që deklarohet e tipit të grupit të përcaktuar.

Shembull

Programi **enum2**, në të cilin definohet dhe shfrytëzohet grupi **java**, i përbërë prej ditëve të javës.

```
// Programi enum2
#include <iostream>
using namespace std;

enum java
{
    hene,
    marte,
    merkure,
    enjte,
    premte,
    shtune,
    diel
};

int main()
{
    java dita;
    dita=marte;

    cout << "\nDitës së martë i shoqërohet numri: "
          << dita
          << "\n\n";
    return 0;
}
```

Në program, përmes deklarimit:

```
java dita;
```

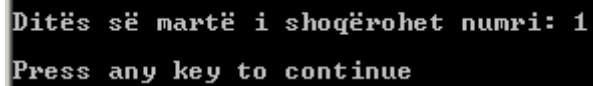
variabla **dita** deklarohet e tipit **java** dhe në të mund të ruhen të dhënat e përfshira brenda kllapave të grupit në fjalë.

Nëse ekzekutohet programi i dhënë, meqë përmes komandës:

```
dita=marte;
```

variablës **dita** të tipit **java** i është shoqëruar anëtar i dytë në grup, të cilit i korrespondon vlera numerike **1**, rezultati në ekran do të duket si në Fig.2.1.

Fig.2.1
Pamja e ekranit pas
ekzekutimit të programit
enum2



```
Ditës së martë i shoqërohet numri: 1
Press any key to continue
```

Variablës **dita** mund t'i shoqërohen vetëm anëtarët e grupit. Kështu, pavarësisht nga vlerat që u përkasin anëtarëve të grupit, shoqërimi direkt i vlerave nuk lejohet, përkatësisht kompjuteri do të lajmërojë gabim, nëse p.sh., në program shkruhet shprehja:

```
dita=1;
```

Anëtarët e grupit, gjatë përcaktimit të tij, mund të shkruhen në disa rreshta (ashtu siç u dha më sipër), ose edhe në vetëm një rresht, p.sh.:

```
enum java {hene,marte,merkure,enjte,premt,e,shtune,die};
```

Degëzimi përmes vlerave të numëruara

Vlerat e numëruara mund të shfrytëzohen për realizimin e degëzimeve të ndryshme edhe atë duke e përdorur komandën **if**, ose komandën **switch**.

Degëzimi me komandën if

Variablat, të cilat deklarohen si variabla të tipit të numëruar në kombinim me operatorët relacionalë, mund të shfrytëzohen për realizimin e degëzimeve të ndryshme përmes komandës **if**.

Shembull

Programi **enum3**, në të cilin tregohet degëzimi përmes komandës **if**, duke e krahasuar variablën e numëruar **koha**.

```
// Programi enum3
#include <iostream>
using namespace std;

enum rezultati
{
    Po,
    Jo
};

int main()
{
    rezultati koha;

    koha=Po;

    cout << "\nRezultati është ";

    if (koha==Po)
        cout << "pozitiv";
    else
        cout << "negativ";

    cout << "\n\n";
    return 0;
}
```

Në program, fillimisht është përcaktuar grupi **rezultati**, me dy anëtarët e tij **Po** dhe **Jo**. Pastaj, pas deklarimit të variablës **koha** të tipit rezultati, me shprehjen:

```
koha=Po;
```

variablës i shoqërohet vlera **Po** e grupit. Në fund, përmes komandës **if**, kontrollohet se a është barazi vlera e variablës **koha** me vlerën **Po** dhe varësisht nga raporti i tyre shtypet fjala **pozitiv** ose **negativ**. Në këtë rast, meqë variablës **koha** i është ndarë vlera **Po**, në ekran do të shtypet fjalia:

Rezultati është pozitiv

ku me ngjyrë të zezë është theksuar fjala që shtypet, duke kaluar në degën e parë të komandës **if**.

Degëzimi me komandën switch

Anëtarët e grupeve zakonisht shfrytëzohen për realizimin e degëzimeve të shumëfishta përmes komandës **switch**.

Shembull

Programi **enum4a**, përmes së cilit tregohet shfrytëzimi i komandës **switch** për degëzim të shumëfishtë në bazë të vlerave të anëtarëve të grupit **java**.

```
// Programi enum4a
#include <iostream>
using namespace std;

enum java
{
    hene,
    marte,
    merkure,
    enjte,
    premte,
    shtune,
    diel
};

int main()
{
    java dita;

    dita=marte;

    cout << "\nDita që u zgjodh është dita e ";
    switch (dita)
    {
        case hene:      cout << "hënë";break;
        case marte:     cout << "martë";break;
        case merkure:   cout << "mërkurë";break;
        case enjte:     cout << "enjte";break;
        case premte:    cout << "premtë";break;
        case shtune:    cout << "shtunë";break;
        case diel:      cout << "diel";break;
    }
    cout << "\n\n";
    return 0;
}
```

Këtu, me komandën **enum** para programit, definohet grupi **java**. Pastaj, duke e shfrytëzuar komandën **switch**, realizohet degëzimi në bazë të vlerave që u shoqërohen anëtarëve që përfshihen në grup. Kështu, meqë përmes shoqërimit:

```
dita=marte;
```

variablës **dita** të tipit **java** i shoqërohet numri rendor **1**, i cili i përket ditës së martë, me komandën për degëzim zgjedhet dega e dytë:

```
case marte:      cout << "martë";break;
```

me çka pjesës së fjalisë:

Dita që u zgjodh është dita e

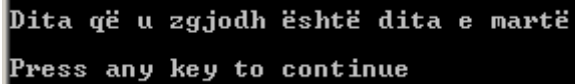
e cila shtypet para komandës **switch**, i shtohet edhe fjala **martë**, për ta fituar formën definitive të fjalisë:

Dita që u zgjodh është dita e **martë**

gjë që shihet edhe në rezultatin që me atë rast shtypet në ekran (shih *Fig.2.2*).

Fig.2.2

*Pamja e ekranit pas ekzekutimit të programit **enum4a***



Rezultati do të jetë i njëjtë nëse programi **enum4a** shkruhet ashtu siç është dhënë në vijim, tek i cili shfrytëzohet variabla **t** e tipit **string**.

```
// Programi enum4b
#include <iostream>
#include <string>
using namespace std;
```

```
enum java
{
    hene,
    marte,
    merkure,
    enjte,
    premte,
    shtune,
    diel
};
int main()
{
```

```

    java dita;
    string t;

    dita=marte;
    switch (dita)
    {
    case hene:      t= "hënë";break;
    case marte:     t= "martë";break;
    case merkure:   t= "mërkurë"; break;
    case enjte:     t= "enjte";break;
    case premte:    t= "premtë";break;
    case shtune:    t= "shtunë";break;
    case diel:      t= "diel";break;
    }
    cout << "\nDita që u zgjodh është dita e "
          << t
          << "\n\n";
    return 0;
}

```

Në program, fillimisht, është deklaruar variabla **t** e tipit **string**, në të cilën ruhen emrat e ditëve të javës. Meqë deklarimi **string** nuk është standard, për ta shfrytëzuar këtë deklarim në ballinën e programit është vendosur komanda paraprocesorike **#include <string>**, sepse në modulën **string** është përcaktuar deklarimi në fjalë. Edhe në këtë rast, pas ekzekutimit të programit të dhënë, rezultati në ekran do të duket si ai që u dha në *Fig.2.2*.

Rezultati i programit të dhënë më sipër nuk ndryshon nëse degët e veçanta të komandës **switch** shkruhen edhe kështu:

```

    switch (dita)
    {
    case 0:  t= "hënë";break;
    case 1:  t= "martë";break;
    case 2:  t= "mërkurë";break;
    case 3:  t= "enjte";break;
    case 4:  t= "premtë";break;
    case 5:  t= "shtunë";break;
    case 6:  t= "diel";break;
    }

```

Në këtë rast, në vend të emrave të ditëve të javës, meqë kemi të bëjmë me konstante, janë shfrytëzuar vlerat e nënkuptuara të tyre.

Disa variabla të numëruara të tipit të njëjtë

Sikurse te variablat e tipeve standarde, brenda një programi mund të deklarohen edhe disa variabla të numëruara të tipit të njëjtë. Gjatë kësaj, komanda përkatëse për deklarim në formë të përgjithshme do të duket:

```
e v1, v2, . . . , vn;
```

ku janë:

e - emri i grupit.

v1, v2, ..., vn - variablat që deklarohen të tipit të grupit të përcaktuar.

Shembull

Programi **enum5**, përmes së cilit tregohet deklarimi i dy variablave të numëruara të tipit **java**.

```
// Programi enum5
#include <iostream>
using namespace std;

enum java
{
    hene,
    marte,
    merkure,
    enjte,
    premte,
    shtune,
    diel
};

int main()
{
    java punuese;
    java pushuese;

    punuese=hene;
    pushuese=shtune;

    cout << "\nDitët e punës fillojnë me numrin: "
         << punuese
         << "\n\nDitët e pushimit fillojnë me numrin: "
         << pushuese
         << "\n\n";

    return 0;
}
```

Këtu, përmes komandave:

```
java punuese;  
java pushuese;
```

variablat **punuese** dhe **pushuese** janë deklaruar si variabla të numëruara të tipit **java**. Në këto variabla mund të ruhen të gjitha vlerat e mundshme që përfshihen në grupin në fjalë, pavarësisht se për cilën ditë të javës kemi të bëjmë.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.2.3.

Fig.2.3
Pamja e ekranit pas
ekzekutimit të programit
`enum5`

```
Ditët e punës fillojnë me numrin: 0  
Ditët e pushimit fillojnë me numrin: 5  
Press any key to continue
```

Nënkuptohet se edhe variablat e numëruara mund të deklarohen duke e shfrytëzuar vetëm një komandë:

```
java punuese, pushuese;
```

plotësisht njëloj siç deklarohen variablat standarde.

Përcaktimi dhe deklarimi njëkohësisht

Gjatë përcaktimit të grupit me të dhëna të numëruara, njëkohësisht mund të bëhet edhe deklarimi i variablës përkatëse. Për këtë qëllim, komanda **enum** shkruhet:

```
enum e  
{  
    a0,  
    a1,  
    ...  
    an  
}  
v;
```

ku janë:

- e** - emri i grupit.
- a0, a1, ..., an** - anëtarët e grupit.
- v** - variabla e tipit të grupit që përcaktohet.

Shembull

Programi **enum6**, përmes së cilit tregohet përcaktimi i grupit **viti**, si dhe njëkohësisht deklarohet variabla **stina** e tipit që definohet.

```
// Programi enum6
#include <iostream>
using namespace std;

enum viti
{
    pranvera,
    vera,
    vjeshta,
    dimri
}
stina;

int main()
{
    stina=vera;

    if (stina==pranvera)
        cout << "\nStina e pranverës";
    else
        if (stina==vera)
            cout << "\nStina e verës";
        else
            if (stina==vjeshta)
                cout << "\nStina e vjeshtës";
            else
                cout << "\nStina e dimrit";
    cout << "\n\n";
    return 0;
}
```

Siç shihet më sipër, para programit është përcaktuar grupi **viti**, duke e shënuar në fund edhe variablën **stina**. Në këtë mënyrë kompjuteri njoftohet se variabla **stina** është deklaruar variabël e tipit **viti** dhe si e tillë shfrytëzohet në program.

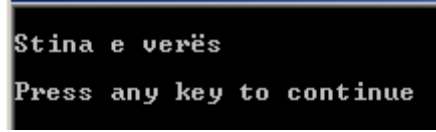
Nëse ekzekutohet programi në fjalë, meqë në fillim të tij paraqitet shprehja:

```
stina=vera;
```

rezultati që fitohet në ekran do të duket si në Fig.2.4.

Fig.2.4

Pamja e ekranit pas ekzekutimit të programit `enum6`



```
Stina e verës
Press any key to continue
```

Gjatë përcaktimit të grupit, njëkohësisht mund të deklarohen edhe më shumë variabla të atij tipit, duke e shkruar komandën **enum** në formë të përgjithshme:

```
enum e
{
    a0,
    a1,
    ...
    an
}
v1, v2, ..., vm;
```

ku janë:

e - emri i grupit.

a0, a1, ..., an - anëtarët e grupit.

v1, v2, ..., vm - variablat e tipit të grupit që përcaktohet.

Shembull

Programi **enum7**, përmes së cilit tregohet përcaktimi i grupit **viti**, si dhe njëkohësisht deklarohen variablat **stina** dhe **koha** të atij tipi.

```
// Programi enum7
#include <iostream>
using namespace std;
```

```
enum viti
{
    pranvera,
    vera,
    vjeshta,
```

```
        dimri
    }
    stina, koha;

int main()
{
    stina=vera;
    koha=dimri;

    if (stina==koha)
        cout << "\nPerioda e njëjtë";
    else
        cout << "\nPerioda ndryshon";
    cout << "\n\n";
    return 0;
}
```

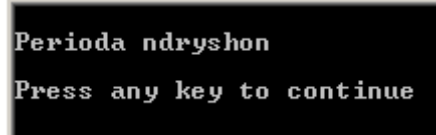
Këtu, menjëherë pas komandës paraprocesorike është përcaktuar grupi **viti**, si dhe njëkohësisht janë deklaruar të tipit **viti** variablat **stina** dhe **koha**. Pastaj, në fillim të tij janë marrë vlerat:

```
stina=vera;
koha=dimri;
```

Pas ekzekutimit të komandës **if**, për vlerat e marra të variablove **stina** dhe **koha**, rezultati që shtypet në ekran do të duket si në *Fig.2.5*.

Fig.2.5

Pamja e ekranit pas ekzekutimit të programit
enum7



```
Perioda ndryshon
Press any key to continue
```

Shoqërimi direkt i vlerave

Vlerat e nënkuptuara të anëtarëve të grupit mund edhe të ndryshohen, duke u shoqëruar direkt vlerat numerike. Ky shoqërim në formë të përgjithshme duket:


```
enum e
{
    a0=k0,
    a1=k1,
    ...
    an=kn
};
```

ku janë:

e - emri i grupit.

a0, a1, ..., an - anëtarët e grupit.

k1,k2,...,kn - konstantet që u shoqërohen anëtarëve të grupit.

Gjatë këtij shoqërimit vlerat e konstanteve zgjedhen sipas dëshirës, por mund të merren edhe të barabarta me vlerat e nënkuptuara të tyre.

Shembull

Programi **enum9**, përmes së cilit tregohet krahasimi i vlerave të anëtarëve të grupit.

```
// Programi enum9
#include <iostream>
using namespace std;
enum nota
{
    pese=5,
    gjashte=6,
    shtate=7,
    tete=8,
    nente=9,
    dhjete=10
};
int main()
{
    nota programim,vizatim;
    programim=tete;
    vizatim=nente;
    cout << "\nNota në programim është "
         << programim
         << "\n";
    cout << "Nota në vizatim është "
         << vizatim
         << "\n";
```

```
    if (programim < vizatim)
        cout << "\nNota në vizatim është më e madhe";
    else
        cout << "\nNota në vizatim s'është më e madhe";

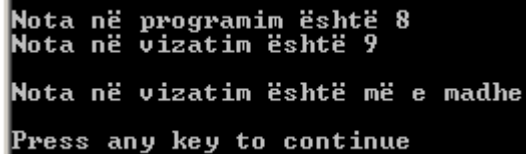
    cout << "\n\n";
    return 0;
}
```

Në program fillimisht është deklaruar grupi **nota**, në të cilin janë përfshirë emrat e notave të mundshme dhe vlerat e tyre. Pastaj, përmes komandave:

```
nota programim,vizatim;
programim=tete;
vizatim=nente;
```

së pari deklarohen variablat **programim** e **vizatim** të tipit **nota**, dhe pastaj atyre u shoqërohen vlerat **tete** dhe **nente**. Në fund, pasi shtypen vlerat e variablave në fjalë, përmes komandës **if** ato krahasohen dhe, duke e pasur parasysh raportin e tyre, shtypet rezultati i cili shihet në Fig.2.9.

Fig.2.9
Pamja e ekranit pas ekzekutimit të
programit **enum9**



```
Nota në programim është 8
Nota në vizatim është 9
Nota në vizatim është më e madhe
Press any key to continue
```

Disa anëtarë të grupit mund të kenë edhe vlera të njëjta.

Shembull

Programi **enum8**, përmes së cilit tregohet shoqërimi direkt i vlerave të anëtarëve të grupit **java**, në të cilin përfshihen ditët e javës.

```
// Programi enum8
#include <iostream>
#include <string>
using namespace std;
```

```
enum java
{
    hene=1,
    marte=1,
```

```

        merkure=1,
        enjte=1,
        premte=1,
        shtune=2,
        diel=2
};
int main()
{
    java dita;
    string t;
    dita=shtune;
    switch (dita)
    {
        case 1:  t= "ditë pune";break;
        case 2:  t= "ditë pushimi";break;
    }
    cout << "\nDita që u zgjodh është "
          << t
          << "\n\n";
    return 0;
}

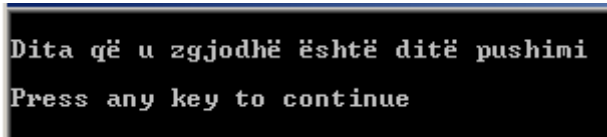
```

Në program, gjatë deklarimit të grupit, anëtarëve që kanë të bëjnë me ditët e javës mes të hënës dhe të premtes u janë shoqëruar vlerat **1**, kurse dy anëtarëve të fundit - vlerat **2**. Pastaj, meqë përmes barazimit:

```
dita=shtune;
```

është zgjedhur anëtari të cilit në grup i është shoqëruar vlera **2**, rezultati që shtypet në ekran do të duket si në Fig.2.6.

Fig.2.6
Pamja e ekranit pas ekzekutimit
të programit `enum8`



```

Dita që u zgjodhë është ditë pushimi
Press any key to continue

```

Vlerat të cilat u shoqërohen anëtarëve të grupit mund të jenë edhe të ndryshme.

Operimi me variabla të numëruara

Variabla e numëruar mund t'i shoqërohet një variable tjetër të numëruar, përmes operatorit të barazimit. Gjithashtu, vlerat e variablave të numëruara mund të shfrytëzohen gjatë llogaritjeve të ndryshme, ose të merren edhe si variabla të unazave.

Barazimi i variablave të numëruara

Mes dy variablave të numëruara, *të cilat janë të tipit të njëjtë*, mund të vendoset shenja e barazimit, në këtë mënyrë:

```
v1=v2;
```

ku janë:

v1, **v2** - variabla të tipit të njëjtë.

Si rezultat i këtij barazimi, variablës **v1** do t'i shoqërohet vlera e variablës **v2**, por me kusht që të dy variablat të jenë të tipit (të grupit) të njëjtë.

Shembull

Programi **enum10**, përmes së cilit tregohet barazimi i variablave **programim** dhe **vizatim**, të cilat janë deklaruar si variabla të tipit **nota**.

```
// Programi enum10
#include <iostream>
using namespace std;
enum nota
{
    pese,
    gjashte,
    shtate,
    tete,
    nente,
    dhjete
};
int main()
{
    nota programim,vizatim;
    programim=tete;
    vizatim=programim;
    cout << "\nNota në vizatim është ";

    switch(vizatim)
    {
        case pese:      cout << "pesë";break;
        case gjashte:   cout << "gjashtë";break;
        case shtate:    cout << "shtatë";break;
        case tete:      cout << "tetë";break;
        case nente:     cout << "nëntë";break;
        case dhjete:    cout << "dhjetë";break;
    }
}
```

```
    cout << "\n\n";
    return 0;
}
```

Në fillim të programit është përcaktuar variabla e numëruar **nota**, në të cilën përfshihen emrat e notave prej pesë deri në dhjetë. Pastaj, përmes komandës:

```
    nota programim,vizatim;
```

janë deklaruar variablat **programim** dhe **vizatim** të tipit **nota**. Me shprehjen:

```
    programim=tete;
```

variablës **programim** i është shoqëruar vlera **tete**. Kurse, përmes barazimit:

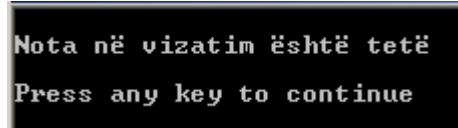
```
    vizatim=programim;
```

vlera e variablës **programim** i shoqërohet edhe variablës **vizatim**.

Pas ekzekutimit të programit në fjalë, rezultati në ekran do të duket si në Fig.2.7.

Fig.2.7

*Pamja e ekranit pas ekzekutimit të programit
enum10*



```
Nota në vizatim është tetë
Press any key to continue
```

Llogaritja me vlera të numëruara

Variablat e numëruara nuk mund të shfrytëzohen në shprehje të ndryshme për llogaritje. Por, duke e shfrytëzuar *operatorin për ndërrim eksplicit të tipit* (ang. explicit type conversion operator) **static_cast**, mund të rriten ose të zvogëlohen vlerat e të dhënave të tipit të numëruar.

Shembull

Programi **enum11**, përmes së cilit tregohet rritja për **2** e vlerës së variablës **lule**, e cila është deklaruar si variabël e tipit të numëruar **ngjyra**.

```
// Programi enum11
#include <iostream>
#include <string>
using namespace std;

enum ngjyra
{
    kuqe,
    kalter,
    bardhe,
    gjelber,
    verdhe,
};

int main()
{
    ngjyra lule;

    string g;
    lule=kalter;
    cout << "\nVlera fillestare: "
          << lule
          << " Ngjyrë e kaltër"
          << "\n";

    lule=static_cast<ngjyra>(lule+2);

    cout << "\nVlera e llogaritur: "
          << lule;
    switch (lule)
    {
        case 0: g="Ngjyrë e kuqe";break;
        case 1: g="Ngjyrë e kaltër";break;
        case 2: g="Ngjyrë e bardhë";break;
        case 3: g="Ngjyrë e gjelbër";break;
        case 4: g="Ngjyrë e verdhë";break;
    }
    cout << " "
          << g
          << "\n\n";
    return 0;
}
```

Në program, fillimisht është përcaktuar grupi i numëruar **ngjyra**, në të cilin përfshihen variabla për disa ngjyra. Pastaj, është deklaruar variabla **lule** e tipit **ngjyra**.

Përmes shprehjes:

```
lule=kalter;
```

është përcaktuar vlera fillestare e variablës **lule**. Në vijim, me shprehjen:

```
lule=static_cast<ngjyra>(lule+2);
```

kjo vlerë është rritur për **2**, gjë që shihet edhe nga rezultati i shtypjes së vlerës fillestare dhe i vlerës së rritur (shih. Fig.2.8).

Fig.2.8
Pamja e ekranit pas
ekzekutimit të programit
enum11

```
Ulera fillestare:  1  Ngjyrë e kaltër
Ulera e llogaritur: 3  Ngjyrë e gjelbër
Press any key to continue
```

Operatori **static_cast** ndryshe quhet *operator i kastës* (ang. cast operator), ose edhe *operatori për kastim të tipit* (ang. type casting operator).

Variablat e numëruara në unaza

Meqë mbi variablat e numëruara nuk lejohet zbatimi direkt i asnjë operacioni aritmetikor, si dhe i operacioneve për rritjen ose zvogëlimin e tyre, për t'i shfrytëzuar si variabla të unazave, rritja ose zvogëlimi i vlerave të tyre bëhet me ndërmjetësimin e *operatorit për ndërrim eksplicit të tipit* (ang. explicit type conversion operator) **static_cast**.

Shembull

Programi **enum12**, përmes së cilit shtypen **5** ngjyrat e përfshira në grupin **ngjyra**.

```
// Programi enum12
#include <iostream>
#include <string>
using namespace std;

enum ngjyra
{
    kuqe,
    kalter,
    bardhe,
    gjelber,
```

```
        verdhe,  
};  
  
int main()  
{  
    ngjyra lule;  
    string g;  
  
    for (lule=kuqe;lule<=verdhe;lule=static_cast<ngjyra>(lule+1))  
    {  
        switch (lule)  
        {  
            case 0: g="Ngjyrë e kuqe";break;  
            case 1: g= "Ngjyrë e kaltër";break;  
            case 2: g="Ngjyrë e bardhë";break;  
            case 3: g="Ngjyrë e gjelbër";break;  
            case 4: g="Ngjyrë e verdhë";break;  
        }  
  
        cout << "\nVlera: "  
              << lule  
              << " "  
              << g  
              << "\n";  
    }  
    cout << "\n";  
    return 0;  
}
```

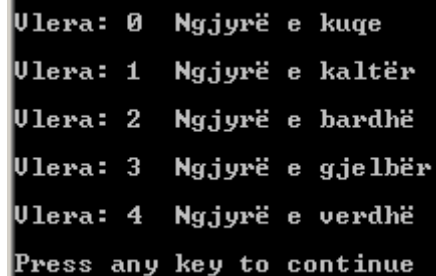
Në program, brenda komandës **for** është shfrytëzuar shprehja:

```
lule=static_cast<ngjyra>(lule+1)
```

me të cilën rritet për **1** vlera e variablës **lule**.

Nëse ekzekutohet programi i dhënë, rezultati që fitohet në ekran do të duket si në Fig.2.10.

Fig.2.10
Pamja e ekranit pas ekzekutimit të programit
enum12



```
Vlera: 0  Ngjyrë e kuqe  
Vlera: 1  Ngjyrë e kaltër  
Vlera: 2  Ngjyrë e bardhë  
Vlera: 3  Ngjyrë e gjelbër  
Vlera: 4  Ngjyrë e verdhë  
Press any key to continue
```


Por, variablat e numëruara mund të shfrytëzohen gjatë përcaktimit të vlerave që marrin pjesë në llogaritje të ndryshme.

Shembull

Programi **enum13**, përmes së cilit tregohet llogaritja e fitimit javor, nëse numri i orëve të punës në **5** ditët e punës së javës ruhet në vektorin **h**, kurse vlera e orës së punës **v** kompjuterit i jepet si vlerë hyrëse.

```
// Programi enum13
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

enum java
{
    hene,
    marte,
    merkure,
    enjte,
    premt
};

int main()
{
    double v,z,h[5]={6.5,8,7.4,2,9.6};
    char t[]="-----";
    string d;
    java dita;
    cout << "\nVlera e orës: ";
    cin >> v;
    cout << "\n      Dita      Orë      Vlera\n"
         << t;
    z=0;
    for (dita=hene;dita<=premt;dita=static_cast<java>(dita+1))
    {
        switch (dita)
        {
            case hene:d="e hënë";break;
            case marte:d="e martë";break;
            case merkure:d="e mërkurë";break;
            case enjte:d="e enjte";break;
            case premt:d="e premtë";break;
        }
    }
    cout << "\n"
         << setw(10)
```

```

        << d
        << fixed
        << setprecision(2)
        << setw(7)
        << h[dita]
        << setw(8)
        << v*h[dita];
    z=z+v*h[dita];
}
cout << "\n"
    << t
    << "\n          Totale:  "
    << fixed
    << setprecision(2)
    << z
    << "\n\n";
return 0;
}

```

Këtu, fillimisht është përcaktuar grupi **java** në të cilin përfshihen pesë ditët e punës në javë. Pastaj, gjatë deklarimit të variablave, për orët e punës në ditët e veçanta, vlerat përkatëse janë shënuar te vektori **h**. Vlera e orës **v** kompjuterit i jepet si vlerë hyrëse përmes tastierës. Pas kësaj, përmes unazës e cila është realizuar duke i shfrytëzuar vlerat e numëruara, është shtypur tabela e orëve të punës gjatë ditëve të veçanta dhe vlerat e tyre. Në fund është shtypur vlera totale, e cila është llogaritur si shumë e vlerave ditore, duke e shfrytëzuar shprehjen:

```
z=z+v*h[dita];
```

Rezultati që me këtë rast shtypet në ekran, për vlerën hyrëse **v=10.5**, do të duket si në Fig.2.11.

Fig.2.11

Pamja e ekranit pas ekzekutimit të programit
enum13

Vlera e orës: 10.5		
Dita	Orë	Vlera
e hënë	6.50	68.25
e martë	8.00	84.00
e mërkurë	7.40	77.70
e enjte	2.00	21.00
e premte	9.60	100.80
Totale:		351.75
Press any key to continue		

Leximi i të dhënave të numëruara

Leximi direkt i të dhënave të numëruara nuk është i mundshëm. Por, indirekt mund të lexohen vlerat në bazë të të cilave pastaj, përmes programit, variabla të numëruara u ndahen vlerat përkatëse.

Shembull

Programi **enum14**, përmes së cilit tregohet procedura e leximit indirekt të vlerave të anëtarëve të grupit.

```
// Programi enum14
#include <iostream>
using namespace std;

enum logjike {Po, Jo, Gabim};

int main()
{
    logjike alfa;

    char h;
    cout << "\nVlera hyrëse P për Po dhe J për Jo: ";
    cin >> h;

    cout << "\nPërgjigjja është ";
    switch (h)
    {
        case 'P':
            alfa=Po;
            cout << "Po";break;

        case 'J':
            alfa=Jo;
            cout << "Jo";break;

        default:
            alfa=Gabim;
            cout << "gabim";break;
    }
    cout << "\nVlera e variablës alfa: "
         << alfa
         << "\n\n";
}
```

Në program, fillimisht është përcaktuar grupi:

```
enum logjike {Po, Jo, Gabim};
```

në të cilin përfshihen **3** variabla. Pastaj, deklarohet variabla **alfa** e tipit **logjike**.

Meqë leximi direkt i vlerave të variablave që përfshihen në grup nuk lejohet, me qëllim të realizimit të leximit është shfrytëzuar variabla ndihmëse **h**, në të cilën mund të lexohet një simbol. Kështu, nëse pas ekzekutimit të programit, përmes tastierës si vlerë hyrëse shkruhet shkronja **P**, rezultati në ekran do të duket si në Fig.2.12.

Fig.2.12

*Pamja e ekranit pas
ekzekutimit të programit
enum14*

```
Ulera hyrëse P për Po dhe J për Jo: P
Përgjigjja është Po
Ulera e variablës alfa: 0
Press any key to continue . . .
```

Këtu, meqë variabla **Po** në grup është vendosur në pozitën e parë, vlera e nënkuptuar e saj është **0**, prandaj edhe shtypet kjo vlerë.

Definimi i disa grupeve njëkohësisht

Në një program njëkohësisht mund të definohen dhe të shfrytëzohen disa grupe. Pastaj, për secilin prej tyre mund të deklarohen variablat përkatëse.

Shembull

Programi **enum15**, përmes së cilit tregohet definimi i dy grupeve dhe shfrytëzimi i tyre.

```
// Programi enum15
#include <iostream>
using namespace std;
```

```
enum perioda
{
    paradite,
    pasdite,
    mbremje,
    pasmesnate,
};
```

```
enum koha
```

```
        {
            dite,
            nate,
        };

int main()
{
    int ora;

    perioda alfa;
    koha beta;

    cout << "\nShënoje orën mes 0 dhe 24: ";
    cin >> ora;

    if ((ora >=5) && (ora <= 12))
    {
        alfa=paradite;
        beta=dite;
    }
    else
        if ((ora >12) && (ora <= 18))
        {
            alfa=pasdite;
            beta=dite;
        }
        else
            if ((ora >18) && (ora <= 24))
            {
                alfa=mbremje;
                beta=nate;
            }
            else
            {
                alfa=pasmesnate;
                beta=nate;
            }

    cout << "\nNë orën e zgjedhur është ";

    switch (alfa)
    {
    case paradite:
        cout << "paraditë";
        break;

    case pasdite:
        cout << "pasditë";
        break;
```

```
    case mbremje:
        cout << "mbrëmje";
        break;

    case pasmesnate:
        cout << "pasmesnatë";
        break;
}

cout << "\nJashtë është ";
if (beta==dite)
    cout << "ditë";
else
    cout << "natë";

cout << "\n\n";
return 0;
}
```

Në program, fillimisht janë përcaktuar grupet **perioda** dhe **koha**, ashtu siç shihet më sipër. Pastaj, përmes komandave:

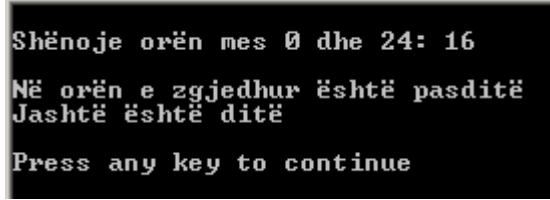
```
perioda alfa;
koha beta;
```

janë deklaruar variablat **alfa** dhe **beta**, të tipeve **perioda** dhe **koha** - përkatësisht.

Në pjesën vijuese të programit është paraparë që, duke ia dhënë kompjuterit si vlerë hyrëse orën aktuale (një numër të plotë mes **0** dhe **24**), përmes komandave **if** të zgjedhen vlerat e variablave **alfa** dhe **beta**, nga vlerat e mundshme të grupeve përkatëse. Zgjedhja bëhet në bazë të intervaleve kohore, duke e kontrolluar vlerën e variablës **ora**, ashtu siç shihet në pjesën përkatëse të programit. Nënkuptohej se intervalet e përcaktuara këtu, realisht mund të merren edhe ndryshe, në çka ka ndikim edhe stina e vitit, së cilës ato i përkasin.

Përmes degëzimeve në pjesën e fundit të programit, duke i pasur parasysh vlerat e përcaktuara të variablave **alfa** dhe **beta**, shtypet mesazhi për periodën kohore të ditës, si dhe informata se a është **ditë**, ose **natë**. Nëse, p.sh., kompjuterit si vlerë hyrëse për orën i jepet vlera **16**, rezultati që shtypet në ekran do të duket si në *Fig.2.13*.

Fig.2.13



```
Shënoje orën mes 0 dhe 24: 16
Në orën e zgjedhur është pasditë
Jashtë është ditë
Press any key to continue
```

Pamja e ekranit pas ekzekutimit të programit enum15

Logjika e funksionimit të programit të dhënë më sipër mbështetet në dy parakushte:

- dita fillon n'orën **5** dhe
- nata - n'orën **18**.

Ngjashëm, në një program mund të definoohen e të shfrytëzohen edhe me shumë grupe me të dhëna të numëruara.

Në një program nuk lejohen grupime në të cilat përfshihen të dhëna të njëjta. P.sh., brenda një programi nuk mund të shfrytëzohen grupimet vijuese:

```
enum obligative{Matematika, Fizika, Kompjuterika, Kimia}
enum zgjedhore{Programimi, Matematika, Fiskultura}
```

sepse **Matematika** paraqitet në të dy grupimet njëkohësisht.

Të dhënat e numëruara në nënprograme

Të dhënat e numëruara mund të shfrytëzohen edhe si parametra të funksioneve. Njëkohësisht, këto të dhëna mund të merren edhe si rezultate nga nënprogramet.

Shembull

Programi **enum16**, i cili paraqet një version të modifikuar të programit **enum14**, ashtu që për shtypje të rezultatit këtu shfrytëzohet funksioni **shtypja**.

```
// Programi enum16
#include <iostream>
using namespace std;
enum logjike {Jo, Po, Gabim};
void shtypja(logjike);
int main()
{
    logjike alfa;
    char h;
    cout << "\nVlera hyrëse P për Po, ose J për Jo: ";
    cin >> h;
    switch (h)
    {
        case 'P': alfa=Po; break;
        case 'J': alfa=Jo; break;
        default: alfa=Gabim; break;
    }
}
```

```
    shtypja(alfa);
    cout << "\n\n";
return 0;
}
void shtypja(logjike alfa)
{
    cout << "\nVlera e variablës: "
          << alfa;
    cout << "\nPërgjigjja është ";
    switch (alfa)
    {
        case Po:cout << "Po";break;
        case Jo:cout << "Jo";break;
        case Gabim:cout << "gabim";break;
    }
    return;
}
```

Në programin e dhënë, përmes komandës:

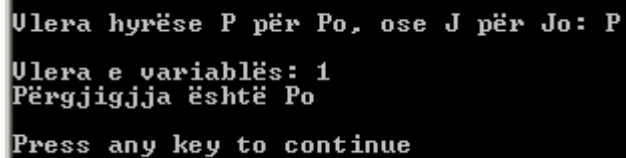
```
enum logjike {Jo,Po,Gabim};
```

është përcaktuar grupi i numëruar **logjike**, me tri variablat e tij: **Jo**, **Po** dhe **Gabim**. Njëkohësisht, në kuadër të programit është definuar nënprogrami **shtypja**, përmes së cilit shtypet teksti që i përgjigjet vlerës së zgjedhur të variablës **alfa**.

Gjatë ekzekutimit të programit të dhënë, kompjuterit fillimisht duhet t'i jepet vlera e variablës së tipit karakter **h**, e cila lexohet. Pastaj në bazë të kësaj vlere, përmes degëzimit **switch(h)**, variablës **alfa** i shoqërohet njëra nga tri vlerat e mundshme (**Po**, **Jo** ose **Gabim**).

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.2.14.

Fig.2.14
Pamja e ekranit pas
ekzekutimit të programit
enum16



```
Ulera hyrëse P për Po, ose J për Jo: P
Ulera e variablës: 1
Përgjigjja është Po
Press any key to continue
```


2

Strukturat

Definimi i strukturave të zakonshme	34
Deklarimi i variablave të tipit të strukturës	36
Qasja te komponentet e strukturës	37
Deklarimi direkt i variablës së strukturës	39
Inicializimi direkt i variablave	44
Llogaritje me variablat e strukturës	46
Ruajtja e rezultateve në strukturë	49
Disa variabla të një strukture	51
Përdorimi i operacionit të shoqërimit	60
Përdorimi i operatorëve relacionalë	62
Disa struktura njëkohësisht	65
Strukturat e ndërthurura	70
Strukturat si parametra të funksioneve	73
Disa nënprograme njëkohësisht	88
Funksionet në komponentet e strukturave	90
Fushat në struktura	124
Fushat e strukturave	128

Tipet standarde të variablave që shfrytëzohen në gjuhën C++ ofrojnë mundësi të shumta për ruajtjen dhe përpunimin e të dhënave. Në program, me deklarin e tipit të një variable, përcaktohet saktë hapësira memoruese, që shfrytëzohet prej saj, si dhe lloji i të dhënave që mund të ruhen në atë hapësirë.

Me qëllim të lehtësimit të punës me grumbuj të të dhënash të tipit të njëjtë, të cilat njihen edhe si të dhëna *homogjene*, shfrytëzohen fushat (vektorët, matricat ose fushat shumëdimensionale). Por, në gjuhën C++ mund të grupohen edhe të dhëna të tipeve të ndryshme, përkatësisht të dhëna *heterogjene*, duke krijuar tipe të reja të të dhënave, të cilat njihen si *struktura* (ang. structure).

Brenda strukturave mund të përfshihen edhe të dhëna të definuara nga vetë shfrytëzuesi, p.sh., siç janë të dhënat e tipeve të numëruara. Por, strukturat në vete mund të përmbajnë edhe funksione përmes të cilëve operohet me të dhënat e përfshira brenda ose jashtë tyre.

Definimi i strukturave të zakonshme

Strukturat, para se të shfrytëzohen, duhet të definohen, duke e përdorur komandën **struct**. Kjo komandë në formë të përgjithshme shkruhet:

```
struct e
{
    t1 x1;
    t2 x2;
    .....
    tn xn;
};
```

ku janë:

e - emri i strukturës.

t1, t2, ..., tn - tipet e të dhënave në komponentet e strukturës.

x1, x2, ..., xn - variablat në komponentet e strukturës.

Variablat që përfshihen në strukturë si dhe tipet e tyre shënohen brenda kllapave dhe njihen si *anëtarë të strukturës*, ose edhe *komponente të strukturës*. Definimi i strukturës përfundon me pikëpresje (;). Nëse, p.sh., duam që të dhënat e tipeve të ndryshme të një personi, siç janë: emri, viti i lindjes dhe qyteti

i banimit, t'i marrim si komponente të strukturës me emër **person**, definimi përkatës do të duket:

```
struct person
{
    char emri[8];
    int viti;
    char qyteti[10];
};
```

Në këtë mënyrë, të dhënave të tipeve të ndryshme u është shoqëruar emri i përbashkët **person**, me çka lehtësohet ruajtja dhe shfrytëzimi i tyre gjatë përpunimit.

Shembull

Programi **struct1**, në të cilin është definuar struktura **person**.

```
// Programi struct1
#include <iostream>
using namespace std;

struct person
{
    char emri[8];
    char qyteti[10];
    int viti;
};

int main()
{
}
```

Siç shihet nga programi i dhënë, këtu struktura është definuar para programit. Në fund të çdo komponente të përfshirë në strukturë shënohet pikëpresja. Kurse, në trupin e programit nuk paraqitet asnjë komandë, prandaj edhe nëse ai ekzekutohet nuk fitohen rezultate.

Definimi i strukturës në fjalë mund të duket edhe:

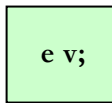
```
struct person
{
    char emri[8],qyteti[10];
    int viti;
};
```

Nga kjo shihet se gjatë definimit të strukturës komponentet e saj përcaktohen duke i shfrytëzuar rregullat që vlejnë për variablat e zakonshme.

Deklarimi i variablave të tipit të strukturës

Me definimin e një strukture kompjuteri nuk rezervon vende në memorien e tij për komponentet e saj, pavarësisht se deklarohen tipet e variablave përkatëse. Por, me strukturën *krijohet një tip i ri i të dhënave*, i cili pastaj mund të shfrytëzohet për deklarimin e tipeve të variablave, përkatësisht për deklarimin e objekteve përkatëse.

Në formë të përgjithshme, deklarimi i variablës së tipit të strukturës së definuar, duket:



```
e v;
```

ku janë:

e - emri i strukturës.

v - variabla e tipit të strukturës **e**.

Siç shihet nga kjo formë e përgjithshme, deklarimi i variablës së tipit të strukturës së definuar bëhet plotësisht njëloj siç deklarohen edhe variablat e tipeve të zakonshme. Për variablën **v** thuhet edhe se paraqet një objekt të strukturës **e**.

Shembull

Programi **struct2**, në të cilin shihet definimi i strukturës **person** dhe shfrytëzimi i saj për deklarimin e variablës **studenti** të tipit **person**.

```
// Programi struct2
#include <iostream>
using namespace std;

struct person
{
    char emri[8], qyteti[10];
    int viti;
};
int main()
{
    person studenti;
}
```

Në program, përmes deklarimit:

```
person studenti;
```

variabla **studenti** deklarohet e tipit **person**, i cili tip në fakt paraqet një kopje të strukturës që është definuar më parë. Pas këtij deklarimi, në memorien e kompjuterit rezervohen vende për variablat të cilat paraqiten në komponentet e strukturës, me radhë të njëjtë me të cilën janë shënuar brenda saj. Nëse ekzekutohet programi në fjalë, meqë në trupin e tij deklarohet variabla **studenti**, e cila nuk shfrytëzohet, kompjuteri do të gjenerojë një mesazh për të na njoftuar me këtë.

Kopja e strukturës e cila krijohet pas deklarimit të një variable të tipit të strukturës, ndryshe quhet *instancë e strukturës* (ang. instance of the structure), kurse variabla quhet *variabël e strukturës* (ang. structure variable), ose edhe *objekt i strukturës* (ang. structure object).

Nga kjo që u tha më sipër mund të shihet qartë se ku qëndron dallimi mes fushave dhe strukturave. Te fushat, p.sh., siç janë vektorët ose matricat, të gjitha të dhënat brenda tyre janë të tipit të njëjtë. Kurse, komponentet e strukturës mund të përmbajnë të dhëna nga më të ndryshmet, të cilat u ngjajnë *regjistrimeve* (ang. record), që shfrytëzohen te bazat e të dhënave.

Qasja te komponentet e strukturës

Për qasje tek anëtarët e fushave të ndryshme shfrytëzohen indeksat përkatëse, të shënuara brenda kllapave. Ndërkaq, qasja te komponentet e strukturës në formë të përgjithshme duket:

v.x

ku janë:

- v** - variabla e strukturës.
- x** - variabla e komponentes së strukturës.
- .** - operatori pikë (ang. dot operator) për qasje te variabla e komponentes së strukturës.

Shembull

Programi **struct3**, përmes së cilit tregohet qasja te komponentet e strukturës **person**.

```
// Programi struct3
#include <iostream>
using namespace std;

struct person
{
    char emri[8], qyteti[10];
    int viti;
};

int main()
{
    person studenti;

    cout << "\nTë dhënat nga tastiera\n\n";

    cout << "Emri .....: ";
    cin >> studenti.emri;
    cout << "Qyteti ...: ";
    cin >> studenti.qyteti;
    cout << "Viti .....: ";
    cin >> studenti.viti;

    cout << "\n\nTë dhënat e lexuara\n";
    cout << "\nEmri .....: "
        << studenti.emri;
    cout << "\nQyteti ...: "
        << studenti.qyteti;
    cout << "\nViti .....: "
        << studenti.viti
        << "\n\n";
    return 0;
}
```

Në program, tri variabla të përfshira në strukturën **person**, pas deklarimit të variablës **studenti** të tipit të kësaj strukture, u qasemi duke i shënuar ato në format:

```
studenti.emri
studenti.qyteti
studenti.viti
```

Nëse ekzekutohet programi i dhënë dhe pas mesazheve përkatëse, përmes tastierës kompjuterit i jepen vlerat hyrëse **Jeta, Shkupi** dhe **1983**, në ekran do ta kemi pamjen e cila shihet në *Fig.3.1*.

Fig. 3.1
Pamja e ekranit pas ekzekutimit të programit
struct3

```
Të dhënat nga tastiera
Emri .....: Jeta
Qyteti ....: Shkupi
Viti .....: 1983

Të dhënat e lexuara
Emri .....: Jeta
Qyteti ....: Shkupi
Viti .....: 1983
Press any key to continue
```

Deklarimi direkt i variablës së strukturës

Gjatë definimit të strukturës, njëkohësisht edhe mund të deklarohet variabla përkatëse e strukturës, gjë që në formë të përgjithshme duket:

```
struct e
{
    t1 x1;
    t2 x2;
    .....
    tn xn;
}
v;
```

ku janë:

- e** - emri i strukturës.
- t1, t2, ..., tn** - tipet e të dhënave në komponentet e strukturës.
- x1, x2, ..., xn** - variablat në komponentet e strukturës.
- v** - variabla e deklaruar e strukturës.

Nëse struktura e tillë krahasohet me definimin e përgjithshëm, që është dhënë në fillim, qartë shihet se në fund të saj është shënuar edhe variabla **v**, me çka ajo deklarohet direkt e tipit të strukturës që definohet.

Shembull

Programi **struct4a**, përmes së cilit definohet struktura **dita** me variablat **a** dhe **b** dhe deklarohet variabla **koha** e tipit të asaj strukture.

```
// Programi struct4a
#include <iostream>
using namespace std;
struct dita
{
    int a;
    float b;
}
koha;

int main()
{
    cout << "\nShfrytëzimi i variablës koha\n"
          << "-----";
    cout << "\n\nVlera e variablës a: ";
    cin >> koha.a;
    cout << "\nVlera e variablës b: ";
    cin >> koha.b;
    cout << "\n\nVlera e lexuar a="
          << koha.a;
    cout << "\nVlera e lexuar b="
          << koha.b
          << "\n\n";

    dita nata;

    cout << "\nShfrytëzimi i variablës nata\n"
          << "-----";
    cout << "\n\nVlera e variablës a: ";
    cin >> nata.a;
    cout << "\nVlera e variablës b: ";
    cin >> nata.b;
    cout << "\n\nVlera e lexuar a="
          << nata.a;
    cout << "\nVlera e lexuar b="
          << nata.b
          << "\n\n";
    return 0;
}
```

Në program, përmes deklarimit:

```
struct dita
{
    int a;
    float b;
}
koha;
```


definohet struktura **dita** dhe njëkohësisht deklarohet variabla me emrin **koha** e tipit të strukturës së definuar. Pastaj lexohen dhe shtypen vlerat e variablave **a** dhe **b** të përfshira brenda komponenteve të variablës **koha** të strukturës, duke i shkruar në formën:

```
koha.a
koha.b
```

Në fund, përmes komandës:

```
dita nata;
```

deklarohet variabla **nata**, e cila e paraqet një variabël tjetër të strukturës **dita**. Për qasje të variablave të komponenteve të strukturës **nata**, ato shkruhen në formën:

```
nata.a
nata.b
```

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.3.2, ku vlerat e variablave të komponenteve të strukturës kompjuterit i jepen përmes tastierës.

```
Shfrytëzimi i variablës koha
-----
Vlera e variablës a: 14
Vlera e variablës b: 8

Vlera e lexuar a=14
Vlera e lexuar b=8

Shfrytëzimi i variablës nata
-----
Vlera e variablës a: 5
Vlera e variablës b: 12

Vlera e lexuar a=5
Vlera e lexuar b=12
Press any key to continue
```

Fig.3.2
Pamja e ekranit pas ekzekutimit të
programit **struct4a**

Gjatë definimit të strukturës dhe deklarimit të variablës përkatëse, mund të shfrytëzohet edhe forma e strukturës e cila nuk ka emër:

```
struct
{
    t1 x1;
    t2 x2;
    .....
    tn xn;
}
v;
```

ku janë:

t1, t2, ..., tn - tipet e të dhënave në komponentet e strukturës.

x1, x2, ..., xn - variablat në komponentet e strukturës.

v - variabla e deklaruar e strukturës.

Nëse definimi i strukturës së tillë krahasohet me definimin e përgjithshëm, që është dhënë në fillim, qartë shihet se këtu struktura nuk ka emër. Por, variabla **v**, e cila është shënuar në fund, deklarohet direkt e tipit të strukturës që definohet.

Shembull

Programi **struct4b**, përmes së cilit tregohet deklarimi direkt i variablës **koha** të strukturës me variablat **a** dhe **b**, e cila deklarohet brenda trupit të programit.

```
// Programi struct4b
#include <iostream>
using namespace std;

int main()
{
    struct
    {
        int a;
        float b;
    }
    koha;

    cout << "\n\nVlera e variablës a: ";
    cin >> koha.a;
    cout << "\nVlera e variablës b: ";
    cin >> koha.b;
    cout << "\n\nVlera e lexuar a="
        << koha.a;
```

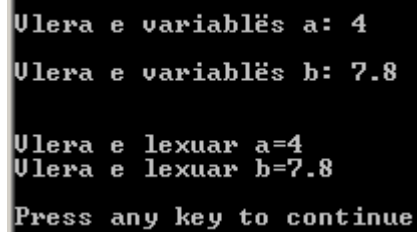
```

        cout << "\nVlera e lexuar b="
              << koha.b
              << "\n\n";
    return 0;
}

```

Meqë struktura e definuar nuk ka emër, ajo nuk mund të shfrytëzohet për deklarimin e variablove të tjera. Por, në program mund të shfrytëzohet vetëm variabla **koha**, e cila është deklaruar direkt gjatë definimit të strukturës.

Nëse ekzekutohet programi i dhënë, për vlera të caktuara hyrëse, të cilat kompjuterit i jepen përmes tastierës, rezultati në ekran do të duket si në Fig.3.3.



```

Vlera e variablës a: 4
Vlera e variablës b: 7.8

Vlera e lexuar a=4
Vlera e lexuar b=7.8
Press any key to continue

```

Fig.3.3

Pamja e ekranit pas ekzekutimit të programit struct4b

Efekt të njëjtë mund të ketë edhe deklarimi i strukturës jashtë trupit të programit.

Shembull

Programi **struct4c**, përmes së cilit tregohet deklarimi direkt i variablës **provimi** të strukturës me variablat **lenda**, **emri** dhe **nota**, e cila deklarohet jashtë trupit të programit.

```

// Programi struct4c
#include <iostream>
using namespace std;

struct
{
    char lenda[15];
    char emri[10];
    int nota;
}
provimi;

int main()
{
    cout << "\n\nTë dhënat që lexohen";
}

```

```
cout << "\n\nLënda: ";
cin >> provimi.lenda;
cout << "\nEmri i studentit: ";
cin >> provimi.emri;
cout << "\nNota në provim: ";
cin >> provimi.nota;

cout << "\n\nTë dhënat e lexuara";

cout << "\n\nLënda: "
      << provimi.lenda;
cout << "\nEmri i studentit: "
      << provimi.emri;
cout << "\nNota në provim: "
      << provimi.nota
      << "\n\n";
return 0;
}
```

Edhe këtu variablat e komponenteve të strukturës **provimi** mund të shfrytëzohen plotësisht njëllëj siç shfrytëzohen variablat e strukturave që deklarohen brenda programit. Si gjatë leximit, ashtu edhe gjatë shtypjes, ato shkruhen në format:

```
provimi.lenda
provimi.emri
provimi.nota
```

Por, edhe në këtë rast nuk mund të deklarohen variabla të tjera të strukturës së definuar, sepse ajo nuk ka emër.

Inicializimi direkt i variablave

Variablat të cilat përfshihen brenda komponenteve të një strukture, gjatë deklarimit të variablës së strukturës përkatëse në program, mund të inicializohen direkt me vlera.

Shembull

Programi **struct5**, përmes së cilit tregohet inicializimi direkt i variablave të strukturës dhe shtypja e vlerave të tyre.

```
// Programi struct5
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet trek = {4.5,7.4};

    cout << "\nVlerat e variablave të strukturës"
          << "\n\nVlera e brinjës a: "
          << trek.a;
    cout << "\nVlera e brinjës b: "
          << trek.b
          << "\n\n";
    return 0;
}
```

Në program, fillimisht, është definuar struktura **brinjet** me dy komponente. Variablat **a** dhe **b** të komponenteve të strukturës në fjalë janë deklaruar të tipit **double**. Pastaj, brenda programit, përmes shprehjes:

```
brinjet trek = {4.5,7.4};
```

është deklaruar variabla **trek** e tipit të strukturës **brinjet**. Njëkohësisht, gjatë deklarimit të kësaj strukture, pas barazimit, brenda kllapave janë përcaktuar edhe vlerat e variablave **a** dhe **b** të komponenteve të strukturës. Në fund, përmes komandave përkatëse, kompjuterit i është urdhëruar që t'i shtypë vlerat **4.5** dhe **7.4** të variablave në fjalë.

Formalisht, definimi i strukturës së përmendur më sipër mund të bëhet edhe kështu:

```
struct brinjet {double a,b};
```

gjë që është e ngjashme me shprehjen:

```
brinjet trek = {4.5,7.4};
```

e cila shfrytëzohet gjatë inicializimit të variablave të përfshira në komponentet e strukturës.

Llogaritje me variablat e strukturës

Variablat e përfshira në strukturë mund të shfrytëzohen gjatë llogaritjeve të ndryshme, plotësisht njëjloj siç shfrytëzohen edhe variablat e zakonshme. Por, gjatë kësaj, para variablave duhet të shënohet edhe emri i strukturës dhe pika si operator për qasje anëtarëve të strukturës.

Shembull

Programi **struct6**, përmes së cilit llogaritet sipërfaqja **s** e trekëndëshit kënddrejtë, brinjët e të cilit përfshihen në komponentet e variablës **trek** të strukturës **brinjet**.

```
// Programi struct6
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet trek = {4.5,7.4};
    double s;

    cout << "\nVlerat e variablave të strukturës"
          << "\n\nVlera e brinjës a: "
          << trek.a;
    cout << "\nVlera e brinjës b: "
          << trek.b;

    s=(trek.a*trek.b)/2;

    cout << "\n\nSipërfaqja e trekëndëshit s="
          << s
          << "\n\n";
    return 0;
}
```

Meqë në matematikë sipërfaqja **s** e trekëndëshit kënddrejtë me brinjët **a** dhe **b** llogaritet përmes shprehjes:

$$s = \frac{a \cdot b}{2}$$

në program shprehja përkatëse për llogaritjen e saj është shkruar:

```
s=(trek.a*trek.b)/2;
```

Rezultati që fitohet në ekran, pas ekzekutimit të programit të dhënë, do të duket si në Fig.3.4.

Fig.3.4

Pamja e ekranit pas ekzekutimit të programit **struct6**

```
Vlerat e variablave të strukturës
Vlera e brinjës a: 4.5
Vlera e brinjës b: 7.4
Sipërfaqja e trekëndëshit s=16.65
Press any key to continue
```

Nëse variablat e strukturës paraqiten në më shumë shprehje, shfrytëzimi i tyre i bën shprehjet më të gjata, sepse para çdo variable duhet të shënohet emri i strukturës dhe pika si operator për qasje tek anëtari i komponentes së strukturës. Me qëllim të shfrytëzimit të shprehjeve të zakonshme, vlerat e variablave të strukturës mund të ruhen edhe në variabla të zakonshme, të cilat nuk dallojnë nga variablat e komponenteve të strukturës.

Shembull

Programi **struct7**, i cili paraqet një version të modifikuar të programit **struct6**, në të cilin është shtuar edhe llogaritja e brinjës **c** të trekëndëshit dhe e perimetrit **p** të tij.

```
// Programi struct7
#include <iostream>
#include <math.h>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet trek = {4.5,7.4};

    double a,b,c,s,p;

    a=trek.a;
    b=trek.b;
```

```
cout << "\nVlerat e variablave të strukturës"
      << "\n\nVlera e brinjës a: "
      << a;
cout << "\nVlera e brinjës b: "
      << b;

s=(a*b)/2;
cout << "\n\nSipërfaqja e trekëndëshit s="
      << s;

c=sqrt(a*a+b*b);
cout << "\nGjatësia e brinjës c="
      << c;

p=a+b+c;
cout << "\nPerimetri i trekëndëshit p="
      << p
      << "\n\n";
return 0;
}
```

Në fillim të programit, përmes shprehjeve:

```
a=trek.a;
b=trek.b;
```

vlerat e variablave të komponenteve të strukturës ruhen te variablat e veçanta **a** dhe **b** (emrat e këtyre variablave mund të zgjedhen lirisht edhe ndryshe).

Në program, për llogaritje të gjatësisë së brinjës **c** është shfrytëzuar shprehja:

$$\mathbf{c} = \sqrt{\mathbf{a}^2 + \mathbf{b}^2}$$

e nxjerrë nga Teorema e Pitagorës:

$$\mathbf{c}^2 = \mathbf{a}^2 + \mathbf{b}^2$$

Për llogaritje të perimetrit **p** të trekëndëshit është shfrytëzuar shprehja e zakonshme:

$$\mathbf{p} = \mathbf{a} + \mathbf{b} + \mathbf{c}$$

Pas ekzekutimit të programit, për vlerat e brinjëve **4.5** dhe **7.4** të cilat kompjuterit i janë dhënë gjatë deklarimit të variablës **trek** të tipit **brinjet**, rezultati në ekran do të duket si në *Fig.3.5*.

Fig.3.5
Pamja e ekranit pas ekzekutimit të programit *struct7*

```
Ulerat e variablave të strukturës
Ulera e brinjës a: 4.5
Ulera e brinjës b: 7.4

Sipërfaqja e trekëndëshit s=16.65
Gjatësia e brinjës c=8.66083
Perimetri i trekëndëshit p=20.5608
Press any key to continue
```

Ruajtja e rezultateve në strukturë

Për ruajtjen e rezultateve të llogaritjeve të ndryshme mund të shfrytëzohen edhe variablat e komponenteve të strukturës.

Shembull

Programi **structA**, përmes së cilit llogaritet vlera e faktorielit:

$$F = (2m - n + 1)!$$

nëse variablat **m** dhe **n** dhe rezultati **F** ruhen në komponentet e strukturës **Alfa**.

```
// Programi structSa
#include <iostream>
using namespace std;

struct Alfa
{
    int m,n;
    double F;
};

int main()
{
    Alfa Fakt;

    int i;

    cout << "\nVlerat hyrëse"
    << "\n\nVariabla m: ";
```

```
cin >> Fakt.m;
cout << "\nVariabla n: ";
cin >> Fakt.n;

Fakt.F=1;
for (i=1;i<=(2*Fakt.m-Fakt.n+1);i++)
    Fakt.F=Fakt.F*i;

cout << "\nVlera e llogaritur F="
      << Fakt.F
      << "\n\n";
return 0;
}
```

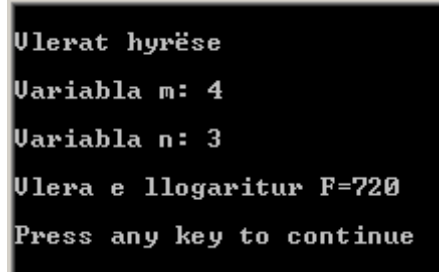
Këtu, fillimisht, është definuar struktura **Alfa**, në komponentet e së cilës paraqiten variablat **m**, **n** dhe **F**. Pastaj deklarohet variabla **Fakt** e tipit të strukturës **Alfa**, në variablat e komponenteve të së cilës ruhen dy vlerat e lexuara **Fakt.m** dhe **Fakt.n**. Në fund, duke e shfrytëzuar pjesën e programit:

```
Fakt.F=1;
for (i=1;i<=(2*Fakt.m-Fakt.n+1);i++)
    Fakt.F=Fakt.F*i;
```

përmes unazës **for** llogaritet vlera e faktorielit. Siç shihet nga komandat e dhëna, në anën e majtë të barazimit paraqitet variabla **Fakt.F**, gjë që d.m.th. se rezultati i llogaritjes së faktorielit përcillet te variabla **F** e strukturës. Meqë edhe në komandën **cout** paraqitet kjo variabël, vlera e faktorielit, që shtypet në ekran, do të duket si në *Fig.3.6*.

Fig.3.6

Pamja e ekranit pas ekzekutimit të programit structA



```
Vlerat hyrëse
Variabla m: 4
Variabla n: 3
Vlera e llogaritur F=720
Press any key to continue
```

Gjatë zgjedhjes së problemit të dhënë struktura shfrytëzohet vetëm me qëllim që të tregohet përdorimi i saj. Zgjedhja do të jetë më e thjeshtë pa e shfrytëzuar strukturën, ashtu siç shihet në vijim.

```
// Programi structSb
#include <iostream>
using namespace std;
```

```

int main()
{
    int m,n,i;
    double F;

    cout << "\nVlerat hyrëse"
          << "\n\nVariabla m: ";
    cin >> m;

    cout << "\nVariabla n: ";
    cin >> n;

    F=1;
    for (i=1;i<=(2*m-n+1);i++)
        F=F*i;

    cout << "\nVlera e llogaritur F="
          << F
          << "\n\n";
    return 0;
}

```

Këtu, variablat që shfrytëzohen janë deklaruar si variabla të zakonshme në fillim të programit.

Nëse ekzekutohet programi i dhënë, për vlerat hyrëse të variablave që lexohen, rezultati nuk do të dallojë aspak nga ai që u dha në *Fig.3.6*.

Disa variabla të një strukture

Struktura e definuar mund të shfrytëzohet për deklarimin e disa variablave brenda programit të tipit të strukturës së definuar. Ky deklarim bëhet pasi të jetë definuar struktura, ose edhe gjatë definimit të strukturës.

Deklarimi pasi është definuar struktura

Për deklarimin e njëkohshëm të disa variablave të strukturës, shfrytëzohet shprehja e formës:

e v1, v2, . . . , vm;

ku janë:

e - emri i strukturës.

v1, v2, ..., vm - variablat të cilat deklarohen të tipit të strukturës **e**.

Kështu, p.sh., deklarimi i variablave **profesori** dhe **studenti** të tipit **person** (që si shembull u përmend më sipër) bëhet njëloj siç deklarohen tipet e variablave të zakonshme:

```
person profesori;  
person studenti;
```

Ky deklarim mund të bëhet më thjesht edhe në një rresht:

```
person profesori, studenti;
```

ku, siç shihet, fillimisht, shënohet tipi dhe pastaj variablat të ndara mes vete me presje. Kjo duket njëloj si edhe gjatë deklarimit, p.sh., të variablave **x** dhe **y** si variabla të zakonshme të tipit **int**:

```
int x,y;
```

Shembull

Programi **struct8**, përmes së cilit tregohet shfrytëzimi i dy variablave të strukturës **person**.

```
// Programi struct8  
#include <iostream>  
using namespace std;  
  
struct person  
{  
    char emri[8], qyteti[10];  
    int viti;  
};  
  
int main()  
{  
    person profesori, studenti;  
  
    cout << "\nTë dhënat nga tastiera\n\n";  
  
    cout << "Qyteti i profesorit ...: ";  
    cin >> profesori.qyteti;  
    cout << "Qyteti i studentit ....: ";  
    cin >> studenti.qyteti;  
    cout << "\n\nTë dhënat e lexuara\n";  
    cout << "\nProfesori është nga "  
        << profesori.qyteti;
```

```

        cout << "\nStudenti është nga "
              << studenti.qyteti
              << "\n\n";
    return 0;
}

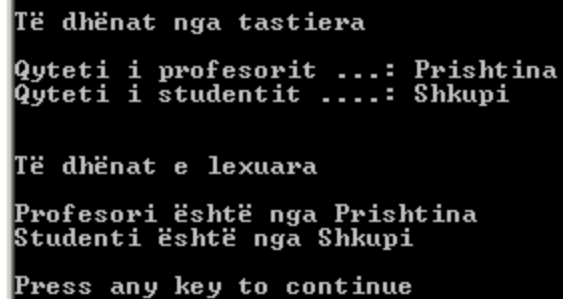
```

Në programin e dhënë, fillimisht, është deklaruar struktura **person** ashtu siç është shpjeguar edhe në shembujt e mësipërm. Pastaj, duke e shfrytëzuar këtë tip të ri të të dhënave, përmes shprehjes:

```
person profesori, student;
```

janë deklaruar dy variabla të tipit **person**. Si rezultat i deklarimit në fjalë, variablat **profesori** dhe **studenti** do t'i kenë strukturat e njëjta me strukturën **person**.

Në pjesën vijuese të programit është paraparë që, pasi të lexohen emrat e qyteteve të profesorit dhe të studentit, ato edhe të shtypen. Pas ekzekutimit të programit, nëse përmes tastierës kompjuterit i jepen qytetet **Prishtina** dhe **Shkupi**, rezultati në ekran do të duket si në Fig.3.7.



```

Të dhënat nga tastiera
Qyteti i profesorit ...: Prishtina
Qyteti i studentit ....: Shkupi

Të dhënat e lexuara
Profesori është nga Prishtina
Studenti është nga Shkupi

Press any key to continue

```

Fig.3.7

*Pamja e ekranit pas ekzekutimit të programit **struct8***

Gjatë shfrytëzimit të variablave të tipit të strukturës, operohet me emrat e variablave të cilat janë përfshirë në definicionin e saj, duke e shënuar para operatorit për qasje në strukturë (para pikës) emrin e variablës përkatëse. Kështu, në programin e dhënë më sipër shihet qartë shfrytëzimi i variablës **emri** te dy variablat **profesori** dhe **studenti**, të cilat janë deklaruar të tipit të strukturës **person**, duke i shkruar në format:

```
profesori.emri
studenti.emri
```

Ngjashëm veprohet edhe nëse shfrytëzohen dy variablat e tjera të strukturës **person**, të cilat duhet të shkruhen:

```
profesori.viti
studenti.viti
```

Shembull

Programi **struct9** përmes së cilit tregohet përdorimi i dy variablave **trek** dhe **kater** të strukturës **brinjet** e cila është shfrytëzuar më parë.

```
// Programi struct9
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet trek = {4.5,7.4};
    brinjet kater = {6,4};

    double st,sk;

    cout << "\nVariabla trek e strukturës brinjet"
         << "\n\nVlera e brinjës a: "
         << trek.a;
    cout << "\nVlera e brinjës b: "
         << trek.b;

    cout << "\n\nVariabla kater e strukturës brinjet";

    cout << "\n\nVlera e brinjës a: "
         << kater.a;
    cout << "\nVlera e brinjës b: "
         << kater.b;

    st=(trek.a*trek.b)/2;
    sk=kater.a*kater.b;

    cout << "\n\nSipërfaqja e trekëndëshit st="
         << st
         << "\nSipërfaqja e katërkëndëshit sk="
         << sk
         << "\n\n";
    return 0;
}
```

Në program janë llogaritur dhe janë shtypur sipërfaqet e trekëndëshit (**st**) dhe të katërkëndëshit kënddrejtë (**sk**), të cilat në matematikë llogariten përmes shprehjeve:

$$s_t = \frac{a \cdot b}{2}$$

$$s_k = a \cdot b$$

Variablat e strukturës **brinjet**, e cila është definuar, janë deklaruar në fillim të programit përmes shprehjeve:

```
brinjet trek = {4.5, 7.4};
brinjet kater = {6, 4};
```

Këtu, gjatë deklarimit, njëkohësisht, brinjëve të trekëndëshit dhe të katërkëndëshit u janë ndarë vlera, duke i shënuar ato brenda kllapave. Rezultati i cili fitohet pas ekzekutimit të programit të dhënë, do të duket si në Fig.3.8.

```
Variabla trek e strukturës brinjet
Vlera e brinjës a: 4.5
Vlera e brinjës b: 7.4

Variabla kater e strukturës brinjet
Vlera e brinjës a: 6
Vlera e brinjës b: 4

Sipërfaqja e trekëndëshit st=16.65
Sipërfaqja e katërkëndëshit sk=24

Press any key to continue
```

Fig.3.8
Pamja e ekranit pas ekzekutimit
të programit **struct9**

Deklarimi dhe inicializimi i dy variablave **trek** dhe **kater** të strukturës **brinjet** mund të bëhet edhe në këtë mënyrë:

```
brinjet trek = {4.5, 7.4}, kater = {6, 4};
```

Deklarimi gjatë definimit të strukturës

Më parë u shpjegue deklarimi i variablës së strukturës gjatë definimit të saj. Plotësisht njëloj do të duket komanda **struct** edhe nëse shfrytëzohet për deklarimin e njëkohshëm të më shumë variablave të një strukture:

```
struct e
{
    t1 x1;
    t2 x2;
    .....
    tn xn;
}
v1, v2, ..., vm;
```

ku janë:

e - emri i strukturës.

t1, t2, ..., tn - tipet e të dhënave në komponentet e strukturës.

x1, x2, ..., xn - variablat në komponentet e strukturës.

v1, v2, ..., vm - variablat të cilat deklarohen të tipit të strukturës **e**.

Definimi i strukturës në fjalë mund të bëhet para programit, ose edhe brenda programit.

Shembull

Pjesa fillestare e programit **struct10a**, i cili e paraqet një version të modifikuar të programit **struct9**, përmes së cilit tregohet deklarimi i variablave **trek** dhe **kater** gjatë definimit të strukturës **brinjet**.

```
// Programi struct10a
#include <iostream>
using namespace std;
struct brinjet
{
    double a,b;
}
trek,kater;

int main()
{
    trek.a = 4.5;
    trek.b = 7.4;
    kater.a = 6;
    kater.b = 4;
    double st,sk;

    .....

    return 0;
```


}

Siç shihet nga pjesa e dhënë e programit, gjatë definimit të strukturës **brinjet**, njëkohësisht, janë deklaruar edhe variablat **trek** dhe **kater** të kësaj strukture. Pastaj, në program, variablat **a** dhe **b** brenda komponenteve të strukturës, për të dy variablat e strukturës, u janë ndarë vlerat në rrugë direkte. Nëse pjesa tjetër e programit merret e njëjtë me atë të programit **struct9**, pas ekzekutimit të tij, rezultati do të duket plotësisht njëjloj si edhe ai i dhënë në *Fig.3.8*.

Gjatë shfrytëzimit të kësaj forme të deklarimit të variablatve të strukturës, variablatve brenda komponenteve të strukturës vlerat mund t'u ndahen edhe direkt.

Shembull

Pjesa fillestare e programit **struct10b**, si një version i modifikuar i programit **struct10a**, përmes së cilit tregohet deklarimi i variablatve **trek** dhe **kater** gjatë definimit të strukturës **brinjet** dhe inicializimi i variablatve të komponenteve përkatëse **a** dhe **b** me vlera.

```
// Programi struct10b
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
}
trek={4.5,7.4},kater={6,4};

int main()
{
    double st,sk;

    .....

    return 0;
}
```

Nga kjo shihet se inicializimi me vlera i variablatve brenda komponenteve të strukturës është i ngjashëm me atë që është dhënë te programi **struct9**, por këtu kjo bëhet gjatë definimit të strukturës. Nëse edhe në këtë rast pjesa tjetër e programit merret e njëjtë me atë të programit **struct9**, pas ekzekutimit të tij, rezultati nuk do të dallojë nga ai që është dhënë në *Fig.3.8*.

Kur shfrytëzohet forma e dhënë, përmes emrit të strukturës, përveç variablave që deklarohen gjatë definimit të strukturës, në program mund të deklarohen edhe variabla të tjera të tipit të saj.

Shembull

Programi **struct11**, tek i cili gjatë definimit të strukturës **brinjet** deklarohen variablat **trek** dhe **kater**, kurse brenda programit deklarohet edhe variabla **rrethi** e kësaj strukture.

```
// Programi struct11
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
}
trek={4.5,7.4}, kater={6,4};

int main()
{
    brinjet rrethi={3.1415926,5};

    double st,sk,sr;

    cout << "\nSipërfaqet e llogaritura:\n";

    st=(trek.a*trek.b)/2;
    sk=kater.a*kater.b;
    sr=rrethi.a*(rrethi.b*rrethi.b);

    cout << "\n\nSipërfaqja e trekëndëshit ... st="
         << st
         << "\nSipërfaqja e katërkëndëshit . sk="
         << sk
         << "\nSipërfaqja e rrethit ..... sr="
         << sr
         << "\n\n";

    return 0;
}
```

Në program, gjatë definimit të strukturës **brinjet**, ashtu si edhe në shembujt **struct10a** dhe **struct10b**, janë deklaruar dhe janë inicializuar variablat **trek** dhe **kater**. Njëkohësisht, por brenda programit, është deklaruar dhe është inicializuar struktura **rrethi** e tipit **brinjet**, në të cilën ruhet vlera e konstantes $\pi=3.1415926$ dhe e rrezes së rrethit $r=5$. Meqë në matematikë sipërfaqja e rrethit llogaritet përmes shprehjes:

$$s = \pi \cdot r^2$$

këtu, pasi vlerat përkatëse ruhen te variablat e komponenteve të variablës **rrethi** të strukturës **brinjet**, shprehja përkatëse është shkruar në formën:

```
sr=rrethi.a*(rrethi.b*rrethi.b);
```

Në vend të pjesës nën kllapa mund të shfrytëzohet funksioni **pow**, kështu:

```
sr=rrethi.a*pow(rrethi.b,2);
```

Nëse ekzekutohet programi i dhënë, rezultati që shtypet në ekran do të duket si në *Fig.3.9*.

Fig.3.9
Pamja e ekranit pas
ekzekutimit të programit
struct11

```
Sipërfaqet e llogaritura:

Sipërfaqja e trekëndëshit ... st=16.65
Sipërfaqja e katërkëndëshit . sk=24
Sipërfaqja e rrethit ..... sr=78.5398

Press any key to continue
```

Deklarimi i variablave të strukturës gjatë definimit të saj mund të bëhet edhe duke e lënë strukturën pa emër, kështu:

```
struct
{
    t1 x1;
    t2 x2;
    .....
    tn xn;
}
v1,v2,...,vm;
```

ku janë:

t1, t2, ..., tn - tipet e të dhënave në komponentet e strukturës.

x1, x2, ..., xn - variablat në komponentet e strukturës.

v1, v2,..., vm - variablat të cilat deklarohen të tipit të strukturës.

Në këtë rast, variablat e deklaruar shfrytëzohen ashtu siç u shpjegua edhe më sipër. Por, këtu nuk mund të deklarohen variabla të tjera të tipit të strukturës së definuar, përveç atyre që janë deklaruar gjatë definimit të saj, sepse struktura nuk ka emër.

Shembull

Programi **struct12**, i ngjashëm me programin **struct11**, tek i cili struktura definohet pa emër dhe njëkohësisht gjatë kësaj deklarohen dhe inicializohen variablat **trek**, **kater** dhe **rrethi**.

```
// Programi struct12
#include <iostream>
#include <math.h>
using namespace std;

struct
{
    double a,b;
}
trek={4.5,7.4},kater={6,4},rrethi={3.1415926,1};

int main()
{
    double st,sk,sr;

    .....

    return 0;
}
```

Këtu, inicializimi i variablave të strukturës nuk është i domosdoshëm. Nëse pjesa tjetër e programit merret plotësisht e njëjtë me atë që shihet te programi **struct11**, rezultati që fitohet në ekran me ekzekutimin e tij do të duket ashtu siç është dhënë në *Fig.3.9*.

Përdorimi i operacionit të shoqërimit

Nëse në program janë deklaruar dy variabla **s1** dhe **s2** të strukturës **s**, përmes operacionit të shoqërimit:

s1=s2;

vlerrat e variablave të komponenteve të variablës **s2** u shoqërohen variablave përkatëse të komponenteve të variablës **s1**.

Shembull

Programi **struct13** përmes së cilit tregohet shoqërimi i vlerave të variablave të komponenteve të variablës **trek** të strukturës **brinjet**, e cila është shfrytëzuar në shembujt e mësipërm, variablave përkatëse të komponenteve të variablës **kater**.

```
// Programi struct13
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet trek = {4.5,7.4};
    brinjet kater;

    kater=trek;

    cout << "\nVlerat e variablave në dy strukturat";
    cout << "\n\nBrinjët e trekëndëshit:"
        << "\n\nBrinja a: "
        << trek.a
        << "\nBrinja b: "
        << trek.b;

    cout << "\n\nBrinjët e katërkëndëshit:"
        << "\n\nBrinja a: "
        << kater.a
        << "\nBrinja b: "
        << kater.b
        << "\n\n";
    return 0;
}
```

Në fillim të programit të dhënë, gjatë deklarimit të variablave **trek** dhe **kater** të strukturës **brinjet**, përmes deklarimit:

```
brinjet trek = {4.5,7.4};
```

variablave të komponenteve të variablës **trek** u janë shoqëruar vlerat përkatëse. Pastaj, përmes shprehjes:

```
kater=trek;
```

përkatësisht me operacionin e shoqërimit, variabla të komponenteve të variablës **kater**:

```
kater.a
kater.b
```

u shoqërohen vlerat e variabla të komponenteve të variablës **trek**. Nëse ekzekutohet programi, rezultati në ekran do të duket si në Fig.3.10.

Fig.3.10
Pamja e ekranit pas
ekzekutimit të programit
struct13

```
Ulerat e variabla në dy strukturat
Brinjët e trekëndëshit:
Brinja a: 4.5
Brinja b: 7.4
Brinjët e katërkëndëshit:
Brinja a: 4.5
Brinja b: 7.4
Press any key to continue
```

Nga rezultati i dhënë shihet se brinjët e trekëndëshit dhe të katërkëndëshit janë të barabarta, gjë që është rezultat i barazimit të dy variabla të strukturës.

Brinjëve të katërkëndëshit në programin e mësipërm mund t'u shoqërohen vlerat e brinjëve të trekëndëshit, nëse në vend të shprehjes:

```
kater=trek;
```

për secilën komponente shfrytëzohet operacioni i barazimit:

```
kater.a=trek.a;
kater.b=trek.b;
```

Përdorimi i operatorëve relacionalë

Operatorët relacionalë nuk mund të zbatohen direkt mbi strukturat, por vetëm mbi komponentet përkatëse të tyre.

Shembull

Programi **struct14**, përmes së cilit tregohet krahasimi i vlerave të variabla të dy komponenteve të strukturës **brinjet**.

```
// Programi struct14
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet kater;
    double s;
    cout << "\nVlera e brinjës a: ";
    cin >> kater.a;
    cout << "\nVlera e brinjës b: ";
    cin >> kater.b;

    s=kater.a*kater.b;

    if (kater.a==kater.b)
        cout << "\nSipërfaqja e katrorit s="
            << s
            << "\n\n";
    else
        cout << "\nSipërfaqja e katërkëndëshit s="
            << s
            << "\n\n";
    return 0;
}
```

Këtu, fillimisht, është definuar struktura **brinjet**, e cila përmban dy komponente me variablat **a** dhe **b**. Pastaj, përmes shprehjes:

```
brinjet kater;
```

në program është deklaruar variabla **kater** e strukturës **brinjet**, në të cilën është menduar të ruhen vlerat e dy brinjëve të katërkëndëshit kënddrejtë. Në vijim, përmes komandave përkatëse, është paraparë të lexohen vlerat e dy brinjëve në fjalë dhe me shprehjen:

```
s=kater.a*kater.b;
```

të llogaritet edhe vlera e sipërfaqes së katërkëndëshit. Kurse, përmes komandës:

```
if (kater.a==kater.b)
```

krahasohen vlerat e dy brinjëve të katërkëndëshit. Nëse brinjët janë të barabarta, rezultati në ekran do të duket si në Fig.3.11, ku, siç shihet, shtypet sipërfaqja e katrorit (meqë dy brinjët janë të barabarta).

Fig.3.11

Pamja e ekranit pas ekzekutimit të programit **struct14**, për dy vlera të njëjta të brinjëve

```
Ulera e brinjës a: 7
Ulera e brinjës b: 7
Sipërfaqja e katrorit s=49
Press any key to continue
```

Por, nëse përmes tastierës, kompjuterit për brinjët i jepen dy vlera të ndryshme, rezultati do të duket si në Fig.3.12.

Fig.3.12

Pamja e ekranit pas ekzekutimit të programit **struct14**, për dy vlera të ndryshme të brinjëve

```
Ulera e brinjës a: 6
Ulera e brinjës b: 5
Sipërfaqja e katërkëndëshit s=30
Press any key to continue
```

Plotësisht njëjloj mund të krahasohen edhe variablat e komponenteve të dy variablave të një strukture.

Shembull

Programi **struct15**, përmes së cilit tregohet krahasimi i vlerave të variablave të komponenteve të dy variablave të një strukture.

```
// Programi struct15
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet kater;
    brinjet trek;

    cout << "\nBrinja a e katërkëndëshit: ";
    cin >> kater.a;
```



```

    cout << "\nBrinja a e trekëndëshit: ";
    cin >> trek.a;

    if (kater.a==trek.a)
        cout << "\nBrinjët e barabarta";
    else
        cout << "\nBrinjët e pabarabarta";
    cout << "\n\n";
    return 0;
}

```

Fillimisht, në program është definuar struktura **brinjet** ashtu siç është shpjeguar edhe më parë. Pastaj, duke e shfrytëzuar strukturën në fjalë, përmes shprehjeve:

```

    brinjet kater;
    brinjet trek;

```

janë deklaruar dy variablat e kësaj strukture, në të cilat është menduar të ruhen vlerat e brinjëve përkatëse të katërkëndëshit dhe trekëndëshit. Në fund, përmes komandës:

```

    if (kater.a==trek.a)

```

gjendet se a janë të barabarta brinjët **a** të katërkëndëshit dhe të trekëndëshit. Nëse, p.sh., për këto dy brinjë kompjuterit përmes tastierës i jepen vlerat **7** dhe **5**, rezultati në ekran do të duket si në Fig.3.13.

Fig.3.13
Pamja e ekranit pas ekzekutimit të
programit **struct15**, për dy vlera të
ndryshme të brinjëve a përkatëse

```

Brinja a e katërkëndëshit: 7
Brinja a e trekëndëshit: 5
Brinjët e pabarabarta
Press any key to continue

```

Disa struktura njëkohësisht

Në një program njëkohësisht mund të definohen edhe disa struktura. Gjatë kësaj çdo strukturë duhet ta ketë emrin dhe komponentet me variablat që përfshihen brenda saj.

Shembull

Programi **struct16**, përmes së cilit tregohet shfrytëzimi i dy strukturave njëkohësisht, strukturës **brinjet** dhe strukturës **rrezja**.

```
// Programi struct16
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

struct rrezja
{
    double r;
};

int main()
{
    brinjet kater = {6,4};
    rrezja rrethi = {5};

    double sk,pk,sr,pr;
    const double pi=3.1415926;

    cout << "\nVlerat e variablave në dy strukturat"
          << "\n\nVlera e brinjës a: "
          << kater.a;
    cout << "\nVlera e brinjës b: "
          << kater.b;
    cout << "\nVlera e rrezes: "
          << rrethi.r;

    sk=(kater.a*kater.b);
    pk=2*kater.a+2*kater.b;

    cout << "\n\nSipërfaqja e katërkëndëshit sk="
          << sk
          << "\nPerimetri i katërkëndëshit pk="
          << pk;

    sr=pi*(rrethi.r*rrethi.r);
    pr=2*pi*rrethi.r;
```

```

        cout << "\nSipërfaqja e rrethit sr="
             << sr

             << "\nPerimetri i rrethit pr="
             << pr
             << "\n\n";
return 0;
}

```

Në program, fillimisht, është definuar struktura:

```

struct brinjet
{
    double a,b;
};

```

dhe pastaj edhe struktura:

```

struct rrezja
{
    double r;
};

```

Si anëtarë të strukturës **brinjet** paraqiten dy brinjët **a** dhe **b** të katërkëndëshit, kurse te struktura **rrezja** përfshihet rrezja **r** e rrethit.

Në matematikë, ashtu siç është shpjeguar edhe më parë, për llogaritjen e sipërfaqes dhe të perimetrit të katërkëndëshit kënddrejtë shfrytëzohen shprehjet:

$$\begin{aligned}
 s_k &= a \cdot b \\
 p_k &= 2 \cdot a + 2 \cdot b
 \end{aligned}$$

Kurse, sipërfaqja dhe perimetri i rrethit llogariten përmes shprehjeve:

$$\begin{aligned}
 s_r &= \pi \cdot r^2 \\
 p_r &= 2 \cdot \pi \cdot r
 \end{aligned}$$

ku vlera e konstantes π është marrë **3.1415926**. Meqë vlerat e variablave **a**, **b** dhe **r** në këtë rast merren nga komponentet e variablave **kater** dhe **rrethi** të strukturave **brinjet** dhe **rrezja**, shprehjet përkatëse për llogaritjen e sipërfaqes dhe të perimetrit të katërkëndëshit dhe të rrethit janë shkruar në këtë formë:

```

sk=(kater.a*kater.b);
pk=2*kater.a+2*kater.b;
sr=pi*(rrethi.r*rrethi.r);

```

```
pr=2*pi*rrethi.r;
```

Nëse ekzekutohet programi i dhënë, rezultati që fitohet në ekran do të duket si në *Fig.3.14*.



```
Ulerat e variablave në dy strukturat
Ulera e brinjës a: 6
Ulera e brinjës b: 4
Ulera e rrezes: 5

Sipërfaqja e katërkëndëshit sk=24
Perimetri i katërkëndëshit pk=20
Sipërfaqja e rrethit sr=78.5398
Perimetri i rrethit pr=31.4159

Press any key to continue
```

Fig.3.14
Pamja e ekranit pas ekzekutimit
të programit **struct16**

Gjatë definimit, brenda një strukture variablat e komponenteve përkatëse duhet të kenë emra, të cilët dallohen mes vete. Por, dy struktura të ndryshme në komponentet e tyre mund të përmbajnë edhe variabla me emra të njëjtë.

Shembull

Programi **struct17**, përmes së cilit tregohet shfrytëzimi i dy strukturave njëkohësisht në komponentet e të cilave paraqiten dy variabla me emra të njëjtë.

```
// Programi struct17
#include <iostream>
using namespace std;

struct Dita
{
    int x;
    float y;
};
struct Nata
{
    double x;
};

int main()
{
    Dita R={5, -8.5};
    Nata F={16.5};
    double z;

    cout << "\nUlerat e përfshira në strukturën R\n";
```

```

    cout << "\nVlera x .....: "
        << R.x;
    cout << "\nVlera y .....: "
        << R.y;
    cout << "\n\nVlera e përfshirë në strukturën F\n";
    cout << "\nVlera x .....: "
        << F.x;

    z=R.x+R.y+F.x;

    cout << "\n\nVlera e llogaritur\n"
        << "\nShuma z .....: "
        << z
        << "\n\n";
return 0;
}

```

Në programin e dhënë, variabla **x** është shfrytëzuar njëkohësisht në komponentet e strukturave **Dita** dhe **Nata**. Por, gjatë shfrytëzimit të vlerave përkatëse, të përfshira në variablat e strukturave në fjalë **R** dhe **F**, përcaktimi i komponenteve lidhet edhe me emrat e strukturave:

```

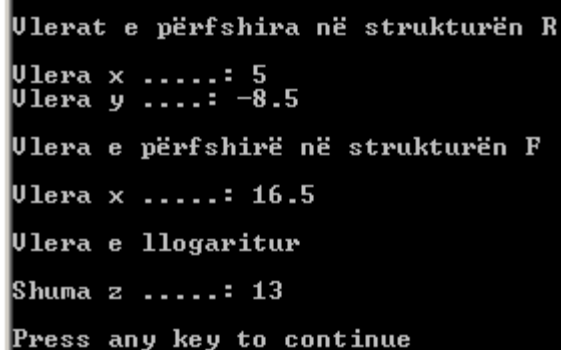
R.x
R.y
F.x

```

gjë që shfrytëzohet si gjatë shtypjes së vlerave ashtu edhe gjatë llogaritjes së shumës:

```
z=R.x+R.y+F.x;
```

Nëse ekzekutohet programi i dhënë, rezultati do të duket si në Fig.3.15.



```

Vlerat e përfshira në strukturën R
Vlera x .....: 5
Vlera y .....: -8.5
Vlera e përfshirë në strukturën F
Vlera x .....: 16.5
Vlera e llogaritur
Shuma z .....: 13
Press any key to continue

```

Fig.3.15
Pamja e ekranit pas ekzekutimit të
programit *struct17*

Strukturat e ndërthurura

Gjatë definimit të një strukture, në komponentet e saj mund të paraqiten variabla të strukturave të tjera, duke krijuar në këtë mënyrë *struktura të ndërthurura*. Strukturat e tilla paraqesin forma më komplekse të të dhënave dhe ndryshe njihen edhe si *struktura hierarkike* (ang. hierarchical structure).

Shembull

Programi **struct18**, përmes së cilit tregohet shfrytëzimi i strukturave **provim** dhe **person**, prej të cilave njëra strukturë paraqitet edhe si komponente e strukturës së dytë.

```
// Programi struct18
#include <iostream>
using namespace std;

struct provim
{
    int nr;
    unsigned nota;
};

struct person
{
    char emri[8];
    provim programimi;
};

int main()
{
    person studenti;

    cout << "\nTë dhënat nga tastiera\n\n";

    cout << "Emri .....: ";
    cin >> studenti.emri;
    cout << "Numri i lëndës ....: ";
    cin >> studenti.programimi.nr;
    cout << "Nota në provim ....: ";
    cin >> studenti.programimi.nota;

    cout << "\n\nTë dhënat e lexuara\n";
    cout << "\nEmri .....: "
```

```

        << studenti.emri;
    cout << "\nNumri i lëndës ....: "
        << studenti.programimi.nr;
    cout << "\nNota në provim ....: "
        << studenti.programimi.nota
        << "\n\n";
return 0;
}

```

Në programin e dhënë definoohen strukturat **provim** dhe **person**. Në strukturën e parë:

```

struct provim
{
    int nr;
    unsigned nota;
};

```

variablat **nr** dhe **nota**, që paraqiten në dy komponentet e saj, p.sh., mund të shfrytëzohen për ruajtjen e numrit rendor të lëndës mësimore në fakultet dhe të notës përkatëse të studentit gjatë provimit. Kurse si komponentë e strukturës së dytë:

```

struct person
{
    char emri[8];
    provim programimi;
};

```

paraqitet variabla **programimi**, e cila është e tipit të strukturës së parë **provim**.

Përmes deklarimit:

```

person studenti;

```

në program është deklaruar variabla **studenti** e tipit të strukturës **person**. Pastaj, variablës **emri** të komponentes së parë të strukturës **provim** i qasemi duke e shkruar në formë të zakonshme:

```

studenti.emri

```

Kurse, për qasje të dy variablat e komponenteve të strukturës **provim**, përkatësisht të variabla **programimi**, e cila përfshihet në strukturën **person**, ato duhet të shkruhen në formën:

```

studenti.programimi.nr
studenti.programimi.nota

```

Nëse ekzekutohet programi i dhënë dhe përmes tastierës kompjuterit i jepet emri i studentit, numri rendor i lëndës Programimi dhe nota me të cilën është notuar studenti në këtë lëndë, rezultati në ekran do të duket si në Fig.3.16.

Fig.3.16

Pamja e ekranit pas ekzekutimit të programit struct18

```
Të dhënat nga tastiera
Emri .....: Agim
Numri i lëndës ....: 34
Nota në provim ....: 9

Të dhënat e lexuara
Emri .....: Agim
Numri i lëndës ....: 34
Nota në provim ....: 9
Press any key to continue
```

Inicializimi me vlera i variablove që paraqiten brenda strukturave të ndërthurura mund të bëhet edhe gjatë deklarimit të variablove përkatëse.

Shembull

Programi **struct18a**, përmes së cilit tregohet inicializimi direkt me vlera i variablove që paraqiten brenda strukturave të ndërthurura **provim** dhe **person**.

```
// Programi struct18a
#include <iostream>
using namespace std;
struct provim
{
    int nr;
    unsigned nota;
};
struct person
{
    char emri[8];
    provim programimi;
};

int main()
{
    person studenti={"Agim",{13,7}};

    cout << "\n\nTë dhënat e inicializuara\n";
    cout << "\nEmri .....: "
         << studenti.emri;
    cout << "\nNumri i lëndës ....: "
```



```

        << studenti.programimi.nr;

        cout << "\nNota në provim ....: "
              << studenti.programimi.nota
              << "\n\n";
    return 0;
}

```

Për inicializimin me vlera gjatë deklarimit të variablës **studenti** të tipit **person** është shfrytëzuar komanda:

```
person studenti={"Agim", {13,7}};
```

Këtu, emri **Agim**, i shënuar nën thonjëza, i përgjigjet variablës **emri**, që paraqitet në komponenten e parë të strukturës **person**. Kurse përmes çiftit të dy numrave **{13,7}** inicializohen variablat **nr** dhe **nota**, që përfshihen në strukturën **provim**, e cila paraqitet si komponente e dytë e strukturës **person**.

Nëse ekzekutohet ky version i programit, rezultati do të duket i njëjtë me atë që është dhënë në *Fig.3.16*.

Nuk ekziston ndonjë kufizim rreth numrit të strukturave, të cilat mund të ndërthuren mes vete. Por, nëse ndërthuren më shumë struktura, nënkuptohet se duhet të shtohet kujdesi për mënyrën e qasjes te komponentet e strukturave të veçanta.

Strukturat si parametra të funksioneve

Variablat e strukturave mund të paraqiten edhe si parametra të funksioneve. Gjatë kësaj, si edhe për të dhënat e zakonshme, parametrat formalë dhe ata aktualë duhet të përputhen mes vete për nga:

- *numri* - sa ka parametra formalë, aq duhet të ketë edhe parametra aktualë;
- *tipi* - tipin e njëjtë duhet ta kenë parametrat formalë dhe parametrat aktualë përkatës;
- *radha e shkrirjes* - parametrin formal në pozitë të caktuar t'i përgjigjet parametër aktual me pozitë të njëjtë.

Shembull

Programi **struct19**, përmes së cilit tregohet shfrytëzimi i strukturës **brinjet** definimi i së cilës është shpjeguar më parë, si parametër i funksionit **Jeta**.

```
// Programi struct19
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

double Jeta(brinjet kater);

int main()
{
    brinjet kater;
    double s;

    cout << "\nBrinja a: ";
    cin >> kater.a;
    cout << "\nBrinja b: ";
    cin >> kater.b;

    s=Jeta(kater);

    cout << "\nSipërfaqja s="
         << s
         << "\n\n";
    return 0;
}

double Jeta(brinjet kater)
{
    double x;
    x=kater.a*kater.b;
    return x;
}
```

Në program është shfrytëzuar nënprogrami **Jeta**, në të cilin si parametër formal paraqitet variabla **kater** e strukturës **brinjet**. Brenda nënprogramit, përmes shprehjes:

$$x=kater.a*kater.b;$$

llogaritet prodhimi i variablave **a** dhe **b** të komponenteve të strukturës **kater**.

Në fillim të programit është shënuar prototipi i funksionit:

```
double Jeta(brinjet kater);
```

përmes së cilit kompjuteri njoftohet se në pjesën vijuese do të definohet nënprogrami **Jeta**, i cili si parametër formal e ka variablën **kater** të tipit të strukturës **brinjet**. Sikur edhe gjatë punës me variabla të tipeve të zakonshëm, prototipi i funksionit mund të deklarohet duke e shënuar vetëm tipin (emrin e strukturës):

```
double Jeta(brinjet);
```

pa variablën përkatëse.

Nënprogrami **Jeta** thirret duke e shfrytëzuar komandën:

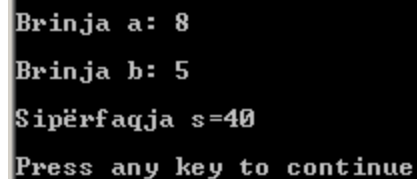
```
s=Jeta(kater);
```

ku si parametër aktual përsëri është shfrytëzuar struktura **kater**. Nëse variablat **a** dhe **b** të komponenteve të strukturës **kater** i paraqesin vlerat e brinjëve të katërkëndëshit kënddrejtë, vlera e cila ruhet te variabla **s** do të jetë sipërfaqja e katërkëndëshit, gjë që në nënprogram përcaktohet përmes llogaritjes së vlerës së variablës **x**.

Pasi të ekzekutohet programi i dhënë dhe përmes tastierës kompjuterit t'i jepen vlerat e brinjëve **a** dhe **b**, rezultati në ekran do të duket ashtu siç shihet në Fig.3.17.

Fig.3.17

Pamja e ekranit pas ekzekutimit të programit struct19



```
Brinja a: 8
Brinja b: 5
Sipërfaqja s=40
Press any key to continue
```

Këtu, si parametër formal dhe parametër aktual është marrë variabla **kater** e strukturës **brinjet**. Por, sikur edhe gjatë operimit me variabla të zakonshme, identifikatorët e parametrave formalë dhe të atyre aktualë mund të merren edhe të ndryshëm.

Shembull

Programi përmes së cilit tregohet shfrytëzimi si parametër formal te funksioni **Jeta** i variablës **Z** të strukturës **brinjet**, kurse si parametër aktual - variabla **kater** e saj.

```
// Programi struct20a
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

double Jeta(brinjet Z);

int main()
{
    brinjet kater;
    double s;

    cout << "\nBrinja a: ";
    cin >> kater.a;
    cout << "\nBrinja b: ";
    cin >> kater.b;

    s=Jeta(kater);

    cout << "\nSipërfaqja s="
         << s
         << "\n\n";
    return 0;
}

double Jeta(brinjet Z)
{
    double x;
    x=Z.a*Z.b;
    return x;
}
```

Siç shihet nga programi i dhënë, si parametër formal të nënprogrami paraqitet variabla **Z** e strukturës **brinjet** (kjo variabël është marrë edhe te prototipi i funksionit). Kurse, gjatë thirrjes së nënprogramit, si parametër aktual i saj shfrytëzohet variabla **kater** e kësaj strukture.

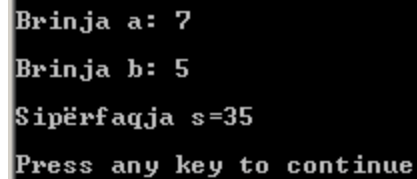
Te prototipi i nënprogramit nuk është e domosdoshme të shkruhet emri i variablës së strukturës, por vetëm tipi i saj, kështu:

```
double Jeta(brinjet);
```

Nëse ekzekutohet programi **struct20a**, për vlera të caktuara hyrëse të brinjëve të katërkëndëshit rezultati do të duket si në Fig.3.18.

Fig.3.18

Pamja e ekranit pas ekzekutimit të programit struct20a



```
Brinja a: 7
Brinja b: 5
Sipërfaqja s=35
Press any key to continue
```

Zgjidhja e problemit të mësipërm do të duket më e thjeshtë, nëse nuk shfrytëzohet struktura, ashtu siç është dhënë në vijim.

```
// Programi struct20b
#include <iostream>
using namespace std;

double Jeta(double a,double b);

int main()
{
    double a,b,s;
    cout << "\nBrinja a: ";
    cin >> a;
    cout << "\nBrinja b: ";
    cin >> b;

    s=Jeta(a,b);

    cout << "\nSipërfaqja s="
         << s
         << "\n\n";
    return 0;
}

double Jeta(double a,double b)
{
    double x;
    x=a*b;
    return x;
}
```

Këtu, parametrat formalë dhe ata aktualë të nënprogrami janë direkt vetë brinjët **a** dhe **b** të katërkëndëshit, pa e marrë si ndërmjetësues strukturën që u përmend më sipër. Në versionin paraprak të programit, struktura është

shfrytëzuar me qëllim të shpjegimit të mënyrës së punës me struktura dhe jo si zgjidhje më optimale e problemit.

Funksione me parametra të përzier

Si parametra të funksionit, përveç variablave të strukturave mund të paraqiten edhe variabla të zakonshme të tipeve standardë.

Shembull

Programi **struct21**, në të cilin si parametra të funksionit **Dita**, përveç variablës së strukturës, paraqiten edhe variabla të tjera të tipeve standarde.

```
// Programi struct21
#include <iostream>
using namespace std;

struct Koha
{
    int g;
    float z,t;
};

void Dita(Koha, float);

int main()
{
    Koha beta;

    cout << "\nVlera g: ";
    cin >> beta.g;
    cout << "Vlera z: ";
    cin >> beta.z;
    cout << "Vlera t: ";
    cin >> beta.t;

    Dita(beta, 5.5);

    return 0;
}

void Dita(Koha beta, float h)
{
    float d,r;

    d=3*h+beta.t;
```

```

    r=beta.g*beta.t+2*h;

    cout << "\nRezultatet: ";
    cout << "\n\nVlera d="
        << d;
    cout << "\nVlera r="
        << r
        << "\n\n";
    return;
}

```

Në programin e dhënë është shfrytëzuar funksioni **Dita**, tek i cili si parametra formalë paraqiten:

variabla **beta** e strukturës **Koha**
 variabla **h** e tipit standard **float**

Duke i shfrytëzuar variablat e komponenteve të strukturës **beta** dhe variablën **h**, në nënprogram llogariten vlerat e shprehjeve:

```

d=3*h+beta.t;
r=beta.g*beta.t+2*h;

```

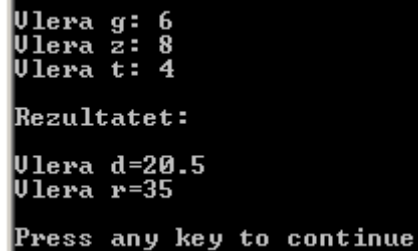
Meqë në nënprogram është paraparë që rezultatet edhe të shtypen, gjatë thirrjes së tij në program nuk përcillet asnjë vlerë, para tij është shënuar fjala **void**.

Në programin kryesor nënprogrami thirret kështu:

```
Dita(beta,5.5);
```

ku si parametra aktualë paraqiten variabla **beta** e strukturës **Koha**, si dhe vlera **5.5** me të cilin zëvendësohet parametri formal **h**.

Rezultati që fitohet në ekran pas ekzekutimit të programit të dhënë, për vlerat hyrëse që kompjuterit i jepen përmes tastierës, do të duket si në Fig.3.19.



```

Vlera g: 6
Vlera z: 8
Vlera t: 4

Rezultatet:

Vlera d=20.5
Vlera r=35

Press any key to continue

```

Fig.3.19

Pamja e ekranit pas ekzekutimit të programit struct21

Ky rezultat fitohet përmes llogaritjeve vijuese:

```
d=3*5.5+4=20.5  
r=6*4+2*5.5=35
```

Gjatë deklarimit të variablës së strukturës e cila përfshihet në parametrat e nënprogramit, variabla në komponentet e saj mund t'u shoqërohen direkt vlerat.

Shembull

Programi **struct22a**, përmes së cilit llogaritet vlera e funksionit:

$$y = 2x + 4 \sum_{i=2}^{n+1} (3i + 4)$$

duke e shfrytëzuar nënprogramin **Nata** për llogaritjen e pjesës së dytë të shprehjes.

```
// Programi struct22a  
#include <iostream>  
using namespace std;  
  
struct Gama  
{  
    int a,b;  
    float x,y;  
};  
  
double Nata(int,Gama);  
  
int main()  
{  
    int n=2;  
  
    Gama Jeta={2,n+1,3,4};  
  
    double x=5,y;  
  
    y=2*x+Nata(4,Jeta);  
  
    cout << "\nRezultati y=" << y << "\n\n";  
    return 0;  
}  
  
double Nata(int k,Gama Jeta)  
{
```



```

int i;
double s, z;
s=0;
for (i=Jeta.a; i<=Jeta.b; i++)
    s=s+(Jeta.x*i+Jeta.y);
z=k*s;
return z;
}

```

Me nënprogramin **Nata** definohet funksioni:

$$z = k \sum_{i=a}^b (x \cdot i + y)$$

Të dhënat **a**, **b**, **x** dhe **y**, të cilat lidhen me llogaritjen e shumës **s** në nënprogram, ruhen te variabla **Jeta** e strukturës **Gama**. Kurse konstantja **k**, me të cilën shumëzohet shuma, paraqitet si parametër i nënprogramit.

Në program, vlerat e variablave jepen direkt, duke i shënuar brenda kllapave, gjatë deklarimit të variablës **Jeta** të strukturës **Gama** kështu:

```
Gama Jeta={2, n+1, 3, 4};
```

Kurse në nënprogram, gjatë llogaritjes së shumës përmes unazës:

```

for (i=Jeta.a; i<=Jeta.b; i++)
    s=s+(Jeta.x*i+Jeta.y);

```

ku siç shihet, vlera fillestare **a** e variablës **i**, vlera kufitare **b** e saj si dhe vlerat e variablave **x** dhe **y** brenda shumës, merren nga variabla **Jeta** e strukturës **Gama**.

Rezultati që fitohet në ekran pas ekzekutimit të programit të dhënë do të duket si në Fig.3.20.

Fig.3.20

Pamja e ekranit pas ekzekutimit të programit
struct22a

```

Rezultati y=102
Press any key to continue

```

Vlera e llogaritur është fituar duke e shfrytëzuar shprehjen:

$$y=2*5+4*\{(3*2+4)+(3*3+4)\}=102$$

Llogaritja e vlerës së funksionit në fjalë është më e thjeshtë, nëse nuk shfrytëzohet struktura. Por, si edhe në shembujt paraprakë, versioni i llogaritjes së vlerës së funksionit pa ndërmjetësimin e strukturës është më i thjeshtë, ashtu siç shihet në vijim.

```
// Programi struct22b
```

```
#include <iostream>
using namespace std;

double Nata(int k,int a,int b,double x,double y);

int main()
{
    int n=2;
    double x=5,y;

    y=2*x+Nata(4,2,n+1,3,4);

    cout << "\nRezultati y="
          << y
          << "\n\n";
return 0;
}

double Nata(int k,int a,int b,double x,double y)
{
    int i;
    double s,z;
    s=0;
    for (i=a;i<=b;i++)
        s=s+(x*i+y);
    z=k*s;
    return z;
}
```

Këtu, duket më qartë shfrytëzimi i parametrave formalë në nënprogram, si dhe zëvendësimi i tyre me parametrat aktualë gjatë llogaritjes së vlerës së funksionit.

Thirrja e përsëritur e funksionit

Nënprogramet, të cilët si parametra formalë përmbajnë edhe variabla të strukturave, mund të thirren më shumë herë brenda një programi.

Shembull

Programi **struct23a**, përmes së cilit llogariten vlerat e funksioneve:

$$y = x + 3 \sum_{i=1}^{n+2} (2i - 5)$$
$$z = \frac{x}{3} + 2 \sum_{i=1}^n (i + g) - 3 \sum_{i=2}^{n+1} (4i - h)$$

```
// Programi struct23a
#include <iostream>
using namespace std;

struct Gama
{
    int a,b;
    float x,y;
};

double Nata(int,Gama);

int main()
{
    int n=2;
    float g=3,h=4;

    Gama Jeta1={1,n+2,2,-5};
    Gama Jeta2={1,n,1,g};
    Gama Jeta3={2,n+1,4,-h};

    double x=5,y,z;

    y=x+Nata(3,Jeta1);

    cout << "\nVlera y="
         << y;

    z=x/3+Nata(2,Jeta2)+Nata(-3,Jeta3);

    cout << "\nVlera z="
         << z
         << "\n\n";
    return 0;
}

double Nata(int k,Gama Jeta)
{
    int i;
    double s,z;
    s=0;
    for (i=Jeta.a;i<=Jeta.b;i++)
        s=s+(Jeta.x*i+Jeta.y);
    z=k*s;
}
```

```
    return z;  
}
```

Në program janë deklaruar **3** variabla të strukturës **Gama**:

```
Gama Jeta1={1,n+2,2,-5};  
Gama Jeta2={1,n,1,g};  
Gama Jeta3={2,n+1,4,-h};
```

deklarim i cili mund të bëhet edhe në një rresht, kështu:

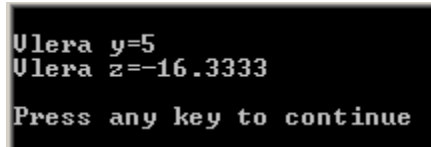
```
Gama Jeta1={1,n+2,2,-5}, Jeta2={1,n,1,g}, Jeta3={2,n+1,4,-h};
```

Në kllapat e secilës nga variablat në fjalë përfshihen vlerat të cilat kanë të bëjnë me **3** shumat që paraqiten në dy shprehjet e funksioneve. Dy vlerat e para të variablave **a** dhe **b** në komponentet e strukturës u përgjigjen kufijve të shumave (kufiri i poshtër dhe kufiri i sipërm). Kurse vlerat e variablave **x** dhe **y** në komponentet e strukturës u përgjigjen konstanteve para variablës **i** dhe në pjesën pas saj, të shprehjeve nën shumat.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në *Fig.3.21*.

Fig.3.21

Pamja e ekranit pas ekzekutimit të programit struct23a



```
Ulera y=5  
Ulera z=-16.3333  
Press any key to continue
```

Në vijim, me qëllim të krahasimit, është dhënë versioni i programit në të cilin nuk shfrytëzohet struktura.

```
// Programi struct23b  
#include <iostream>  
using namespace std;  
  
double Nata(int k,int a,int b,float x,float y);  
  
int main()  
{  
    int n=2;  
    float g=3,h=4;  
  
    double x=5,y,z;
```

```

    y=x+Nata(3,1,n+2,2,-5);

    cout << "\nVlera y="
          << y;
    z=x/3+Nata(2,1,n,1,g)+Nata(-3,2,n+1,4,-h);

    cout << "\nVlera z="
          << z
          << "\n\n";
return 0;
}

double Nata(int k,int a,int b,float x,float y)
{
    int i;
    double s,z;
    s=0;
    for (i=a;i<=b;i++)
        s=s+(x*i+y);
    z=k*s;
    return z;
}

```

Në programin e dhënë shumë qartë duket shfrytëzimi i parametrave formalë gjatë definimit të procedurës së llogaritjes së shumës të nënprogrami **Nata**, si dhe zëvendësimi i tyre me parametra aktualë gjatë tri thirrjeve të funksionit.

Disa struktura si parametra të funksioneve

Në një funksion, si parametra njëkohësisht mund të shfrytëzohen dy e më shumë struktura.

Shembull

Programi **struct24a**, përmes së cilit llogaritet vlera e funksionit:

$$h = 3 \cdot \max(a, b, c) + (2m + n - 1)! - 5$$

duke e shfrytëzuar nënprogramin **Jeta** për llogaritjen dhe shtypjen e vlerës maksimale mes tre numrave **a**, **b** dhe **c**, si dhe vlerës së faktorielit.

```

// Programi struct24a
#include <iostream>
using namespace std;

```

```
struct Alfa
{
    double a,b,c;
};

struct Beta
{
    int m,n;
};

double Jeta(Alfa,Beta);

int main()
{
    int m=1,n=2;
    double h;

    Alfa Dita={2,4,3};
    Beta Nata={m,n};

    h=Jeta(Dita,Nata);

    cout << "\nVlera e funksionit h="
          << h
          << "\n\n";
    return 0;
}

double Jeta(Alfa Dita,Beta Nata)
{
    int i;
    double x,F,h;

    if (Dita.a>Dita.b)
    {
        if (Dita.a>Dita.c)
            x=Dita.a;
    }
    else
    {
        if (Dita.b>Dita.c)
            x=Dita.b;
        else
            x=Dita.c;
    }

    cout << "\nVlera më e madhe x="
```

```

        << x;

    F=1;
    for (i=1;i<=(2*Nata.m+Nata.n-1);i++)
        F=F*i;
    cout << "\nVlera e faktorielit F="
        << F;

    h=3*x+F-5;

    return h;
}

```

Përmes deklarimit të strukturave përkatëse, variablat **a**, **b** dhe **c** janë përfshirë në komponentet e strukturës **Alfa**, kurse variablat **m** dhe **n** - në strukturën **Beta**. Gjatë deklarimit të variablave **Dita** dhe **Nata** të strukturave:

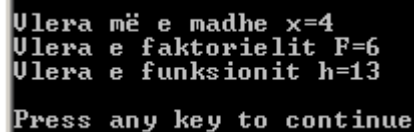
```

Alfa Dita={2,4,3};
Beta Nata={m,n};

```

variablat e komponenteve përkatëse edhe janë inicializuar me vlera. Pastaj, këto dy variabla janë shfrytëzuar si parametra të funksionit **Jeta**.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.3.22.



```

Vlera më e madhe x=4
Vlera e faktorielit F=6
Vlera e funksionit h=13
Press any key to continue

```

Fig.3.22

Pamja e ekranit pas ekzekutimit të programit **struct24a**

Vlera maksimale **x** dhe faktorieli **F** llogariten dhe shtypen në nënprogram. Kurse, vlera e funksionit **h** llogaritet në programin kryesor duke e shfrytëzuar edhe nënprogramin **Jeta**, ku si parametra aktualë përsëri paraqiten variablat **Dita** dhe **Nata** të strukturave **Alfa** dhe **Beta**.

Nëse programi i dhënë shkruhet duke mos i shfrytëzuar strukturat, do të jetë më i thjeshtë, ashtu siç është dhënë në vijim.

```

// Programi struct24b
#include <iostream>
using namespace std;

double Jeta(double a,double b,double c,int m,int n);

int main()
{

```

```
    int m=1,n=2;
    double h;

    h=Jeta(2,4,3,m,n);

    cout << "\nVlera e funksionit h="
          << h
          << "\n\n";
return 0;
}

double Jeta(double a,double b,double c,int m,int n)
{
    int i;
    double x,F,h;

    if (a>b)
    {
        if (a>c)
            x=a;
    }
    else
    {
        if (b>c)
            x=b;
        else
            x=c;
    }

    cout << "\nVlera më e madhe x="
          << x;

    F=1;
    for (i=1;i<=(2*m+n-1);i++)
        F=F*i;

    cout << "\nVlera e faktorielit F="
          << F;

    h=3*x+F-5;

    return h;
}
```

Këtu, **5** variablat të cilat në programin paraprak ishin përfshirë në komponentet e strukturave **Alfa** dhe **Beta**, janë vendosur direkt si parametra të nënprogramit **Jeta**.

Disa nënprograme njëkohësisht

Variablat e strukturave pa ndonjë kufizim mund të përdoren njëkohësisht në më shumë nënprograme.

Shembull

Programi **struct25**, përmes së cilit llogaritet vlera e funksionit nga detyra paraprahe:

$$h = 3 \cdot \max(a, b, c) + (2m + n - 1)! - 5$$

por duke i shfrytëzuar nënprogramet **max** dhe **fakt** përmes të cilëve gjendet vlera maksimale mes tre numrave **a**, **b** dhe **c**, si dhe vlera e faktorialit.

```
// Programi struct25
#include <iostream>
using namespace std;
struct Alfa
{
    double a,b,c;
};
struct Beta
{
    int m,n;
};
double max(Alfa);
double fakt(Beta);

int main()
{
    int m=1,n=2;
    double h;
    Alfa Dita={2,4,3};
    Beta Nata={m,n};

    h=3*max(Dita)+fakt(Nata)-5;

    cout << "\nVlera e funksionit h="
         << h
         << "\n\n";
    return 0;
}
```

```
double max(Alfa Dita)
{
    double x;

    if (Dita.a>Dita.b)

    {
        if (Dita.a>Dita.c)
            x=Dita.a;
        }
    else
    {
        if (Dita.b>Dita.c)
            x=Dita.b;
        else
            x=Dita.c;
        }
    cout << "\nVlera më e madhe x="
        << x;
    return x;
}

double fakt(Beta Nata)
{
    int i;
    double F;
    F=1;
    for (i=1;i<=(2*Nata.m+Nata.n-1);i++)
        F=F*i;
    cout << "\nVlera e faktorielit F="
        << F;
    return F;
}
```

Edhe këtu variablat **a**, **b** dhe **c** përfshihen në komponentet e strukturës **Alfa**, kurse variablat **m** dhe **n** - në strukturën **Beta**. Pastaj, variablat **Dita** dhe **Nata** të strukturave shfrytëzohen si parametra të funksioneve **max** dhe **fakt**.

Nëse ekzekutohet programi i dhënë, rezultati do të duket ashtu siç është dhënë në *Fig.3.22*.

Funksionet në komponentet e strukturave

Përveç variablave, në komponentet e strukturave mund të paraqiten edhe funksionet. Gjatë kësaj, në formë të përgjithshme struktura shkruhet kështu:



```

struct e
{
    t1 y1;
    t2 y2;
    .....
    tn yn;
};

```

ku janë:

e - emri i strukturës.

t1, t2, ..., tn - tipet e të dhënave ose të funksioneve në komponentet e strukturës.

y1, y2, ..., yn - variablat ose funksionet në komponentet e strukturës.

Në program, variabla e strukturës së tillë deklarohet plotësisht njëjloj siç deklarohen variablat e strukturave të zakonshme. Kurse, edhe funksionet që paraqiten në komponentet e strukturës thirren ashtu siç shfrytëzohen variablat brenda komponenteve të saj.

Funksionet pa parametra formalë

Nëse gjatë llogaritjeve brenda funksioneve shfrytëzohen vetëm variablat të cilat përfshihen në komponentet e strukturave ku ato definojnë, në kllapat e tyre nuk shënohet asnjë parametër formalë. Por, definimi i funksioneve mund të bëhet brenda vetë strukturës, ose edhe jashtë saj.

Definimi brenda strukturës

Funksionet brenda strukturës definojnë plotësisht njëjloj siç definojnë edhe funksionet e zakonshme.

Shembull

Programi **struct26**, përmes së cilit llogaritet vlera e funksionit:

$$y = 2x + a - 1$$

Në program është definuar struktura **Delta**, në komponentet e së cilës, përveç variablave **a** dhe **x**, paraqitet edhe funksioni **Omega**.

```

// Programi struct26
#include <iostream>
using namespace std;

```

```
struct Delta
{
    double a,x;
    double Omega()
    {
        return 2*x+a-1;
    };
};

int main()
{
    Delta Alfa;
    double y;

    cout << "\nVlerat hyrëse"
          << "\n\nVariabla x: ";
    cin >> Alfa.x;
    cout << "\nVariabla a: ";
    cin >> Alfa.a;

    y=Alfa.Omega();

    cout << "\nVlera e llogaritur y="
          << y
          << "\n\n";
    return 0;
}
```

Siç shihet nga programi, përmes funksionit **Omega**:

```
double Omega()
{
    return 2*x+a-1;
};
```

brenda strukturës **Delta**, përcaktohet llogaritja e vlerës së funksionit **y**. Meqë variablat **x** dhe **a** që shfrytëzohen në funksion janë variabla të komponenteve të strukturës, brenda kllapave të funksionit nuk është shënuar asnjë parametër.

Në fillim të programit është deklaruar variabla **Alfa** e tipit të strukturës **Delta**. Kjo variabël shfrytëzohet për qasje te komponentet e zakonshme të strukturës:

```
Alfa.x
Alfa.a
```

si dhe te komponentja me funksionin **Omega**. Vlera e funksionit **y** është llogaritur duke e thirrur funksionin në fjalë përmes shprehjes:

```
y=Alfa.Omega();
```

Nëse ekzekutohet programi i dhënë, për vlerat hyrëse të variablave **x** dhe **a**, rezultati do të duket si në Fig.3.23.

Fig.3.23

Pamja e ekranit pas ekzekutimit të programit struct26

```
Vlerat hyrëse
Variabla x: 8
Variabla a: 6
Vlera e llogaritur y=21
Press any key to continue
```

Vlera e funksionit **y** është llogaritur në këtë mënyrë:

$$y = 2 \cdot 8 + 6 - 1 = 21$$

Funksioni, i cili përfshihet brenda strukturës, mund të përmbajë edhe më shumë komanda, përkatësisht të definohet plotësisht njëjloj siç definohen edhe funksionet e zakonshme. Kështu, p.sh., struktura nga shembulli i mësipërm mund të duket:

```
struct Delta
{
    double a,x;
    double Omega()
    {
        double y;
        y=2*x+a-1;
        return y;
    };
};
```

Këtu, në kuadër të komponentes në të cilën përfshihet funksioni është deklaruar variabla ndihmëse **y**, ku ruhet vlera e llogaritur, e cila njëkohësisht shënohet në vazhdim të komandës **return**. Nënkuptohe se në vend të variablës

y, gjatë definimit të funksionit, mund të shfrytëzohet çfarëdo variabël tjetër, pavarësisht se realisht funksioni që definohet është quajtur si **y**.

Gjatë definimit të funksionit, i cili paraqitet në komponenten e një strukture, mund të shfrytëzohen edhe degëzime dhe unaza të ndryshme.

Shembull

Programi **struct27**, përmes së cilit llogaritet vlera e funksionit:

$$h = \frac{x}{2} + 3 \sum_{\substack{i=1 \\ (i \neq 4)}}^{n+1} (2i + a)$$

Në program është definuar struktura **Beta**, brenda komponenteve të së cilës, përveç variablave **a**, **x** dhe **n**, paraqitet edhe funksioni **Nata**.

```
// Programi struct27
#include <iostream>
using namespace std;

struct Beta
{
    double a,x;
    int n;
    double Nata()
    {
        int i;
        double h,s=0;
        for (i=1;i<=(n+1);i++)
            if (i!=4)
                s=s+(2*i+a);
        h=x/2+3*s;
        return h;
    };
};

int main()
{
    Beta Dita;
    double h;

    cout << "\nVlerat hyrëse"
          << "\n\nVariabla x: ";
    cin >> Dita.x;
    cout << "\nVariabla a: ";
    cin >> Dita.a;
    cout << "\nVariabla n: ";
    cin >> Dita.n;
```

```

    h=Dita.Nata();

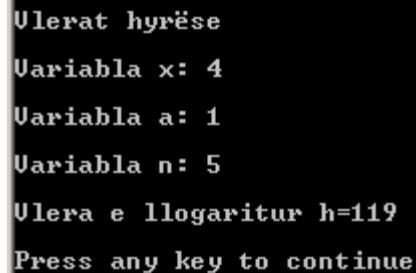
    cout << "\nVlera e llogaritur h="
         << h
         << "\n\n";
return 0;
}

```

Funksioni **Nata**, i cili është definuar brenda komponentes së strukturës **Beta**, shfrytëzohet për përcaktimin e llogaritjes së vlerës së variablës **h**. Në funksion shfrytëzohen variablat ndihmëse **i**, **h** dhe **s**, si dhe unaza e realizuar përmes komandës **for**. Brenda unazës paraqitet edhe komanda **if** me të cilën pengohet që shumës t'i shtohet anëtari për **i=4**, ashtu siç është përcaktuar me shprehjen e dhënë të funksionit.

Në fillim të programit është deklaruar variabla **Dita** e tipit të strukturës **Beta**. Kjo variabël shfrytëzohet gjatë leximit të vlerave të variablave të komponenteve të strukturës si dhe gjatë thirrjes së funksionit **Nata** për llogaritjen e vlerës së variablës **h**.

Nëse ekzekutohet programi i dhënë, për vlerat hyrëse, të cilat kompjuterit i jepen përmes tastierës, rezultati në ekran do të duket si në Fig.3.24.



```

Ulerat hyrëse
Variabla x: 4
Variabla a: 1
Variabla n: 5
Vlera e llogaritur h=119
Press any key to continue

```

Fig.3.24

Pamja e ekranit pas ekzekutimit të programit struct27

Nëse në programin e dhënë më sipër në komponenten e strukturës **Beta** definohet llogaritja e vetëm shumës **s**, e cila paraqitet te funksioni **h**, struktura do të duket si në vijim.

```

struct Beta
{
    double a,x;
    int n;
    double Nata()
    {
        int i;
        double s=0;
        for (i=1;i<=(n+1);i++)

```

```
        if (i!=4)
            s=s+(2*i+a);
        return s;
    };
};
```

Pas kësaj, vlera e funksionit **h** do të llogaritet duke e shfrytëzuar shprehjen:

```
h=Dita.x/2+3*Dita.Nata();
```

Struktura nuk është e thënë të shfrytëzohet për definimin e të gjitha llogaritjeve komplekse që nevojiten gjatë zgjidhjes së një problemi.

Shembull

Programi **structK** përmes së cilit llogaritet vlera e funksionit:

$$g = \begin{cases} 3a + (n + 1)! & \text{për } x < 0.8 \\ \frac{a}{2} + 4 \prod_{k=1}^n (kx + b) & \text{për } x \geq 0.8 \end{cases}$$

Në program është definuar struktura **Delta**, brenda komponenteve të së cilës, përveç variablave **a**, **b**, **x** dhe **n**, paraqitet edhe funksioni **Gama** përmes së cilit definohet llogaritja vetëm e prodhimit.

```
// Programi structK
#include <iostream>
using namespace std;

struct Delta
{
    double a,b,x;
    int n;
    double Gama()
    {
        int k;
        double P=1;
        for (k=1;k<=n;k++)
            P=P*(k*x+b);
        return P;
    };
};

int main()
{
    Delta Koha;
    double g;
```



```

cout << "\nVlerat hyrëse"
      << "\n\nVariabla x: ";
cin >> Koha.x;
cout << "\nVariabla a: ";
cin >> Koha.a;
cout << "\nVariabla n: ";
cin >> Koha.n;

if (Koha.x<0.8)
{
    int i;
    double F=1;

    for (i=1;i<=(Koha.n+1);i++)
        F=F*i;
    g=3*Koha.a+F;
}
else
{
    cout << "\nVariabla b: ";
    cin >> Koha.b;
    g=Koha.a/2+4*Koha.Gama();
}

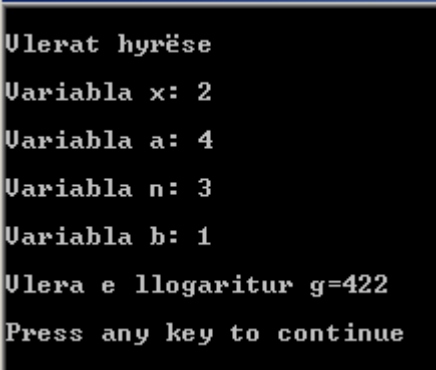
cout << "\nVlera e llogaritur g="
      << g
      << "\n\n";
return 0;
}

```

Këtu, definimi i funksionit **g**, si dhe llogaritjet e nevojshme kryhen brenda programit. Kurse, prej strukturës shfrytëzohet nënprogrami **Gama**, gjatë llogaritjes së vlerës së prodhimit, duke e thirrur në shprehjen e funksionit:

```
g=Koha.a/2+4*Koha.Gama();
```

Rezultati në ekran, për vlerat hyrëse të cilat kompjuterit i jepen përmes tastierës, do të duket si në *Fig.3.25*.



```

Vlerat hyrëse
Variabla x: 2
Variabla a: 4
Variabla n: 3
Variabla b: 1
Vlera e llogaritur g=422
Press any key to continue

```

*Fig.3.25**Pamja e ekranit pas ekzekutimit të programit **structK***

Komandat për lexim mund të eliminohen nga programi, nëse vlerat e nevojshme për variablat e veçanta kompjuterit i jepen gjatë deklarimit të variablës së strukturës, ashtu siç shihet në vijim.

Shembull

Versioni **structKz** i programit **structK** për llogaritjen e vlerës së funksionit të dhënë në detyrën paraprake, por tek i cili variablat u shoqërohen vlerat gjatë deklarimit të variablës së strukturës.

```
// Programi structKz
#include <iostream>
using namespace std;

struct Delta
{
    double a,b,x;
    int n;
    double Gama()
    {
        int k;
        double P=1;
        for (k=1;k<=n;k++)
            P=P*(k*x+b);
        return P;
    };
};

int main()
{
    Delta Koha={4,1,2,3};
    double g;
    if (Koha.x<0.8)
    {
        int i;
        double F=1;
        for (i=1;i<=(Koha.n+1);i++)
            F=F*i;
        g=3*Koha.a+F;
    }
}
```

```

    else
    {
        g=Koha.a/2+4*Koha.Gama();
    }

    cout << "\nVlera e llogaritur g="
          << g
          << "\n\n";
return 0;
}

```

Nëse ekzekutohet programi i dhënë, rezultati edhe në këtë rast do të jetë i njëjtë me atë që u dha në *Fig.3.25*.

Këtu, siç është theksuar edhe më parë, vlerat që shënohen brenda kllapave gjatë deklarimit të variablës së strukturës duhet të përputhen plotësisht me radhën e shkruarjes së variablave përkatëse në strukturë. Por, në këtë rast nuk ka rëndësi pozita e komponenteve me variablat që përfshihen brenda tyre ndaj pozitës së funksionit. Kështu, në programin e dhënë, për strukturën në fjalë mund të shfrytëzohen edhe versionet vijuese.

a.

```

struct Delta
{
    double a,b,x;
    double Gama()
    {
        int k=1;
        double P=1;
        for (k=1;k<=n;k++)
            P=P*(k*x+b);
        return P;
    };
    int n;
};

```

b.

```

struct Delta
{
    double Gama()
    {
        int k=1;
        double P=1;
        for (k=1;k<=n;k++)
            P=P*(k*x+b);
        return P;
    };
};

```

```
};  
double a,b,x;  
int n;  
};
```

Në të dy këto raste, deklarimi i variablës së strukturës dhe inicializimi me vlera i variablave në komponentet e strukturës mund të bëhet siç u dha edhe më sipër:

```
Delta Koha={4,1,2,3};
```

meqë variablat në strukturë vetëm e kanë ndryshuar pozitën ndaj funksionit **Gama**, por e kanë ruajtur radhën e shkruarjes së tyre.

Definimi jashtë strukturës

Funksionet që paraqiten si komponente të strukturës mund të definohen edhe jashtë saj, gjë që është me rëndësi sidomos kur kemi të bëjmë me funksione më komplekse. Gjatë kësaj, *brenda strukturës patjetër vendoset vetëm prototipi i funksionit*.

Me qëllim që brenda nënprogramit të shfrytëzohen direkt variablat e komponenteve të strukturës (pa ua shtuar emrin e variablës së strukturës dhe pikën), gjatë definimit të funksionit, në titullin e tij shënohet edhe emri i strukturës, kështu:

```
t e::f();
```

ku janë:

t - tipi i funksionit.
e - emri i strukturës.
f - emri i funksionit

Shembull

Programi **struct27a** si version i programit **struct27**, përmes së cilit llogaritet vlera e funksionit:

$$h = \frac{x}{2} + 3 \sum_{\substack{i=1 \\ (i \neq 4)}}^{n+1} (2i + a)$$

Në program është definuar struktura **Beta**, brenda komponenteve të së cilës, përveç variablave **a**, **x** dhe **n**, paraqitet edhe funksioni **Nata**.

```
// Programi struct27a
```

```

#include <iostream>
using namespace std;

struct Beta
{
    double a,x;
    int n;
    double Nata();
};

int main()
{
    Beta Dita;
    double h;

    cout << "\nVlerat hyrëse"
          << "\n\nVariabla x: ";
    cin >> Dita.x;
    cout << "\nVariabla a: ";
    cin >> Dita.a;
    cout << "\nVariabla n: ";
    cin >> Dita.n;

    h=Dita.Nata();

    cout << "\nVlera e llogaritur h="
          << h
          << "\n\n";
    return 0;
}

double Beta::Nata()
{
    int i;
    double h,s=0;
    for (i=1;i<=(n+1);i++)
        if (i!=4)
            s=s+(2*i+a);
    h=x/2+3*s;
    return h;
};

```

Këtu, si komponente e strukturës paraqitet edhe prototipi i funksionit:

```
double Nata();
```

Njëkohësisht, gjatë definimit të funksionit, titulli i nënprogramit është shkruar në formën:

```
double Beta::Nata()
```

me çka mundësohet që brenda tij do të përdoren variablat **a**, **x** dhe **n**, të cilat paraqiten në komponentet e strukturës, duke i shkruar si variabla të zakonshme (pa parashtesën dhe pikën).

Nëse ekzekutohet programi i dhënë, rezultati do të duket si në *Fig. 3.24* të dhënë më sipër, për vlerat hyrëse të cilat kompjuterit i jepen përmes tastierës.

Prej funksionit i cili vendoset në komponenten e strukturës mund të mos merret asnjë rezultat. Si edhe në rastet e shfrytëzimit të funksioneve të zakonshme, para tij duhet të shënohet fjala **void**.

Shembull

Programi **structD**, përmes së cilit llogaritet vlera e funksionit:

$$z = \begin{cases} 2k + a - 1 & \text{për } k = 3 \\ k + 3a & \text{për } k = 6 \\ k^2 + 2a & \text{për } k = 9 \end{cases}$$

Në program është definuar struktura **Omega**, brenda komponenteve të së cilës, përveç variablave **k** dhe **a** është vendosur edhe prototipi i funksionit **Jeta**. Ky funksion shfrytëzohet për definimin dhe pastaj edhe për shtypjen e vlerës së funksionit **z**, nëse vlerat e variablave **a** dhe **k** kompjuterit i jepen përmes tastierës.

```
// Programi structD
#include <iostream>
using namespace std;

struct Omega
{
    double a;
    int k;
    void Jeta();
};

int main()
{
    Omega Libri;
```

```

        cout << "\nVlera e variablës a: ";
        cin >> Libri.a;
        cout << "\nVlera e variablës k: ";
        cin >> Libri.k;

        Libri.Jeta();

return 0;
}

void Omega::Jeta()
{

    double z;
    switch(k)
    {
        case 3:
            z=2*k+a-1;
            break;

        case 6:
            z=k+3*a;
            break;

        case 9:
            z=k*k+2*a;
            break;

        default:
            cout << "\nGabim vlera e variablës k";
            goto Fundi;
    }
    cout << "\nVlera e llogaritur z="
        << z;
Fundi:
    cout << "\n\n";

return;
};

```

Meqë prej nënprogramit **Jeta** nuk merret asnjë vlerë, përkatësisht në vazhdim të komandës **return** të tij nuk shënohet asnjë variabël, para titullit të nënprogramit është shkruar fjala **void**. Rezultati **z**, që shtypet në ekran, varet nga vlerat hyrëse të cilat kompjuterit i jepen përmes tastierës. Kështu, p.sh., nëse për variablat **a** dhe **k**, të cilat paraqiten në komponentet e strukturës, kompjuterit i jepen vlerat **8** dhe **6**, rezultati do të duket si në Fig.3.26.

```

Vlera e variablës a: 8
Vlera e variablës k: 6
Vlera e llogaritur z=30
Press any key to continue

```

Fig.3.26

Pamja e ekranit pas ekzekutimit të programit **structD**, kur vlera e variablës *k* është e saktë

Por, nëse për **k** kompjuterit i jepet ndonjë vlerë, e cila nuk është në grupin e vlerave **3**, **6** dhe **9**, p.sh., vlera **5**, rezultati do të duket si në Fig.3.27.

Fig.3.27

Pamja e ekranit pas ekzekutimit të programit **structD**, kur vlera e variablës *k* është gabim

```
Ulera e variablës a: 8
Ulera e variablës k: 5
Gabim vlera e variablës k
Press any key to continue
```

Funksionet me parametra formalë

Nëse funksionet që paraqiten në komponentet e strukturave shfrytëzojnë *variabla që nuk përfshihen në strukturë*, ato duhet të shënohen si parametra formalë brenda kllapave, pas emrit të tyre. Siç u dha edhe më sipër, definimi i funksioneve të tilla mund të bëhet brenda strukturës, ose jashtë saj.

Definimi brenda strukturës

Kur funksioni që përfshihet në strukturë përmban parametra formalë, brenda kllapave në titullin e tij, përveç tipeve, shënohen edhe variablat e parametrave formalë përkatës.

Shembull

Programi **structR** përmes së cilit llogaritet vlera e funksionit:

$$g = \begin{cases} x + 3 & \text{për } x > z \\ 2x + (a + 2b - 1)! & \text{për } x \leq z \end{cases}$$

Në program është definuar struktura **Delta**, brenda komponenteve të së cilës, përveç variablave **a** dhe **b**, paraqitet edhe funksioni **Gama**.

```
// Programi structR
#include <iostream>
using namespace std;
struct Delta
{
```



```

int a,b;
double Gama(double x,double z)
{
    int i;
    double F,g;
    if (x>z)
        g=x+3;
    else
    {
        F=1;
        for (i=1;i<=(a+2*b-1);i++)
            F=F*i;
        g=2*x+F;
    }
    return g;
};

int main()
{
    Delta Jeta;
    double x,z,g;

    cout << "\nVlerat hyrëse"
          << "\n\nVariabla x: ";
    cin >> x;
    cout << "\nVariabla a: ";
    cin >> Jeta.a;
    cout << "\nVariabla b: ";
    cin >> Jeta.b;
    cout << "\nVariabla z: ";
    cin >> z;

    g=Jeta.Gama(x,z);

    cout << "\nVlera e llogaritur g="
          << g
          << "\n\n";
    return 0;
}

```

Funksioni **Gama** brenda komponentes së strukturës **Delta** i përmban edhe parametrat formalë **x** e **z**, si dhe tipet e tyre. Këta dy parametra shfrytëzohen gjatë llogaritjeve brenda funksionit, por nuk janë variabla të komponenteve të strukturës në fjalë. Nga ana tjetër, variablat **a** dhe **b**, të cilat shfrytëzohen në llogaritje, meqë përfshihen në komponentet e strukturës, nuk janë shënuar si parametra formalë brenda kllapave të funksionit.

Në program, gjatë llogaritjes së vlerës së funksionit **g**, është shfrytëzuar shprehja:

```
g=Jeta.Gama(x,z);
```

ku vlerat e parametrave aktualë **x** dhe **z** në program janë shënuar si variabla të zakonshme, sepse ato nuk janë variabla të komponenteve të strukturës.

Nëse ekzekutohet programi i dhënë, rezultati do të duket si në Fig.3.28, për vlerat hyrëse të cilat janë zgjedhur lirisht.

Fig.3.28

Pamja e ekranit pas ekzekutimit të programit structR

```
Ulerat hyrëse
Variabla x: 4
Variabla a: 5
Variabla b: 1
Variabla z: 5
Ulera e llogaritur g=728
Press any key to continue
```

Për parametrat formalë që shfrytëzohen gjatë definimit të funksioneve që paraqiten si komponente të strukturave, dhe për parametrat aktualë gjatë thirrjes së tyre, vlejné principe të ngjashme me ato të funksioneve të zakonshme. Përkatësisht, ato duhet të përputhen për nga *tipi*, *numri* dhe *radha e shkruarjes*.

Funksioni që paraqitet si komponente e strukturës njëkohësisht mund të shfrytëzojë variabla të komponenteve të strukturës si dhe variabla të tjera jashtë saj. Nënkuptohe se brenda kllapave të tij duhet të shënohen vetëm variablat të cilat nuk figurojnë në komponentet e strukturës.

Shembull

Programi **structA1**, përmes së cilit llogariten vlerat e funksioneve:

$$g = ax^2 + bx + c$$

$$h = 2x + 3 \cdot \min(x, y, z)$$

Në program është definuar struktura **Alfa**, brenda komponenteve të së cilës, përveç variablave **a, b, c** dhe **x**, paraqitet edhe funksioni **min** përmes së cilit përcaktohet gjetja e vlerës minimale mes tri variablave **x, y** dhe **z**.

```
// Programi structA1
#include <iostream>
using namespace std;

struct Alfa
{
    double a,b,c,x;
    double min(double y,double z)
    {
        double r;
        if ((x<y) && (x<z))
            r=x;
        else
            if (y<z)
                r=y;
            else
                r=z;

        cout << "\nVlera minimale r="
              << r;
        return r;
    };
};

int main()
{
    Alfa Jeta={5,2,6,7};
    double g,h;
    g=Jeta.a*Jeta.x*Jeta.x+Jeta.b*Jeta.x+Jeta.c;
    h=2*Jeta.x+3*Jeta.min(6,4);

    cout << "\nVlera e funksionit g="
          << g
          << "\nVlera e funksionit h="
          << h
          << "\n\n";
    return 0;
}
```

Në funksionin **min**, i cili paraqitet si komponente e strukturës, variablat **y** dhe **z** janë shënuar brenda kllapave pas emrit të tij, sepse nuk janë variabla të strukturës. Gjithashtu, gjatë deklarimit të variablës **Jeta** të strukturës **Alfa**:

```
Alfa Jeta={5,2,6,7};
```

variabla ve që paraqiten brenda strukturës u janë shoqëruar vlerat: **a=5**, **b=2**, **c=6** dhe **x=7**. Ndërkaq gjatë thirrjes së funksionit **min**, në shprehjen:

```
h=2*Jeta.x+3*Jeta.min(6,4);
```

parametrave formalë **y** dhe **z** u janë shoqëruar vlerat: **y=6** dhe **z=4**. Rezultati që fitohet në ekran do të duket si në Fig.3.29.

Fig.3.29

Pamja e ekranit pas ekzekutimit të programit structA1

```
Vlera minimale r=4
Vlera e funksionit g=265
Vlera e funksionit h=26
Press any key to continue
```

Mesazhi për vlerën minimale shtypet përmes komandës:

```
cout << "\nVlera minimale r="
      << r;
```

e cila është vendosur në nënprogram, menjëherë pasi gjendet vlera minimale mes vlerave të variabla ve **x**, **y** dhe **z**.

Definimi jashtë strukturës

Siç u tha edhe më parë, funksionet që paraqiten në komponentet e strukturës mund të definohen edhe jashtë saj. Në atë rast, brenda strukturës shënohen prototipet e tyre. Gjatë kësaj, si parametra formalë brenda kllapave të funksioneve patjetër duhet të shënohen variablat të cilat nuk përfshihen në strukturë, ose edhe tipet e tyre, gjë që nuk është e domosdoshme.

Shembull

Versioni **structRb** i programit **structR**, që është dhënë më sipër, përmes së cilit llogaritet vlera e funksionit:



$$g = \begin{cases} x + 3 & \text{për } x > z \\ 2x + (a + 2b - 1)! & \text{për } x \leq z \end{cases}$$

Këtu, funksioni **Gama**, i cili paraqitet si komponente e strukturës **Delta**, definohet jashtë strukturës.

```
// Programi structRb
#include <iostream>
using namespace std;

struct Delta
{
    int a,b;
    double Gama(double x,double z);
};

int main()
{
    Delta Jeta;
    double x,z,g;

    cout << "\nVlerat hyrëse"
          << "\n\nVariabla x: ";
    cin >> x;
    cout << "\nVariabla a: ";
    cin >> Jeta.a;
    cout << "\nVariabla b: ";
    cin >> Jeta.b;
    cout << "\nVariabla z: ";
    cin >> z;

    g=Jeta.Gama(x,z);

    cout << "\nVlera e llogaritur g="
          << g
          << "\n\n";
    return 0;
}

double Delta::Gama(double x,double z)
{
    int i;
    double F,g;
    if (x>z)
        g=x+3;
    else
```

```
{
    F=1;
    for (i=1; i<=(a+2*b-1); i++)
        F=F*i;
    g=2*x+F;
}
return g;
};
```

Siç shihet nga programi i dhënë, si komponente e strukturës paraqitet prototipi i funksionit **Gama**, i shkruar në formën:

```
double Gama(double x, double z);
```

Njëkohësisht, funksioni në fjalë është definuar në pjesën e fundit të programit, duke e shënuar titullin e tij në formën:

```
double Delta::Gama(double x, double z)
```

Këtu, para titullit të nënprogramit paraqitet edhe emri i strukturës **Delta**, me çka kompjuteri njoftohet se variablat **a** dhe **b**, të cilat shfrytëzohen brenda nënprogramit, janë variabla të komponenteve të kësaj strukture.

Si zakonisht, prototipi i funksionit mund të shkruhet edhe duke i shënuar brenda kllapave vetëm tipet e variablave, pa i shënuar edhe variablat, kështu:

```
double Gama(double, double);
```

Shfrytëzimi në strukturë i funksioneve të tjera

Te funksionet që paraqiten në komponentet e strukturave mund të shfrytëzohen funksione standarde që përfshihen në module të ndryshme të gjuhës programuese C++, ose edhe funksione që definohen pavarësisht nga struktura.

Shembull

Programi përmes së cilit llogaritet vlera e funksionit:

$$g = \begin{cases} 2 \sin(4x) - b & \text{për } x < a \\ x^b + 3 & \text{për } x \geq a \end{cases}$$

duke i deklaruar variablat dhe funksionin në komponentet e strukturës **Gama**.

```
// Programi struct28
#include <iostream>
#include <math.h>
using namespace std;

struct Gama
{
    double x,a;
    int b;
    double Libri();
};

int main()
{
    double g;
    Gama Jeta = {0.5,2.3,3};

    cout << "\nVlera e variablës x: "
          << Jeta.x;
    cout << "\nVlera e variablës a: "
          << Jeta.a;
    cout << "\nVlera e variablës b: "
          << Jeta.b;

    g=Jeta.Libri();

    cout << "\n\nVlera e funksionit g="
          << g
          << "\n\n";
    return 0;
}

double Gama::Libri()
{
    double g;
    if (x<a)
        g=2*sin(4*x)-b;
    else
        g=pow(x,b)+3;
    return g;
};
```

Llogaritja e vlerës së funksionit **g** është përcaktuar si komponente e strukturës, përmes funksionit **Libri**. Brenda funksionit, për llogaritjen e vlerës **x^b** është shfrytëzuar funksioni standard **pow**, nga moduli **math.h**, i cili është shënuar në fillim të programit përmes komandës paraprosesorike:

```
#include <math.h>
```

Ngjashëm shfrytëzohet edhe funksioni trigonometrik **sin** nga moduli në fjalë.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket ashtu siç është dhënë në Fig.3.30.

Fig.3.30
Pamja e ekranit pas ekzekutimit të
programit **struct28**

```
Vlera e variablës x: 0.5
Vlera e variablës a: 2.3
Vlera e variablës b: 3

Vlera e funksionit g=-1.18141
Press any key to continue
```

Shfrytëzimi i funksioneve të definuara jashtë strukturës, gjatë përcaktimit të funksioneve në komponentet e strukturës, kërkon një kujdes të veçantë rreth parametrave që nuk paraqiten si variabla të komponenteve të strukturës.

Shembull

Programi **structDa**, përmes së cilit llogaritet vlera e funksionit:

$$g = ax + b \sum_{\substack{i=1 \\ (i \neq 3,4)}}^n (2i + c)$$

duke i deklaruar variablat **a**, **b** dhe **x** në komponentet e strukturës **Alfa**. Llogaritja e shumës përcaktohet me funksionin **Shuma** jashtë strukturës. Kurse, për llogaritjen e vlerës së funksionit **h**, shfrytëzohet funksioni **Gama**, i cili vendoset brenda strukturës në fjalë.

```
// Programi structDa
#include <iostream>
using namespace std;

double Shuma(int n,double c);

struct Alfa
{
    double a,b,x;
    double Gama(int n,double c);
};

int main()
```



```

{
    double g;

    Alfa R={2,4,3};

    cout << "\nTë dhënat e inicializuara\n";
    cout << "\nVariabla a: "
          << R.a;
    cout << "\nVariabla b: "
          << R.b;
    cout << "\nVariabla x: "
          << R.x
          << "\n";

    g=R.Gama(4,3);

    cout << "\nVlera e funksionit g="
          << g
          << "\n\n";
return 0;
}

double Alfa::Gama(int n,double c)
{
    double r;
    r=a*x+b*Shuma(n,c);
return r;
}

double Shuma(int n,double c)
{
    int i;
    double s=0;
    for (i=1;i<=n;i++)
        if (i!=3 && i!=4)
            s=s+(2*i+c);
return s;
}

```

Meqë për llogaritjen e shumës nevojiten variablat **n** dhe **c**, të cilat nuk paraqiten në komponentet e strukturës, ato figurojnë brenda kllapave të funksionit **Shuma** si variabla të zakonshme. Ashtu shënohen edhe te shprehja:

```
r=a*x+b*Shuma(n,c);
```

gjatë thirrjes së funksionit në fjalë. Këto dy variabla paraqiten si parametra formalë edhe te funksioni **Gama**, i cili është vendosur si komponente e strukturës, meqë brenda këtij funksioni thirret funksioni **Shuma**.

Rezultati që fitohet pas ekzekutimit të programit të dhënë do të duket si në Fig.3.31.

Fig.3.31

Pamja e ekranit pas ekzekutimit të programit **structDa**

```
Të dhënat e inicializuara
Variabla a: 2
Variabla b: 4
Variabla x: 3

Vlera e funksionit g=54
Press any key to continue
```

Nëse parametrat e funksioneve që nuk i takojnë strukturës, por të cilët thirren nga funksionet në komponentet e strukturës, janë variabla të komponenteve të strukturës, nuk duhet të shënohen si parametra formalë te funksionet brenda strukturës.

Shembull

Programi **structDb**, si version i programit **structDa**, por tek i cili edhe variablat **c** dhe **n** merren si komponente të strukturës **Alfa**.

```
// Programi structDb
#include <iostream>
using namespace std;

double Shuma(int n,double c);

struct Alfa
{
    double a,b,x,c;
    int n;
    double Gama();
};

int main()
{
    double g;

    Alfa R={2,4,3,3,4};
```

```

        cout << "\nTë dhënat e inicializuara\n"
              << "\nVariabla a: "
              << R.a
              << "\nVariabla b: "
              << R.b
              << "\nVariabla c: "
              << R.c
              << "\nVariabla x: "
              << R.x
              << "\nVariabla n: "
              << R.n
              << "\n";

        g=R.Gama();

        cout << "\nVlera e funksionit g="
              << g
              << "\n\n";

    return 0;
}

double Alfa::Gama()
{
    double r;
    r=a*x+b*Shuma(n,c);
    return r;
}

double Shuma(int n,double c)
{
    int i;
    double s=0;
    for (i=1;i<=n;i++)
        if (i!=3 && i!=4)
            s=s+(2*i+c);
    return s;
}

```

Nga analiza e këtij versioni të programit shihet se, meqë variablat **c** dhe **n** janë përfshirë në komponentet e strukturës, te funksioni **Gama** ato nuk duhet të shënohen si parametra. Prandaj, edhe gjatë thirrjes së funksionit në fjalë, për ta llogaritur vlerën e funksionit **g**:

```
g=R.Gama();
```

nuk shënohet asnjë parametër aktual. Por, meqë funksioni **Shuma** definohet pavarësisht nga struktura, variablat **n** dhe **c** duhet patjetër të paraqiten si parametra.

Nëse ekzekutohet programi i dhënë, rezultati do të jetë i njëjtë me atë që u dha në *Fig.3.31*.

Disa funksione brenda strukturës

Brenda një strukture njëkohësisht mund të paraqiten disa komponente me funksione. Definimi dhe shfrytëzimi i tyre nuk dallon aspak nga rastet kur struktura përmban vetëm një komponente me funksion.

Shembull

Programi **struct29a**, përmes së cilit llogaritet sipërfaqja dhe perimetri i katërkëndëshit kënddrejtë. Për llogaritje shfrytëzohen funksionet **siperfaqja** dhe **perimetri**, të cilët janë komponente të strukturës **brinjet**.

```
// Programi struct29a
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;

    double siperfaqja()
    {
        return a*b;
    };

    double perimetri()
    {
        return 2*a+2*b;
    };
};

int main()
{
    double s,p;

    brinjet kater = {6,4};

    s=kater.siperfaqja();
    p=kater.perimetri();
```

```

    cout << "\nVlera e brinjës a: "
        << kater.a;
    cout << "\nVlera e brinjës b: "
        << kater.b;

    cout << "\n\nSipërfaqja e katërkëndëshit s="
        << s
        << "\nPerimetri i katërkëndëshit p="
        << p
        << "\n\n";
return 0;
}

```

Në program, brenda strukturës **brinjet** si komponente paraqiten edhe dy funksione. Me funksionin **siperfaqja** definohet llogaritja e sipërfaqes së katërkëndëshit, kurse për llogaritjen e perimetrit të tij është definuar funksioni **perimetri**. Meqë funksionet janë vendosur si komponente të strukturës, deklarimi i parametrave formalë përkatës brenda kllapave është i panevojshëm.

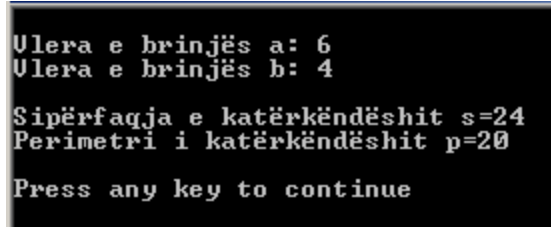
Funksionet në fjalë brenda programit thirren duke i shfrytëzuar shprehjet:

```

s=kater.siperfaqja();
p=kater.perimetri();

```

Nëse ekzekutohet programi i dhënë, rezultati do të duket si në Fig.3.32.



```

Vlera e brinjës a: 6
Vlera e brinjës b: 4

Sipërfaqja e katërkëndëshit s=24
Perimetri i katërkëndëshit p=20

Press any key to continue

```

Fig.3.32
Pamja e ekranit pas ekzekutimit të
programit **struct29a**

Funksionet që deklarohen si komponente të strukturës mund të përmbajnë edhe më shumë komanda. Kështu, struktura në programin **struct29a** mund të deklarohet:

```

struct brinjet
{
    double a,b;
    double siperfaqja()
    {
        double s;
        s=a*b;
        return s;
    }
}

```

```
};

double perimetri()
{
    double p;
    p=2*a+2*b;
    return p;
};
};
```

Brenda funksioneve, të cilat paraqiten si komponente të funksioneve që përfshihen në strukturë, përmes komandave:

```
double s;
```

dhe

```
double p;
```

variablat **s** dhe **p** janë deklaruar si *variabla lokale* dhe, si të tilla, mund të shfrytëzohen vetëm brenda nënprogrameve.

Edhe në rastet kur si komponente të strukturave paraqiten dy e më shumë funksione, definimi i tyre mund të bëhet jashtë strukturës. Kështu, programi i dhënë më sipër, me ndryshimet e përmendura, do të duket si në vijim.

```
// Programi struct29b
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
    double siperfaqja();
    double perimetri();
};

int main()
{
    double s,p;
    brinjet kater = {6,4};
    s=kater.siperfaqja();
    p=kater.perimetri();
    cout << "\nVlera e brinjës a: "
          << kater.a;
    cout << "\nVlera e brinjës b: "
          << kater.b;
```

```

        cout << "\n\nSipërfaqja e katërkëndëshit s="
              << s
              << "\nPerimetri i katërkëndëshit p="
              << p
              << "\n\n";
    return 0;
}

double brinjet::siperfaqja()
{
    double s;
    s=a*b;
    return s;
};

double brinjet::perimetri()
{
    double p;
    p=2*a+2*b;
    return p;
};

```

Nëse ekzekutohet programi i dhënë, rezultati nuk do të dallohet nga ai që u dha më sipër.

Disa struktura brenda funksioneve

Funksionet që paraqiten si komponente të strukturës, përveç variablaeve të komponenteve të strukturës, gjatë llogaritjeve mund të shfrytëzojnë edhe variabla të komponenteve të strukturave të tjera që definojnë në program. Këto variabla, te prototipi i funksionit duhet të deklarohen si parametra formalë të tipeve përkatëse. Kurse, gjatë thirrjes së funksioneve në programin kryesor, parametrat formalë zëvendësohen me variablat e komponenteve të strukturave të tjera.

Shembull

Programi **structF1** përmes së cilit llogaritet vlera e funksionit:

$$g = \begin{cases} ax^2 + bx + c & \text{për } a < b \\ ax + 3c - d & \text{për } a \geq b \end{cases}$$

ku variablat **a**, **b** dhe **c** vendosen te komponentet e strukturës **Alfa**, kurse variablat **x** dhe **d** - te komponentet e strukturës **Beta**. Për llogaritjen e vlerës së funksionit **g** shfrytëzohet funksioni **Gama**, i cili paraqitet si komponente e strukturës **Alfa**.

```
// Programi structF1
```

```
#include <iostream>
using namespace std;

struct Alfa
{
    double a,b,c;
    double Gama(double x,double d);
};

struct Beta
{
    double x,d;
};

int main()
{
    double g;

    Alfa Dita = {1,2,3};
    Beta Nata={4,5};

    cout << "\nVlerat e variablave"
         << "\n\nVariabla a: "
         << Dita.a
         << "\nVariabla b: "
         << Dita.b
         << "\nVariabla c: "
         << Dita.c
         << "\nVariabla x: "
         << Nata.x
         << "\nVariabla d: "
         << Nata.d;

    g=Dita.Gama(Nata.x,Nata.d);

    cout << "\n\nVlera e funksionit g="
         << g
         << "\n\n";
    return 0;
}

double Alfa::Gama(double x,double d)
{
    double g;
    if (a<b)
        g=a*x*x+b*x+c;
    else
        g=a*x+3*c-d;
```



```
return g;
};
```

Këtu, te prototipi i funksionit:

```
double Gama(double x,double d);
```

si parametra formalë të zakonshëm janë shënuar emrat e variablave **x** dhe **d**, të cilat përfshihen në komponentet e strukturës **Beta**. Pastaj, gjatë thirrjes së funksionit **Gama**, për ta llogaritur vlerën e funksionit **g**:

```
g=Dita.Gama(Nata.x,Nata.d);
```

si parametra aktualë janë shfrytëzuar variablat e komponenteve të strukturës **Beta**.

Nëse ekzekutohet programi i dhënë, rezultati do të duket si në Fig.3.33.

Fig.3.33

Pamja e ekranit pas ekzekutimit të programit
structF1

```
Vlerat e variablave
Variabla a: 1
Variabla b: 2
Variabla c: 3
Variabla x: 4
Variabla d: 5

Vlera e funksionit g=27
Press any key to continue
```

Vlera e funksionit është llogaritur me shprehjen:

$$g = ax^2 + bx + c$$

sepse **a<b**, gjë që shihet edhe nga rezultati.

Nënkuptohet se funksioni që paraqitet në komponenten e njërës strukturë, përveç variablave të komponenteve të strukturës tjetër, mund të shfrytëzojë edhe

vlera të variablave të zakonshme. Për këtë qëllim, te prototipi i funksionit variablat duhet të shënohen si parametra formalë.

Shembull

Programi **structF2**, përmes së cilit tabelohen vlerat e funksioneve:

$$y = a \cdot \sin(bx)$$

$$z = \frac{c}{d} \cdot \ln(ex)$$

për vlera të ndryshme të variablës **x**, mes vlerës fillestare **1** dhe vlerës përfundimtare **5**, duke e ndryshuar atë me hapin **0.5**. Në program janë definuar strukturat **Ro** dhe **Fi**. Në komponentet e strukturës **Ro**, përveç variablave **a**, **b** dhe **x**, paraqiten edhe funksionet **FunkY** dhe **FunkZ**. Kurse, struktura **Fi** në komponentet e saj i përfshin vetëm variablat **c** dhe **d**.

```
// Programi structF2
#include <iostream>
#include <iomanip>
#include <math.h>
using namespace std;

struct Ro
{
    double a,b,x;
    double FunkY();
    double FunkZ(double c,double d,double e);
};

struct Fi
{
    double c,d;
};

int main()
{
    double e=3,y,z;
    char g[]="-----";

    Ro R = {2,5};
    Fi F={6,4};

    cout << "\nVlerat e variablave"
         << "\n\nVariabla a: "
         << R.a
         << "\nVariabla b: "
```

```
        << R.b
        << "\nVariabla c: "
        << F.c
        << "\nVariabla d: "
        << F.d
        << "\nVariabla e: "
        << e
        << "\n\n";

    cout << "      x          y          z"
    << "\n"
    << g
    << "\n";
for (R.x=1;R.x<=5;R.x=R.x+0.5)
{
    y=R.FunkY();
    z=R.FunkZ(F.c,F.d,e);

    cout << fixed
        << setprecision(2)
        << setw(6)
        << R.x
        << setw(8)
        << y
        << setw(8)
        << z

        << "\n";
}
    cout << g
    << "\n\n";
return 0;
}

double Ro::FunkY()
{
    double y;
    y=a*sin(b*x);
return y;
};

double Ro::FunkZ(double c,double d,double e)
{
    double z;
    z=c/d*log(e*x);
return z;
};
```

Funksioni **FunkY** nuk ka asnjë parametër, meqë variablat në shprehjen e funksionit **y**, që definoohen përmes tij, bëjnë pjesë në komponentet e strukturës **Ro**, ku edhe funksioni paraqitet në njëërën prej komponenteve të saj. Brenda kllapave të funksionit **FunkZ** paraqiten **3** parametra formalë, të cilët marrin pjesë në llogaritjen e vlerës së funksionit **z**, por nuk janë variabla në komponentet e strukturës **Ro**.

Meqë variablat **c** dhe **d** të funksionit **z** përfshihen në komponentet e strukturës **Fi**, gjatë thirrjes së funksionit **FunkY** në program, ato zëvendësohen me parametrat aktualë **F.c** dhe **F.d**:

```
z=R.FunkZ(F.c,F.d,e);
```

pasi variabla **F** është deklaruar e tipit të strukturës **Fi**.

Këtu, vlerat e funksioneve **y** dhe **z** llogariten dhe shtypen brenda unazës së variablës **x**. Si rezultat në ekran fitohet tabela e cila shihet në *Fig.3.34*, për vlera të ndryshme të variablës **x** që përfshihet në komponenten e strukturës **Ro** (shkruhet si **F.x**, sepse variabla **F** është e tipit të kësaj strukture).

Vlerat e variablave

```
Variabla a: 2
Variabla b: 5
Variabla c: 6
Variabla d: 4
Variabla e: 3
```

x	y	z
1.00	-1.92	1.65
1.50	1.88	2.26
2.00	-1.09	2.69
2.50	-0.13	3.02
3.00	1.30	3.30
3.50	-1.95	3.53
4.00	1.83	3.73
4.50	-0.97	3.90
5.00	-0.26	4.06

Press any key to continue

Fig.3.34

Pamja e ekranit pas ekzekutimit të programit **structF2**

Fushat në struktura

Në komponentet e strukturave mund të vendosen edhe fusha, p.sh., siç janë vektorët. Rezervimi i vendeve të nevojshme për fushat (numri maksimal i mundshëm i vendeve) bëhet përmes deklarimit si konstante para definimit të strukturave. Por, njëkohësisht brenda strukturave, në komponente të veçanta vendosen variablat në të cilat ruhen dimensionet aktuale të fushave.

Shembull

Programi **structV1** përmes së cilit gjendet anëtarë maksimal në vektorin e dhënë **A(n)**, i cili është vendosur në komponenten e strukturës **Vektori**.

```
// Programi structV1
#include <iostream>
#include <iomanip>
using namespace std;

const int m=10;

struct Vektori
{
    int n;
    double A[m];
};

int main()
{
    int i;
    double x;
    char g[]="-----";

    Vektori Vlera = {5,2.7,4.1,3.5,12.6,8.3};

    cout << "\nAnëtarët e vektorit\n"
    << "\n   i      A[i]\n"
    << g
    << "\n";

    for (i=0;i<Vlera.n;i++)
        cout << setw(4)
        << i
        << setw(10)
        << Vlera.A[i]
        << "\n";

    cout << g
```

```

        << "\n";

        x=Vlera.A[0];
        for (i=1;i<Vlera.n;i++)
            if (Vlera.A[i]>x)
                x=Vlera.A[i];

        cout << "\nVlera maksimale x="
              << x
              << "\n\n";
    return 0;
}

```

Në komponenten e parë të strukturës **Vektori** është vendosur variabla **n**, në të cilën do të ruhet numri aktual i anëtarëve të vektorit, i cili shfrytëzohet në program. Kurse në komponenten e dytë të tij përfshihet vektori **A** me **m**-anëtar, ku **m** e paraqet numrin maksimal të mundshëm të tij, i cili numër këtu është deklaruar paraprakisht si konstante me vlerë **10**.

Gjatë deklarimit të variabës **Vlera** të strukturës **Vektori**:

```
Vektori Vlera = {5,2.7,4.1,3.5,12.6,8.3};
```

bëhet inicializimi me vlera, së pari i variablës **n** (numri i parë **5**) dhe pastaj edhe i anëtarëve të vektorit **A**.

Në program, përmes unazës së parë **for** shtypen anëtarët e vektorit, duke i shkruar në formën **Vlera.A[i]**. Kjo formë e shkruarjes shfrytëzohet edhe më poshtë, gjatë gjetjes së anëtarit maksimal, duke e zbatuar algoritmin e zakonshëm për gjetje të anëtarit të caktuar në vektor.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket ashtu siç shihet në Fig.3.35.



```

Anëtarët e vektorit
  i      A[i]
-----
  0      2.7
  1      4.1
  2      3.5
  3     12.6
  4      8.3
-----

Vlera maksimale x=12.6
Press any key to continue

```

Fig.3.35

Pamja e ekranit pas ekzekutimit të programit *structV1*

Gjatë operimit me anëtarët e fushave, brenda strukturës mund të shfrytëzohen edhe funksione.

Shembull

Programi **structV2**, si version i programit **structV1**, tek i cili për gjetjen e anëtarit maksimal **x** të vektorit **A(n)** shfrytëzohet funksioni **Max**, që vendoset në komponenten e strukturës **Vektori**.

```
// Programi structV2
#include <iostream>
#include <iomanip>
using namespace std;

const int m=10;

struct Vektori
{
    int n;
    double A[m];
    double Max();
};

int main()
{
    int i;
    Vektori Vlera = {5,2.7,4.1,3.5,12.6,8.3};
    cout << "\nAnëtarët e vektorit\n"
         << "\n   i      A[i]"
         << "\n-----\n";

    for (i=0;i<Vlera.n;i++)
        cout << setw(4)
              << i
              << setw(10)
              << Vlera.A[i]
              << "\n";
    cout << "-----\n"
         << "\nVlera maksimale x: "
         << Vlera.Max()
         << "\n\n";

    return 0;
}

double Vektori::Max()
{
    double x=A[0];
    int i;
    for (i=1;i<n;i++)
```

```
        if (A[i]>x)
            x=A[i];
return x;
};
```

Për gjetjen e anëtarit maksimal **x** brenda vektorit **A(n)** përmes funksionit **Max**, këtu është shfrytëzuar procedura e zakonshme, ashtu siç shpjegohet në vijim.

- Përmes deklarimit **x=A[0]**, si vlerë fillestare për anëtarin maksimal merret anëtari i parë i vektorit.
- Duke i krahasuar të gjithë anëtarët e vektorit, sa herë që gjendet se një anëtar është më i madh se vlera aktuale e variablës **x**, përmes shprehjes:

x=A[i];

te variabla **x** ruhet vlera e re maksimale.

- Në fund, vlera maksimale brenda vektorit është vlera që ruhet te variabla **x**.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të jetë i njëjtë me atë që u dha në *Fig.3.35*.

Fushat e strukturave

Ngjashëm si fushat e zakonshme, mund të krijohen edhe fusha të strukturave. Gjatë kësaj, në një element të fushës përfshihen të dhënat e të gjitha komponenteve përkatëse të strukturës.

Shembull

Programi **structFS1**, përmes së cilit tregohet deklarimi si vektor me **n**-anëtarë i strukturës **studenti**, e cila është e tipit **person**.

```
// Programi structFS1
#include <iostream>
#include <iomanip>
using namespace std;

struct person
{
    char emri[8],qyteti[10];
```



```

    int viti;
};

int main()
{
    const int n=3;
    int i;

    person studenti[n];

    cout << "\nTë dhënat nga tastiera\n\n";

    for (i=0;i<n;i++)
    {
        cout << "Emri .....: ";
        cin >> studenti[i].emri;
        cout << "Qyteti ...: ";
        cin >> studenti[i].qyteti;
        cout << "Viti .....: ";
        cin >> studenti[i].viti;
        cout << "\n";
    }
    cout << "\nTë dhënat e lexuara\n\n"
        << "      Emri      Qyteti   Viti"
        << "\n-----\n"
        << right;

    for (i=0;i<n;i++)
    {
        cout << setw(8)
            << studenti[i].emri;
        cout << setw(13)
            << studenti[i].qyteti;
        cout << setw(7)
            << studenti[i].viti
            << "\n";
    }
    cout << "-----"
        << "\n\n";
    return 0;
}

```

Në fillim të programit është definuar struktura **person**, me komponentet të cilat i përmbajnë variablat **emri**, **vendi** dhe **viti**. Pastaj, përmes komandës:

```
person studenti[n];
```

variabla e strukturës **person** deklarohet si vektor **studenti**. Si rezultat, çdo anëtar i vektorit përmban **3** të dhëna, emrin e studentit, vendin dhe vitin e lindjes. Në këtë mënyrë, nëse të dhënat e regjistruara vërehen nga jashtë, ato do të mund të vendoseshin në një matricë me **n**-rreshta dhe **3**-kolona.

Për qasje në variablat e komponenteve të veçanta të strukturës **studenti**, për anëtarin e **i**-të të vektorit, ato shkruhen në formën:

```
studenti[i].vendi
studenti[i].qyteti
studenti[i].viti
```

Nëse ekzekutohet programi i dhënë dhe përmes tastierës kompjuterit i jepen të dhënat e kërkuara për **3** studentë (meqë **n=3**), në ekran do ta kemi pamjen e dhënë në *Fig.3.36*.

```
Të dhënat nga tastiera
Emri .....: Agim
Qyteti ....: Prishtina
Viti .....: 1984

Emri .....: Besa
Qyteti ....: Shkupi
Viti .....: 1985

Emri .....: Fatmir
Qyteti ....: Prizreni
Viti .....: 1983

Të dhënat e lexuara
```

Emri	Qyteti	Viti
Agim	Prishtina	1984
Besa	Shkupi	1985
Fatmir	Prizreni	1983

```
Press any key to continue
```

Fig.3.36

*Pamja e ekranit pas ekzekutimit të programit **structFS1***

Variablat e përfshira në komponentet e strukturës që është deklaruar si fushë mund të inicializohen direkt në program.

Shembull

Programi **structFS2**, përmes së cilit tregohet inicializimi direkt i variablave të strukturës **java**, e cila është deklaruar si vektor i tipit **dita**.

```
// Programi structFS2
#include <iostream>
#include <iomanip>
using namespace std;

struct dita
{
    int nr;
    double temp;
};

int main()
{
    int i;
    double s;

    dita java[7]={1,10.5,
                  2,12.7,
                  3,15.4,
                  4,11.8,
                  5,12.3,
                  6,13.9,
                  7,12.5};

    cout << "\nTë dhënat e strukturës java\n"
          << "\n Dita      Temperatura\n"
          << "-----\n";

    for (i=0;i<7;i++)
    {
        cout << setw(4)
              << java[i].nr
              << " "
              << setw(10)
              << java[i].temp
```

```

        << "\n";
    }
    cout << "-----"
        << "\n\n";

    s=0;
    for (i=0;i<7;i++)
        s=s+java[i].temp;

    cout << "Temperatura mesatare: "
        << s/7
        << "\n\n";
return 0;
}

```

Këtu, pas deklarimit si vektor të strukturës **java** e tipit **dita**, ajo është inicializuar me vlera, plotësisht njëloj siç do të inicializohej matrica me **7**-rreshta dhe dy kolona. Kolona e parë i përgjigjet variablës **nr**, kurse në kolonën e dytë përfshihen vlerat e temperaturave ditore **temp**.

Pas inicializimit, të dhënat e temperaturave javore shtypen si tabelë, duke i shkruar komponentet e strukturës në formën:

```

java[i].nr
java[i].temp

```

Në fund, përmes unazës **for**, llogaritet shuma **s** e temperaturave të **7** ditëve të javës dhe shtypet e pjesëtuar me **7**, ashtu që të fitohet temperatura mesatare javore.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.3.37.

Të dhënat e strukturës java	
Dita	Temperatura
1	10.5
2	12.7
3	15.4
4	11.8
5	12.3
6	13.9
7	12.5

Temperatura mesatare: 12.7286	
Press any key to continue	

Fig.3.37
 Pamja e ekranit pas ekzekutimit të
 programit *structFS2*

Në vend të detyrave me sipërfaqe dhe perimetër të merren det. me shuma dhe det. tjera më serioze. Korrigjimet të bëhen për libër.

===== Ndoshta të fshihet =====
Shembull: struktura që e kam marrë prej shembullit class1. Meqë te funksioni nuk kam shënuar parametra, kjo e nënkupton se shfrytëzohen variablat e strukturës. Kompjuteri njohtimin e merrë edhe prej `Nata::Maksimale()`.

```
// Programi structF
#include <iostream>
using namespace std;

struct Nata
{
    double Maksimale();
    double a,b,c;
};

int main()
{
    Nata Vlera={7.4,14.2,9.6};
    double z;
    z=Vlera.Maksimale();
    cout << z;
    return 0;
}

double Nata::Maksimale()
{
    double g;
    if ((a>b) && (a>c))
        g=a;
    else
        if (b>c)
            g=b;
        else
            g=c;
    return g;
}
```

Unazat mund të shfrytëzohen prej fillimit!!!

17-545
Strukturat e ndërthurura
17-551
f128 Programmers Notebook

Strukturat si parametra të funksioneve
17-546

f.398 Pascal, f.540 Malik.

Strukturat dhe unionet f214 Practical C++

Programmer's Notebook

f114 Struktura mund të përmbajë edhe variabla të tipeve
që përcaktohen nga vetë shfrytëzuesi (p.sh. var. të numëruara

Daitel f406

17-540

Variablat e strukturës (Daitel nuk e shfrytëzon si objekt)

Komponentet e strukturës

Variablat e komponenteve të strukturës

Me deklarimin formohet kopja ose instanca e strukturës.

- versioni kur te një funksion shfrytëzohen disa struktura (njëra është strukturë ku është përfshirë funksioni)

- te dy funksione shfrytëzohen dy struktura (njëra është strukturë ku është përfshirë funksioni).

- në të dy rastet me parametra formal të variablave jashtë strukturës.

Fushat në struktura: 17-547

Fushat e strukturave: 11-220

Konstruktorët duhet të vlejë edhe për strukturat.

Vlerat e variablave të cilat brenda strukturës janë deklaruar si private, mund të merren edhe *përmes referencës* (operatori **&**). Kjo ka rëndësi sidomos kur nevojitet që përmes një funksioni prej komponenteve të strukturës të merren vlerat e disa variablave.

Shih te klasat dhe te pointerët.

17-300

Edhe te strukturat kjo ka rëndësi sepse parametrat aktualë nuk do ta kenë pikën.

Në ribotim të jepet një shembull me referenca edhe te strukturat.

	3	
Klasat		
Definimi i klasave të zakonshme 134		
Deklarimi i objekteve 136		
Qasja te komponentet e klasës 137		
Forma e përgjithshme e klasave 139		
Definimi i funksioneve jashtë klasës 143		
Forma të tjera të inicializimit të variablove 145		
Shfrytëzimi i vlerave të variablove private 149		
Llogaritjet me variablat e klasës 153		
Shfrytëzimi i funksioneve brenda klasës 164		
Konstruktorët 172		
Destruktorët 188		
Trashëgimia 191		
Operatori i shoqërimit tek objektet 204		
Krahasimi i variablove të klasës 206		
Fushat brenda klasave 212		
Fushat e objekteve 214		
Dukshmëria e klasave dhe e objekteve 217		

Kur flitet për *programimin e orientuar në objekte* (ang. object-oriented programming), ose shkurt - *programimin me objekte*, gjithnjë mendohet në *klasat* si dhe në *objektet* që deklarohen me shfrytëzimin e tyre. Klasat paraqesin një tip tjetër të strukturave, në të cilat bashkërisht vendosen të dhënat dhe funksionet që i shfrytëzojnë ato të dhëna. Por, në gjuhën C++, *strukturat dhe klasat kanë një dallim të vogël*. Derisa *qasja e nënkuptuar* (ang. default access) të strukturat është *publike* (ang. public), të klasat kjo qasje e nënkuptuar është *private* (ang. private), gjë që do të shpjegohet në pjesët vijuese.

Përmes klasave jepet një mundësi e shkruarjes së programeve, të cilët sipas nevojës, lehtë ndryshohen, duke i ndryshuar vetëm klasat. Gjatë kësaj, problemi që zgjidhet copëtohet në klasa dhe në deklarimin e objekteve përkatëse, gjë që ka një rëndësi të veçantë kur kemi të bëjmë me programe komplekse, me çka zvogëlohet mundësia e gabimeve.

Programimi i zakonshëm, në të cilin shfrytëzohen vetëm funksionet, ndryshe quhet edhe *programim procedural*. Përmes programimit me objekte, më lehtë modelohet bota reale, krahasuar me programimin procedural, meqë brenda objekteve përfshihen funksionet dhe të dhënat të cilat ato i shfrytëzojnë.

Në fillim të pjesës vijuese, me qëllim të krahasimit të strukturave dhe të klasave, do të shfrytëzohen shembujt e programeve elementare, të cilët janë marrë gjatë shpjegimit të strukturave.

Definimi i klasave të zakonshme

Të gjitha format e definimit të strukturave vlejnë edhe për definimin e klasave të zakonshme. Kështu, p.sh., le ta marrim strukturën e cila në formë të përgjithshme duket si në vijim:

```
struct e
{
    t1 x1;
    t2 x2;
    .....
    tn xn;
};
```

ku janë:

- e** - emri i strukturës.
- t1, t2, ..., tn** - tipet e të dhënave në komponentet e strukturës.
- x1, x2, ..., xn** - variablat në komponentet e strukturës.

Definimi i strukturës së dhënë si klasë do të bëhet:

```
class e
{
public:
    t1 x1;
    t2 x2;
    .....
    tn xn;
};
```

ku janë:

- e** - emri i klasës.
- t1, t2, ..., tn** - tipet e të dhënave në komponentet e klasës.
- x1, x2, ..., xn** - variablat në komponentet e klasës.

Nëse krahasohet forma e përgjithshme e klasës me formën e përgjithshme të strukturës, që u dhanë më sipër, qartë shihet se:

- në vend të fjalës **struct**, këtu është shfrytëzuar fjala **class** dhe
- para komponenteve të klasës është shënuar fjala **public**.

Me fjalën **public** të shënuar para komponenteve të klasës, atyre mund t'u qasemi dhe t'i shfrytëzojmë në program. Fjala **public**, e cila njihet si *specifikuesit e qasjes* (ang. access specifier), nuk shfrytëzohej te struktura, sepse, siç u tha edhe në fillim, qasja e tillë te struktura është e nënkuptuar, përkatësisht *struktura është klasë me qasje publike*. Kurse, nëse te klasa nuk shfrytëzohet specifikuesi **public**, ai do të nënkuptohet nga kompjuteri si **private**, dhe qasja direkte nga jashtë është e pamundshme.

Shembull

Programi **class1**, në të cilin është definuar klasa **person**, ku përfshihen të dhënat e tipeve të ndryshme të një personi, siç janë emri, viti i lindjes dhe qyteti i banimit.

```
// Programi class1
#include <iostream>
using namespace std;

class person
{
public:
    char emri[8];
    char qyteti[10];
    int viti;
};

int main()
{
}
```

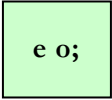
Shembulli i programit të dhënë është i ngjashëm me programin **struct1**, i shfrytëzuar gjatë shpjegimit të definimit të strukturave. Nëse krahasohet programi **class1** me programin **struct1**, siç u tha edhe më sipër, para komponenteve të klasës është shtuar fjala **public:**. Meqë në trupin e programit të dhënë nuk përfshihen komanda, me ekzekutimin e tij nuk do të merret asnjë rezultat.

Deklarimi i objekteve

Pas definimit të një klase kompjuteri nuk rezervon vende në memorie për komponentet që përfshihen brenda klasës, pavarësisht se deklarohen tipet e variablave përkatëse. Por, me klasën *krijohet një tip i ri*, i cili pastaj mund të *shfrytëzohet për deklarimin e objekteve* të asaj klase.

Definimi i klasës tregon vetëm se si objekti duket, kurse pas deklarimit në program, krijohet *objekti i klasës*, ose, siç thuhet, krijohet *instanca e klasës*.

Deklarimi i objekteve të klasës bëhet plotësisht njëloj siç deklarohen variablat e strukturave, ose edhe variablat e tipeve të zakonshme. Por, këtu, në vend të variablës *deklarohet një objekt*. Në formë të përgjithshme ky deklarim duket:



e o;

ku janë:

- e** - emri i klasës.
- o** - objekti i tipit të klasës **e**.

Shembull

Programi **class2**, në të cilin shihet definimi i klasës **person** dhe shfrytëzimi i saj për deklarimin e objektit **studenti** të tipit të klasës **person**.

```
// Programi class2
#include <iostream>
using namespace std;

class person
{
public:
    char emri[8], qyteti[10];
    int viti;
};

int main()
{
    person studenti;
}
```

Këtu, përmes shprehjes:

```
person studenti;
```

deklarohet objekti **studenti** i klasës **person**, i cili në fakt paraqet një kopje të klasës që është definuar më parë. Pas këtij deklarimi, në memorien e kompjuterit rezervohen vende për variablat të cilat paraqiten në komponentet e klasës. Nëse ekzekutohet programi i dhënë, meqë në trupin e tij deklarohet dhe nuk shfrytëzohet objekti **studenti**, kompjuteri do të gjenerojë një mesazh për të na njoftuar se **studenti** është variabël lokale që nuk i referohemi (që nuk shfrytëzohet).

Qasja te komponentet e klasës

Komponenteve të klasës mund t'u qasemi pasi të jetë deklaruar objekti përkatës. Për qasje në komponente të klasës shfrytëzohen shprehjet e formës:

o.x

ku janë:

o - objekti i deklaruar i klasës.

- x** - variabla ose funksioni në komponenten e klasës.
- .** - operatori pikë (ang. dot operator) për qasje variablës ose funksionit të komponentes së klasës.

Shembull Programi **class3**, përmes së cilit tregohet qasja te komponentet e klasës **person**.

```
// Programi class3
#include <iostream>
using namespace std;
class person
{
public:
    char emri[8], qyteti[10];
    int viti;
};

int main()
{
    person studenti;

    cout << "\nTë dhënat nga tastiera\n\n";
    cout << "Emri .....: ";
    cin >> studenti.emri;
    cout << "Qyteti ...: ";
    cin >> studenti.qyteti;
    cout << "Viti .....: ";
    cin >> studenti.viti;

    cout << "\n\nTë dhënat e lexuara\n";
    cout << "\nEmri .....: "
        << studenti.emri;
    cout << "\nQyteti ...: "
        << studenti.qyteti;
    cout << "\nViti .....: "
        << studenti.viti
        << "\n\n";

    return 0;
}
```

Në program, si edhe te strukturat, tri variabla të përfshira në objektin **studenti**, i cili është deklaruar i klasës **person**, u qasemi duke i shënuar ato në format:

```
studenti.emri
studenti.qyteti
```

```
studenti.viti
```

Nëse ekzekutohet programi i dhënë dhe pas mesazheve përkatëse, përmes tastierës kompjuterit i jepen vlerat hyrëse **Valmira**, **Ohri** dhe **1983**, në ekran do ta kemi pamjen e cila shihet në Fig.4.1.

Fig.4.1

Pamja e ekranit pas ekzekutimit të programit
class3

```
Të dhënat nga tastiera
Emri .....: Valmira
Qyteti ....: Ohri
Viti .....: 1983

Të dhënat e lexuara
Emri .....: Valmira
Qyteti ....: Ohri
Viti .....: 1983

Press any key to continue
```

Forma e përgjithshme e klasave

Zakonisht, kur flitet për klasat, nënkuptohet se komponentet e tyre përmbajnë *variabla* dhe *funksione* të tipeve të caktuara. Specifikuesit e qasjes së tyre, përveç **public**, që u shpjegua më sipër, mund të jenë edhe **private** ose **protected**. Këtu, fillimisht, do të flitet për dy tipet e para të specifikuesve, kurse për specifikuesin **protected** do të bëhet fjalë më vonë.

Forma e përgjithshme e definimit të klasës mund të duket:

```
class e
{
    private:
        t1 y1;
        t2 y2;
        .....
        tn yn;
    public:
        tp zp;
        tq zq;
        .....
        ts zs;
};
```

ku janë:

e - emri i klasës.

t1, t2, ..., ts - tipet e variablave ose të funksioneve në komponentet e klasës.

y1, y2, ..., yn - variablat ose funksionet në komponentet e klasës, të deklaruar si publike.

zp, zq, ..., zs - variablat ose funksionet në komponentet e klasës, të deklaruar si private.

Radha e shkruarjes së specifikuesve të qasjes brenda definicionit të klasës nuk ka rëndësi, si dhe specifikuesit e veçantë mund të shfrytëzohen edhe disa herë, gjë që nuk është e nevojshme.

Variablat e tipeve të caktuara që përfshihen në klasë, njihen si *komponente të dhënash* (ang. data components), ose *anëtarë të dhënash* (ang. data members). Kurse funksionet që përfshihen në klasë njihen si *komponente funksionesh* (ang. function components), ose *anëtarë funksionesh* (ang. member functions), ose edhe *metoda* (ang. methods). Të gjitha komponentet ose anëtarët brenda klasës me një fjalë mund të quhen *komponente të klasës* (ang. class components), ose *anëtarë të klasës* (ang. class members).

Zakonisht, komponentet me të dhëna deklarohen si private, kurse komponentet e funksioneve - si publike. Por, kjo nuk është e thënë, sepse brenda klasave njëkohësisht mund të deklarohen funksione dhe variabla private dhe publike.

Deklarimi i komponenteve të klasës si private nuk ka të bëjë me atë se të dhënat janë sekrete dhe si të tilla nuk duhet të shihen nga të tjerët. Por, kjo lidhet me pengimin e shfrytëzimit direkt të tyre me qëllim të eliminimit të gabimeve të mundshme gjatë shkruarjes së programeve të gjata.

Komponentet, të cilat brenda klasës deklarohen si private, nuk mund t'u qasemi direkt nga jashtë. Ato mund të shfrytëzohen direkt vetëm brenda funksioneve të klasës, pavarësisht se a kemi të bëjmë me funksione private ose publike. Kurse qasja te komponentet që deklarohen si publike është e lirë, si brenda klasës ashtu edhe në program.

Shembull

Programi **class4a**, përmes së cilit definohet klasa **Jeta**, në komponentet e së cilës paraqiten variablat **m** dhe **a**, njëra e deklaruar si private, kurse tjetra - si publike, si dhe funksionet **vlera** dhe **shtypja** të deklaruar si publike.

```
// Programi class4a
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;

    void vlera(int k)
    {
        m=k;
    }

    void shtypja()
    {
        cout << "\nVlera e variablës private m="
              << m
              << "\n";
    }
};

int main()
{
    Jeta Dita;

    Dita.vlera(77);
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;

    Dita.shtypja();
    cout << "\nVlera e variablës publike a="
          << Dita.a
          << "\n\n";
    return 0;
}
```

Meqë variabla **m** është deklaruar si variabël private, asaj nuk mund t'i qasemi direkt nga jashtë. Prandaj, për ta inicializuar atë me një vlerë shfrytëzohet funksioni **vlera**, me parametrin formal **k**, të cilin mund ta shfrytëzojmë nga jashtë sepse është deklaruar si funksion publik. Brenda funksionit paraqitet shprehja **m=k**, përmes së cilës vlera e variablës **k** vendoset te variabla **m**.

Nga ana tjetër, variabla **a** dhe funksionet **vlera** dhe **shtypja** janë deklaruar si publike, gjë që e nënkupton se atyre mund t'u qasemi direkt nga jashtë, përkatësisht nga programi.

Në fillim të programit kryesor, përmes komandës:

```
Jeta Dita;
```

deklarohet objekti **Dita** i klasës **Jeta**. Pastaj, duke e thirrur funksionin **vlera**:

```
Dita.vlera(77);
```

ku në vend të parametrit formal **k** është shënuar parametri aktual **77**, variablës **m** i shoqërohet kjo vlerë. Kurse, përmes komandave:

```
cout << "\nLeximi i vlerës së variablës a: ";  
cin >> Dita.a;
```

lexohet vlera e variablës **a**, e cila brenda klasës është deklaruar si publike, vlerë të cilën kompjuterit duhet t'ia japim përmes tastierës.

Për shtypje të vlerës së variablës **m**, meqë nuk mund t'i qasemi direkt, sepse është deklaruar si private, shfrytëzohet funksioni **shtypja**, i cili është deklaruar si funksion publik, duke e thirrur kështu:

```
Dita.shtypja();
```

Funksioni **shtypja** ka qasje te variabla **m**, pavarësisht se ajo është deklaruar si private, sepse ai gjendet në njërin nga komponentet e klasës.

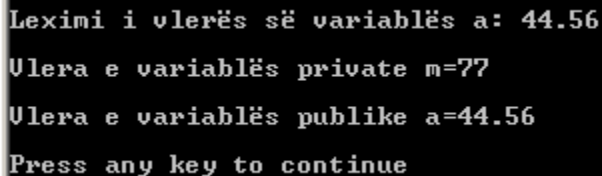
Në fund të programit, për shtypje të vlerës së variablës **a**, lirisht është shfrytëzuar komanda:

```
cout << "\nVlera e variablës publike a=" << Dita.a << "\n\n";
```

sepse në të kemi qasje direkte për shkak të deklarimit të saj si publike.

Nëse, p.sh., përmes tastierës kompjuterit për variablën **a** ia japim vlerën hyrëse **44.56**, në ekran do ta kemi pamjen e dhënë në *Fig.4.2*.

Fig.4.2



```
Leximi i vlerës së variablës a: 44.56  
Vlera e variablës private m=77  
Vlera e variablës publike a=44.56  
Press any key to continue
```

Pamja e ekranit pas ekzekutimit të programit class4a

Funksioni **shtypja** mund të shfrytëzohet edhe për shtypjen e vlerës së variablës **a**, ashtu siç është dhënë në vijim në versionin **class4b** të programit.

```
// Programi class4b
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;
    void vlera(int k)
    {
        m=k;
    }
    void shtypja()
    {
        cout << "\nVlera e variablës private m="
              << m
              << "\n\nVlera e variablës publike a="
              << a
              << "\n\n";
    }
};

int main()
{
    Jeta Dita;
    Dita.vlera(77);
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;
    Dita.shtypja();
    return 0;
}
```

Nëse ekzekutohet ky version i programit, për vlerë hyrëse të njëjtë, rezultati në ekran do të duket si ai që u dha në *Fig.4.2*.

Forma e përgjithshme e dhënë më sipër për definimin e klasave, plotësisht njëjloj mund të shfrytëzohet edhe te strukturat, përfshirë edhe specifikuesit e qasjes **public**, **private** dhe **protected**.

Definimi i funksioneve jashtë klasës

Siç u shpjegua te strukturat, edhe te klasat funksionet mund të definohen jashtë trupit të klasave. Por, brenda definicionit të tyre duhet të *shënohen prototipet e funksioneve*.

Shembull

Programi **class4c** si version i programit **class4b**, tek i cili brenda klasës **Jeta** janë deklaruar vetëm prototipet e funksioneve **vlera** dhe **shtypja**, kurse definimi i tyre është bërë jashtë definicionit të klasës.

```
// Programi class4c
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;
    void vlera(int k);
    void shtypja();
};

int main()
{
    Jeta Dita;
    Dita.vlera(77);
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;
    Dita.shtypja();
    return 0;
}

void Jeta::vlera(int k)
{
    m=k;
}

void Jeta::shtypja()
{
    cout << "\nVlera e variablës private m="
         << m
         << "\n\nVlera e variablës publike a="
         << a
```

```

    << "\n\n";
}

```

Nga definicionet e funksioneve shihet se, plotësisht njëloj si edhe te funksionet që përfshihen në struktura, para emrave të tyre shënohet edhe emri **Jeta** i klasës, i shoqëruar me *operatorin për zhbërthim të dukjes* (ang. scope resolution operator), që shënohet me katër pika (::).

Kjo formë e shkruarjes së funksioneve jashtë definicionit të klasës, sidomos kur funksionet përmbajnë më shumë komanda, është më praktike për shkak të dukshmërisë më të mirë të definicionit të klasës.

Funksionet që përfshihen brenda klasave mund të jenë edhe të tipit **inline**. Kështu, p.sh., nëse funksioni **vlera** te programi i fundit **class4c** merret i tipit **inline**, prototipi i tij brenda definicionit të klasës **Jeta** duhet të shkruhet në formën:

```
inline void vlerat(int k, double x);
```

Gjithashtu, titulli përkatës gjatë definimit të funksionit duhet të shkruhet:

```
inline void Jeta::vlerat(int k, double x)
```

Shfrytëzimi i funksioneve të tipit **inline** lidhet me rritjen e shpejtësisë së ekzekutimit të tyre. Por, gjatë kësaj rritet edhe programi, sepse, kur thirren funksionet e këtij tipi, ato bëhen pjesë e programit.

Forma të tjera të inicializimit të variablave

Për inicializimin e variablave që paraqiten si anëtarë të klasës mund të shfrytëzohen edhe mënyra të tjera. Njëra mënyrë është ajo e leximit nga jashtë të vlerave të tyre, duke definuar brenda klasave funksione me komanda për lexim. Por, nëse variablat vendosen vetëm në pjesën publike të klasave, inicializimi i tyre mund të bëhet edhe gjatë deklarimit të objekteve.

Inicializimi përmes leximit

Një mënyrë tjetër e inicializimit të variablave, në veçanti atyre të deklaruara si private, përveç asaj që u shfrytëzua më sipër, është ajo e leximit të vlerave të tyre. Për këtë qëllim, komandat për lexim duhet të përfshihen në funksionin e veçantë brenda klasës, duke e deklaruar atë si funksion publik.

Shembull

Programi **class5a** si version i programit **class4c**, tek i cili brenda klasës **Jeta** është definuar funksioni **leximi** si publik, përmes së cilit lexohet vlera e variablës private **m**.

```
// Programi class5a
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;
    void leximi();
    void shtypja();
};

int main()
{
    Jeta Dita;
    Dita.leximi();
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;
    Dita.shtypja();
    return 0;
}

void Jeta::leximi()
{
    cout << "\nLeximi i vlerës së variablës m: ";
    cin >> m;
}

void Jeta::shtypja()
{
    cout << "\nVlera e variablës private m="
         << m
         << "\n\nVlera e variablës publike a="
         << a
         << "\n\n";
}
```

Këtu, brenda klasës, si publik është definuar funksioni **leximi**, përmes së cilit lexohet vlera e variablës **m**. Meqë funksioni përfshihet në klasë, ai ka qasje

direkte në këtë variabël. Pastaj, ky funksion thirret në fillim të programit kryesor përmes komandës:

```
Dita.leximi();
```

Nëse ekzekutohet programi i dhënë dhe përmes tastierës si vlera hyrëse për variablat **m** dhe **a** kompjuterit i jepen vlerat **77** dhe **44.56**, rezultati do të duket si në Fig.4.3.

Fig.4.3

*Pamja e ekranit pas
ekzekutimit të programit
class5a*

```
Leximi i vlerës së variablës m: 77
Leximi i vlerës së variablës a: 44.56
Vlera e variablës private m=77
Vlera e variablës publike a=44.56
Press any key to continue
```

Inicializimi gjatë deklarimit të objekteve

Nëse variablat e përfshira në pjesën me të dhëna të klasës deklarohen vetëm si publike, inicializimi i tyre mund të bëhet edhe gjatë deklarimit të objektit përkatës.

Shembull

Versioni **class5b** i programit **class5a**, tek i cili variablat **a** dhe **m** janë përfshirë në pjesën publike të klasës **Jeta**, kurse inicializimi i tyre me vlera bëhet gjatë deklarimit të objektit përkatës **Dita**.

```
// Programi class5b
#include <iostream>
using namespace std;

class Jeta
{
public:
    int m;
    double a;
    void shtypja();
};

int main()
{
    Jeta Dita={77,44.56};
```

```
        Dita.shtypja();
return 0;
}

void Jeta::shtypja()
{
    cout << "\nVlera e variablës private m="
          << m
          << "\n\nVlera e variablës publike a="
          << a
          << "\n\n";
}
```

Siç shihet edhe në program, gjatë deklarimit të objektit **Dita**:

```
Jeta Dita={77,44.56};
```

pas barazimit, brenda kllapave janë shënuar vlerat **77** dhe **44.56**, të cilat u përgjigjen variablave **m** dhe **a**. Nëse ekzekutohet programi i dhënë, përmes funksionit **shtypja**, në ekran do të shtypen vlerat e variablave në fjalë, ashtu siç shihet edhe në pjesën e poshtme të *Fig.4.3*.

Variablat që definojnë si anëtarë publikë të klasës mund të inicializohen me vlera edhe prej programit kryesor.

Shembull

Versioni **class5c** i programit **class5b**, tek i cili variablat **a** dhe **m** janë përfshirë në pjesën publike të klasës **Jeta**, kurse inicializimi i tyre bëhet duke i lexuar vlerat përmes komandave përkatëse në programin kryesor.

```
// Programi class5c
#include <iostream>
using namespace std;

class Jeta
{
public:
    int m;
    double a;
    void shtypja();
};

int main()
{
    Jeta Dita;
    cout << "\nLeximi i vlerës së variablës m: ";
```

```

        cin >> Dita.m;
        cout << "\nLeximi i vlerës së variablës a: ";
        cin >> Dita.a;
        Dita.shtypja();
    return 0;
}

void Jeta::shtypja()
{
    cout << "\nVlera e variablës private m="
         << m
         << "\n\nVlera e variablës publike a="
         << a
         << "\n\n";
}

```

Meqë variablat **m** dhe **a** përfshihen në komponentet publike të klasës, lejohet operimi me këto dy variabla në programin kryesor, duke e shfrytëzuar emrin e objektit përkatës dhe pikën si operator për qasje, kështu:

```

Dita.m
Dita.a

```

Nëse gjatë ekzekutimit të programit të dhënë më sipër, përmes tastierës kompjuterit i jepen dy vlerat hyrëse **77** dhe **44.56**, në ekran do ta kemi pamjen e dhënë në *Fig.4.3*.

Shfrytëzimi i vlerave të variablave private

Qasja direkte në program të variablave të cilat brenda klasës janë deklaruar si private nuk lejohet. Por, për shfrytëzimin e tyre, brenda klasës duhet të definohen funksione të veçanta, të deklaruara si publike, përmes së cilave mundësohet marrja e vlerave të variablave në fjalë. Gjatë kësaj funksionet mund të jenë *pa parametra formale* ose me *parametra referente*.

Funksionet pa parametra formale

Nëse përmes funksioneve merret vlera vetëm e një variable që përfshihet në komponentet private të klasave, atëherë variabla duhet të shënohet te komanda **return** përkatëse, pa shfrytëzuar parametra formale.

Shembull

Programi **class5b** si version i programit **class5a**, tek i cili brenda klasës **Jeta** është definuar funksioni **marrja** në një komponente publike, përmes së cilit në program, nga komponentja private merret vlera e variablës **m**.


```
// Programi class5b
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;
    void leximi();
    void shtypja();
    int merre();
};

int main()
{
    int k;
    Jeta Dita;
    Dita.leximi();
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;
    k=Dita.merre();
    cout << "\nVlera e variablës private m="
        << k
        << "\n";
    Dita.shtypja();
    return 0;
}

void Jeta::leximi()
{
    cout << "\nLeximi i vlerës së variablës m: ";
    cin >> m;
}

void Jeta::shtypja()
{
    cout << "\nVlera e variablës publike a="
        << a
        << "\n\n";
}
```

```
int Jeta::merre()
{
    return m;
}
```

Siç shihet në fund të pjesës së programit ku janë definuar funksionet, brenda funksionit **merre** është vendosur vetëm komanda:

```
return m;
```

me çka mundësohet marrja e vlerës së variablës **m**, e cila është deklaruar si private. Funksioni në fjalë është marrë i tipit **int**, sepse vlera e variablës **m** që nxirret si rezultat është e këtij tipi. Ky funksion, në program thirret përmes shprehjes:

```
k=Dita.merre();
```

dhe pas kësaj, te variabla **k** përcillet vlera e variablës **m**, e cila me ndërmjetësimin e funksionit **leximi**, përmes tastierës i është dhënë kompjuterit si vlerë hyrëse.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket plotësisht njëloj me atë që u dha në Fig.4.3.

Funksionet me parametra referentë

Vlerat e variablave të cilat brenda klasës janë deklaruar në komponentet private, mund të merren edhe duke shfrytëzuar *parametra referentë*. Kjo ka rëndësi sidomos kur nevojitet që përmes një funksioni prej komponenteve të klasës të merren vlerat e disa variablave njëkohësisht.

Kur pas tipit të parametrin formal të një funksioni shënohet simboli **&**, variabla përkatëse bëhet *parametër referent*. Pastaj, nëse gjatë thirrjes së funksionit, në vend të parametrin formal referent si parametër aktual shënohet një variabël e zakonshme, adresa e parametrin aktual i shoqërohet parametrin formal. Në këtë mënyrë, gjatë ekzekutimit të programit, *parametri formal e shfrytëzon vlerën që është vendosur në adresën e parametrin aktual përkatës*.

Shembull

Programi **class5c** si version i programit **class5b**, tek i cili brenda klasës **Jeta** është definuar funksioni **merre** si publik, përmes së cilit në program, nga komponenta private, duke e shfrytëzuar parametrin referent, merret vlera e variablës **m**.

```
// Programi class5c
#include <iostream>
```

```
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;
    void leximi();
    void shtypja();
    void merre(int& k);
};

int main()
{
    int k;
    Jeta Dita;
    Dita.leximi();
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;
    Dita.merre(k);
    cout << "\nVlera e variablës private m="
         << k
         << "\n";
    Dita.shtypja();
    return 0;
}

void Jeta::leximi()
{
    cout << "\nLeximi i vlerës së variablës m: ";
    cin >> m;
}

void Jeta::shtypja()
{
    cout << "\nVlera e variablës publike a="
         << a
         << "\n\n";
}

void Jeta::merre(int& k)
{
    k=m;
    return;
}
```

Këtu, meqë parametri formal i funksionit **merre** është parametër referent, gjatë thirrjes së tij përmes shprehjes:

```
Dita.merre(k);
```

te parametri formal **k**, përmes adresës përkatëse përcillet vlera e variablës aktuale **k**, e cila mund të mos quhet me të njëjtin emër.

Te prototipi i funksionit, i cili është shënuar brenda definicionit të klasës, nuk është e domosdoshme të shënohet edhe variabla **k**, por prototipi i funksionit mund të shkruhet edhe kështu:

```
void Jeta::merre(int&)
```

Rezultati që do të fitohet, nëse ekzekutohet programi i dhënë për vlera hyrëse të njëjta, do të duket si në *Fig.4.3*.

Llogaritjet me variablat e klasës

Variablat që paraqiten në komponentet e klasës mund të shfrytëzohen gjatë llogaritjeve të ndryshme, si në program ashtu edhe në funksionet që përfshihen brenda klasës.

Llogaritjet në program

Shfrytëzimi i variablave të klasës në program varet nga ajo se a janë deklaruar komponentet përkatëse si publike ose private. Nëse variablat që janë anëtarë të klasës përfshihen në komponente publike, në program mund të shfrytëzohen pa asnjë pengesë, duke e shënuar para tyre emrin e objektit përkatës dhe pikën. Nga ana tjetër, meqë qasja direkte të variablave të klasës, të cilat janë përfshirë në komponente private, nuk është e lejueshme, ato mund të merren vetëm përmes funksioneve brenda klasës, ashtu siç u shpjegua më sipër.

Shembull

Programi **class6a**, si version i programit **class5b**, tek i cili është shtuar edhe llogaritja e vlerës së funksionit:

$$y = (m + 1)! + 3a$$

ku **m** dhe **a** janë variablat në komponentet e klasës **Jeta**.

```
// Programi class6a
#include <iostream>
using namespace std;
```

```
class Jeta
{
```

```
private:
    int m;
public:
    double a;
    void leximi();
    void shtypja();
    int merre();
};

int main()
{
    int i,k;
    double F,y;
    Jeta Dita;
    Dita.leximi();
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;
    Dita.shtypja();

    k=Dita.merre();

    F=1;
    for (i=1;i<=(k+1);i++)
        F=F*i;
    y=F+3*Dita.a;
    cout << "\nVlera e funksionit y="
        << y
        << "\n\n";
    return 0;
}

void Jeta::leximi()
{
    cout << "\nLeximi i vlerës së variablës m: ";
    cin >> m;
}

void Jeta::shtypja()
{
    cout << "\nVlera e variablës private m="
        << m
        << "\n\nVlera e variablës publike a="
        << a
        << "\n";
}

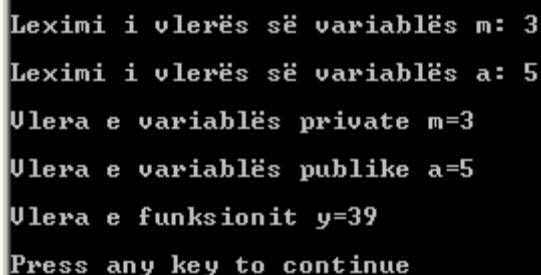
int Jeta::merre()
{
    return m;
}
```

```
}

```

Në pjesën e programit për llogaritjen e vlerës së funksionit **y** janë shfrytëzuar vlerat e variablave që paraqiten në komponentet me të dhëna të klasës **Jeta**. Vlera e variablës **m**, në program është marrë përmes funksionit **merre** dhe ruhet te variabla **k**. Kurse vlera e variablës **a**, meqë brenda klasës është deklaruar si publike, është marrë direkt prej klasës, duke e shënuar si **Dita.a**.

Për vlera hyrëse të caktuara, rezultati në ekran do të duket si në *Fig.4.4*.



```
Leximi i vlerës së variablës m: 3
Leximi i vlerës së variablës a: 5
Vlera e variablës private m=3
Vlera e variablës publike a=5
Vlera e funksionit y=39
Press any key to continue
```

Fig.4.4

Pamja e ekranit pas ekzekutimit të programit class6a

Në program, për ta ruajtur vlerën e variablës **m**, e cila paraqitet në komponenten e klasës, përsëri mund të shfrytëzohet variabla **m**. Tipi i kësaj variable, si edhe për variablën **k**, që u shfrytëzua më sipër, patjetër duhet të deklarohet. Pas kësaj, programi kryesor i programit **class6** do të duket si në vijim.

```
int main()
{
    int i,m;
    double F,y;
    Jeta Dita;
    Dita.leximi();
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;
    m=Dita.merre();
    cout << "\nVlera e variablës private m="
         << m
         << "\n";
    Dita.shtypja();

    F=1;
    for (i=1;i<=(m+1);i++)
        F=F*i;
    y=F+3*Dita.a;
    cout << "Vlera e funksionit y="
         << y

```

```
        << "\n\n";
return 0;
}
```

Vlerat e variablove nga komponentet e klasës mund të merret përmes referencës.

Shembull

Programi **class6b**, si version i programit **class6a**, tek i cili është shtuar edhe llogaritja e vlerës së funksionit:

$$y = (m + 1)! + 3a$$

ku gjatë shoqërimit të vlerave të variablove **m** dhe **a** në komponentet e klasës **Jeta** shfrytëzohet referenca.

```
// Programi class6b
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;
    void leximi();
    void merre(int& k, double& x);
};

int main()
{
    int i, m;
    double a, F, y;
    Jeta Dita;
    Dita.leximi();
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;
    Dita.merre(m, a);
    cout << "\nVlera e variablës publike a="
        << a
        << "\n"
        << "\nVlera e variablës private m="
        << m
        << "\n\n";
    F=1;
    for (i=1; i<=(m+1); i++)
        F=F*i;
}
```

```

        y=F+3*a;
        cout << "\nVlera e funksionit y="
              << y
              << "\n\n";
    return 0;
}
void Jeta::leximi()
{
    cout << "\nLeximi i vlerës së variablës m: ";
    cin >> m;
}
void Jeta::merre(int& k,double& x)
{
    k=m;
    x=a;
}

```

Këtu, përmes funksionit **merre**, prej komponenteve të klasës merren njëkohësisht vlerat e dy variablave **m** dhe **a**. Gjatë kësaj, parametrat **k** dhe **x** që janë shënuar brenda kllapave të funksionit, meqë janë deklaruar si variabla referente, njihen edhe si *parametra referentë*.

Nëse ekzekutohet programi i dhënë, rezultati përsëri do të duket si në *Fig.4.4*.

Lidhur me shfrytëzimin e variablave referente më detajisht do të flitet në kapitullin e veçantë, i cili është dhënë në pjesën vijuese të librit.

Llogaritjet përmes funksioneve të klasës

Përmes funksioneve që definojnë brenda klasave definojnë llogaritje të ndryshme, duke i shfrytëzuar variablat që paraqiten në komponentet e tyre. Gjatë kësaj, rezultatet e llogaritjeve mund të ruhen te variablat e komponenteve të klasës, ose në variabla të zakonshme brenda programit ku deklarohen objektet e klasave në fjalë.

Rezultatet te variablat e programit

Nëse llogaritjet e ndryshme kryhen përmes funksioneve të cilat janë anëtarë të klasës, rezultatet përkatëse mund të përcillen te variablat e programit, duke i thirrur funksionet në program.

Shembull

Programi **class7**, tek i cili brenda klasës **Llogaritja** janë definuar funksionet **vlerat** dhe **funkY**. Përmes funksionit të parë merren dhe shtypen vlerat e variablave **m** dhe **a** të deklaruar si `private`. Kurse, funksioni **funkY** shfrytëzohet për definimin e llogaritjes së vlerës së funksionit:



$$y = (m + 1)! + 3a$$

ku **m** dhe **a** janë variablat në komponentet e klasës.

```
// Programi class7
#include <iostream>
using namespace std;

class Llogaritja
{
private:
    int m;
    double a;
public:
    void vlerat(int k,double x);
    double funkY();
};

int main()
{
    double y;
    Llogaritja Koha;
    Koha.vlerat(2,9);
    y=Koha.funkY();
    cout << "\nVlera e funksionit y="
          << y
          << "\n\n";
    return 0;
}

void Llogaritja::vlerat(int k,double x)
{
    m=k;
    a=x;
    cout << "\nVlera e variablës m="
          << m
          << "\n\nVlera e variablës a="
          << a
          << "\n";
}

double Llogaritja::funkY()
{
    int i;
    double F=1,y;
    for (i=1;i<=(m+1);i++)
        F=F*i;
```

```

        y=F+3*a;
return y;
}

```

Këtu, përmes funksionit **funkY**, i cili përfshihet në klasë, përcaktohet llogaritja e vlerës së funksionit **y**. Gjatë kësaj, brenda tij shfrytëzohen variablat **m** dhe **a**, të cilat përfshihen në komponentet e klasës. Në llogaritje shfrytëzohen variablat ndihmëse **F** dhe **i**, duke i deklaruar si variabla lokale.

Pas deklarimit të objektit **Koha** të klasës **Llogaritja** në fillim të programit, duke e shfrytëzuar funksionin **vlerat**, përmes komandës:

```
Koha.vlerat(2,9);
```

variablave **m** dhe **a** u shoqërohen vlerat **2** dhe **9**, meqë parametrat formalë **k** dhe **x** të këtij funksioni janë zëvendësuar me këta dy numra (si parametra aktualë). Njëkohësisht, përmes komandave që përfshihen brenda funksionit **funkY**, shtypen në ekran vlerat e këtyre dy variablave. Në fund të programit, duke e thirrur funksionin:

```
y=Koha.funkY();
```

llogaritet vlera e funksionit **y** dhe me komandat vijuese ajo vlerë edhe shtypet, për ta fituar rezultatin si në *Fig.4.5*.

Fig.4.5

Pamja e ekranit pas ekzekutimit të programit class7

```

Vlera e variablës m=2
Vlera e variablës a=9
Vlera e funksionit y=33
Press any key to continue

```

Gjatë llogaritjeve përmes funksioneve që përfshihen në klasë, përveç variablave në komponentet e klasës mund të shfrytëzohen edhe variabla jashtë klasës. Në këto raste, variablat e tilla duhet të paraqiten në listën e parametrave formalë të funksioneve për llogaritje.

Shembull

Programi **classJ**, përmes së cilit llogaritet vlera e funksionit

$$z = \begin{cases} \frac{a}{m} + 2b - 1 & \text{për } a < 4 \\ a + \frac{b}{2} + m & \text{për } a \geq 4 \end{cases}$$

ku **m** dhe **a** janë variablat në komponentet e klasës **Jeta**. Për llogaritje të vlerës së variablës **z**, brenda klasës shfrytëzohet



funkcioni **llogaritja**, me parametrin formal **b**, përmes së cilit merret nga jashtë vlera e variablës përkatëse, meqë nuk bën pjesë në variablat e komponenteve të klasës.

```
// Programi classJ
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
    double a;
public:
    void vlerat(int k,double x);
    double llogaritja(double b);
};

int main()
{
    Jeta Dita;
    int k;
    double b,x,z;
    cout << "\nVlerat që lexohen"
          << "\n\nVariabla k: ";
    cin >> k;
    cout << "\nVariabla x: ";
    cin >> x;
    cout << "\nVariabla b: ";
    cin >> b;
    Dita.vlerat(k,x);

    z=Dita.llogaritja(b);

    cout << "\nVlera e llogaritur z="
          << z
          << "\n\n";

    return 0;
}

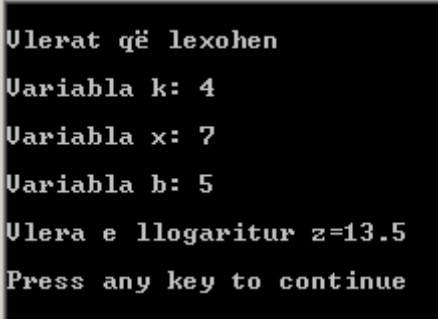
void Jeta::vlerat(int k,double x)
{
    m=k;
    a=x;
}
```

```
double Jeta::llogaritja(double b)
{
    double z;
    if (a<4)
        z=a/m+2*b-1;

    else
        z=a+b/2+m;
    return z;
}
```

Te funksioni **llogaritja**, vlerat e variablave **a** dhe **m** merren direkt nga komponentet përkatëse të klasës. Kurse, variabla **b** është deklaruar si parametër formal i funksionit, meqë vlera e saj merret jashtë klasës.

Nëse ekzekutohet programi i dhënë, për vlera hyrëse të caktuara, të cilat kompjuterit i jepen përmes tastierës, rezultati do të duket si në Fig.4.6.



```
Vlerat që lexohen
Variabla k: 4
Variabla x: 7
Variabla b: 5
Vlera e llogaritur z=13.5
Press any key to continue
```

Fig.4.6

Pamja e ekranit pas ekzekutimit të programit classJ

Rezultatet te variablat e klasës

Rezultati që fitohet përmes llogaritjeve brenda një funksioni, që është anëtar i klasës, mund të ruhet direkt te variablat e klasës, pavarësisht se a janë deklaruar ato si publike ose private. Por, marrja e tyre në programin kryesor do të dallojë varësisht se a janë deklaruar si publike ose private.

Shembull

Programi **class8a**, si version i programit **class7**, tek i cili për ruajtjen e vlerës së funksionit **z**, brenda klasës, shfrytëzohet një variabël e veçantë e deklaruar si publike.

```
// Programi class8a
#include <iostream>
using namespace std;
```

```
class Llogaritja
```

```
{
private:
    int m;
    double a;

public:
    double z;
    void vlerat(int k,double x);
    void funkZ();
};

int main()
{
    Llogaritja Koha;
    Koha.vlerat(2,9);
    Koha.funkZ();
    cout << "\nVlera e funksionit z="
          << Koha.z
          << "\n\n";
    return 0;
}

void Llogaritja::vlerat(int k,double x)
{
    m=k;
    a=x;
    cout << "\nVlera e variablës m="
          << m
          << "\n\nVlera e variablës a="
          << a
          << "\n";
}

void Llogaritja::funkZ()
{
    int i;
    double F=1;
    for (i=1;i<=(m+1);i++)
        F=F*i;
    z=F+3*a;
    return;
}
```

Nëse programi i dhënë krahasohet me versionin **class7**, të dhënë më sipër, do të vërehen ndryshimet vijuese.

- Brenda klasës është deklaruar si publike variabla **z**.

- Te funksioni **funkZ** më nuk deklarohet variabla lokale **z**, në të cilën te versioni **class7** i programit ruhej rezultati i llogaritjes.
- Para funksionit **funkZ** është shkruar fjala **void** në vend të tipit **double**, që ishte shënuar te versioni **class7** i programit, sepse këtu rezultati i llogaritjes përcillet te variabla **z**, që përfshihet brenda klasës. Prandaj, edhe në vazhdim të komandës **return** të këtij funksioni më nuk figuron variabla **z**.
- Për ta llogaritur vlerën e funksionit **z**, në programin kryesor thirret funksioni **funkZ** përmes komandës:

```
Koha.funkZ();
```

pasi paraprakisht te variablat **m** dhe **a**, duke e thirrur funksionin **vlerat** si edhe më parë:

```
Koha.vlerat(2,9);
```

përcillen vlerat **2** dhe **9**.

- Në fund, për shtypjen e vlerës së variablës **z**, e cila ruhet brenda klasës si publike, te komanda për shtypje shfrytëzohet shprehja **Koha.z**.

Nëse ekzekutohet programi **class8a**, rezultati do të duket plotësisht njëjloj si ai që u dha në *Fig.4.5*.

Nëse variabla **z** te programi **class8a**, brenda klasës **Llogaritja** deklarohet si private, për marrjen e rezultatit që ruhet në të duhet të shfrytëzohet një funksion i veçantë, p.sh., **merre**, ashtu siç shihet në vijim.

```
// Programi class8b
#include <iostream>
using namespace std;

class Llogaritja
{
private:
    int m;
    double a;
    double z;
public:
    void vlerat(int k,double x);
    void funkZ();
    double merre();
};
```

```
int main()
{
    Llogaritja Koha;
    Koha.vlerat(2,9);
    Koha.funkZ();
    cout << "\nVlera e funksionit z="
          << Koha.merre()
          << "\n\n";
    return 0;
}
void Llogaritja::vlerat(int k,double x)
{
    m=k;
    a=x;
    cout << "\nVlera e variablës m="
          << m
          << "\n\nVlera e variablës a="
          << a
          << "\n";
}

void Llogaritja::funkZ()
{
    int i;
    double F=1;
    for (i=1;i<=(m+1);i++)
        F=F*i;
    z=F+3*a;
    return;
}

double Llogaritja::merre()
{
    return z;
}
```

Këtu, në fakt, funksioni **merre** është thirrur brenda komandës për shtypje **cout**, duke e shfrytëzuar formën **Koha.merre()**. Nëse ekzekutohet ky version i programit, rezultati përsëri do të duket plotësisht njëjloj si ai që shihet në *Fig.4.5*.

Gjatë deklarimit të pjesës së klasës me komponentet private nuk është e thënë që variablat **a** dhe **z** të deklarohen veç. Por, ato mund të deklarohen edhe kështu:

```
private:
    int m;
    double a,z;
```

Shfrytëzimi i funksioneve brenda klasës

Funksionet e përfshira në komponentet e klasës mund të shfrytëzohen për llogaritje brenda vetë klasës, pavarësisht se a janë përfshirë në komponentet private ose në komponentet publike të saj.

Funksionet në komponentet publike

Shfrytëzimi i funksioneve brenda klasës ose edhe jashtë saj, përkatësisht në programin kryesor, nëse janë përfshirë në komponentet e deklaruara si publike, nuk ka asnjë kufizim.

Shembull

Programi **classF1**, përmes së cilit llogaritet vlera e funksionit:

$$g = 3x + 4 \sum_{i=1}^{n+1} (2i + a)$$

nëse variablat **x**, **n** dhe **a** deklarohen në komponenten private të klasës **Alfa**. Kurse, për llogaritje të shumës shfrytëzohet funksioni **shuma** i përfshirë në komponentet publike të tij. Gjithashtu, brenda strukturës së klasës përfshihen edhe funksionet **Vlerat** dhe **FunkG** - njëri për leximin e vlerave të variablave që janë përfshirë në komponentet private të klasës, kurse tjetri për llogaritjen e vlerës së funksionit **g**.

```
// Programi classF1
#include <iostream>
using namespace std;

class Alfa
{
private:
    int n;
    double a,x;
public:
    void Vlerat();
    double Shuma();
    double FunkG();
};

int main()
{
    Alfa Dita;
    Dita.Vlerat();
}
```



```

        cout << "\nVlera e llogaritur g="
            << Dita.FunkG()
            << "\n\n";
return 0;
}

void Alfa::Vlerat()
{
    cout << "\nVlerat e variablave"
        << "\n\nVariabla n: ";
    cin >> n;
    cout << "\nVariabla a: ";
    cin >> a;
    cout << "\nVariabla x: ";
    cin >> x;
}

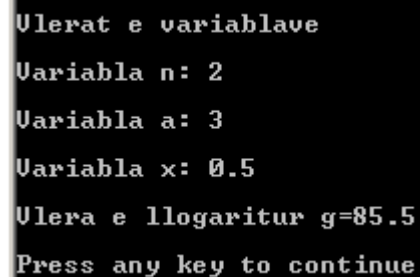
double Alfa::Shuma()
{
    int i;
    double s=0;
    for (i=1;i<=(n+1);i++)
        s=s+(2*i+a);
return s;
}

double Alfa::FunkG()
{
    double g;
    g=3*x+4*Shuma();
return g;
}

```

Këtu, brenda klasës është definuar funksioni **Shuma**, për llogaritjen e vlerës së shumës që paraqitet në shprehjen e funksionit **g**. Pastaj, ai shfrytëzohet te funksioni **FunkG** për llogaritjen e vlerës së funksionit **g**, i cili përfshihet në komponenten e klasës.

Rezultati në ekran, për vlera hyrëse të caktuara, do të duket si ai që është dhënë në *Fig.4.7*.



```

Vlerat e variablave
Variabla n: 2
Variabla a: 3
Variabla x: 0.5
Vlera e llogaritur g=85.5
Press any key to continue

```

Fig.4.7

Pamja e ekranit pas ekzekutimit të programit `classF1`

Brenda klasës **Alfa** të programi i mësipërm mund të përfshihet edhe rezultati, përkatësisht variabla **g**. Në këtë rast, programi me ndryshimet e nevojshme do të duket ashtu siç është dhënë në vijim.

```
// Programi classF2
#include <iostream>
using namespace std;

class Alfa
{
private:
    int n;
    double a,x;
public:
    double g;
    void Vlerat();
    double Shuma();
    void FunkG();
};

int main()
{
    Alfa Dita;
    Dita.Vlerat();
    Dita.FunkG();
    cout << "\nVlera e llogaritur g="
         << Dita.g
         << "\n\n";
    return 0;
}

void Alfa::Vlerat()
{
    cout << "\nVlerat e variablave"
         << "\n\nVariabla n: ";
    cin >> n;
    cout << "\nVariabla a: ";
    cin >> a;
    cout << "\nVariabla x: ";
    cin >> x;
}

double Alfa::Shuma()
{
    int i;
```

```
double s=0;
for (i=1;i<=(n+1);i++)
    s=s+(2*i+a);
return s;
}

void Alfa::FunkG()
{
    g=3*x+4*Shuma();
return;
}
```

Këtu, meqë rezultati **g** ruhet brenda strukturës së klasës, para funksionit përkatës shënohet fjala **void**. Gjithashtu, rezultati në programin kryesor shtypet duke e shfrytëzuar shprehjen **Dita.g**, përmes së cilës vlera **g** merret nga komponentja publike e klasës së objektit **Dita**.

Nëse ekzekutohet ky version i programit, për vlera hyrëse të njëjta, rezultati do të duket si ai që u dha më sipër, në *Fig.4.7*.

Funksionet në komponentet private

Kur funksionet vendosen në komponentet private, shfrytëzimi i tyre brenda klasës nuk ka kufizim. Por, thirrja direkte e tij jashtë klasës, përkatësisht në programin kryesor, nuk është e mundshme.

Shembull

Programi **classF3** përmes së cilit llogaritet vlera e funksionit:

$$z = (2m + 1)! + 3a$$

nëse variablat **m** dhe **a** deklarohen në komponenten private të klasës **Gama**. Kurse, për llogaritje të faktorielit shfrytëzohet funksioni **Fakt**, i cili përfshihet në një komponente private të klasës. Gjithashtu, brenda strukturës së klasës përfshihen edhe funksionet **Vlerat** dhe **FunkZ** - njëri për marrjen e vlerave të variablave që janë përfshirë në komponentet private të klasës, kurse tjetri për llogaritjen e vlerës së funksionit **z**.

```
// Programi classF3
#include <iostream>
using namespace std;

class Gama
{
private:
```

```

        int m;
        double a;
        double Fakt(int n);
public:
    void vlerat(int k,double x);
    double funkZ();
};

int main()
{
    double z;
    Gama Koha;
    Koha.vlerat(2,9);
    z=Koha.funkZ();
    cout << "\nVlera e funksionit z="
          << z
          << "\n\n";
    return 0;
}

void Gama::vlerat(int k,double x)
{
    m=k;
    a=x;
    cout << "\nVlera e variablës m="
          << m
          << "\n\nVlera e variablës a="
          << a
          << "\n";
}

double Gama::Fakt(int n)
{
    int i;
    double F=1;
    for (i=1;i<=n;i++)
        F=F*i;
    return F;
}

double Gama::funkZ()
{
    double z;
    z=Fakt(m+1)+3*a;
    return z;
}

```

Përmes funksionit **Fakt**, këtu është definuar llogaritja e funksionit **F=n!**, duke e marrë variablën **n** si parametër formal. Gjatë llogaritjes së vlerës së

funksionit **z**, meqë funksioni **Fakt** është në njëër nga komponentet e klasës, ai është thirrur direkt, duke e zëvendësuar parametrin formal **n** me atë aktual **m+1**.

Nëse ekzekutohet programi i dhënë, për vlerat aktuale **2** dhe **9** të variablave **m** dhe **a**, të cilat kompjuterit i jepen direkt gjatë thirrjes së funksionit **vlerat**, rezultati do të duket si në Fig.4.8.

```
Vlera e variablës m=2
Vlera e variablës a=9
Vlera e funksionit z=33
Press any key to continue
```

Fig.4.8

Pamja e ekranit pas ekzekutimit të programit `classF3`

Deklarimi i disa objekteve

Duke e shfrytëzuar klasën e definuar, brenda një programi, njëkohësisht mund të deklarohen më shumë objekte. Objektet e deklaruara shfrytëzohen si objekte të veçanta, të pavarura nga njëri a tjetri.

Shembull

Programi `classM`, përmes së cilit llogaritet vlera e funksionit:

$$y = 3a + (2m - 1)!$$

nëse variablat **m** dhe **a** deklarohen në komponenten private të klasës **Alfa**.

```
// Programi classM
#include <iostream>
using namespace std;

class Alfa
{
private:
    int m;
    double a;
public:
    double y;
    void Vlerat(int k, double x);
    void Funk();
};

int main()
{
    cout << "\nObjekti Dita"
          << "\n-----"
```

```

        << "\n";
    Alfa Dita;
    Dita.Vlerat(2,9);
    Dita.Funk();
    cout << "\nVlera e funksionit y="
        << Dita.y;

    cout << "\n\n\nObjekti Nata"
        << "\n-----"
        << "\n";
    Alfa Nata;
    Nata.Vlerat(3,7);
    Nata.Funk();
    cout << "\nVlera e funksionit y="
        << Nata.y
        << "\n\n";
return 0;
}

void Alfa::Vlerat(int k,double x)
{
    m=k;
    a=x;
    cout << "\nVlera e variablës m="
        << m
        << "\n\nVlera e variablës a="
        << a
        << "\n";
}

void Alfa::Funk()
{
    int i;
    double F=1;
    for (i=1;i<=(2*m-1);i++)
        F=F*i;
    y=3*a+F;
}

```

Përmes funksionit **Vlerat**, i cili përfshihet në komponenten publike të klasës, inicializohen variablat **m** dhe **a** të përfshira në komponentet private të klasës **Alfa**. Kurse, për llogaritjen e vlerës së funksionit **y**, i cili përfshihet në komponenten publike të klasës, shfrytëzohet funksioni **Funk**.

Në program janë deklaruar objektet **Dita** dhe **Nata** të klasës **Alfa**. Gjatë kësaj, për vlerat hyrëse, të cilat u janë shoqëruar variablave **m** dhe **a** të objekteve përkatëse përmes funksionit **Vlerat**, janë llogaritur dy vlera të funksionit **y**, duke e shfrytëzuar funksionin **Funk**, ashtu siç shihet në *Fig.4.9*.

Fig.4.9
Pamja e ekranit pas ekzekutimit të programit
classM

```
Objekti Dita
-----
Ulera e variablës m=2
Ulera e variablës a=9
Ulera e funksionit y=33

Objekti Nata
-----
Ulera e variablës m=3
Ulera e variablës a=7
Ulera e funksionit y=141
Press any key to continue
```

Objektet **Dita** dhe **Nata** të klasës **Alfa** mund të deklarohen edhe me një komandë në fillim të programit, kështu:

```
Alfa Dita,Nata;
```

Ngjashëm mund të veprohet edhe nëse njëkohësisht brenda programit deklarohen më shumë objekte.

Konstruktorët

Me qëllim të inicializimit të variablave të cilat përfshihen në komponentet e klasës, brenda saj definoen funksione të veçanta, të cilat njihen si *konstruktorë*.

Këto funksione ekzekutohen automatikisht, kur deklarohen objektet përkatëse të klasave në fjalë. Konstruktorët dallohen nga funksionet e zakonshme brenda klasës, sepse *kanë emra të njëjtë me klasat* dhe para emrave të tyre si dhe te komandat e fundit **return** nuk shënohet asgjë.

Konstruktorët mund të kenë ose mund të mos kenë parametra formalë.

Konstruktorët pa parametra formalë

Nëse përcaktimi i vlerave të variablave në komponentet private bëhet brenda konstruktorëve, ato mund të mos kenë parametra formalë.

Shembull

Programi **class9**, përmes së cilit llogaritet sipërfaqja **s** dhe perimetri **p** i rrethit me rreze **r**. Për inicializimin e konstantes **pi**, brenda klasës **rrethi**, e cila definohet në program shfrytëzohet konstruktori përkatës.

```
// Programi class9
#include <iostream>
using namespace std;

class rrethi
{
private:
    double pi,r,s,p;
public:
    rrethi();
    void rrezja(double x);
    void llogaritja();
    void shtypja();
};

int main()
{
    rrethi Alfa;
    double x;
    cout << "\nRrezja e rrethit x: ";
    cin >> x;
    Alfa.rrezja(x);
    Alfa.llogaritja();
    Alfa.shtypja();
    return 0;
}

rrethi::rrethi()
```



```
{
    pi=3.1415926;
}

void rrethi::rrezja(double x)
{
    r=x;
}

void rrethi::llogaritja()
{
    s=pi*r*r;
    p=2*pi*r;
    return;
}

void rrethi::shtypja()
{
    cout << "\nVlerat e llogaritura"
          << "\n\nSipërfaqja s="
          << s
          << "\n\nPerimetri p="
          << p
          << "\n\n";
}
```

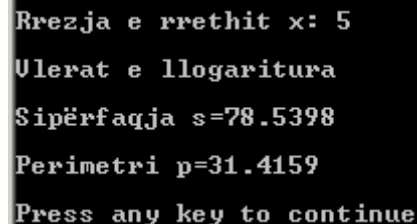
Në program, meqë klasa është quajtur me emrin **rrethi**, edhe konstruktori për inicializimin e konstantes **pi**, e cila brenda klasës është deklaruar si **private**, quhet njëllë. Në këtë mënyrë, në momentin kur në program deklarohet objekti **Alfa** i klasës **rrethi**:

```
rrethi Alfa;
```

automatikisht ekzekutohet funksioni **rrethi** dhe konstantes **pi** i shoqërohet vlera përkatëse. Pastaj, pasi përmes tastierës kompjuterit t'i jepet vlera e variablës **x**, kur thirret funksioni **rrezja**:

```
Alfa.rrezja(x);
```

variablës **r**, e cila e paraqet rrezen e rrethit, i shoqërohet kjo vlerë. Në fund, në program thirren funksionet **llogaritja** dhe **shtypja**, përmes të cilave së pari llogariten sipërfaqja **s** dhe perimetri **p** i rrethit, dhe pastaj vlerat e llogaritura edhe shtypen. Rezultati që fitohet në ekran, për vlerën hyrëse **5** të variablës **x**, do të duket si në *Fig.4.10*.



```
Rrezja e rrethit x: 5
Vlerat e llogaritura
Sipërfaqja s=78.5398
Perimetri p=31.4159
Press any key to continue
```

Fig.4.10

*Pamja e ekranit pas ekzekutimit të programit **class9***

Nuk është e thënë që vlerat e variablave private brenda klasës të jenë vetëm konstante. Ato mund të lexohen si vlera të çfarëdoshme edhe nga jashtë.

Shembull

Programi **class10a**, përmes së cilit llogaritet sipërfaqja **s** dhe perimetri **p** i drejkëndëshit me brinjët **a** dhe **b**. Shoqërimi i vlerave për variablat e brinjëve **a** dhe **b**, brenda klasës **kater** që definohet në program, bëhet përmes konstruktorit përkatës.

```
// Programi class10a
#include <iostream>
using namespace std;

class kater
{
private:
    double a,b,s,p;
public:
    kater();
    void llogaritja();
    void shtypja();
};

int main()
{
    kater Omega;
    Omega.llogaritja();
    Omega.shtypja();
    return 0;
}

kater::kater()
{
    cout << "\nVlerat e lexuara"
          << "\n\nBrinja a: ";
    cin >> a;
    cout << "\nBrinja b: ";
```

```

        cin >> b;
    }

    void kater::llogaritja()
    {
        s=a*b;
        p=2*(a+b);
        return;
    }

    void kater::shtypja()
    {
        cout << "\nVlerat e llogaritura"
              << "\n\nSipërfaqja s="
              << s
              << "\n\nPerimetri p="
              << p
              << "\n\n";
    }

```

Këtu, brenda konstruktorit **kater** janë përfshirë komandat për leximin e vlerave të variablave **a** dhe **b**, të cilat paraqiten në komponentet private të klasës. Pas deklarimit të objektit **Omega**, janë thirrë dy funksionet e përfshira brenda klasës përkatëse:

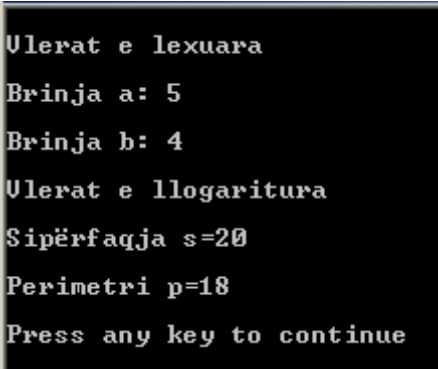
```

Omega.llogaritja();
Omega.shtypja();

```

ashtu që së pari të llogariten sipërfaqja dhe perimetri i drejtkëndëshit dhe pastaj të shtypen ato vlera.

Nëse ekzekutohet programi i dhënë dhe për brinjët e drejtkëndëshit, përmes tastierës kompjuterit i jepen vlerat **5** dhe **4**, rezultati në ekran do të duket si në Fig.4.11.



```

Vlerat e lexuara
Brinja a: 5
Brinja b: 4
Vlerat e llogaritura
Sipërfaqja s=20
Perimetri p=18
Press any key to continue

```

Fig.4.11

Pamja e ekranit pas ekzekutimit të programit class10a

Konstruktorët me parametra formalë

Përmes konstruktorëve, si edhe përmes funksioneve të zakonshme, variablat brenda klasës mund t'u shoqërohen edhe vlera të çfarëdoshme.

Shembull

Versioni **class10b** i programit **class10a**, tek i cili konstruktori **kater** ka edhe dy parametra formalë **x** dhe **y**.

```
// Programi class10b
#include <iostream>
using namespace std;

class kater
{
private:
    double a,b,s,p;
public:
    kater(double x,double y);
    void llogaritja();
    void shtypja();
};

int main()
{
    double x,y;
    cout << "\nVlerat e lexuara"
          << "\n\nBrinja a: ";
    cin >> x;
    cout << "\nBrinja b: ";
    cin >> y;

    kater Omega(x,y);
    Omega.llogaritja();
    Omega.shtypja();
    return 0;
}

kater::kater(double x,double y)
{
    a=x;
    b=y;
}

void kater::llogaritja()
{
    s=a*b;
    p=2*(a+b);
}
```

```
return;
}

void kater::shtypja()
{
    cout << "\nVlerat e llogaritura"
          << "\n\nSipërfaqja s="
          << s
          << "\n\nPerimetri p="
          << p
          << "\n\n";
}
```

Këtu, parametrat formalë të konstruktorit **kater** shfrytëzohen për përcjelljen brenda tij të vlerave **x** dhe **y**, të cilat lexohen në programin kryesor. Pastaj, këto vlera u shoqërohen variablave **a** dhe **b** të klasës, përmes shprehjeve:

```
a=x;
b=y;
```

të cilat përfshihen te konstruktori. Meqë në momentin e deklarimit të objektit të klasës **kater** automatikisht thirret edhe konstruktori, deklarimi i objektit duhet të shoqërohet edhe me vlerat aktuale të këtyre dy parametrave. Për këtë arsye, deklarimi i objektit **Omega** i klasës **kater** është bërë në formën:

```
kater Omega(x,y);
```

ku variablat **x** dhe **y** paraqesin parametra aktualë, vlerat e të cilëve janë lexuar paraprakisht.

Rezultati që fitohet në ekran, për vlerat hyrëse **5** dhe **4** të brinjëve **a** dhe **b**, do të duket si në *Fig.4.11*.

Në literaturë takohet edhe një formë tjetër e definimit të konstruktorëve. Kjo formë, p.sh., për konstruktorin i cili u shfrytëzua te programi i mësipërm mund të duket:

```
kater::kater(double x,double y):a(x),b(y)
{
}
```

Edhe pas definimit të konstruktorit në këtë mënyrë, deklarimi i objektit **Omega** të klasës **kater** në program nuk do të ndryshojë.

Gjatë përcaktimit të vlerave duke e shfrytëzuar këtë formë të shkruarjes së konstruktorëve, brenda kllapave të variablave mund të shkruhen vlera numerike konkrete.

Shembull

Programi **class11**, përmes së cilit llogaritet vlera e funksionit:

$$y = ax + (m + 1)!$$

nëse variablat **m** dhe **x** përfshihen në komponentet private të klasës **Alfa**, kurse variabla **a** merret si variabël e jashtme.

```
// Programi class11
#include <iostream>
using namespace std;

class Alfa
{
private:
    int m;
    double x;
public:
    Alfa();
    double Funk(double a);
};

int main()
{
    double y;
    Alfa Dita;
    y=Dita.Funk(6.5);
    cout << "\nVlera e llogaritur y="
         << y
         << "\n\n";

    return 0;
}

Alfa::Alfa():m(3),x(5)
{
    cout << "\nVlera e variablës m: "
         << m
         << "\n\nVlera e variablës x: "
         << x
         << "\n";
}
```

```

}

double Alfa::Funk(double a)
{
    double F=1,y;
    int i;
    for (i=1;i<=(m+1);i++)
        F=F*i;
    y=a*x+F;
    return y;
}

```

Në këtë rast, përmes konstruktorit janë inicializuar direkt variablat **m** dhe **x** me vlerat **3** dhe **5** - përkatësisht, duke i shënuar:

```
Alfa::Alfa(): m(3), x(5)
```

kurse gjatë thirrjes së funksionit **Funk**:

```
y=Dita.Funk(6.5);
```

parametri formal **a** është zëvendësuar me vlerën aktuale **6.5**.

Rezultati në ekran pas ekzekutimit të programit të dhënë do të duket si në *Fig.4.12*.

```

Vlera e variablës m: 3
Vlera e variablës x: 5
Vlera e llogaritur y=56.5
Press any key to continue

```

Fig.4.12

Pamja e ekranit pas ekzekutimit të programit
class11

Llogaritjet brenda konstruktorëve

Përveç inicializimit të variablave me vlera, brenda konstruktorëve mund të kryhen edhe llogaritje të ndryshme. Por, rezultatet e llogaritjeve duhet të përcillen te variablat e përfshira në komponentet e klasave.

Shembull

Programi **class12a**, përmes së cilit llogariten vlerat e funksionit:

$$y = 3x + k \sum_{i=1}^{n+1} (2i + a)$$

Variablat **x**, **k**, **a** dhe **n**, si dhe rezultati i llogaritjes **y**, ruhen në klasën **Omega**. Kurse, për inicializimin e variablave dhe për llogaritje shfrytëzohet konstruktori përkatës.

```

// Programi class12a
#include <iostream>
using namespace std;

class Omega
{
private:
    int n,k;
    double x,a;
public:
    double y;
    Omega();
};

int main()
{
    Omega Delta;
    cout << "\nVlera e llogaritur y="
          << Delta.y
          << "\n\n";
    return 0;
}

Omega::Omega()
{
    cout << "\nVlerat inicializuese"
          << "\n\nVariabla n: ";
    cin >> n;
    cout << "\nVariabla x: ";
    cin >> x;
    cout << "\nVariabla a: ";
    cin >> a;
    cout << "\nVariabla k: ";
    cin >> k;

    int i;
    double s=0;
    for (i=1;i<=(n+1);i++)
        s=s+(2*i+a);
    y=3*x+k*s;
}

```

Këtu, brenda konstruktorit **Omega** fillimisht është paraparë inicializimi me vlera i variablave që përfshihen në komponentet e klasës përmes leximit të tyre. Pastaj, është vazhduar me llogaritjen e shumës **s** dhe të vlerës së funksionit **y**. Rezultati që fitohet në ekran, për vlera hyrëse të zgjedhura lirisht, do të duket si në *Fig.4.13*.


```

Ulerat inicializuese
Variabla n: 3
Variabla x: 2.3
Variabla a: 1.7
Variabla k: 3
Vlera e llogaritur y=87.3
Press any key to continue

```

Fig.4.13

Pamja e ekranit pas ekzekutimit të programit **class12a**

Rezultati **y**, qëllimisht është ruajtur në komponenten publike të klasës, ashtu që gjatë shtypjes të merret direkt, duke e shfrytëzuar shprehjen **Delta.y**.

Nëse gjatë ekzekutimit të programit vlerat e variablave të caktuara brenda klasës nuk ndryshojnë, ato mund të inicializohen direkt gjatë deklarimit të objektit përkatës.

Shembull

Versioni **class12b** i programit **class12a**, tek i cili për inicializimin e variablave **k** dhe **a** janë shfrytëzuar parametrat formalë **p** dhe **q** të konstruktorit **Omega**.

```

// Programi class12b
#include <iostream>
using namespace std;

class Omega
{
private:
    int n,k;
    double x,a;
public:
    double y;
    Omega(int p,double q);
};

int main()
{
    Omega Delta(3,1.7);

    cout << "\n\nVlera e llogaritur y="
         << Delta.y
         << "\n\n";
    return 0;
}

```

```

Omega::Omega(int p,double q)
{
    n=p;
    a=q;
    cout << "\nVlerat inicializuese"
          << "\n\nVariabla x: ";
    cin >> x;
    cout << "\nVariabla k: ";
    cin >> k;
    cout << "\nVariabla a: "
          << a
          << "\n\nVariabla n: "
          << n;

    int i;
    double s=0;
    for (i=1;i<=(n+1);i++)
        s=s+(2*i+a);
    y=3*x+k*s;
}

```

Këtu, gjatë deklarimit të objektit **Omega** të klasës **Delta**:

```
Omega Delta(3,1.7);
```

janë inicializuar variablat **n** dhe **a**, duke i shënuar parametrat aktualë **3** dhe **1.7**, në vend të parametrave formalë **p** dhe **q**.

Nëse ekzekutohet ky version i programit, për vlera hyrëse të njëjta, rezultati në ekran nuk do të ndryshojë nga ai që u dha në *Fig.4.13*.

Disa konstruktorë njëkohësisht

Brenda një klase mund të përfshihen edhe disa konstruktorë njëkohësisht. Kjo lidhet me krijimin e disa objekteve të klasës në fjalë, ashtu që objektet e ndryshme të mund ta shfrytëzojnë njërin nga konstruktorët e definuar. *Thirrja automatike* e konstruktorit të caktuar lidhet me deklarin e objektit përkatës dhe *varet nga numri i parametrave aktualë ose nga tipi i tyre*. Në këtë rast, meqë njëkohësisht definohen disa konstruktorë, flitet për *konstruktorë të mbingarkuar* (ang. overloading).

Thirrja në bazë të numrit të parametrave

Nëse konstruktorët e përfshirë në definicionin e klasës dallohen mes vete për nga numri i parametrave, përfshirë edhe konstruktorin pa parametra, kompjuteri do ta thërrasë automatikisht konstruktorin i cili ka aq parametra

formalë sa janë shënuar brenda kllapave gjatë deklarimit të objektit përkatës në program.

Shembull

Programi **class13**, përmes së cilit llogariten vlerat e funksioneve:

$$y = 5x + 3!$$

$$z = \frac{x}{2} + (m + 1)!$$

duke shfrytëzuar dy konstruktorë për përcaktimin e vlerave që lidhen me faktorialët të cilët duhet të llogariten.

```
// Programi class13
#include <iostream>
using namespace std;

class faktoriel
{
private:
    int m;
    double x;
public:
    double g;
    faktoriel();
    faktoriel(int k);
};

int main()
{
    faktoriel Dita;
    cout << "\nVlera e llogaritur y="
          << Dita.g;
    faktoriel Nata(3);
    cout << "\nVlera e llogaritur z="
          << Nata.g
          << "\n\n";
    return 0;
}

faktoriel::faktoriel()
{
    m=3;
    cout << "\nKonstruktori i parë"
          << "\n-----"
          << "\n\nVariabla x: ";
}
```

```

        cin >> x;
        double F=1;
        int i;
        for (i=1;i<=m;i++)
            F=F*i;
        g=5*x+F;
    }

faktoriel::faktoriel(int k)
{
    m=k;
    cout << "\n\nKonstruktori i dytë"
          << "\n-----"
          << "\nVariabla x: ";
    cin >> x;
    double F=1;
    int i;
    for (i=1;i<=(m+1);i++)
        F=F*i;
    g=x/2+F;
}

```

Brenda klasës **faktoriel** këtu shfrytëzohen dy konstruktorë me emrin **faktoriel**, të cilët janë vendosur në komponentet publike dhe shkruhen:

```

faktoriel();
faktoriel(int k);

```

Përmes parametrin **k**, në konstruktorin e dytë përcaktohet vlera e variablës **m**, sepse brenda tij paraqitet shprehja **m=k**. Kurse, në konstruktorin e parë është marrë **m=3**, pa pasur mundësi që vlera e variablës **m** të merret edhe ndryshe.

Përmes konstruktorëve të dhënë, përveç që përcaktohet vlera e variablës **m**, llogaritet edhe njëra prej vlerave të funksioneve **y** dhe **z**, e cila ruhet brenda klasës te variabla **g**. Se cila vlerë do të llogaritet, varet nga prezenca ose mosprezenca e parametrin **k** gjatë deklarimit të objektit përkatës të klasës.

Në program, duke e shfrytëzuar klasën **faktoriel**, janë deklaruar objektet **Dita** dhe **Nata**. Meqë gjatë deklarimit të objektit të parë:

```

faktoriel Dita;

```

nuk shoqërohet asnjë vlerë, automatikisht do të aktivizohet konstruktori i parë, përmes së cilit llogaritet vlera e funksionit **y**. Kurse, gjatë deklarimit të objektit të dytë:

```

faktoriel Nata(3);

```

meqë brenda kllapave është shënuar vlera **3**, e cila i përket parametrin formal **k**, automatikisht aktivizohet konstruktori i dytë dhe do të llogaritet vlera e funksionit **z**.

Nëse ekzekutohet programi i dhënë, rezultati do të duket si në Fig.4.14.

Fig.4.14

Pamja e ekranit pas ekzekutimit të programit `class13`

```
Konstruktori i parë
-----
Variabla x: 5
Vlera e llogaritur y=31

Konstruktori i dytë
-----
Variabla x: 5
Vlera e llogaritur z=26.5
Press any key to continue
```

Thirrja në bazë të tipit të parametrave

Nëse konstruktorët e përfshirë në definicionin e klasës dallohen mes vete për nga tipi i parametrave, kompjuteri do ta thërrasë automatikisht konstruktorin me tip të njëjtë të parametrave formalë me parametrat aktualë që janë shënuar brenda kllapave gjatë deklarimit të objektit përkatës në program.

Shembull

Programi **class14**, përmes së cilit llogariten vlerat e funksioneve:

$$y = 3 \sum_{i=1}^n \left[\frac{i}{2} + 1 \right]$$

$$z = 2 \sum_{i=1}^{n+1} [3i + 4]$$

duke shfrytëzuar dy konstruktorë **Shuma** për përcaktimin e llogaritjes së vlerave të shumave të veçanta.

```
// Programi class14
#include <iostream>
using namespace std;
```

```
class Shuma
{
private:
    int n;
public:
```

```

        double s;
        Shuma(double a);
        Shuma(int a);
};

int main()
{
    char v[]="-----";
    double y;
    cout << "\nObjekti Alfa\n"
          << v;
    Shuma Alfa(0.5);
    y=3*Alfa.s;
    cout << "\nVlera e llogaritur y="
          << y;

    double z;
    cout << "\n\n\nObjekti Beta\n"
          << v;

    Shuma Beta(3);
    z=2*Beta.s;
    cout << "\nVlera e llogaritur z="
          << z
          << "\n\n";
    return 0;
}

Shuma::Shuma(double a)
{
    cout << "\n\nVariabla n: ";
    cin >> n;
    int i;
    s=0;
    for (i=1;i<=n;i++)
        s=s+(a*i+1);
}

Shuma::Shuma(int a)
{
    cout << "\n\nVariabla n: ";
    cin >> n;
    int i;
    s=0;
    for (i=1;i<=(n+1);i++)
        s=s+(a*i+4);
}

```

Këtu, konstruktorët **Shuma** shfrytëzohen për inicializimin e vlerave të variablave **n** përmes leximit. Gjithashtu, llogariten vlerat e shumave, duke e shënuar, në vend të konstantes para variablës **i**, parametrin **a**, kështu:

```
a · i + 1
a · i + 4
```

Meqë, për llogaritjen e vlerës së funksionit **y**, gjatë deklarimit të objektit përkatës:

```
Shuma Alfa(0.5);
```

parametri formal **a** është zëvendësuar me numrin dhjetor **0.5**, i cili i përgjigjet thyesës **1/2** para variablës **i**, kompjuteri do ta aktivizojë konstruktorin e parë **Shuma**, tek i cili ky parametër është deklaruar i tipit **double**. Kurse, në rastin e llogaritjes së vlerës së funksionit **z**, kompjuteri do ta shfrytëzojë konstruktorin e dytë **Shuma**, sepse gjatë deklarimit të objektit përkatës:

```
Shuma Beta(3);
```

parametri formal **a** i tipit **int**, te ky konstruktor zëvendësohet me numrin e plotë **3**.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në *Fig.4.15*.

```
Objekti Alfa
-----
Variabla n: 2
Vlera e llogaritur y=10.5

Objekti Beta
-----
Variabla n: 2
Vlera e llogaritur z=60
Press any key to continue
```

Fig.4.15

Pamja e ekranit pas ekzekutimit të programit class14

Destruktorët

Me qëllim të lirimit të hapësirës memoruese, të cilën e shfrytëzon një objekt, pasi objekti më nuk nevojitet, brenda klasës definohet një funksion i veçantë, i cili njihet si *destruktor*. Si konstruktori, edhe emri i destruktorigj është i

njëjtë me emrin e klasës, por para tij vendoset simboli `~`. Brenda kllapave të trupit të destruktorit vendoset komanda **delete**, në vazhdim të së cilës shënohen variablat e klasës të cilat duhet të fshihen, të ndara mes vete me presje. Kështu, p.sh., destruktori që vendoset brenda klasës **Alfa** mund ta ketë formën:

```
~Alfa()
{
    cout << "Aktivizimi i destruktorit";
    delete m,a;
}
```

Destruktori mund të shfrytëzohet vetëm për fshirje të objekteve të deklaruara si pointer.

Kur përfundon ekzekutimi i programit, nëse nuk shfrytëzohen destruktore, kompjuteri automatikisht i fshin objektet e shfrytëzuara brenda tij. Prandaj, edhe shfrytëzimi i destruktoreve të programet e zakonshme nuk ka ndonjë rëndësi të veçantë.

Disa klasa njëkohësisht

Një program mund të përmbajë edhe disa klasa njëkohësisht. Për definimin dhe shfrytëzimin e tyre vlejnë rregullat sikur edhe për një klasë të vetme.

Shembull

Programi **classK2**, përmes së cilit llogariten sipërfaqja **sd** dhe perimetri **pd** i drejkëndëshit me brinjët **a** dhe **b**, si dhe sipërfaqja **sr** dhe perimetri **pr** i rrethit me rreze **r**. Gjatë llogaritjeve shfrytëzohen dy klasa të veçanta **drejtk** dhe **rrethi**, në komponentet private të së cilave ruhen brinjët e drejkëndëshit dhe konstantja **pi** me rrezen **r** të rrethit. Kurse, funksionet për marrjen e vlerave të nevojshme (**vleratD** dhe **vleratR**), si dhe funksionet për llogaritje (**llogaritD** dhe **llogaritR**) janë definuar si anëtarë publikë të klasave.

```
// Programi classK2
#include <iostream>
using namespace std;

class drejtk
{
private:
    double a,b;
public:
    void vleratD(double x,double y);
    void llogaritD();
```



```
};

class rrethi
{
private:
    double pi,r;
public:
    void vleratR(double x,double y);
    void llogaritR();
};

int main()
{
    drejtk D;
    rrethi R;
    D.vleratD(7,4);
    D.llogaritD();
    R.vleratR(3.1415926,5);
    R.llogaritR();

    return 0;
}

void drejtk::vleratD(double x,double y)
{
    a=x;
    b=y;
    cout << "\nBrinjët e drejtkëndëshit"
        << "\n\nBrinja a="
        << a
        << "\nBrinja b="
        << b
        << "\n";
}

void drejtk::llogaritD()
{
    double sd,pd;
    sd=a*b;
    pd=2*(a+b);
    cout << "\nSipërfaqja e drejtkëndëshit sd="
        << sd
        << "\nPerimetri i drejtkëndëshit pd="
        << pd
        << "\n";
}

void rrethi::vleratR(double x,double y)
{

```

```

        pi=x;
        r=y;
        cout << "\nRrezja e rrethit r="
              << r
              << "\n";
    }

    void rrethi::llogaritR()
    {
        double sr,pr;
        sr=pi*r*r;
        pr=2*pi*r;
        cout << "\nSipërfaqja e rrethit sr="
              << sr
              << "\nPerimetri i rrethit pr="
              << pr
              << "\n\n";
    }

```

Te funksionet e veçanta të përfshira në klasa janë shfrytëzuar edhe komanda për shtypje të vlerave hyrëse si dhe vlerat e rezultateve që fitohen gjatë llogaritjeve. Në këtë mënyrë, programi kryesor është thjeshtuar krejtësisht. Në të fillimisht deklarohen objektet **D** dhe **R**:

```

    drejtk D;
    rrethi R;

```

të klasave **drejtk** dhe **rrethi**. Pastaj janë thirrur funksionet:

```

    D.vleratD(7,4);
    D.llogaritD();
    R.vleratR(3.1415926,5);
    R.llogaritR();

```

për marrje të vlerave, llogaritje dhe shtypje të rezultateve.

Rezultati në ekran do të duket si në *Fig.4.16*.

Fig.4.16

```

Brinjët e drejtkëndëshit
Brinja a=7
Brinja b=4

Sipërfaqja e drejtkëndëshit sd=28
Perimetri i drejtkëndëshit pd=22

Rrezja e rrethit r=5

Sipërfaqja e rrethit sr=78.5398
Perimetri i rrethit pr=31.4159

Press any key to continue

```

Pamja e ekranit pas ekzekutimit të programit `classK2`

Ngjashëm si në shembullin e mësipërm, në një program mund të definohen dhe të shfrytëzohen edhe më shumë klasa njëkohësisht.

Trashëgimia

Përmes mekanizmit të *trashëgimisë* (ang. inheritance), anëtarët e klasës ekzistuese mund të shfrytëzohen gjatë definimit të klasave të reja. Kjo ka një rëndësi të veçantë, sepse mundësohet plotësimi me komponente i një klase ekzistuese, gjë që mund të shfrytëzohet gjatë përmirësimit të programeve që janë në shfrytëzim, ose të përfshira në biblioteka të ndryshme programesh të gatshme.

Klasa ekzistuese njihet si *klasë bazë* (ang. base class), kurse *klasa e re* i trashëgon karakteristikat e klasës bazë dhe quhet *klasë e nxjerrë* (ang. derived class).

Klasa e re krijohet duke e plotësuar klasën ekzistuese me anëtarë të tjerë, qofshin ato anëtarë të dhënash ose funksione. Gjatë kësaj, klasa bazë nuk pëson asnjë ndryshim. Njëkohësisht, klasa e nxjerrë mund të shfrytëzohet si klasë bazë për nxjerrjen e klasave të tjera. Një mundësi e tillë e krijimit të klasave të reja, si prej klasave bazë ashtu edhe prej klasave të nxjerra, përmes mekanizmit të trashëgimisë, paraqet një *organizim hierarkik të klasave*, gjë që programimin e orientuar në objekte e bën edhe më të fuqishëm. Organizimi i tillë hierarkik, p.sh., mund të duket si ai që është dhënë në Fig.4.17, ku prej klasës bazë **Person** janë nxjerrë klasat **Profesori** dhe **Studenti**. Pastaj, prej klasës **Studenti** janë nxjerrë klasat **Provimet** dhe **Pagesat**.

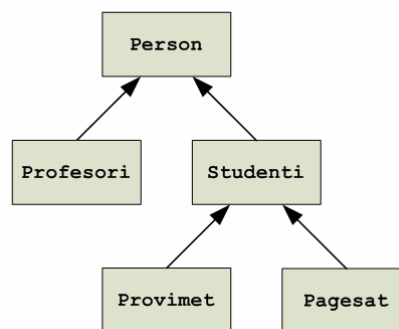


Fig.4.17

Dukja e një organizimi hierarkik të klasave

Këtu, nënkuptohet se hierarkia mund të vazhdohet më tutje edhe pas klasës së nxjerrë **Profesori**. Gjatë kësaj, shigjetat janë të drejtuara prej klasave të nxjerra kah klasa bazë dhe përmes tyre tregohet se prej cilës klasë janë nxjerrë klasat e veçanta.

Nëse analizohet hierarkia e dhënë, do të shihet se klasat e nxjerra, në fakt, e plotësojnë klasën bazë me të dhëna dhe funksione të tjera. Kështu, p.sh., nëse

te klasa **Person** përfshihen të dhënat bazike, siç janë emri, vendi, datëlindja etj., përmes klasës së nxjerrë **Studenti** të dhënat bazë plotësohen, p.sh., me të dhënat për fakultetin, drejtimin e zgjedhur etj. Më tutje, të gjitha këtyre të dhënave mund t'u shtohen edhe ato për provimet e studentëve (klasa e nxjerrë **Provimet**) dhe për pagesat e tyre (klasa e nxjerrë **Pagesat**).

Meqë shembulli i programit ku do të paraqitet hierarkia e dhënë më sipër, ose edhe pjesë të saj, kërkon më shumë hapësirë, në vijim do të jepen disa shembuj më elementarë.

Shembull

Programi **classT1a**, në të cilin definohet klasa **Alfa** si klasë bazë dhe klasa **Beta** - si klasë e nxjerrë prej saj.

```
// Programi classT1a
#include <iostream>
using namespace std;

class Alfa
{
private:
    int a;
public:
    double b;
    void Jeta()
    {
        a=5;
        b=8;
        cout << "\nVlerat te klasa Alfa"
              << "\n\n    a="
              << a
              << "\n    b="
              << b
              << "\n\n";
    }
};

class Beta:public Alfa
{
private:
    double c;
public:
    int d;
    void Dita()
    {
```

```
        b=2;
        c=4;
        d=7;
        cout << "Vlerat te klasa Beta"
              << "\n\n    b="
              << b
              << "\n    c="
              << c
              << "\n    d="
              << d
              << "\n";
    }
};

int main()
{
    Alfa X;
    Beta Y;
    X.Jeta();
    Y.Dita();
    X.Jeta();
    return 0;
}
```

Në program, fillimisht është definuar klasa **Alfa**. Te funksioni **Jeta** i cili është deklaruar si publik, variabla **a** dhe **b** të klasës u janë shoqëruar vlerat përkatëse **5** dhe **8**. Njëkohësisht, përmes komandës **cout** është urdhëruar shtypja e këtyre vlerave. Pastaj është definuar klasa **Beta**, në të cilën, përveç komponenteve të saj, janë deklaruar si publike edhe komponentet publike të klasës **Alfa**, gjë që arrihet përmes deklarimit **public Alfa**, kështu:

```
class Beta:public Alfa
```

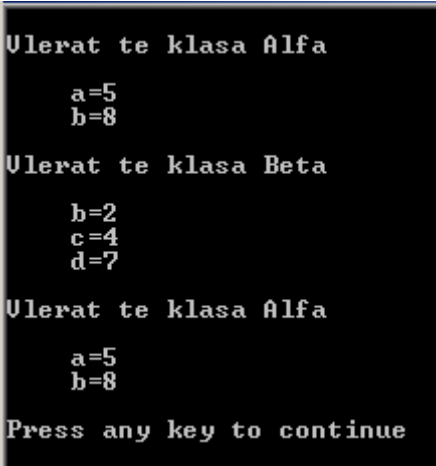
Klasa **Beta** llogaritet si klasë e nxjerrë nga klasa **Alfa**, meqë brenda saj shfrytëzohen edhe komponentet publike të klasës **Alfa**.

- Brenda klasës **Alfa** mund të shfrytëzohen vetëm variablat **a** dhe **b**, si dhe funksioni **Jeta**. Por, nuk mund të shfrytëzohen variablat **c** e **d**, si dhe funksioni **Dita**, të cilët paraqiten në komponentet e klasës së nxjerrë **Beta**.
- Meqë komponentet publike të klasës **Alfa** janë deklaruar njëkohësisht si publike edhe te klasa **Beta**, te kjo klasë mund të shfrytëzohet edhe variabla **b**, si dhe funksioni **Jeta**.
- Brenda klasës **Beta** nuk mund të shfrytëzohet variabla **a**, e cila është anëtar i klasës **Alfa**, por e deklaruar si **private**.

Në program, fillimisht janë deklaruar objektet **X** dhe **Y** të klasave **Alfa** dhe **Beta**. Pastaj, përmes komandave:

```
X.Jeta();
Y.Dita();
X.Jeta();
```

dy herë është thirrë funksioni **Jeta** dhe një herë funksioni **Dita**, të cilët përfshihen në komponentet e klasave përkatëse. Rezultati i cili fitohet në ekran do të duket si ai që është dhënë në *Fig.4.18*.



```
Vlerat te klasa Alfa
a=5
b=8
Vlerat te klasa Beta
b=2
c=4
d=7
Vlerat te klasa Alfa
a=5
b=8
Press any key to continue
```

Fig.4.18

Pamja e ekranit pas ekzekutimit të programit `classT1a`

Nga kjo shihet se, si rezultat i thirrjes dy herë të funksionit **Jeta**, vlerat që përcaktohen brenda klasës **Alfa** janë shtypur dy herë.

Në shembullin e dhënë, deklarimi i objektit **X** në programin kryesor është i panevojshëm, sepse pjesa publike e klasës **Alfa** përfshihet në klasën **Beta**. Kështu, programi kryesor mund të shkruhet edhe si në vijim.

```
int main()
{
    Beta Y;
    Y.Jeta();
    Y.Dita();
}
```

```
        Y.Jeta();  
return 0;  
}
```

Definimi i funksioneve jashtë klasave

Me qëllim të rritjes së dukshmërisë së klasave, funksionet që shfrytëzohen brenda tyre edhe në këtë rast mund të definohen jashtë strukturave të tyre, duke i shfrytëzuar format e zakonshme.

Shembull

Versioni **classT1b** i programit **classT1a**, në të cilin definimi i funksioneve bëhet jashtë strukturave të klasave.

```
// Programi classT1b  
#include <iostream>  
using namespace std;  
  
class Alfa  
{  
private:  
    int a;  
public:  
    double b;  
    void Jeta();  
};  
  
class Beta:public Alfa  
{  
private:  
    double c;  
public:  
    int d;  
    void Dita();  
};  
  
int main()  
{  
    Beta Y;  
    Y.Jeta();  
    Y.Dita();  
    Y.Jeta();  
return 0;  
}
```

```

}

void Alfa::Jeta()
{
    a=5;
    b=8;
    cout << "\nVlerat te klasa Alfa"
          << "\n\n      a="
          << a
          << "\n      b="
          << b
          << "\n\n";
}

void Beta::Dita()
{
    b=2;
    c=4;
    d=7;
    cout << "Vlerat te klasa Beta"
          << "\n\n      b="
          << b
          << "\n      c="
          << c
          << "\n      d="
          << d
          << "\n";
}

```

Nëse ekzekutohet programi i dhënë, rezultati do të duket si në *Fig.4.18*.

Shfrytëzimi i anëtarëve të mbrojtur

Anëtarët e klasës bazë do të jenë edhe *të mbrojtur*, duke e shfrytëzuar specifikuesin e qasjes **protected**. Këta anëtarë mund të shfrytëzohen plotësisht njëjloj si edhe anëtarët të cilët përfshihen nën specifikuesin **private**. Por, për dallim nga ata nën specifikuesin **private**, komponentet nën specifikuesin **protected** mund të *shfrytëzohen edhe te klasat e nxjerra* (ato nën specifikuesin **private** nuk mund të shfrytëzohen te klasat e nxjerra).

Anëtarëve të klasës bazë me specifikues të qasjes së mbrojtur nuk mund t'u qasemi nga jashtë sikur edhe atyre me specifikues të qasjes private. Por, anëtarët e klasës, të cilët janë deklaruar si të mbrojtur, mund të shfrytëzohen edhe brenda klasave të nxjerra, gjë që nuk është e mundur për anëtarët me specifikues të qasjes private.

Shembull

Versioni **classT1c** i programit **classT1b**, në të cilin variabla **b** është përfshirë nën specifikuesin e qasjes së mbrojtur.


```
// Programi classT1c
#include <iostream>
using namespace std;

class Alfa
{
private:
    int a;
protected:
    double b;
public:
    void Jeta();
};

    class Beta:public Alfa
    {
    private:
        double c;
    public:
        int d;
        void Dita();
    };

int main()
{
    Beta Y;
    Y.Jeta();
    Y.Dita();
    Y.Jeta();
return 0;
}
```

Pjesa tjetër e programit, që nuk është dhënë, në të cilën përcaktohen funksionet **Jeta** dhe **Dita**, mbetet e njëjtë me atë që shihet te programi **classT1b**. Kjo do të thotë se variabla **b** në komponenten e mbrojtur të klasës **Alfa** shfrytëzohet edhe te klasa e nxjerrë **Beta**.

Nëse ekzekutohet programi i dhënë, rezultati do të jetë i njëjtë me atë që është dhënë në *Fig.4.18*.

Shfrytëzimi i anëtarëve gjatë trashëgimisë

Anëtarët e klasës bazë bëhen edhe anëtarë të klasës së nxjerrë. Te klasa e nxjerrë mund t'u qasemi direkt anëtarëve publikë dhe anëtarëve të mbrojtur të klasës bazë. Por, nuk lejohet qasja direkte tek anëtarët e mbrojtur të klasës bazë,

prej klasës së nxjerrë. Qasja direkte te anëtarët privat të klasës bazë është e mundshme vetëm brenda klasës bazë.

Anëtarët publikë të klasës bazë trashëgohen te klasa e nxjerrë si anëtarë publikë, kurse anëtarët privatë ose të mbrojtur, varësisht nga specifikuesi i qasjes që zgjedhet gjatë definimit të klasës së nxjerrë, ashtu siç është dhënë në vijim.

1.

```
class Alfa
{
    ...
};
class Beta:public Alfa
{
    ...
};
```

Këtu, anëtarët publikë të klasës **Alfa** trashëgohen te klasa e nxjerrë **Beta** si anëtar publik.

2.

```
class Alfa
{
    ...
};
class Beta:private Alfa
{
    ...
};
```

Në këtë rast, anëtarët publikë të klasës **Alfa** te klasa e nxjerrë **Beta** trashëgohen si anëtar privat.

3.

```
class Alfa
{
    ...
};
class Beta:Alfa
{
    ...
};
```

Meqë këtu nuk është shënuar specifikuesi i qasjes, kompjuteri e nënkupton atë si **private**. Prandaj, në këtë rast, si edhe në rastin paraprak, anëtarët publikë të klasës **Alfa** trashëgohen si anëtarë privatë te klasa e nxjerrë **Beta**.

4.

```

class Alfa
{
    ...
};
class Beta:protected Alfa
{
    ...
};

```

Në këtë rast, pasi specifikuesi i qasjes për anëtarët e trashëguar është **protected**, anëtarët publikë të klasës **Alfa** do të jenë anëtarë të mbrojtur të klasa e nxjerrë **Beta**.

Pas definimit të klasës së nxjerrë, vlejnë rregullat e qasjes për klasat e zakonshme, si tek anëtarët e saj, ashtu edhe tek anëtarët e trashëguar nga klasa bazë, gjë që në formë përmbledhëse shihet te tabela e cila është dhënë në Fig.4.19.

Specifikuesi i qasjes	Qasja brenda klasës bazë	Qasja brenda klasës së nxjerrë	Qasja jashtë klasës
public	po	po	po
private	po	jo	jo
protected	po	po	jo

Fig.4.19 Mundësitë e qasjes tek anëtarët e klasave

Klasa bazë nuk ka qasje tek anëtarët e klasës së nxjerrë, përkatësisht prej saj nuk mund të shihen komponentet e pjesës tjetër të hierarkisë së klasave në të cilën ajo përfshihet.

Në literaturë për *klasën bazë* dhe për *klasën e nxjerrë* shfrytëzohen edhe termat *klasë prind* dhe *klasë fëmijë*, ose edhe *superklasë* dhe *subklasë*.

Çdo klasë e nxjerrë bëhet klasë bazë për klasat të cilat nxirren prej saj.

Ridefinimi i funksioneve të klasës bazë

Funksionet në komponentet publike të klasës bazë mund të ridefinohen te klasat e nxjerra prej saj. Përkatësisht, në klasën e nxjerrur mund të ridefinohen funksione me emra të njëjtë, numër dhe tip të njëjtë të parametrave. Por, ky ridefinim i funksioneve vlen vetëm për klasën e nxjerrë dhe jo edhe për klasën bazë.

Shembull

Programi **classT2**, përmes së cilit tregohet ridefinimi te klasa e nxjerrë **Beta** i funksionit **Nata**, përmes së cilit te klasa **Alfa**

llogaritet vlera e shprehjes:

$$g = 3a + 2b - 1$$

kurse te klasa e nxjerrë **Beta** vlera në fjalë llogaritet me shprehjen:

$$g = 2b + 3c - d$$

```
// Programi classT2
#include <iostream>
using namespace std;
class Alfa
{
public:
    double a,b,g;
    Alfa();
    void Nata();
};

class Beta:public Alfa
{
private:
    double c;
public:
    int d;
    Beta();
    void Nata();
};

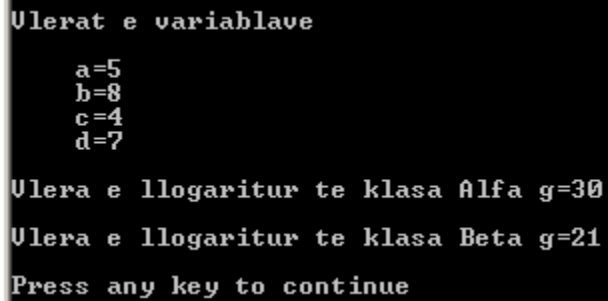
int main()
{
    Alfa X;
    Beta Y;
    X.Nata();
    cout << "\nVlera e llogaritur te klasa Alfa g="
          << X.g;
    Y.Nata();
    cout << "\n\nVlera e llogaritur te klasa Beta g="
          << Y.g
          << "\n\n";
    return 0;
}

Alfa::Alfa()
{
    a=5;
    b=8;
```

```
}  
  
Beta::Beta()  
{  
    c=4;  
    d=7;  
    cout << "\nVlerat e variablave"  
        << "\n\n    a=" << a  
        << "\n    b=" << b  
        << "\n    c=" << c  
        << "\n    d=" << d  
        << "\n";  
}  
  
void Alfa::Nata()  
{  
    g=3*a+2*b-1;  
}  
  
void Beta::Nata()  
{  
    g=2*b+3*c-d;  
}
```

Llogaritja e vlerës së variablës **g**, si në klasën bazë **Alfa** ashtu edhe në klasën e nxjerrë **Beta**, është definuar brenda funksionit **Nata**, i cili përfshihet në komponentet publike përkatëse.

Nëse ekzekutohet programi i dhënë, rezultati që fitohet në ekran do të duket si në *Fig.4.20*.



```
Vlerat e variablave  
  
a=5  
b=8  
c=4  
d=7  
  
Vlera e llogaritur te klasa Alfa g=30  
Vlera e llogaritur te klasa Beta g=21  
Press any key to continue
```

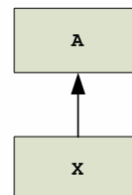
Fig.4.20
Pamja e ekranit pas
ekzekutimit të programit
classT2

Trashëgimia e shumëfishtë

Gjatë krijimit të klasave përmes trashëgimisë, klasa e re mund të nxirret prej një klase bazë, ose edhe prej më shumë klasave bazë njëkohësisht. Në këto dy raste flitet për *trashëgimi të njëfishtë* (ang. single inheritance) dhe *trashëgimi të shumëfishtë* (ang. multiple inheritance). Te shembujt që u dhanë në pjesën paraprake kemi të bëjmë me trashëgiminë e njëfishtë, sepse anëtarët e trashëguar të klasës së nxjerrë janë anëtarë të vetëm një klase bazë, p.sh, ashtu siç shihet në Fig.4.21.

Fig.4.21

Paraqitja skematike e trashëgimisë së njëfishtë



Këtu, klasa e nxjerrë **X** i trashëgon vetëm anëtarët e klasës bazë **A**, gjë që, siç është dhënë më parë, në program shkruhet si në vijim.

```

class A
{
    ...
};

class X:public A
{
    ...
};
  
```

Specifikuesi i qasjes te klasa e nxjerrë **X**, për anëtarët e trashëguar nga klasa **A**, siç u pa në shembujt paraprakë, përveç **public**, mund të jenë edhe **private** dhe **protected**.

Një shembull i trashëgimisë së shumëfishtë, kur klasa **X** është nxjerrë nga dy klasat bazë **A** dhe **B**, skematikisht mund të paraqitet si në Fig.4.22.

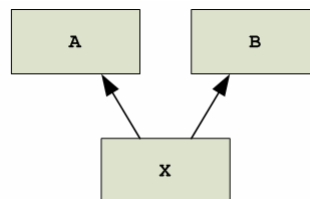


Fig.4.22

Paraqitja skematike e trashëgimisë së shumëfishtë

Gjatë definimit të trashëgimisë nga shembulli në fjalë, definimi i relacioneve të klasave do të duket si në vijim.

```
class A
{
    ...
};

class B
{
    ...
};

class X:public A,public B
{
    ...
};
```

Nënkuptohet se edhe këtu specifikuesit e qasjes të klasa e nxjerrë **X**, për anëtarët e trashëguar nga klasat **A** dhe **B**, si të rastet e trashëgimisë së njëfishtë, përveç **public**, mund të jenë edhe **private** dhe **protected**.

Trashëgimia e shumëfishtë mund të jetë edhe komplekse, kur si klasa bazë paraqiten edhe klasa të nxjerra, p.sh. si ajo që shihet në Fig.4.23.

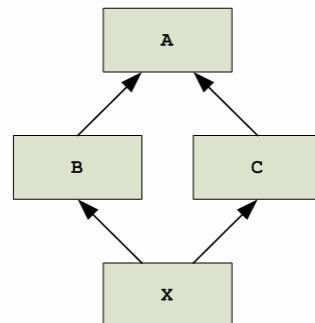


Fig.4.23
Trashëgimi e shumëfishtë komplekse

Në program, definimi i relacioneve të klasave në këtë rast do të mund të shkruhej ashtu siç është dhënë në vijim.

```
class A
{
    ...
};
```

```

class B:public A
{
    ...
};

class C:public A
{
    ...
};

class X:public B,public C
{
    ...
};

```

Operatori i shoqërimit tek objektet

Nëse në program janë deklaruar dy objekte **x1** dhe **x2** të klasës **x**, përmes operatorit të shoqërimit:

```
x2=x1;
```

vlerat e variablave të komponenteve të objektit **x1** u shoqërohen variablave përkatëse të komponenteve të objektit **x2**.

Shembull

Programi **classR1**, përmes së cilit tregohet shoqërimi i vlerave të variablave të komponenteve të objektit **kater1** të klasës **brinjet**, variablave përkatëse të komponenteve të objektit **kater2**.

```

// Programi classR1
#include <iostream>
using namespace std;

class brinjet
{

```



```
private:
    double a,b;
public:
    void vlerat(double x,double y);
    void shtypja();
};

int main()
{
    brinjet kater1,kater2;
    kater1.vlerat(7.5,4.2);
    cout << "\nVlerat origjinale";
    kater1.shtypja();
    kater2=kater1;
    cout << "vlerat e kopjuara";
    kater2.shtypja();
    return 0;
}

void brinjet::vlerat(double x,double y)
{
    a=x;
    b=y;
}

void brinjet::shtypja()
{
    cout << "\n\n    a="
        << a
        << "\n    b="
        << b
        << "\n\n";
}
```

Në program, fillimisht janë deklaruar objektet **kater1** dhe **kater2** të klasës **brinjet**. Me thirrjen e funksionit **vlerat**:

```
kater1.vlerat(7.5,4.2);
```

variabla **a** dhe **b** të objektit **kater1** u shoqërohen vlerat **7.5** dhe **4.2**. Pastaj, pasi shtypen këto vlera, duke e thirrur funksionin **shtypja**:

```
kater1.shtypja();
```

përmes operatorit të shoqërimit, i cili zbatohet mbi dy objektet:

```
kater2=kater1;
```

vlerat e variablave të objektit **kater1** u shoqërohen variablave përkatëse të objektit **kater2**, përkatësisht vlerat e variablave të njërit objekt kopjohen te variablat përkatëse të objektit tjetër. Kjo shihet edhe pas shtypjes së vlerave, duke e thirrur funksionin **shtypja** të objektit **kater2**:

```
kater2.shtypja();
```

gjë që është dhënë në Fig.4.24.

Fig.4.24

Pamja e ekranit pas ekzekutimit të programit **classR1**

```
Vlerat origjinale
    a=7.5
    b=4.2

Vlerat e kopjuara
    a=7.5
    b=4.2

Press any key to continue
```

Krahasimi i variablave të klasës

Objektet nuk mund të krahasohen direkt mes vete. Por, mund të krahasohen vlerat e variablave të përfshira brenda tyre.

Shembull

Versioni **classR2** i programit **classR1**, tek i cili krahasohen variablat brenda objekteve **X** dhe **Y** të klasës **brinjet**.

Këtu është ndryshuar programi kryesor dhe definicioni i klasës (variablat **a** dhe **b** janë deklaruar si publike), ashtu siç shihet në vijim, kurse pjesa e nënprogrameve mbetet sikurse që është dhënë te versioni **classR1**.

```
// Programi classR2
#include <iostream>
using namespace std;

class brinjet
{
public:
    double a,b;
    void vlerat(double x,double y);
    void shtypja();
};
```

```
int main()
{
    brinjet X,Y;

    X.vlerat(7.5,4.2);
    Y.vlerat(6.3,4.2);

    cout << "\nVlerat në objektin X";
    X.shtypja();

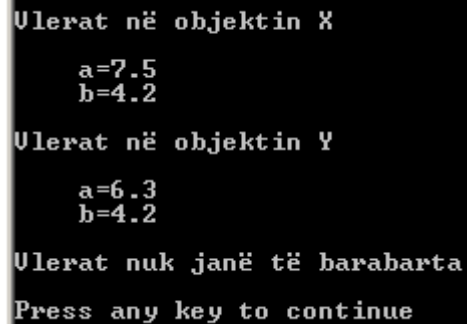
    cout << "Vlerat në objektin Y";
    Y.shtypja();

    if ((X.a==Y.a) && (X.b==Y.b))
        cout << "Vlerat janë të barabarta";
    else
        cout << "Vlerat nuk janë të barabarta"
               << "\n\n";
    return 0;
}
```

Me pjesën e fundit të programit krahasohet barazia e variablave përkatëse tek objektet **X** dhe **Y**. Meqë gjatë thirrjes së funksionit **vlerat** për dy objektet e veçanta:

```
X.vlerat(7.5,4.2);
Y.vlerat(6.3,4.2);
```

variablave **a** dhe **b** u shoqërohen vlera të ndryshme, rezultati në ekran do të duket si në Fig.4.25.



```
Vlerat në objektin X
    a=7.5
    b=4.2
Vlerat në objektin Y
    a=6.3
    b=4.2
Vlerat nuk janë të barabarta
Press any key to continue
```

Fig.4.25
Pamja e ekranit pas ekzekutimit të
programit `classR2`

Klasat e ndërthurura

Brenda klasave mund të definojen klasa të tjera, për të krijuar struktura të klasave të ndërthurura. Te strukturat e tilla, klasa në të cilën përfshihet një klasë njihet si *klasë e jashtme*, kurse klasa e përfshirë në të njihet si *klasë e brendshme*. Gjatë kësaj, klasat e brendshme mund të vendosen në pjesën private ose në pjesën publike të klasës së jashtme.

Shembull

Programi **classN**, në të cilin brenda komponentes publike të klasës së jashtme **Dita** definohet klasa e brendshme **Nata**.

```
// Programi classN
#include <iostream>
using namespace std;

class Dita
{
private:
    void MesazhiA();
public:
    int a;
    void vleraA(int x);

    class Nata
    {
    private:
        void MesazhiB();
    public:
        int b;
        void vleraB(int y);
    };
};

int main()
{
    Dita Alfa;
    Dita::Nata Beta;
    Alfa.vleraA(55);
    cout << "\nVlera e variablës a="
         << Alfa.a;
    Beta.vleraB(33);
    cout << "\nVlera e variablës b="
         << Beta.b
         << "\n\n";
    return 0;
}

void Dita::MesazhiA()
{
```

```

        cout << "\nKlasa e jashtme Dita\n";
    }
    void Dita::vleraA(int x)
    {
        a=x;
        MesazhiA();
    }

    void Dita::Nata::MesazhiB()
    {
        cout << "\n\nKlasa e brendshme Nata\n";
    }

    void Dita::Nata::vleraB(int y)
    {
        b=y;
        MesazhiB();
    }

```

Këtu, gjatë definimit të funksioneve të klasës së brendshme dy herë shfrytëzohet *operatori për zëbërthim të dukjes (::)*:

```

void Dita::Nata::MesazhiB()
void Dita::Nata::vleraB(int y)

```

Gjithashtu, ky operator shfrytëzohet edhe gjatë deklarimit të objektit **Beta** të klasës së brendshme **Nata**, në këtë mënyrë:

```
Dita::Nata Beta;
```

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në *Fig.4.26*.

Fig.4.26
Pamja e ekranit pas ekzekutimit të programit
classN

```

Klasa e jashtme Dita
Vlera e variablës a=55

Klasa e brendshme Nata
Vlera e variablës b=33
Press any key to continue

```

Këtu, variablat **a** dhe **b** nuk mund të shfrytëzohen te njëra ose te klasa tjetër. Gjithashtu, emri i klasës së brendshme **Nata**, meqë nuk shihet jashtë klasës, mund të shfrytëzohet si emër edhe i ndonjë klase tjetër, jashtë klasave të ndërthurura.

Objektet brenda klasave

Brenda klasave mund të deklarohen dhe të shfrytëzohen objekte të klasave të tjera. Deklarimi bëhet në pjesën e anëtarëve privatë ose publikë të klasës.

Shembull

Programi **classH1**, në të cilin brenda komponentes private të klasës **Nata** deklarohet objekti **Alfa** i klasës **Dita**.

```
// Programi classH1
#include <iostream>
using namespace std;

class Dita
{
private:
    void MesazhiA();
public:
    int a;
    void vleraA(int x);
};

class Nata
{
private:
    void MesazhiB();
    Dita Alfa;
public:
    int b;
    void vleraB(int y);
};

int main()
{
    Nata Beta;
    Beta.vleraB(56);
    cout << "\n\n\nProgrami kryesor"
          << "\n\nVlera e variablës b: "
          << Beta.b
          << "\n\n";
    return 0;
}

void Dita::MesazhiA()
{
```

```
        cout << "\nKlasa Dita\n";
    }
    void Dita::vleraA(int x)
    {
        a=x;
        MesazhiA();
        cout << "\nVlera e variablës a: "
              << a;
    }

    void Nata::MesazhiB()
    {
        cout << "\n\nKlasa Nata\n";
    }

    void Nata::vleraB(int y)
    {
        b=y;
        Alfa.vleraA(78);
        MesazhiB();
        cout << "\nVlera e variablës a: "
              << Alfa.a;
    }
}
```

Në program, fillimisht është definuar klasa **Dita**. Pastaj, brenda klasës **Nata** është deklaruar objekti **Alfa** i klasës **Dita**, i cili shfrytëzohet gjatë thirrjes së funksionit **vleraA**:

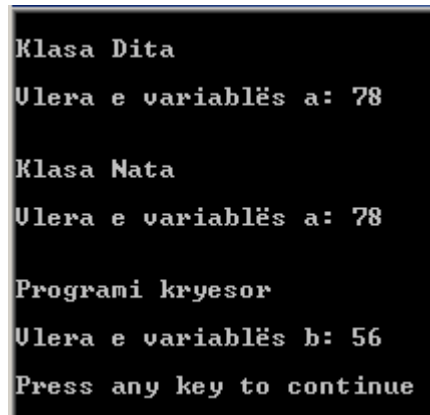
```
Alfa.vleraA(78);
```

brenda funksionit **vleraB**, ku edhe shtypet vlera e variablës **a**, duke e shkruar brenda komandës për shtypje si **Alfa.a**, pasi fillimisht thirret funksioni **MesazhiB**.

Në programin kryesor deklarohet edhe objekti **Beta** i klasës **Nata**, ku shtypet vlera e variablës **b**, duke e shkruar brenda komandës për shtypje si **Beta.b**, pasi fillimisht thirret funksioni:

```
Beta.vleraB(56);
```

Rezultati në ekran do të duket ashtu siç është dhënë në *Fig.4.27*.



```
Klasa Dita
Vlera e variablës a: 78

Klasa Nata
Vlera e variablës a: 78

Programi kryesor
Vlera e variablës b: 56
Press any key to continue
```

Fig.4.27

Pamja e ekranit pas ekzekutimit të programit `classH1`

Fushat brenda klasave

Në komponentet e klasave mund të përfshihen edhe fusha, p.sh., siç janë vektorët. Rezervimi i vendeve të nevojshme për fushat (numri maksimal i mundshëm i anëtarëve të tyre) bëhet përmes deklarimit si konstante para definimit të klasave. Por, njëkohësisht brenda klasave, në komponente të veçanta vendosen variablat të cilat ruhen dimensionet aktuale të fushave.

Shembull

Programi **classV1**, përmes së cilit gjendet anëtari minimal në vektorin e dhënë **A(n)**, i cili përfshihet në komponenten e klasës **Vektori**.

```
// Programi classV1
#include <iostream>
#include <iomanip>
using namespace std;

const int m=10;

class Vektori
{
public:
    int n;
    double A[m];
    double Min();
};

int main()
{
    int i;
    Vektori Jeta = {6,3.2,14.5,-7.6,1.4,-3.2,8.3};
    cout << "\nAnëtarët e vektorit\n"
         << "\n   i      A[i]"
         << "\n-----\n";

    for (i=0;i<Jeta.n;i++)
        cout << setw(4)
```



```
        << i
        << setw(10)
        << Jeta.A[i]
        << "\n";
    cout << "-----\n"
    << "\nVlera minimale x: "
    << Jeta.Min()
    << "\n\n";

return 0;
}

double Vektori::Min()
{
    double x=A[0];
    int i;
    for (i=1;i<n;i++)
        if (A[i]<x)
            x=A[i];
return x;
}
```

Programi i dhënë është një verzion i modifikuar i programit të ngjashëm, i cili është dhënë gjatë shpjegimit të strukturave. Këtu, në vend të fjalës **struct** është shënuar fjala **class**, si dhe komponentet janë deklaruar si publike, duke e shënuar para tyre fjalën **public**.

Numri maksimal i mundshëm i anëtarëve të vektorit është përcaktuar përmes komandës:

```
const int m=10;
```

meqë brenda klasës vektorit është deklaruar kështu:

```
double A[m];
```

Por, pasi gjatë deklarimit të objekteve të klasës në fjalë mund të shfrytëzohen vektorë me numër të ndryshëm anëtarësh, në pjesën me të dhëna të saj është shfrytëzuar edhe variabla **n**, përmes së cilës përcaktohet numri aktual i anëtarëve të vektorit. Në këtë mënyrë, gjatë deklarimit të objektit **Jeta** të tipit vektor, në grumbullin me vlera, të cilat janë shënuar brenda kllapave:

```
Vektori Jeta = {6, 3.2, 14.5, -7.6, 1.4, -3.2, 8.3};
```

vlera e parë **6** i përket variablës **n**.

Për gjetjen e anëtarit minimal të vektorit **A(n)**, përmes algoritmit të njohur, brenda klasës është definuar funksioni **Min**.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.4.28.

Anëtarët e vektorit	
i	A[i]
0	3.2
1	14.5
2	-7.6
3	1.4
4	-3.2
5	8.3
Vlera minimale x: -7.6	
Press any key to continue	

Fig.4.28

Pamja e ekranit pas ekzekutimit të programit **classV1**

Fushat e objekteve

Grupet e të dhënave të një klase të caktuar mund të vendosen në një fushë të objektit përkatës. Gjatë kësaj, në një element të fushës përfshihen të dhëna të cilat u përgjigjen komponenteve të klasës në fjalë.

Shembull

Programi **classF1**, përmes së cilit tregohet deklarimi si vektor me **n**-anëtarë i objektit **studenti** të klasës **person**.

```
// Programi structV2
#include <iostream>
using namespace std;

class person
{
public:
    char emri[8],qyteti[10];
    int viti;
};

int main()
{
    const int n=2;
    int i;

    person studenti[n];
```

```

    cout << "\nTë dhënat nga tastiera\n\n";

    for (i=0;i<n;i++)
    {
        cout << "Emri .....: ";
        cin >> studenti[i].emri;
        cout << "Qyteti ...: ";
        cin >> studenti[i].qyteti;
        cout << "Viti .....: ";
        cin >> studenti[i].viti;
        cout << "\n";
    }

    cout << "\nTë dhënat e lexuara\n";

    for (i=0;i<n;i++)
    {
        cout << "\nEmri .....: "
              << studenti[i].emri;
        cout << "\nQyteti ...: "
              << studenti[i].qyteti;
        cout << "\nViti .....: "
              << studenti[i].viti
              << "\n";
    }
    cout << "\n";
    return 0;
}

```

Edhe ky program paraqet një verzion të modifikuar të programit të ngjashëm, i cili është dhënë gjatë shpjegimit të strukturave.

Në program është paraparë të lexohen të dhënat që ruhen në komponentet e klasës **person** dhe pastaj të njëjtat edhe të shtypen në ekran. Rezultati pas ekzekutimit të programit, p.sh., mund të duket ashtu siç është dhënë në Fig.4.29.



```

Të dhënat nga tastiera

Emri .....: Besa
Qyteti ...: Shkupi
Viti .....: 1983

Emri .....: Fatmir
Qyteti ...: Struga
Viti .....: 1985

Të dhënat e lexuara

Emri .....: Besa
Qyteti ...: Shkupi
Viti .....: 1983

Emri .....: Fatmir
Qyteti ...: Struga
Viti .....: 1985

Press any key to continue

```

*Fig.4.29**Pamja e ekranit pas ekzekutimit të programit `classv2`*

Dukshmëria e klasave dhe e objekteve

Anëtarët e klasës janë lokalë dhe duken vetëm brenda klasës. Për t'i shfrytëzuar anëtarët e klasës jashtë saj, duhet të shfrytëzohet objekti i klasës dhe pika si operator për qasje tek anëtari i klasës.

Për dukshmërinë e objekteve vlejnë principet e njohura për dukshmërinë e variablave lokale dhe globale.

	4	
Pointerët - Deklarimi i pointerëve 218 - Adresat e variablove 220 - Vlera në adresën e variablës 222 - Shoqërimi i vlerave 224 - Operatorët inverzë 227 - Llogaritjet përmes pointerëve 231 - Operimi me vlerat e pointerëve 239 - Pointerët gjatë operimit me fusha 242 - Fusha pointerësh 265 - Pointerët si parametra të funksioneve 267 - Pointerët në funksione 279 - Pointerët në struktura 286 - Pointerët në objekte 289		

Për ruajtjen e të dhënave në memorien e kompjuterit shfrytëzohen variablat. Kjo mundëson që operimi me të dhëna të bëhet duke operuar me variablat në të cilat ruhen ato. Me qëllim të rezervimit të hapësirës së nevojshme memoruese për variablat, para se të shfrytëzohen deklarohen tipet e tyre. Njëkohësisht, çdo lokacioni në hapësirën memoruese të kompjuterit i shoqërohet një numër, i cili e paraqet *adresën e lokacionit*.

Nga ana tjetër, kompjuteri gjatë operimit me të dhëna i shfrytëzon adresat e lokacioneve në të cilat ato janë vendosur, përkatësisht adresat e variablave përkatëse. Por, që edhe programuesi të ketë qasje në këto adresa, në gjuhën C++ shfrytëzohen *variabla të tipit tregues*, ose, siç thuhet ndryshe - *variabla pointer* (ang. pointer), ose shkurt vetëm *pointer*. Këto variabla quhen kështu, sepse përmes tyre merren adresat e variablave, përkatësisht vlerat që ruhen brenda tyre tregojnë te variablat.

Kur flitet për pointerët, mund të shtrohet pyetja e arsyeshmërisë së përdorimit të tyre, përkatësisht e adresave të variablave, kur mund të operohet direkt me vetë variablat. Pointerët, në raste të caktuara kanë një përparësi, sepse përmes adresave, pa kufizime, mund të shfrytëzohen vlerat e variablave në pjesë të ndryshme të programit.

Deklarimi i pointerëve

Pointerët, si edhe variablat e tipeve të tjera, para se të shfrytëzohen duhet të deklarohen. Ky deklarim u përngjan deklarimeve të variablave të zakonshme. Por këtu, për t'i dalluar nga variablat e zakonshme, para pointerëve shënohet simboli *****, gjë që në formë të përgjithshme duket:

```
t *p;
```

ku janë:

p - variabla e tipit pointer.

t - tipi i variablës **p**

Tipi **t** i variablës **p** duhet të përputhet me tipin e variablës adresa e së cilës do të ruhet në të. Kështu, p.sh., përmes komandës:

```
int *a;
```

deklarohet variabla **a** e tipit pointer, e cila tregon në një variabël të tipit **int**.

Gjatë deklarimit, simboli ***** mund të shkruhet kudo mes tipit dhe variablës, p.sh., në njërën nga këto forma:

```
int *a;
int * a;
int  * a;
int* a;
```

Meqë deklarimet, kur simboli ***** shënohet larg variablës, mund të shkaktojnë konfuzion, zakonisht praktikohet që ai të shkruhet pranë variablës, ose pranë tipit të variablës, ashtu siç shihet në shembullin e parë dhe në shembullin e fundit të dhënë më sipër.

Si tipe të variablave pointerë shfrytëzohen të gjithë tipet e variablave të zakonshme. Kështu, p.sh., si pointer mund të deklarohen variabla të cilat tregojnë në variabla të tipeve **int**, **double**, **char** etj., në këtë mënyrë:

```
int *z;
double *k;
char *h;
```

ku, përmbajtja e variablës **z** tregon te një variabël, përkatësisht lokacion memorues të tipit **int**, përmbajtja e variablës **k** tregon në një variabël të tipit **double**, kurse përmbajtja e variablës **h** tregon në një variabël të tipit **char**. Në një pointer të tipit **int** mund të ruhen vetëm adresa të variablave të tipit **int**, gjë që vlen edhe për tipet e tjera të pointerëve.

Ngjashëm si edhe gjatë deklarimit të variablave të zakonshme, në një deklarim njëkohësisht mund të përfshihen disa variabla të tipit pointer, p.sh., kështu:

```
int *g, *h;
```

Gjithashtu, bashkë me variablat e tipit pointer mund të përfshihen edhe deklarime të tipeve të zakonshëm të variablave. Kështu, p.sh., variablat e dy deklarimeve të veçanta:

```
int x,y;
int *h,*p;
```

mund të përfshihen në një deklarim të vetëm:

```
int x,y,*h,*p;
```

ku dy variablat e para **x** dhe **y** paraqesin variabla të zakonshme, kurse dy të fundit **h** dhe **p** - variabla të tipit pointer.

Rekomandohet që gjatë deklarimit të variablave të tipit pointer ato të inicializohen me vlerën zero, si *null pointer*, p.sh., kështu:

```
int *x=0;
```

ose duke e shfrytëzuar *konstanten zero* (**NULL**):

```
int *x=NULL;
```

Në fakt, *lejohet inicializimi direkt i pointerëve vetëm me vlerën zero*. Kurse, nëse tentohet që inicializimi direkt të bëhet me ndonjë vlerë tjetër, kompjuteri do të na lajmërojë gabim.

Mosinicializimi i pointerit me një vlerë të caktuar paraqet rrezik, sepse vlera e çfarëdoshme e tij mund të tregojë edhe në hapësirën e pjesës me programe të kompjuterit, ashtu që ndryshimi i të dhënave që ruhen në këtë hapësirë mund të shkaktojë dëme serioze softverike.

Adresat e variablave

Duke e shfrytëzuar operatorin **&**, përmes shprehjes së formës:

```
p=&v;
```

adresa e variablës së zakonshme **v** vendoset te variabla **p** e tipit pointer.

Shembull

Programi **point1**, përmes së cilit tregohet marrja dhe shtypja e adresës së variablës **x**, duke e shfrytëzuar pointerin **a**.

```
// Programi point1
#include <iostream>
using namespace std;
int main()
{
    int *a,x;

    x=357;
    cout << "\nVlera e variablës x: "
          << x
          << "\n";

    a=&x;
```



```

        cout << "\nAdresa te pointeri a: "
              << a
              << "\n\n";
return 0;
}

```

Nëse ekzekutohet programi i dhënë, rezultati që shtypet në ekran do të duket si në *Fig.5.1*.

Fig.5.1
Pamja e ekranit pas ekzekutimit të
programit *point1*

```

Vlera e variablës x: 357
Adresa te pointeri a: 0012FEC8
Press any key to continue

```

Siç shihet nga figura e dhënë, këtu fillimisht është shtypur vlera e variablës **x** dhe pastaj edhe adresa e saj. Kjo adresë ruhet te pointeri **a** dhe është marrë përmes operatorit **&**, duke e shfrytëzuar shprehjen:

```
a=&x;
```

Pamja e pjesës së hapësirës memoruese, ku vendosen të dhënat, pas secilit nga deklaratimet vijuese:

1. `int *a, x;`
2. `x=357;`
3. `a=&x;`

do të duket si në *Fig.5.2*.

	1.	2.	3.	
00000000				
00000001				
...				
0012FEC8		357	357	x
...				
0012FED4			0012FEC8	a
...				

Fig.5.2 Pamja e hapësirës memoruese pas secilës nga sekuencat e tri komandave

Me deklarin e parë, në memorie rezervohen vende për variablën **x** (në adresën **0012FEC8**) dhe për pointerin **a** (në adresën **0012FED4**). Pas komandës së dytë, në hapësirën memoruese përkatëse regjistrohet numri **357**, i cili në fakt e paraqet vlerën që i shoqërohet variablës **x**. Kurse, pas komandës së tretë, te

hapësira memoruese që i përket pointerit **a** vendoset adresa **0012FEC8**, e cila i përket variablës **x**.

Vlera në adresën e variablës

Nëse dihet adresa e një variable, përmes procesit të *deadresimit* ose të *dereferimit* (ang. dereferencing), mund të merret vlera e variablës. Për këtë qëllim shfrytëzohet operatori *****, kështu:

```
v=*p;
```

pas së cilës, te variabla **v** do të vendoset vlera e variablës adresa e së cilës ruhet te pointeri **p**.

Shembull

Programi **point2**, përmes së cilit tregohet shtypja e vlerës së variablës ndërmjetësuese **h**, duke e shfrytëzuar adresën e variablës **g**, e cila ruhet te pointeri **a**.

```
// Programi point2
#include <iostream>
using namespace std;
int main()
{
    int *a,g,h;

    g=357;
    cout << "\nVlera e variablës g: "
         << g
         << "\n";

    a=&g;

    cout << "\nAdresa te variabla a: "
         << a
         << "\n";

    h=*a;

    cout << "\nVlera e variablës h: "
         << h
         << "\n\n";
    return 0;
}
```

Në program, përmes shprehjes:

```
a=&g;
```

te pointeri **a** ruhet adresa e variablës **g**. Pastaj, duke e shfrytëzuar shprehjen:

```
h=*a;
```

te variabla **h** vendoset vlera e variablës, adresa e së cilës ruhet te pointeri **a**, përkatësisht vlera e variablës **g**.

Pas ekzekutimit të programit të dhënë, rezultati në ekran do të duket si në Fig.5.3.

Fig.5.3

Pamja e ekranit pas ekzekutimit të programit *point2*

```
Ulera e variablës g: 357
Adresa te variabla a: 0012FEC8
Ulera e variablës h: 357
Press any key to continue
```

Prej këtu shihet se te variabla **h** ruhet vlera e variablës **g**, duke e përcjellë atë përmes adresës e cila ruhet te pointeri **a**.

Pamja e hapësirës memoruese përkatëse pas sekuencës së komandave:

1. **g=357;**
2. **a=&g;**
3. **h=*a;**

do të duket si në Fig.5.4.

	1.	2.	3.	
00000000				
00000001				
...				
0012FEBC			357	h
...				
0012FEC8	357	357	357	g
...				
0012FED4		0012FEC8	0012FEC8	a
...				

Fig.5.4 Pamja e hapësirës memoruese pas sekuencës së tri komandave

Përmes komandës së parë **g=357**, te lokacioni me adresë **0012FEC8**, vendoset vlera **357**. Pastaj, pas komandës së dytë **a=&g**, adresa e variablës **g** vendoset te pointeri **a**. Në fund, me komandën e tretë **h=*a**, te variabla **h** vendoset vlera, e cila gjendet në adresën që ruhet te pointeri **a**.

Shoqërimi i vlerave

Variablave të tipeve të zakonshëm, me ndërmjetësimin e pointerëve, në të cilët ruhen adresat përkatëse, mund t'u shoqërohen vlera të konstanteve, ose vlera të variablave të tjera.

Shoqërimi i vlerave të konstanteve

Variablës, adresa e së cilës ruhet te pointeri **p**, mund t'i shoqërohet një vlerë konstante **k** edhe në rrugë indirekte, kështu:

```
*p=k;
```

Shembull

Programi **point3a**, përmes së cilit tregohet vendosja indirekte e vlerës **649**, te variabla **x**, duke e shfrytëzuar adresën e variablës **x**, e cila ruhet te pointeri **a**.

```
// Programi point3a
#include <iostream>
using namespace std;
int main()
{
    int x=837, *a;
    cout << "\nVlera fillestare e variablës x: "
          << x;

    a=&x;

    cout << "\n\nAdresa e variablës x te pointeri a: "
          << a
          << "\n";

    *a=649;

    cout << "\nVlera e ndryshuar e variablës x: "
          << x
          << "\n\n";
    return 0;
}
```

Këtu, fillimisht, te pointeri **a** ruhet adresa e variablës **x**. Pastaj, pasi shtypet vlera fillestare e variablës **x** dhe adresa e cila ruhet te pointeri **a**, përmes shprehjes:

```
*a=649;
```

te variabla **x** vendoset vlera e re **649**, gjë që shihet edhe te rezultati që shtypet në ekran (Fig.5.5).

Fig.5.5
Pamja e ekranit pas
ekzekutimit të
programit point3a

```
Ulera fillestare e variablës x: 837
Adresa e variablës x te pointeri a: 0012FED4
Ulera e ndryshuar e variablës x: 649
Press any key to continue
```

Pamja përkatëse e hapësirës memoruese pas sekuencës së komandave:

1. `int x=837, *a;`
2. `a=&x;`
3. `*a=649;`

do të duket si në Fig.5.6.

	1.	2.	3.	
00000000				
00000001				
...				
0012FEC8		0012FED4	0012FED4	a
...				
0012FED4	837	837	649	x

Fig.5.6 Pamja e hapësirës memoruese pas sekuencës së tri komandave

Përmes komandës së parë në sekuencën e dhënë, te lokacioni që i përket variablës **x** (lokacioni me adresë **0012FED4**), vendoset vlera **837**. Pastaj, me komandën **a=&x**, adresa e variablës **x** vendoset te lokacioni i rezervuar për pointerin **a**. Në fund, duke e shfrytëzuar komandën ***a=649**, në lokacionin me adresë **0012FED4** (lokacionin, adresa e të cilit ruhet te pointeri **a**), vendoset vlera **649**.

Shoqërimi i vlerave të variablave

Variablës, adresa e së cilës ruhet te pointeri **p**, mund t'i shoqërohet vlera e variablës tjetër **v** në rrugë indirekte, duke e shfrytëzuar shprehjen:

```
*p=v;
```

Shembull

Programi **point3b**, përmes së cilit tregohet vendosja indirekte e vlerës së variablës **x** te variabla **y**, duke e shfrytëzuar adresën e variablës **y**, e cila ruhet te pointeri **a**.

```
// Programi point3b
#include <iostream>
using namespace std;
int main()
{
    int x=837,y,*a;
    cout << "\nVlera e variablës x: "
          << x;

    a=&y;

    cout << "\n\nAdresa e variablës y: "
          << a
          << "\n";

    *a=x;

    cout << "\nVlera e variablës y: "
          << y
          << "\n\n";
    return 0;
}
```

Rezultati i cili fitohet në ekran pas ekzekutimit të programit të dhënë do të duket si në Fig.5.7.

Fig.5.7
Pamja e ekranit pas ekzekutimit të
programit **point3b**

```
Vlera e variablës x: 837
Adresa e variablës y: 0012FEC8
Vlera e variablës y: 837
Press any key to continue
```

Siç shihet nga rezultati i fituar, variablës **y** i është shoqëruar vlera e variablës **x**. Për këtë qëllim, te pointeri **a** ruhet adresa e variablës **x**, duke e shfrytëzuar shprehjen:

```
a=&y;
```

Pastaj, përmes shprehjes:

```
*a=x;
```

me ndërmjetësimin e adresës së variablës **y**, te kjo variabël përcillet vlera e variablës **x**.

Pamja e hapësirës memoruese në këtë rast do të jetë e ngjashme me atë që u tregua më sipër, përmes tabelës që shihet në *Fig. 5.6*.

Operatorët inverzë

Operatorët **&** dhe ***** mes vete llogariten si *operatorë inverzë*, gjë që vërtetohet edhe përmes shembujve të programeve që janë dhënë në vijim.

Shembull

Programi **point7**, në të cilin te pointeri **a** ruhet adresa e variablës **x**. Pastaj shtypen dhe krahasohen vlerat **x** dhe ***a**.

```
// Programi point7
#include <iostream>
using namespace std;
int main()
{
    int x, *a;
    a=&x;
    x=837;

    cout << "\nMeqë:"
         << "\n\nVlera  x: "
         << x
         << "\nVlera *a: "
         << *a
         << "\n";
    cout << "\nRezultati i krahasimit është ";

    if (x == *a)
        cout << "true";
    else
        cout << "false";

    cout << "\n\n";
    return 0;
}
```

Në program, përmes shprehjes:

```
a=&x;
```

fillimisht, adresa e variablës **x** vendoset te pointeri **a**. Pastaj, pasi shtypet vlera e variablës **x** dhe vlera ***a**, duke e shfrytëzuar komandën:

```
if (x == *a)
```

këto dy vlera krahasohen. Meqë variabla **a** është pointer në të cilin ruhet adresa e variablës **x**, përmes ***a** merret vlera e variablës **x**. Prandaj, raporti i vlerave që krahasohen me komandën **if** në fjalë, do të jetë **true**, gjë që qartë shihet në rezultatin që fitohet në ekran pas ekzekutimit të programit (shih Fig.5.8).

Fig.5.8
Pamja e ekranit pas ekzekutimit të programit **point7**

```
Meqë
Vlera x: 837
Vlera *a: 837

Rezultati i krahasimit është true
Press any key to continue
```

Në program mund të shfrytëzohen edhe kombinimet e operatorëve ***** dhe **&**.

Shembull

Programi **pointX1**, në të cilin te pointeri **a** ruhet adresa e variablës **x**. Pastaj shtypen vlerat e të gjitha kombinimeve të mundshme të operatorëve **&** dhe *****.

```
// Programi pointX1
#include <iostream>
using namespace std;
int main()
{
    int x, *a;
    x=453;

    a=&x;
    cout << "\n a=&x:  "
         << a;

    x=*a;
    cout << "\n\n x=*a:  "
         << x;
```



```

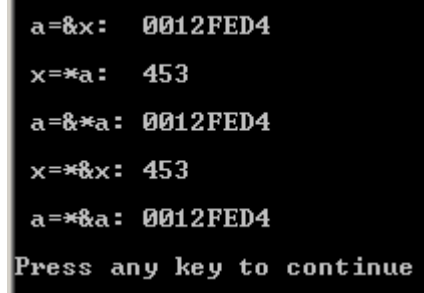
a=&*a;
cout << "\n\n a=&*a: "
      << a;

x=&*x;
cout << "\n\n x=&*x: "
      << x;

a=&*a;
cout << "\n\n a=&*a: "
      << a
      << "\n\n";
return 0;
}

```

Rezultati, i cili shtypet në ekran pas ekzekutimit të programit të dhënë, do të duket si në *Fig.5.9*.



```

a=&x: 0012FED4
x=&a: 453
a=&*a: 0012FED4
x=&*x: 453
a=&*a: 0012FED4
Press any key to continue

```

Fig.5.9

Pamja e ekranit pas ekzekutimit të programit pointX1

Këtu, përmes kombinimit të operatorëve **&**, te shprehja **a=&*a**, në fakt merret adresa e variablës **x** që ruhet te pointeri **a**. Kjo duket më e qartë, nëse shfrytëzohet forma e shprehjes ***(&a)**, ku me **&a** nënkuptohet adresa e pointerit **a**, kurse me operatorin ***** para saj, nga adresa në fjalë merret vlera, përkatësisht adresa që ruhet te pointeri **a**. Ngjashëm, adresa **a** e variablës **x** merret edhe përmes kombinimit të operatorëve **&** te shprehja **a=&*a**.

Si më sipër mund të analizohet edhe kombinimi i operatorëve **&**, te shprehja **x=&*x**, përmes së cilës merret vlera e variablës **x**.

Nga rezultatet e fitura më sipër, si përfundim mund të nxirret se operatorët **&** dhe *****, gjatë punës me pointerë, janë *operatorë inverzë mes vete*. Gjithashtu, si përmbledhëse mund të shkruhen shprehjet e raporteve mes operatorëve dhe adresës **a** të variablës **x**:

```
a=&x
x=*a
a=&*a
x=*&x
a=*&a
```

Kombinimet e operatorëve mund të shfrytëzohen gjatë shtypjes së adresave, gjatë shfrytëzimit të vlerave të variablave, ose dhe gjatë krahasimit të tyre.

Shembull

Programi **point8**, në të cilin te pointeri **a** ruhet adresa e variablës **x**. Pastaj shtypen dhe krahasohen vlerat **a** e ***&a**, si dhe **x** e ***&x**.

```
// Programi point8
#include <iostream>
using namespace std;
int main()
{
    int x,*a;
    x=837;
    a=&x;

    cout << "\nMeqë"
         << "\n\nVlera   a: "
         << a
         << "\nVlera *&a: "
         << *&a
         << "\n";
    cout << "\nRezultati i krahasimit të parë është ";

    if (a == *&a)
        cout << "true";
    else
        cout << "false";

    cout << "\n\nMeqë"
         << "\n\nVlera   x: "
         << x
         << "\nVlera *&x: "
         << *&x
         << "\n";
    cout << "\nRezultati i krahasimit të dytë është ";

    if (x == *&x)
```

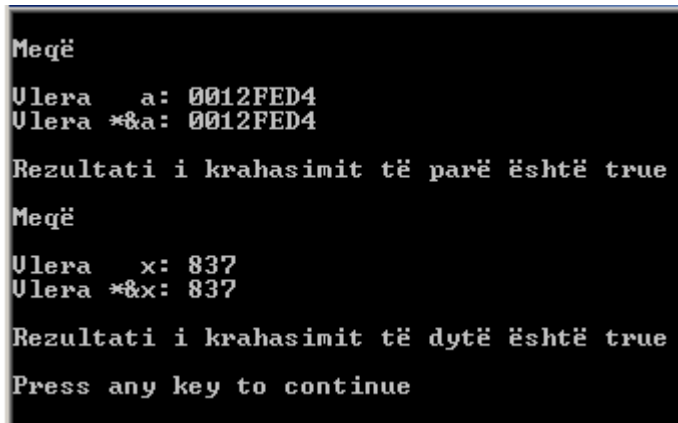
```

        cout << "true";
    else
        cout << "false";

    cout << "\n\n";
    return 0;
}

```

Nëse ekzekutohet programi i dhënë, rezultati që shtypet në ekran do të duket si në Fig.5.10.



```

Meqë
Vlera  a: 0012FED4
Vlera *a: 0012FED4

Rezultati i krahasimit të parë është true
Meqë
Vlera  x: 837
Vlera *x: 837

Rezultati i krahasimit të dytë është true
Press any key to continue

```

Fig.5.10
Pamja e ekranit pas
ekzekutimit të programit
point8

Llogaritjet përmes pointerëve

Gjatë llogaritjeve të ndryshme, përveç operimit direkt me variabla, mund të shfrytëzohet edhe *operimi me ndërmjetësimin e pointerëve*.

Shembull

Programi përmes së cilit, nëse $x > y$ gjendet shuma z e numrave x dhe y , përndryshe z llogaritet si diferencë e tyre. Vlerat e variablave x dhe y kompjuterit i jepen si vlera hyrëse përmes tastierës.

a. Llogaritjet direkte

```

// Programi pointA1a
#include <iostream>
using namespace std;
int main()
{
    int x,y,z;

```

```
cout << "\nVlera e variablës x: ";
cin >> x;

cout << "\nVlera e variablës y: ";
cin >> y;

if (x>y)
{
    z=x+y;
    cout << "\nRezultati i shumës z="
        << z;
}
else
{
    z=x-y;
    cout << "\nRezultati i diferencës z="
        << z;
}
cout << "\n\n";
return 0;
}
```

Përmes programit të dhënë, varësisht nga raporti i vlerave të variablave hyrëse **x** dhe **y**, llogaritet shuma:

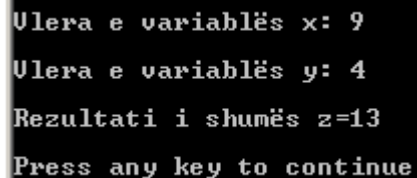
z=x+y;

përkatësisht diferenca e tyre:

z=x-y;

Këtu, siç shihet nga shprehjet e dhëna, gjatë llogaritjeve janë shfrytëzuar variablat e zakonshme **x** dhe **y**.

Për vlera hyrëse të caktuara, rezultati në ekran do të duket ashtu siç është dhënë në Fig.5.11.



```
Vlera e variablës x: 9
Vlera e variablës y: 4
Rezultati i shumës z=13
Press any key to continue
```

Fig.5.11

Pamja e ekranit pas ekzekutimit të programit pointA1a

b. Llogaritjet përmes pointerëve

```
// Programi pointA1b
#include <iostream>
using namespace std;
int main()
{
    int x,y,z,*a,*b;

    cout << "\nVlera e variablës x: ";
    cin >> x;

    a=&x;

    cout << "\nVlera e variablës y: ";
    cin >> y;

    b=&y;

    if (x>y)
    {
        z=*a+*b;
        cout << "\nRezultati i shumës z="
            << z;
    }
    else
    {
        z=*a-*b;
        cout << "\nRezultati i diferencës z="
            << z;
    }
    cout << "\n\n";
    return 0;
}
```

Këtu, gjatë llogaritjes së shumës **z** janë shfrytëzuar pointerët përkatës **a** dhe **b** të variablave **x** dhe **y**, si dhe shprehja:

$$z=*a+*b;$$

Njëkohësisht, llogaritja e diferencës së dy vlerave hyrëse **x** dhe **y** është përcaktuar përmes shprehjes:

$$z=*a-*b;$$

ku në vend të variablave **x** dhe **y** janë shfrytëzuar pointerët përkatës.

Nëse ekzekutohet programi i dhënë dhe përmes tastierës, si vlera hyrëse kompjuterit i jepen numrat **4** dhe **7**, rezultati në ekran do të duket si në Fig.5.12.

Fig.5.12

Pamja e ekranit pas ekzekutimit të programit pointA1b

```
Ulera e variablës x: 4
Ulera e variablës y: 7
Rezultati i diferencës z=-3
Press any key to continue
```

Pointerët mund të shfrytëzohen edhe gjatë llogaritjeve brenda unazave të ndryshme.

Shembull

Programi përmes së cilit tregohet shfrytëzimi i pointerit gjatë llogaritjes së shumës **s** të numrave natyrorë mes numrave **1** dhe **n**, por të cilët janë të plotpjesëtueshëm me numrin **x**.

a. Llogaritja direkte

```
// Programi point9a
#include <iostream>
using namespace std;
int main()
{
    int i,n,x,k;
    double s=0;
    cout << "\nUlera e variablës n: ";
    cin >> n;
    cout << "\nUlera e variablës x: ";
    cin >> x;

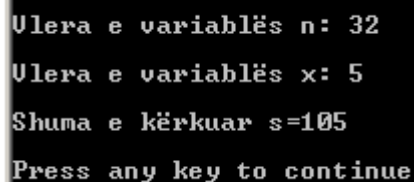
    for (i=1;i<=n;i++)
    {
        k=i%x;
        if (k==0)
            s=s+i;
    }

    cout << "\nShuma e kërkuar s="
        << s
        << "\n\n";
    return 0;
}
```

Në program, pasi përmes tastierës kompjuterit t'i jepen vlerat e variablave **n** dhe **x**, duke e shfrytëzuar unazën **for** realizohet shtimi i anëtarëve, të cilët e përcaktojnë vlerën e shumës. Por, me qëllim të zgjedhjes së vlerave të variablës **i** që plotpjesëtohen me **x**, shfrytëzohet shprehja:

```
k=i%x;
```

Kjo shprehje do të ketë vlerën **0** nëse **i** plotpjesëtohet me **x**, gjë që përdoret te komanda **if** për degëzimin përkatës. Kështu, p.sh., nëse përmes tastierës kompjuterit si vlera hyrëse për variablat **n** dhe **x** i jepen numrat **32** dhe **5**, rezultati në ekran do të duket si në Fig.5.13.



```
Vlera e variablës n: 32
Vlera e variablës x: 5
Shuma e kërkuar s=105
Press any key to continue
```

Fig.5.13

Pamja e ekranit pas ekzekutimit të programit point9a

Këtu, vlera e shumës është llogaritur duke i shtuar anëtarët që plotpjesëtohen me **5**, kështu:

$$s = 5 + 10 + 15 + 20 + 25 + 30 = 105$$

b. Llogaritje përmes pointerit

```
// Programi point9b
#include <iostream>
using namespace std;
int main()
{
    int i,n,x,k;
    double s,*a;

    a=&s;

    cout << "\nVlera e variablës n: ";
    cin >> n;
    cout << "\nVlera e variablës x: ";
    cin >> x;

    *a=0;
    for (i=1;i<=n;i++)
    {
        k=i%x;
```

```
        if (k==0)
            *a=*a+i;
    }

    cout << "\nShuma e kërkuar s="
         << s
         << "\n\n";
    return 0;
}
```

Në këtë rast, në vend të shprehjeve për vlerën fillestare:

```
s=0;
```

dhe për shtimin e anëtarëve të shumës:

```
s=s+i;
```

janë shfrytëzuar shprehjet:

```
*a=0;
```

dhe

```
*a=*a+i;
```

Kështu, gjatë llogaritjes së shumës, si para unazës ashtu edhe brenda saj, variablës **s** i qasemi përmes pointerit përkatës **a**. Pas ekzekutimit të programit në fjalë, rezultati në ekran nuk do të ndryshojë nga ai që u dha në *Fig.5.13*.

Vlera e llogaritur mund të shihet si rezultat në ekran edhe nëse kompjuterit i urdhërohet që ta shtypë vlerën ***a**.

Shembull

Versioni **point9c** i programit paraprak, në të cilin është urdhëruar shtypja brenda unazës **for** e vlerave parciale të shumës, duke i shtypur njëkohësisht vlerat **s** dhe ***a**.

```
// Programi point9c
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int i,n,x,k;
    double s,*a;
    char h[]="-----";

    a=&s;
```



```

cout << "\nVlera e variablës n: ";
cin >> n;
cout << "\nVlera e variablës x: ";
cin >> x;
cout << "\n      i      s      *a\n"
      << h
      << endl;
*a=0;
for (i=1;i<=n;i++)
{
    k=i%x;
    if (k==0)
    {
        *a=*a+i;
        cout << setw(5)
              << i
              << setw(8)
              << s
              << setw(8)
              << *a
              << endl;
    }
}

cout << h
      << "\nShuma e kërkuar s="
      << s
      << "\n\n";
return 0;
}

```

Pas ekzekutimit të programit të dhënë, rezultati në ekran do të duket si në Fig.5.14.



```

Vlera e variablës n: 32
Vlera e variablës x: 5

```

i	s	*a
5	5	5
10	15	15
15	30	30
20	50	50
25	75	75
30	105	105

```

Shuma e kërkuar s=105
Press any key to continue

```

Fig.5.14

Pamja e ekranit pas ekzekutimit të programit point9c

Siç shihet nga rezultati, gjatë shtimit të anëtarëve të veçantë, shuma **s** dhe vlera ***a** janë të barabarta, sepse në fakt kemi të bëjmë me rritjen e vlerës në lokacionin e njëjtë të memories.

Edhe variabla e unazës mund të merret si pointer.

Shembull

Versioni **point9d** i programit paraprak, në të cilin në vend të variablës **i** të unazës shfrytëzohet pointeri përkatës **j** në këtë variabël.

```
// Programi point9d
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int i, *j, n, x, k;
    double s=0, *a;
    char h[]="-----";
    cout << "\nVlera e variablës n: ";
    cin >> n;
    cout << "\nVlera e variablës x: ";
    cin >> x;
    cout << "\n      *j          s          *a\n"
         << h
         << endl;
    a=&s;
    j=&i;
    for (*j=1; *j<=n; *j=*j+1)
    {
        k=*j%x;
        if (k==0)
        {
            *a=*a+*j;
            cout << setw(5)
                 << *j
                 << setw(8)
                 << s
                 << setw(8)
                 << *a
                 << endl;
        }
    }

    cout << h
         << "\nShuma e kërkuar s="
         << s
```

```

        << "\n\n";
return 0;
}

```

Këtu, për dallim nga verzionet paraprake të programit, para unazës është shfrytëzuar shprehja:

```
j=&i;
```

përmes së cilës kompjuteri njoftohet se pointeri **j** do ta përmbajë adresën e variablës **i**, përkatësisht përmes kombinimit ***j** mund të merret vlera e variablës **i**. Por, duhet pasur kujdes në shfrytëzimin e kombinimit ***j**, para se të përcaktohet vlera e variablës **i**, meqë në momentin e shkruarjes së shprehjes në fjalë kjo vlerë nuk është përcaktuar.

Nëse ekzekutohet programi i dhënë, për vlerat hyrëse që u shfrytëzuan më sipër, rezultati do të jetë i njëjtë me atë që shihet në *Fig.5.14*, por në titull, në vend të variablës **i**, do të figurojë variabla ***j**.

Operimi me vlerat e pointerëve

Vlerat e pointerëve mund të rriten, të zvogëlohen, të krahasohen, ose përmes operatorit të barazimit, ato vlera edhe mund t'u shoqërohen pointerëve të tjerë.

Rritja dhe zvogëlimi i vlerave

Vlerat e pointerëve mund të rriten ose të zvogëlohen në rrugë të zakonshme, duke shfrytëzuar shprehje të formave:

```

p=p+k
p=p-k

```

pas së cilave, pointeri **p** rritet ose zvogëlohet për **k · b** bajtë, ku **b** e paraqet numrin e bajtëve për pointerin e tipit të zgjedhur.

Shembull

Programi **pointZ1**, përmes së cilit tregohet rritja e vlerës së pointerit **a** brenda një unaze.

```

// Programi pointZ1
#include <iostream>
using namespace std;
int main()
{

```

```

int i,x,*a;
a=&x;
cout << "\n    i        a"
      << "\n-----"
      << "\n";
i=1;
for (i=1;i<=6;i++)
{
    cout << "    "
          << i
          << "    "
          << a
          << "\n";
    a=a+1;
}
cout << "\n";

return 0;
}

```

Në program, përmes shprehjes **a=a+1**, e cila është vendosur brenda unazës **for**, vlera e pointerit **a** rritet me hapin **1**. Meqë pointeri **a** është i tipit **int**, përmes shprehjes në fjalë, adresa që ruhet në pointer rritet për **4** bajtë. Kjo shihet edhe nga tabela e dhënë në Fig.5.15, e cila shtypet gjatë ekzekutimit të programit.

Fig.5.15

Pamja e ekranit pas ekzekutimit të programit *pointZ1*

i	a
1	0012FEC8
2	0012FECC
3	0012FED0
4	0012FED4
5	0012FED8
6	0012FEDC

Press any key to continue

Këtu, vlera fillestare e adresës **0012FEC8** e cila ruhet te pointeri **a**, është e barabartë me adresën e variablës **x** dhe përcaktohet me shprehjen:

```
a=&x;
```

Pas kësaj, përmes komandës:

```
a=a+1;
```

adresës **a** i shtohen **4** bajtë:

```
0012FEC8
+      4
```

0012FECC

Nënkuptohe se, nëse variabla **x** zgjedhet e tipit **double**, edhe pointeri përkatës **a** duhet të zgjedhet i tipit të njëjtë. Gjatë kësaj, përmes shprehjes **a=a+1**, adresat e pointerit do të rriten nga **8** bajtë, sepse të dhënat e tipit **double** ruhen në **8** bajtë.

Shoqërimi dhe krahasimi i vlerave

Përmes operatorit të zakonshëm të barazimit:

p=q

pointerit **p** mund t'i shoqërohet vlera e pointerit **q**, por vetëm nëse pointerët janë të tipeve të njëjtë. Gjithashu, duke i shfrytëzuar operatorët përkatës, përmes komandës **if** mund të krahasohen mes vete vlerat e dy pointerëve.

Shembull

Programi **pointZ2**, përmes së cilit tregohet barazimi dhe krahasimi i vlerave të pointerëve **p** dhe **q**, pasi fillimisht te pointeri **p** të ruhet adresa e variablës **x**.

```
// Programi pointZ2
#include <iostream>
using namespace std;
int main()
{
    double x, *p, *q;
    x=23.4;
    p=&x;
    cout << "\nAdresa te pointeri p: "
          << p;
    q=p;
    cout << "\n\nAdresa te pointeri q: "
          << q;
    if (p==q)
        cout << "\n\nPointerët janë të barabartë";
    else
        cout << "\n\nPointerët nuk janë të barabartë";
    cout << "\n\n";
    return 0;
}
```

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si ai që është dhënë në Fig.5.16.

Fig.5.16
 Pamja e ekranit pas ekzekutimit të
 programit `pointZ2`

```
Adresa te pointeri p: 0012FECC
Adresa te pointeri q: 0012FECC
Pointerët janë të barabartë
Press any key to continue
```

Në programin e dhënë, krahasimi për barazim mes pointerëve mund të bëhet edhe duke e shkruar komandën **if** në këtë formë:

```
if (p!=q)
    cout << "\n\nPointerët nuk janë të barabartë";
else
    cout << "\n\nPointerët janë të barabartë";
```

Gjithashtu, mund të shfrytëzohen edhe operatorët e tjerë të krahasimit të pointerëve, të cilët përdoren gjatë krahasimit të variablave të zakonshme.

Pointerët gjatë operimit me fusha

Për operim me anëtarët e fushave mund të përdoren edhe pointerët, ashtu siç është shpjeguar në vijim.

Operimi me anëtarët e vektorëve

Vektorët mund të shfrytëzohen duke i përdorur pointerët, në të cilët ruhen adresat e anëtarëve të tyre, ose vetëm adresa e anëtarit të parë, ku fillon vendosja e tyre në memorien e kompjuterit. Gjithashtu, gjatë operimit me anëtarët e vektorëve përmes indekseve përkatëse, edhe indeksat mund të merren si pointerë.

Pointerët në anëtarët e vektorëve

Nëse duam që gjatë operimit, p.sh., me anëtarët e vektorit **v**, atyre t'u qasemi me ndërmjetësimin e pointerit **p**, për përcjelljen e adresës së anëtarit të **i**-të te pointeri mund të shfrytëzohet shprehja:

```
p=&v[i];
```

ku, p.sh., indeksi **i** e paraqet variablën e unazës përkatëse.

Shembull Programi **pointA1**, përmes së cilit tregohet shfrytëzimi i pointerit **a** gjatë shtypjes së anëtarëve të vektorit **Z(n)**.

```
// Programi pointA1
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int n=5;
    int *a,i,Z[n]={27,12,-23,9,35};
    char h[]="-----";

    cout << "\n Indeksi   Anëtari   Adresa\n"
         << h
         << endl;

    for (i=0;i<n;i++)
    {
        a=&Z[i];
        cout << setw(6)
              << i
              << setw(10)
              << Z[i]
              << setw(12)
              << a
              << endl;
    }
    cout << h
         << "\n";
    return 0;
}
```

Këtu, brenda unazës **for** është vendosur shprehja:

```
a=&Z[i];
```

përmes së cilës adresa e anëtarit të **i**-të të vektorit vendoset te pointeri **a**.

Njëkohësisht, shtypen vlerat e variablave **i**, **Z[i]** dhe **a**, për ta fituar tabelën me rezultate, e cila është dhënë në

Fig.5.17.

Fig.5.17

Pamja e ekranit pas ekzekutimit të programit **pointA1**

Indeksi	Anëtari	Adresa
0	27	0012FEA0
1	12	0012FEA4
2	-23	0012FEA8
3	9	0012FEAC
4	35	0012FEB0

Press any key to continue		

Siç shihet nga tabela, anëtar i parë është vendosur tek adresa **0012FEA0**, e cila këtu është dhënë në formë heksadecimale. Pastaj, adresa e anëtarit të dytë rritet për **4**, përkatësisht gjendet përmes mbledhjes:

$$\begin{array}{r} 0012FEA0 \\ + 4 \\ \hline 0012FEA4 \end{array}$$

Kjo është rezultat i asaj se për vendosjen e anëtarit të parë **A[0]** të vektorit shfrytëzohen **4**-bajtë, meqë vektori është deklaruar i tipit **int**. Ngjashëm kompjuteri vepron edhe gjatë zgjedhjes së adresave të anëtarëve të tjerë të vektorit.

Në hapësirën memoruese, anëtarët e vektorit **A** vendosen ashtu siç shihet në Fig.5.18.

00000000		
00000001		
...		
0012FEA0	27	Z[0]
0012FEA4	12	Z[1]
0012FEA8	-23	Z[2]
0012FEAC	9	Z[3]
0012FEB0	35	Z[4]
...		

Fig.5.18 Pamja e hapësirës memoruese pas vendosjes së anëtarëve të vektorit

Pointeri në anëtarin e parë të vektorit

Anëtarëve të veçantë të vektorit **v** mund t'u qasemi përmes pointerit përkatës **p**, edhe nëse pointerit i shoqërohet adresa ku fillon vendosja e anëtarëve të vektorit:

p=v;

e cila në fakt e paraqet adresën e anëtarit të parë të tij.

Shembull

Versioni **pointA2** i programit **pointA1**, tek i cili për deklarimin e pointerit **a** si tregues në vektorin **Z(n)** shfrytëzohet forma e parë që u dha më sipër.

```
// Programi pointA2
#include <iostream>
```



```

#include <iomanip>
using namespace std;
int main()
{
    const int n=5;
    int *a,i,Z[n]={27,12,-23,9,35};
    char h[]="-----";

    cout << "\n Indeksi   Anëtari   Adresa\n"
         << h
         << endl;
    a=Z;
    for (i=0;i<n;i++)
    {
        cout << setw(6)
              << i
              << setw(10)
              << Z[i]
              << setw(12)
              << a+i
              << endl;
    }

    cout << h
         << "\n";
    return 0;
}

```

Në fillim të programit, përmes komandës:

```
a=Z;
```

te pointeri **a** përcillet adresa ku fillon vendosja e vektorit **Z** në memorien e kompjuterit. Si rezultat, p.sh., te pointeri do të ruhet adresa **0012FEA0**, ku vendoset anëtari i parë **Z[0]** i vektorit, gjë që mund të vërtetohet nëse shfrytëzohet komanda për shtypje të vlerës së variablës **a**.

Pas kësaj, rezultati i ekzekutimit të programit do të jetë i njëjtë me atë që u dha në Fig.5.17. Me qëllim të shtypjes së adresave në të cilat fillon vendosja e anëtarëve të veçantë të vektorit, te komanda për shtypje, vlerës fillestare të pointerit **a** i shtohet edhe indeksi i anëtarit përkatës (**a+i**). Kjo do të thotë se gjatë rritjes së vlerës së pointerit për **1**, adresa që ruhet në të rritet për **4**, gjë që i përgjigjet numrit të bajtëve në të cilët ruhet një anëtar i vektorit, meqë ai është deklaruar i tipit **int**. Kështu, p.sh., nëse jemi pozicionuar tek anëtari i dytë i vektorit, pasi indeksi i tij është **1**, me rritjen për **1** përmes shprehjes **a+1**, te pointeri **a** do të ruhet adresa **0012FEA8**, e cila i përket anëtarit të tretë **Z[2]** të

vektorit. Kompjuteri vepron ngjashëm edhe gjatë zbritjes së vlerës së pointerit, përkatësisht pas zbritjes **a-1**, kalohet në adresën e anëtarit paraprak të vektorit.

Adresa e anëtarit të parë të vektorit mund të vendoset te pointeri **p**, edhe nëse shfrytëzohet shprehja:

```
p=&v[0];
```

Kështu, te shembulli i programit të dhënë më sipër, për inicializimin e pointerit **a** që të tregojë te vektori, përkatësisht tek anëtari i parë i tij, në vend të komandës:

```
a=Z;
```

duhet të shfrytëzohet komanda:

```
a=&Z[0];
```

Pointeri në anëtarin e parë të vektorit mund të shfrytëzohet edhe gjatë llogaritjeve të ndryshme në të cilat marrin pjesë anëtarët e vektorëve.

Shembull

Programi **pointL1**, përmes së cilit tregohet shfrytëzimi i pointerit **a** gjatë llogaritjes së shumës së anëtarëve të vektorit **R(n)**.

```
// Programi pointL1
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int n=6;
    int *a,i,R[n]={15,8,4,17,6,29};
    double s;
    char h[]="-----";
    cout << "\n Indeksi Anëtari Shuma  Adresa\n"
         << h
         << endl;
    a=&R[0];
    s=0;
    for (i=0;i<n;i++)
    {
        s=s+*(a+i);
        cout << setw(6)
              << i
              << setw(8)
              << R[i]
```

```

        << setw(7)
        << s
        << setw(10)
        << &R[i]
        << endl;
    }
    cout << h
        << "\nShuma përfundimtare s="
        << s
        << "\n\n";
    return 0;
}

```

Nëse ekzekutohet programi i dhënë, rezultati që shtypet në ekran do të duket si në Fig.5.19, ku në dy kolonat e fundit të tabelës janë shtypur vlerat e shumave parciale dhe adresat e anëtarëve të vektorit.

Indeksi	Anëtar	Shuma	Adresa
0	15	15	0012FE9C
1	8	23	0012FEA0
2	4	27	0012FEA4
3	17	44	0012FEA8
4	6	50	0012FEAC
5	29	79	0012FEB0

Shuma përfundimtare s=79

Press any key to continue

Fig.5.19

Pamja e ekranit pas ekzekutimit të programit `pointL1`

Në program, duke e shfrytëzuar shprehjen:

```
a=&R[0];
```

pointeri **a** është inicializuar me adresën e anëtarit të parë të vektorit, që siç shihet edhe më sipër, është adresa **0012FE9C**. Pastaj, brenda unazës në të cilën llogaritet shuma **s**, anëtarët e veçantë të vektorit i shtohen shumës, duke e shfrytëzuar shprehjen:

```
s=s+*(a+i);
```

Këtu, në fakt, përmes pjesës së shprehjes ***(a+i)**, shumës i shtohet vlera e anëtarit **R[i]**, e cila gjendet në lokacionin me adresë **a+4·i**, ku numri **4** lidhet me atë se pointeri është deklaruar i tipit **int** dhe ka të bëjë me të dhëna që ruhen në **4**-bajtë, ashtu siç u shpjegua më parë. Efekti i shprehjes në fjalë është i njëjtë sikur të shfrytëzohej shprehja:

```
s=s+R[i];
```

Kështu, p.sh., adresa e anëtarit të fundit të vektorit llogaritet duke ia shtuar adresës **0012FE9C**, në të cilën fillon vendosja e anëtarit të parë, vlerën heksadecimale **14** të numrit dhjetor **$4 \cdot i = 4 \cdot 5 = 20$** :

```
0012FE9C
+      14
-----
0012FEB0
```

Indekset si pointerë

Indekset e anëtarëve të vektorëve mund të shfrytëzohen për përcaktimin e pointerëve në anëtarët e tyre, pa përdorur një variabël të veçantë të tipit pointer.

Shembull

Programi **pointL2**, përmes së cilit tregohet shfrytëzimi i pointerit **a** gjatë radhitjes së anëtarëve të vektorit **R(n)** sipas madhësisë, prej anëtarit më të madh kah ai më i vogli.

```
// Programi pointL2
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int n=6;
    int i,j,*a,x,R[n]={15,8,4,17,6,12};

    cout << "\nVektori i paradhitur\n"
          << "\n R={";
    for (i=0;i<n;i++)
        cout << setw(4)
              << R[i];
    cout << " }\n";

    a=&R[0];

    for (i=1;i<n-1;i++)
        for (j=i+1;j<n;j++)
            if (*(a+i) < *(a+j))
            {
                x=*(a+i);
                *(a+i)=*(a+j);
                *(a+j)=x;
            }
}
```

```

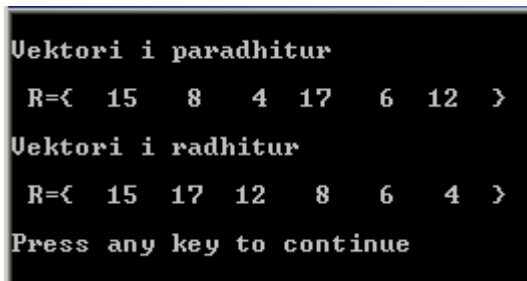
    cout << "\nVektori i radhitur\n"
        << "\n R={";

    for (i=0;i<n;i++)
        cout << setw(4)
            << *(a+i);
    cout << " }\n\n";
return 0;
}

```

Këtu, krahasimi i anëtarëve të vektorit përmes komandës **if** realizohet duke i shfrytëzuar adresat përkatëse, të cilat përcaktohen përmes adresës së anëtarit të parë të tij, e cila ruhet te pointeri **a**. Për këtë qëllim shfrytëzohen indeksat e tyre **i** dhe **j**, përkatësisht variablat e dy unazave, brenda së cilave krahasohen anëtarët e vektorit. Varësisht nga rezultati i krahasimit, në hapësirën memoruese ndërrohen vendet e anëtarëve që krahasohen, me ndërmjetësimin e variablës **x**.

Rezultati që shtypet në ekran pas ekzekutimit të programit të dhënë do të duket si në Fig.5.20.



```

Vektori i paradhitur
R={ 15  8  4 17  6 12 }
Vektori i radhitur
R={ 15 17 12  8  6  4 }
Press any key to continue

```

Fig.5.20

Pamja e ekranit pas ekzekutimit të programit **pointL2**

Gjatë shfrytëzimit të indekseve të anëtarëve të vektorëve për përcaktimin e pointerëve në vlerat e tyre, mund të përdoret edhe një variabël e veçantë e tipit pointer.

Shembull

Programi **pointL3**, përmes së cilit gjendet shuma **s** e anëtarëve të vektorit **R(n)** me indeks tek, nëse gjatë llogaritjes edhe indeksat deklarohen si pointerë.

```

// Programi pointL3
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{

```

```

const int n=6;
int *a,i,*k,R[n]={15,8,4,17,6,29};
double s;
char h[]="-----";

cout << "\n Indeksi   Anëtari       Shuma\n"
      << h
      << endl;

a=&R[0];

s=0;

for (i=1;i<n;i=i+2)
{
    k=&i;
    s=s+*(a+k);

    cout << setw(6)
          << i
          << setw(10)
          << R[*k]
          << setw(12)
          << s
          << endl;
}
cout << h
     << "\nShuma përfundimtare s="
     << s
     << "\n\n";
return 0;
}

```

Këtu, te shprehja për llogaritjen e shumës:

```
s=s+*(a+k);
```

në vend të indeksit, është shfrytëzuar pointeri **k** në indeksin **i**. Pointeri **k** inicializohet brenda unazës, sepse inicializimi jashtë unazës mund të bëhet vetëm për pointerin e vektorit.

Pas ekzekutimit të programit të dhënë, rezultati në ekran do të duket si në Fig.5.21.

Fig.5.21

Indeksi	Anëtari	Shuma
1	8	8
3	17	25
5	29	54
Shuma përfundimtare s=54		
Press any key to continue		

Pamja e ekranit pas ekzekutimit të programit pointL3

Pointerët pa i deklaruar

Operimi me anëtarët e vektorëve mund të bëhet edhe me kombinimin e operatorëve ***&**, pa e deklaruar ndonjë pointer të veçantë.

Shembull

Programi **pointL4**, përmes së cilit gjendet anëtari më i vogël **x** në vektorin e dhënë **R(n)**, duke e shfrytëzuar kombinimin e operatorëve ***&**.

```
// Programi pointL4
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int n=6;
    int i,x,R[n]={15,8,4,17,6,12};

    char h[]="-----";

    cout << "\n Indeksi    Anëtari\n"
         << h
         << endl;

    for (i=0;i<n;i++)
        cout << setw(5)
              << i
              << setw(12)
              << R[i]
              << endl;

    cout << h
         << endl;

    x=&R[0];
    for (i=1;i<n;i++)
        if (&R[i] < x)
            x=&R[i];
    cout << "Anëtari më i vogël x="
         << x
         << "\n\n";
    return 0;
}
```

Këtu, përmes shprehjes së formës:

```
x=&R[0];
```

variablës **x** i shoqërohet vlera e anëtarit të parë të vektorit, sepse me **&R[0]** merret adresa e anëtarit në fjalë, kurse me ***&R[0]** - vlera e tij. Ngjashëm kompjuteri e nënkupton edhe shprehjen ***&R[i]**, e cila paraqitet dy herë brenda unazës.

Pas ekzekutimit të programit, rezultati në ekran do të duket ashtu siç është dhënë në Fig.5.22.

Indeksi	Anëtari
0	15
1	8
2	4
3	17
4	6
5	12
Anëtari më i vogël x=4	
Press any key to continue	

Fig.5.22

Pamja e ekranit pas ekzekutimit të programit **pointL4**

Edhe variablat e unazave mund të përcaktohen me kombinimin e operatorëve ***&**.

Shembull

Programi **pointL5**, përmes së cilit gjendet numri **k** i anëtarëve tekë të vektorit **F(n)**, duke e shfrytëzuar për të gjitha variablat kombinimin e operatorëve ***&**.

```
// Programi pointL5
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int n=6;
    int i,k,F[n]={17,8,11,5,10,3};
    char h[]="-----";

    cout << "\n Indeksi   Anëtari\n"
         << h
         << endl;

    *&k=0;
    for (*&i=0;*&i<n;*&i=*&i+1)
    {
```



```

        if ((*&F[*&i]%2) != 0)
            *&k=*&k+1;
        cout << setw(6)
              << *&i
              << setw(10)
              << F[*&i]
              << setw(12)
              << endl;
    }
    cout << h
         << "\nNumri i anëtarëve tekë k="
         << k
         << "\n\n";
    return 0;
}

```

Edhe në këtë program nuk është deklaruar asnjë pointer. Por, pointerët shfrytëzohen duke i futur në përdorim kombinimet e operatorëve ***&**, përfshirë edhe variablën e unazës. Rezultati në ekran, pas ekzekutimit të programit të dhënë, do të duket si në *Fig.5.23*.

Indeksi	Anëtarë
0	17
1	8
2	11
3	5
4	10
5	3

Numri i anëtarëve tekë k=4	
Press any key to continue	

Fig.5.23

Pamja e ekranit pas ekzekutimit të programit pointL5

Nëse në programin e mësipërm duam që të shtypen vetëm anëtarët tekë të vektorit, unaza përkatëse do të shkruhet:

```

for (*&i=0;*&i<n;*&i=*&i+1)
    if ((*&F[*&i]%2) != 0)
    {
        *&k=*&k+1;
        cout << setw(6)
              << *&i
              << setw(10)
              << F[*&i]
              << setw(12)
              << endl;
    }

```

Pas këtyre ndryshimeve, rezultati në ekran do të duket ashtu siç shihet në Fig.5.24.

Fig.5.24

Pamja e ekranit pas ekzekutimit të programit `pointL5` me ndryshimet e theksuara

Indeksi	Anëtarë
0	17
2	11
3	5
5	3
Numri i anëtarëve tekë k=4	
Press any key to continue	

Operimi me anëtarë të matricave

Gjatë operimit me anëtarët e matricave, si edhe te vektorët, mund të shfrytëzohen pointerët në të cilët ruhen adresat e anëtarëve, ose vetëm adresat e anëtarëve të parë të tyre, në të cilat fillojnë vendosjet e matricave në memorien e kompjuterit. Gjithashtu, edhe indeksat e anëtarëve të matricave mund të merren si pointerë, me ndërmjetësimin e variablave të veçanta të tipit pointerë, ose edhe direkt.

Në vijim, përmes shembujve, do të tregohen opsionet e mundshme të shfrytëzimit të pointerëve gjatë operimit me anëtarët e matricave.

Shtypja e anëtarëve të matricave

Çfarëdo operacioni që të ndërmerret mbi anëtarët e matricave, patjetër duhet të shfrytëzohen dy unaza të veçanta, përmes së cilave përcaktohen indeksat e anëtarëve të tyre. Gjatë shtypjes së anëtarëve të matricave, me qëllim që matricat në ekran ta kenë pamjen e zakonshme, pas shtypjes së çdo rreshti të tyre duhet të urdhërohet kalimi në një rresht të ri.

Shembull

Versione të programit përmes së cilit shtypen anëtarët e matricës së dhënë **A(m, n)**, të shkruar në formë të zakonshme dhe në format e tjera të mundshme me përdorim të pointerëve.

a. Forma e zakonshme

```
// Programi pointM1a
#include <iostream>
#include <iomanip>
using namespace std;
```

```

int main()
{
    const int m=3,n=4;
    int A[m][n]={5,4,7,-2},
                {3,1,-4,9},
                {1,2,-3,6}};

    int i,j;

    cout << "\n Matrica A\n\n";

    for (i=0;i<m;i++)
    {
        for (j=0;j<n;j++)
        {
            cout << setw(5)
                << A[i][j];
        }
        cout << endl;
    }

    cout << "\n";
    return 0;
}

```

Rezultati në ekran, pas ekzekutimit të programit të dhënë, do të duket si në Fig.5.25.

Fig.5.25

Pamja e ekranit pas ekzekutimit të programit pointM1a

```

Matrica A
  5    4    7   -2
  3    1   -4    9
  1    2   -3    6
Press any key to continue

```

b. Pointeri në anëtarin e parë të matricës

Sikurse te vektorët, edhe te matricat mund të operohet me anëtarët e tyre, duke e shfrytëzuar pointerin i cili tregon në anëtarin e parë të saj. Kështu, p.sh., adresa e anëtarit të parë të matricës **v** mund të vendoset te pointeri **p**, nëse shfrytëzohet shprehja:

```
p=&v[0][0];
```

Shfrytëzimi i pointerit në fjalë, i shënuar si **a**, shihet në versionin vijues të programit për shtypje të matricës **R**.

```
// Programi pointM1b
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int m=3,n=4;
    int A[m][n]={{5,4,7,-2},
                 {3,1,-4,9},
                 {1,2,-3,6}};
    int i,j,k=0,*a;

    cout << "\n Matrica A\n\n";

    a=&A[0][0];

    for (i=0;i<m;i++)
    {
        for (j=0;j<n;j++)
        {
            cout << setw(5)
                  << *(a+k);
            k=k+1;
        }
        cout << endl;
    }

    cout << "\n";
    return 0;
}
```

Këtu, pointeri **a** është inicializuar fillimisht me adresën e anëtarit të parë të matricës:

```
a=&A[0][0];
```

Pastaj, për t'i shtypur vlerat e veçanta të anëtarëve të matricës, shfrytëzohet shprehja:

```
cout << setw(5)
      << *(a+k);
```

ku **k** e paraqet *numërorin e anëtarëve të matricës*. Në fakt, me shprehjen ***(a+k)** kompjuterit i urdhërohet që ta shtypë vlerën e cila gjendet tek adresa **a**, por e rritur për **k**-herë nga **4** bajtë.

Adresat e anëtarëve të matricës **R** në memorien e kompjuterit, p.sh., mund të duken ashtu siç është dhënë në Fig.5.26.

00000000		
00000001		
...		
0012FF1C	5	A[0][0]
0012FF20	4	A[0][1]
0012FF24	7	A[0][2]
0012FF28	-2	A[0][3]
0012FF3C	3	A[1][0]
...	...	...
0012FF40	2	A[2][1]
0012FF44	-3	A[2][2]
0012FF48	6	A[2][3]

Fig.5.26 Pamja e hapësirës memoruese pas vendosjes së anëtarëve të matricës

Pas ekzekutimit të programit të dhënë, rezultati do ta ketë pamjen e njëjtë me atë që është dhënë në Fig.5.25.

c. Anëtarët e matricës si pointerë të padeklaruar

```
// Programi pointM1c
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int m=3,n=4;
    int A[m][n]={5,4,7,-2},
                {3,1,-4,9},
                {1,2,-3,6}};

    int i,j;

    cout << "\n Matrica A\n\n";

    for (i=0;i<m;i++)
    {
        for (j=0;j<n;j++)
        {
            cout << setw(5)
                  << *&A[i][j];
        }
        cout << endl;
    }

    cout << "\n";
}
```

```
return 0;
}
```

Këtu, nuk është deklaruar asnjë pointer, por gjatë shtypjes së anëtarëve të matricës shfrytëzohet kombinimi i operatorëve ***&**, efekti i të cilëve është shpjeguar më parë. Edhe në këtë rast, pas ekzekutimit të programit të dhënë, rezultati që shtypet do të jetë i njëjtë me atë që u dha në *Fig.5.25*.

d. Të gjitha variablat si pointerë të padeklaruar

```
// Programi pointM1d
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int m=3,n=4;
    int A[m][n]={5,4,7,-2},
                {3,1,-4,9},
                {1,2,-3,6}};

    int i,j;
    cout << "\n Matrica A\n\n";

    for (*&i=0;*&i<m;*&i=*&i+1)
    {
        for (*&j=0;*&j<n;*&j=*&j+1)
        {
            cout << setw(5)
                  << *&A[*&i][*&j];
        }
        cout << endl;
    }

    cout << "\n";
    return 0;
}
```

Në fillim të programit, edhe në këtë version asnjë variabël nuk është deklaruar si pointer. Por, gjatë zgjedhjes së vlerave të variablave të unazave **i** e **j**, si dhe gjatë shtypjes së anëtarëve të matricës **A** shfrytëzohen kombinimet e operatorëve ***&**, ashtu siç është shpjeguar edhe më lart. Pas ekzekutimit të programit të dhënë, rezultati do të duket si ai që është dhënë në *Fig.5.25*.

Gjetja e anëtarit të caktuar në matricë

Nga anëtarët e matricës së dhënë mund të gjendet anëtari më i madh, anëtari më i vogël, anëtari më i vogël ose më i madh për nga vlera absolute, ose edhe indeksat e anëtarëve në fjalë. Gjithashtu, mund të gjenden anëtarët në fjalë

në kufij të caktuar vlerash, p.sh., vetëm në grumbullin e anëtarëve me vlera pozitive, ose të anëtarëve me vlera negative etj.

Shembull

Programi **pointM2**, përmes së cilit gjendet anëtari më i madh për nga vlera absolute të matrica e dhënë **A(m, n)**, si dhe indeksset përkatëse (pozicioni i tij në matricë).

```
// Programi pointM2
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int m=3,n=4;
    int A[m][n]={5,4,7,-2,
                {3,1,-4,9},
                {1,2,-3,6}};
    int i,j,g,h,x;

    cout << "\n Matrica A\n\n";

    x=abs(*&A[0][0]);
    g=0;
    h=0;

    for (i=0;i<m;i++)
    {
        for (j=0;j<n;j++)
        {
            if (abs(*&A[i][j]) > x)
            {
                x=abs(*&A[i][j]);
                g=i;
                h=j;
            }

            cout << setw(5)
                 << *&A[i][j];
        }
        cout << endl;
    }

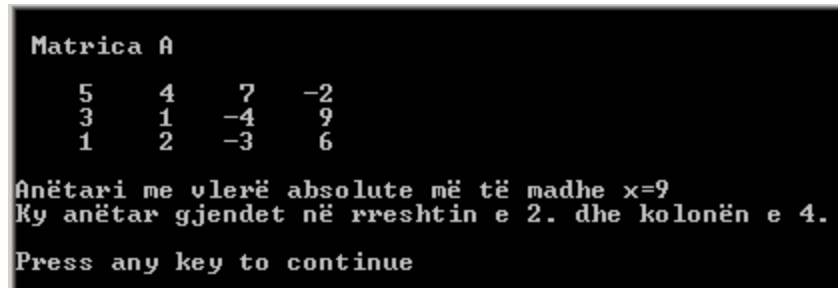
    cout << "\nAnëtari me vlerë absolute më të madhe x="
         << x;
    cout << "\nKy anëtar gjendet në rreshtin e "
         << g+1
```

```

        << ". dhe kolonën e "
        << h+1
        << ".\n\n";
return 0;
}

```

Në program, për operim me anëtarët e matricës shfrytëzohet kombinimi i operatorëve ***&**. Kurse indekset **i** dhe **j** të anëtarëve të matricës janë shkruar si variabla të zakonshme. Rezultati në ekran, pas ekzekutimit të programit të dhënë, do të duket ashtu siç është dhënë në Fig.5.27.



```

Matrica A
5    4    7   -2
3    1   -4    9
1    2   -3    6

Anëtari me vlerë absolute më të madhe x=9
Ky anëtar gjendet në rreshtin e 2. dhe kolonën e 4.
Press any key to continue

```

Fig.5.27 Pamja e ekranit pas ekzekutimit të programit **pointM2**

Formimi i vektorit nga anëtarët e matricës

Anëtarët e matricave, ose vetëm ato me vlera të caktuara, me qëllim të shfrytëzimit të mëtejshëm, mund të vendosen në vektor.

Shembull

Programi **pointM3**, përmes së cilit nga anëtarët e matricës së dhënë **A(m,n)** formohet vektori **B**, duke i marrë vetëm ata që janë më të mëdhenj se **2** dhe më të vegjël se **8**.

```

// Programi pointM3
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int m=3,n=4;
    int A[m][n]={5,4,7,-2},
              {3,1,-4,9},
              {1,2,-3,6}};
    int i,j,k,B[m*n],*a,*b;

    cout << "\n Matrica A\n\n";

```



```

k=-1;

for (i=0;i<m;i++)
{
    for (j=0;j<n;j++)
    {
        a=&A[i][j];
        if ((*a>2) && (*a<8))
        {
            k=k+1;
            b=&B[k];
            *b=*a;
        }

        cout << setw(5)
              << *a;
    }
    cout << endl;
}

cout << "\nVektori i formuar"
      << "\n\nB={";
for (i=0;i<k;i++)
    cout << setw(4)
          << *&B[i];
cout << "  }"
     << "\n\n";
return 0;
}

```

Në programin e dhënë, gjatë operimit me anëtarët e matricës **A** dhe vektorit **B** janë shfrytëzuar pointerët **a** dhe **b**, të deklaruar në fillim të programit. Me qëllim të përcjelljes së adresave të anëtarëve të matricës **A** te pointeri **a**, brenda unazave është shfrytëzuar shprehja:

```
a=&A[i][j];
```

Pastaj, ky pointer është përdorur te komanda **if**:

```
if ((*a>2) && (*a<8))
```

me qëllim të zgjedhjes së anëtarëve që kanë vlera në kufijtë mes vlerave **2** dhe **8**. Kur gjendet një anëtar që i plotëson dy kushtet e dhëna, së pari rritet numërori i anëtarëve që përfshihen në vektor:

```
k=k+1;
```

dhe pastaj, për vendosjen e anëtarit të gjetur në vektor, së pari përcaktohet pointeri përkatës:

```
b=&B[k];
```

dhe në adresën që ruhet te ky pointer, përcillet vlera e matricës **A**, duke e lexuar adresën përkatëse te pointeri **a**:

```
*b=*a;
```

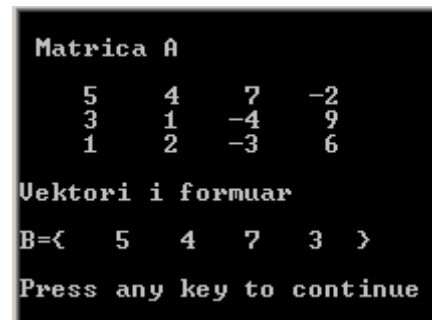
Në fund të unazës së jashtme është paraparë shtypja e anëtarëve të matricës **A**, me ndërmjetësimin e pointerit **a**:

```
cout << setw(5)
      << *a;
```

Gjatë shtypjes së anëtarëve të vektorit të formuar **B** në fund të programit shfrytëzohet kombinimi i operatorëve ***&**, në këtë mënyrë:

```
cout << setw(4)
      << *&B[i];
```

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket ashtu siç është dhënë në Fig.5.28.



```
Matrica A
  5   4   7  -2
  3   1  -4   9
  1   2  -3   6

Vektori i formuar
B={  5   4   7   3 }

Press any key to continue
```

Fig.5.28

Pamja e ekranit pas ekzekutimit të programit *pointM3*

Shuma e anëtarëve të matricës

Pointerët mund të shfrytëzohen edhe gjatë llogaritjes së shumës së të gjithë anëtarëve të matricës, ose të shumës së anëtarëve të caktuar të saj, duke i mbledhur vlerat reale të anëtarëve, ose vlerat absolute të tyre.

Shembull

Programi **pointM4**, përmes së cilit gjendet shuma **s** e vlerave

 absolute të anëtarëve të matricës së dhënë **A(m, n)**.

```
// Programi pointM4
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int m=3,n=4;
    int R[m][n]={{5,4,7,-2},
                 {3,1,-4,9},
                 {1,2,-3,6}};
    int i,j,k=0,s=0,*a;

    cout << "\n Matrica R\n\n";

    a=&R[0][0];

    for (i=0;i<m;i++)
    {
        for (j=0;j<n;j++)
        {
            cout << setw(5)
                  << R[i][j];
            s=s+abs(*(a+k));
            k=k+1;
        }
        cout << endl;
    }

    cout << "\nShuma e llogaritur s="
          << s
          << "\n\n";
    return 0;
}
```

Në fillim të programit, përmes shprehjes:

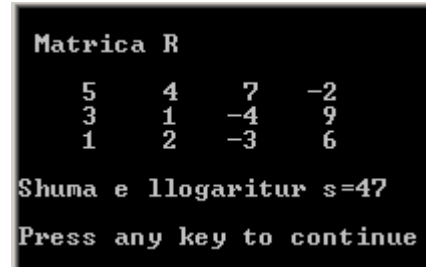
```
a=&R[0][0];
```

te pointeri **a** vendoset adresa e anëtarit të parë të matricës. Pastaj, për shtimin e vlerës absolute të anëtarëve të matricës te variabla e shumës **s**, shfrytëzohet shprehja:

```
s=s+abs(*(a+k));
```

Këtu, përmes pjesës së shprehjes **abs(*(a+k))**, shumës i shtohet vlera absolute e anëtarit të **k**-të të matricës. Vlera e numërorit **k** në fakt i shtohet adresës që ruhet te pointeri **a**, e shumëzuar me **4** (aq sa bajtë shfrytëzohen për vendosjen e anëtarëve të matricës, meqë janë zgjedhur të tipit **int**).

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.5.29.



```
Matrica R
5    4    7   -2
3    1   -4    9
1    2   -3    6
Shuma e llogaritur s=47
Press any key to continue
```

Fig.5.29

Pamja e ekranit pas ekzekutimit të programit **pointM4**

Pointerët në stringje

Pointerët mund të shfrytëzohen edhe gjatë punës me stringje, duke i deklaruar ato të tipit **char**.

Shembull

Programi **pointS1**, përmes së cilit lexohet dhe shtypet fjalia me më së shumti **n=20** karaktere. Në program është deklaruar pointeri **a**, i cili shfrytëzohet gjatë shtypjes së anëtarëve të vektorit **x**, ku ruhet fjalia e lexuar.

```
// Programi pointS1
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int n=20;
    char x[n], *a;
    int i;
    cout << "\nTeksti që lexohet: ";
    cin.getline(x,n);
    cout << "\nTeksti i lexuar\n\n";

    a=&x[0];
```

```

    for (i=0;x[i]!='\0';i++)
        cout << setw(4+i)
              << *(a+i)
              << endl;

    cout << "\n";
    return 0;
}

```

Te programi i dhënë, fillimisht pointeri **a** është deklaruar i tipit **char**, meqë adresa që ruhet në të i përket një variable të këtij tipi. Pastaj, adresa e anëtarit të parë të vektorit **x** përcillet te pointeri me shprehjen:

```
a=&x[0];
```

Teksti i lexuar përmes komandës:

```
cin.getline(x,n);
```

shtypet duke e shfrytëzuar pointerin e formës ***(a+i)**, kurse numri i simboleve në vektorin **x**, tek unaza **for**, e cila shfrytëzohet për shtypje, lidhet me kushtin **x[i]!='\0'**.

Rezultati që shtypet në ekran do të duket si në Fig.5.30.

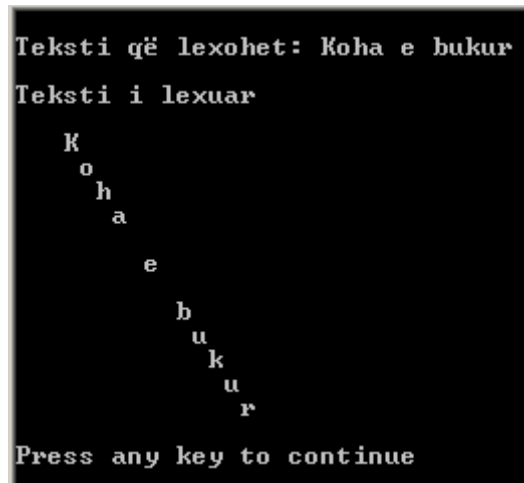


Fig.5.30
Pamja e ekranit pas ekzekutimit të
programit **points1**

Fusha pointerësh

Për të vendosur stringje brenda fushave, p.sh., siç janë emrat e ndryshëm, fushat duhet të deklarohen si dydimensionale, përkatësisht si matrica. Por, nëse

fushat e tilla ruhen në një vektor të deklaruar si *fushë pointerësh* (ang. array of pointers), hapësira memoruese do të jetë minimale.

Shembull

Programi **pointS2**, përmes së cilit emrat e stinëve të vitit ruhen në matricën **stina** dhe njëkohësisht të njëjtat shtypen në ekran.

```
// Programi pointS2
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int m=4,n=9;
    int i,j;
    char stina[m][n]={"Pranvera",
                      "Vera",
                      "Vjeshta",
                      "Dimri"};

    cout << "\nStinët e vitit\n\n";

    for (i=0;i<m;i++)
    {
        cout << "    ";
        for (j=0;j<n;j++)
            cout << stina[i][j];
        cout << "\n";
    }
    cout << endl;
    return 0;
}
```

Nëse ekzekutohet programi, rezultati në ekran do të duket ashtu siç është dhënë në Fig.5.31.

Fig.5.31

Pamja e ekranit pas ekzekutimit të programit **pointS2**

Stinët e vitit

Pranvera
Vera
Vjeshta
Dimri

Press any key to continue

Në program, edhe përkundër asaj se emrat e stinëve kanë gjatësi të ndryshme, për secilin prej tyre në matricën **stina** janë rezervuar **n=9** vende, aq

sa nevojiten për vendosjen e emrit më të gjatë (**Pranvera**), përfshirë simbolin e fundit **'\0'**.

Humbja e hapësirës memoruese, duke shfrytëzuar matrica, ashtu siç u dha në shembullin paraprak, mund të eliminohet nëse emrat e stinëve të vitit ruhen në vektor të deklaruar si fusha pointerësh.

Shembull

Versioni **points3** i programit **points2**, tek i cili, për ruajtje të emrave të stinëve të vitit, shfrytëzohet vektori **stina** i deklaruar si pointer.

```
// Programi points3
#include <iostream>
using namespace std;
int main()
{
    const int m=4;
    int i,j;
    char *stina[m]={"Pranvera",
                   "Vera",
                   "Vjeshta",
                   "Dimri"};

    cout << "\nStinët e vitit\n\n";

    for (i=0;i<m;i++)
    {
        cout << "    ";
        for (j=0;*(stina[i]+j)!='\0';j++)
            cout << *(stina[i]+j);
        cout << "\n";
    }
    cout << endl;
    return 0;
}
```

Me deklarimin e fushës **stina** si pointer, në fakt formohet një vektor pointerësh, në anëtarët e të cilit ruhen adresat ku fillojnë vendosjet e emrave të stinëve të veçanta.

Rezultati i programit të dhënë do të duket i njëjtë me atë që u dha në *Fig.5.31*.

Pointerët si parametra të funksioneve

Rëndësi të veçantë ka përdorimi i pointerëve si parametra funksionesh, sepse përmes tyre mundësohet përcjellja direkte e vlerave prej programit kryesor në funksione dhe e rezultateve të llogaritjeve të ndryshme - prej funksioneve të programi kryesor.

Mundësitë themelore

Në vijim, përmes shembujve do të jepen disa versione të përcjelljes së vlerave në nënprograme dhe marrjes së rezultateve nga nënprogramet, duke i shfrytëzuar pointerët.

Shembull

Programi **pointF1a**, përmes së cilit llogaritet vlera e funksionit:

$$y = x + 3$$

duke e shfrytëzuar edhe funksionin **dita** me parametrin formal **x** të tipit pointer.

```
// Programi pointF1a
#include <iostream>
using namespace std;

double dita(int *x);

int main()
{
    int x;
    double y;
    cout << "\nVlera e lexuar x: ";
    cin >> x;
    y=dita(&x);
    cout << "\nVlera e llogaritur y="
         << y
         << "\n\n";
    return 0;
}
double dita(int *x)
{
    double y;
```



```

    y=*x+3;
    return y;
}

```

Këtu, meqë parametri formal i funksionit është deklaruar si pointer ***x**, gjatë shfrytëzimit të tij në programin kryesor për llogaritjen e vlerës së funksionit:

```
y=dita(&x);
```

si parametër aktual **&x** është marrë adresa e variablës **x**.

Pas ekzekutimit të programit të dhënë, rezultati në ekran do të duket ashtu siç është dhënë në *Fig.5.32*.

Fig.5.32

Pamja e ekranit pas ekzekutimit të programit pointF1a

```

Vlera e lexuar x: 8
Vlera e llogaritur y=11
Press any key to continue

```

Për parametrat formalë të nënprogrami mund të mos shfrytëzohen identifikatorë të njëjtë me ato të parametrave aktualë përkatës.

Shembull

Versioni **pointF1b** i programit **pointF1a**, tek i cili dallohet variabla e parametrin formal dhe ajo e parametrin aktual.

```

// Programi pointF1b
#include <iostream>
using namespace std;

double dita(int *a);

int main()
{
    int x;
    double y;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    y=dita(&x);

    cout << "\nVlera e llogaritur y="
         << y
         << "\n\n";
    return 0;
}

```

```
double dita(int *a)
{
    double g;
    g=*a+3;
    return g;
}
```

Në nënprogram, si parametër formal është shfrytëzuar pointeri **a**. Kurse gjatë thirrjes së tij në programin kryesor, parametri formal në fjalë është zëvendësuar me adresën e variablës **x**. Njëkohësisht, në nënprogram definohet llogaritja e vlerës së funksionit **g**, i cili është shënuar edhe te komanda **return**. Rezultati në ekran, për vlerën hyrëse të njëjtë do të duket si ai që shihet në Fig.5.32.

Rezultati që fitohet përmes llogaritjes në nënprogram mund edhe të shtypet, pa e përcjellë atë në programin kryesor.

Shembull

Versioni **pointF1c** i programit **pointF1a**, te i cili rezultati i llogaritjes shtypet në nënprogram.

```
// Programi pointF1c
#include <iostream>
using namespace std;

void dita(int *x);

int main()
{
    int x;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    dita(&x);

    return 0;
}

void dita(int *x)
{
    double y;

    y=*x+3;

    cout << "\nVlera e llogaritur y="
         << y
```

```

        << "\n\n";
    return;
}

```

Këtu, meqë vlera e llogaritur shtypet në nënprogram, variabla **y** nuk është shënuar në vazhdim të komandës **return** dhe para emrit të funksionit është shënuar fjala **void**.

Shembull

Versioni **pointF2a** i programit **pointF1a**, tek i cili rezultati i llogaritjes përcillet në programin kryesor duke e shënuar variablën përkatëse si pointer të dytë.

```

// Programi pointF2a
#include <iostream>
using namespace std;

void dita(int *a, double *g);

int main()
{
    int x;
    double y;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    dita(&x, &y);

    cout << "\nVlera e llogaritur y="
         << y
         << "\n\n";
    return 0;
}

void dita(int *a, double *g)
{
    *g = *a + 3;
    return;
}

```

Brenda kllapave të funksionit këtu janë shfrytëzuar si parametra formale pointerët **a** dhe **g**. Gjatë shfrytëzimit të funksionit në programin kryesor:

```
dita(&x,&y);
```

përmes adresës përkatëse **&y**, vlera e llogaritur e variablës **g**, prej nënprogramit përcillet direkt te variabla **y** e funksionit.

Nëse ekzekutohet programi i dhënë, rezultati do të duket si ai që është dhënë në *Fig.5.32*.

Nuk ka asnjë arsye që të mos shfrytëzohen identifikatorë të njëjtë për parametrat formalë dhe parametrat aktualë përkatës.

Shembull

Versioni **pointF2b** i programit **pointF2a**, tek i cili emrat e parametrave formalë dhe atyre aktualë nuk dallohen mes vete.

```
// Programi pointF2b
#include <iostream>
using namespace std;

void dita(int *x,double *y);

int main()
{
    int x;
    double y;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    dita(&x,&y);

    cout << "\nVlera e llogaritur y="
         << y
         << "\n\n";
    return 0;
}

void dita(int *x,double *y)
{
    *y=*x+3;
    return;
}
```

Në vijim, sa herë që nuk është e nevojshme, parametrat formalë dhe ata aktualë do të quhen njëloj.

Nuk është e thënë që të gjithë parametrat e funksioneve të merren patjetër pointerë.

Shembull

Programi **pointF3**, i cili shfrytëzohet për llogaritjen e vlerës së funksionit **y**, i përcaktuar me shprehjen e dhënë më parë. Por, këtu vlera e variablës hyrëse **x** në nënprogram është marrë si variabël e zakonshme.

```
// Programi pointF3
#include <iostream>
using namespace std;

void dita(int x,double *y);

int main()
{
    int x;
    double y;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    dita(x,&y);

    cout << "\nVlera e llogaritur y="
         << y
         << "\n\n";
    return 0;
}

void dita(int x,double *y)
{
    *y=x+3;
    return;
}
```

Në program, variabla **y**, tek e cila ruhet vlera dalëse nga nënprogrami, është marrë si pointer, sepse vetëm në këtë mënyrë përcillet te programi kryesor pa e shënuar në vazhdim të komandës **return**.

Mundësi të tjera

Nënprogramet mund të shfrytëzohen për llogaritje të vlerave të disa funksioneve, ose edhe procedurat llogaritëse mund të jenë më komplekse.

Shembull

Programi **pointF4**, përmes së cilit llogariten vlerat e funksioneve:

$$\begin{aligned}y &= x + 3 \\ z &= 4 * x - 1\end{aligned}$$

Në program, për definimin e funksioneve **y** dhe **z** shfrytëzohet funksioni **jeta**, me parametra formalë të tipit pointer.

```
// Programi pointF4
#include <iostream>
using namespace std;

void jeta(int *x,double *y,double *z);

int main()
{
    int x;
    double y,z;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    jeta(&x,&y,&z);

    cout << "\nVlera e llogaritur y="
         << y
         << "\n\nVlera e llogaritur z="
         << z
         << "\n\n";
    return 0;
}

void jeta(int *x,double *y,double *z)
{
    *y=*x+3;
    *z=4*(*x)-1;
    return;
}
```

Këtu, si parametra formalë të funksionit **jeta** janë marrë pointerët e variablës **x** dhe të rezultateve të funksioneve **y** e **z**.

```
void jeta(int *x,double *y,double *z)
```

Gjatë thirrjes së funksionit në fjalë në programin kryesor:

```
jeta(&x,&y,&z);
```

si parametra aktualë janë marrë adresat e variablave përkatëse hyrëse (**&x**) dhe dalëse (**&y, &z**).

Rezultati, pas ekzekutimit të programit të dhënë, për vlerën hyrëse të variablës **x** do të duket si në Fig.5.33.

Fig.5.33

Pamja e ekranit pas ekzekutimit të programit **pointF4**

```
Vlera e lexuar x: 8
Vlera e llogaritur y=11
Vlera e llogaritur z=31
Press any key to continue
```

Nënprogramet me pointerë si parametra formalë mund të shfrytëzohen edhe për llogaritje të vlerave për funksione të çfarëdoshme.

Shembull

Programi **pointF5**, përmes së cilit llogariten vlerat e funksionit:

$$y = 4x + 3 \sum_{i=1}^{n+1} (2i + 1)$$

Në program, për definimin e funksionit **y** përdoret funksioni **alfa**, në të cilin si parametra formalë shfrytëzohen pointerët e variablave **x, n** dhe **y**.

```
// Programi pointF5
#include <iostream>
using namespace std;
```

```
void alfa(double *x,int *n,double *y);
```

```
int main()
{
    int n;
```

```
double x,y;
cout << "\nVariabla x: ";
cin >> x;
cout << "\nVariabla n: ";
cin >> n;

alfa(&x,&n,&y);

cout << "\nVlera e llogaritur y="
      << y
      << "\n\n";
return 0;
}

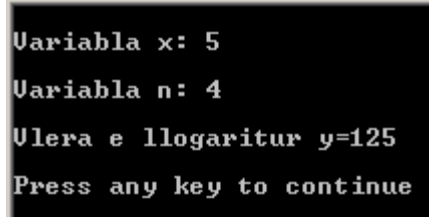
void alfa(double *x,int *n,double *y)
{
    int i;
    double s=0;
    for (i=1;i<=*n+1;i++)
        s=s+(2*i+1);
    *y=4*(*x)+3*s;
return;
}
```

Këtu, si parametra formalë paraqiten pointerët e dy variablave hyrëse **x** dhe **n**, si dhe pointeri i rezultatit **y**.

Nëse ekzekutohet programi i dhënë, për vlerat hyrëse, të cilat kompjuterit i jepen përmes tastierës, rezultati do të duket ashtu siç është dhënë në Fig.5.34.

Fig.5.34

Pamja e ekranit pas ekzekutimit të programit pointF4



```
Variabla x: 5
Variabla n: 4
Vlera e llogaritur y=125
Press any key to continue
```

Në programin e dhënë më sipër, nënprogrami mund të shfrytëzohet vetëm për llogaritjen e shumës, ose vlera e llogaritur e funksionit **y** mund të shtypet edhe brenda nënprogramit.

Vektorët si pointerë

Për përcjellje të anëtarëve të vektorëve në nënprograme mund të shfrytëzohen si pointerë adresat e anëtarëve të parë të tyre, ose edhe vetë vektorët.

Shembull

Programi **pointF6a**, përmes së cilit nga anëtarët e vektorit të



dhënë **A(n)** formohet vektori i ri **B(n)**, duke e shfrytëzuar funksionin **vekt**.

```
// Programi pointF6a
#include <iostream>
#include <iomanip>
using namespace std;

const int n=5;

void vekt(int *a);

int main()
{
    int A[n]={5,18,7,12,9}, *a;

    a=&A[0];
    vekt(a);
    return 0;
}

void vekt(int *a)
{
    int i,B[n];
    cout << "\n Anëtarët e vektorëve"
    << "\n\n          A          B"
    << "\n  -----"
    << "\n";
    for (i=0;i<n;i++)
    {
        B[i]=*(a+i)+3;
        cout << setw(8)
        << *(a+i)
        << setw(8)
        << B[i]
        << endl;
    }
    cout << "\n";
    return;
}
```

Si parametër formal i nënprogramit këtu paraqitet vetëm pointeri **a**, tek i cili ruhet adresa e anëtarit të parë të vektorit:

```
a=&A[0];
```

Pastaj, përmes kësaj adrese në nënprogram llogariten adresat e anëtarëve të tjerë të vektorit, duke e shfrytëzuar shprehjen **$*(a+i)$** , gjë që shihet edhe te shprehja për llogaritjen e anëtarëve të vektorit **B**:

$$B[i]=*(a+i)+3;$$

si edhe te komanda për shtypje të anëtarëve të vektorit **A**.

Rezultati në ekran, pas ekzekutimit të programit të dhënë, do të duket si në Fig.5.35.

Anëtarët e vektorëve	
A	B
5	8
18	21
7	10
12	15
9	12
Press any key to continue	

Fig.5.35

Pamja e ekranit pas ekzekutimit të programit **pointF6a**

Përcjellja e vlerave të anëtarëve të vektorëve në nënprogram mund të bëhet edhe duke e deklaruar vektorin si pointer.

Shembull

Versioni **PointF6b** i programit **pointF6a**, tek i cili si parametër i funksionit **vekt** merret vektori **A(n)**.

```
// Programi pointF6b
#include <iostream>
#include <iomanip>
using namespace std;

const int n=5;

void vekt(int *A);

int main()
{
    int A[n]={5,18,7,12,9};

    vekt(A);
    return 0;
}

void vekt(int *A)
{
    int i,B[n];
    cout << "\n Anëtarët e vektorëve"
```

```

        << "\n\n          A          B"
        << "\n  -----"
        << "\n";
    for (i=0;i<n;i++)
    {
        B[i]=A[i]+3;
        cout << setw(8)
              << A[i]
              << setw(8)
              << B[i]
              << endl;
    }
    cout << "\n";
return;
}

```

Këtu, vektori **A** paraqitet si parametër formal i nënprogramit **vekt**, duke e deklaruar atë si pointer:

```
void vekt(int *A)
```

Kurse gjatë shfrytëzimit të funksionit, si parametër aktual i tij shfrytëzohet vetë vektori:

```
vekt(A);
```

Në këtë mënyrë, brenda nënprogramit anëtarët e veçantë të vektorit shfrytëzohen direkt, duke i shënuar ato si **A[i]**.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si ai që është dhënë në Fig.5.35.

Pointerët në funksione

Gjatë ekzekutimit të programeve, funksionet që përfshihen brenda tyre vendosen në hapësira memoruese të caktuara. Adresa ku fillon vendosja e një funksioni në memorien e kompjuterit quhet *adresë e funksionit* (ang. function address). Për ruajtje të kësaj adrese mund të shfrytëzohet *pointeri në funksion* (ang. pointer to function), i cili njihet edhe si *pointer funksioni* (ang. function pointer). Pas shoqërimit të adresës në fjalë te pointeri i funksionit, ai mund të shfrytëzohet për thirrjen e funksionit në fjalë.

Shembull

Programi **pointF7a**, përmes së cilit llogaritet vlera e funksionit:

$$y = 3x + 4$$

duke e shfrytëzuar edhe funksionin **jeta** me parametrin formal

 **x** të tipit **double**.

```
// Programi pointF7a
#include <iostream>
using namespace std;

void jeta(double x);

int main()
{
    void (*a)(double x);
    a=jeta;

    double x;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    (*a)(x);

    return 0;
}

void jeta(double x)
{
    double y;
    y=3*x+4;
    cout << "\nVlera e llogaritur y="
         << y
         << "\n\n";
    return;
}
```

Në program është definuar funksioni **jeta**, përmes së cilit llogaritet dhe shtypet vlera e funksionit **y**. Me qëllim të thirrjes së funksionit përmes pointerit **a**, në pjesën e programit kryesor, përmes shprehjes:

```
void (*a)(double x);
```

është deklaruar pointeri në fjalë, duke e shënuar brenda kllapave, por i shoqëruar edhe me pjesën e parametrave të funksionit. Pointerit i shoqërohet adresa e funksionit **jeta**, duke e shfrytëzuar shprehjen:

```
a=jeta;
```

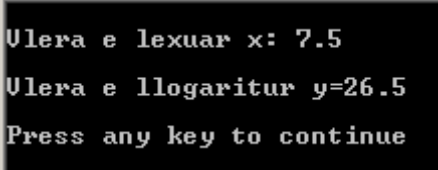
Pas kësaj, funksioni **jeta** thirret përmes pointerit **a**, gjatë së cilës i shoqërohet edhe parametri aktual **x**, në këtë mënyrë:

```
(*a)(x);
```

Nëse ekzekutohet programi i dhënë, për vlerën hyrëse **x**, e cila kompjuterit i jepet përmes tastierës, rezultati do të duket si në *Fig.5.36*.

Fig.5.36

Pamja e ekranit pas ekzekutimit të programit pointF7a



```
Vlera e lexuar x: 7.5
Vlera e llogaritur y=26.5
Press any key to continue
```

Përveç funksioneve, njëkohësisht edhe parametrat e tyre mund të merren si pointerë.

Shembull

Versioni **PointF7b** i programit **pointF7a**, tek i cili parametri **x** merret si pointer.

```
// Programi pointF7b
#include <iostream>
using namespace std;

void jeta(double *x);

int main()
{
    void (*a)(double *x);
    a=jeta;

    double x;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    (*a)(&x);

    return 0;
}

void jeta(double *x)
{
    double y;
    y=3*(*x)+4;
    cout << "\nVlera e llogaritur y="
```

```
        << y
        << "\n\n";
    return;
}
```

Ky version i programit dallohet nga versioni **pointF7a** vetëm në parametrin **x** të funksionit, i cili gjatë definimit të tij shënohet si ***x**, kurse gjatë thirrjes - si **&x**. Pas ekzekutimit të programit, rezultati në ekran do të duket si ai që është dhënë në *Fig.5.36*.

Rezultati i funksionit mund të përcillet edhe te programi kryesor, duke e shënuar në vazhdim të komandës **return**.

Shembull

Versioni **PointF7c** i programit **pointF7a**, tek i cili rezultati i llogaritjes brenda funksionit **jeta** shfrytëzohet te programi kryesor.

```
// Programi pointF7c
#include <iostream>
using namespace std;

double jeta(double x);

int main()
{
    double (*a)(double x);
    a=jeta;

    double x,y;
    cout << "\nVlera e lexuar x: ";
    cin >> x;

    y=(*a)(x);

    cout << "\nVlera e llogaritur y="
        << y
        << "\n\n";
    return 0;
}

double jeta(double x)
{
    double y;
    y=3*x+4;
    return y;
}
```

Këtu, funksioni **jeta** është deklaruar i tipit **double**, sepse prej tij merret rezultati i funksionit **y**, duke e shënuar variablën përkatëse në vazhdim të komandës **return**. Gjithashtu, funksioni thirret përmes shprehjes:

```
y=( *a)(x);
```

Rezultati i llogaritjes, për vlerën hyrëse të variablës **x** si ajo që u zgjodh te versionet paraprake të programit, në ekran do të duket ashtu siç është dhënë në *Fig.5.36*.

Fusha pointerësh në funksione

Pointerët në funksione mund të formojnë fusha pointerësh të funksioneve. Në këtë mënyrë lehtësohet thirrja e funksioneve, duke e përcaktuar indeksin e funksionit që thirret.

Shembull

Programi **PointF8a**, përmes së cilit tregohet shfrytëzimi i fushës së pointerëve në funksione, duke e formuar fushën **a** të 3 pointerëve në funksionet **jeta**, **dita** dhe **nata**.

```
// Programi pointF8a
#include <iostream>
using namespace std;

void jeta(double x);
void dita(double x);
void nata(double x);

int main()
{
    int i;
    void (*a[3])(double)={jeta,dita,nata};
    for (i=0;i<=2;i++)
        (*a[i])(5.6);
    cout << "\n";
    return 0;
}

void jeta(double x)
{
    cout << "\nThirrja e funksionit jeta"
        << "\nVlera e llogaritur y="
        << 2*x+1
```

```

        << "\n";
    return;
}

void dita(double x)
{
    cout << "\nThirrja e funksionit dita"
          << "\nVlera e llogaritur z="
          << x+2
          << "\n";
    return;
}

void nata(double x)
{
    cout << "\nThirrja e funksionit nata"
          << "\nVlera e llogaritur v="
          << 4*x+3
          << "\n";
    return;
}

```

Në program, fusha e pointerëve në **3** funksione është deklaruar:

```
void (*a[3])(double)={jeta,dita,nata};
```

ku, siç shihet, emrat e funksioneve **jeta**, **dita** dhe **nata**, të cilëve u përkasin pointerët **a[1]**, **a[2]** dhe **a[3]**, janë përfshirë brenda kllapave. Këtu është paraparë që të thirren të **3** funksionet përmes pointerëve në fjalë, duke e shënuar brenda unazës **for** komandën për thirrje:

```
(*a[i])(5.6);
```

ku vlera **5.6** i përket variablës **x**, e cila shfrytëzohet nga funksionet e veçanta.

Nëse ekzekutohet programi i dhënë, rezultati do të duket si në Fig.5.37.

Fig.5.37

Pamja e ekranit pas ekzekutimit të programit pointF8a

```

Thirrja e funksionit jeta
Vlera e llogaritur y=12.2

Thirrja e funksionit dita
Vlera e llogaritur z=7.6

Thirrja e funksionit nata
Vlera e llogaritur v=25.4

Press any key to continue

```


Prej këtu shihet se rezultatet në ekran shtypen përmes ekzekutimit të **3** funksioneve të veçanta, me radhën e zgjedhur gjatë deklarimit të fushës së pointerëve në funksione.

Funksionet do të ekzekutohen edhe nëse brenda unazës **for** pointerët shkruhen në formën:

```
a[i](5.6);
```

pa i shfrytëzuar kllapat dhe operatorin *****.

Nënkuptohe se funksionet, pointerët e të cilëve janë përfshirë në fushë, mund të thirren edhe sipas dëshirës, duke e përcaktuar vlerën e indeksit përkatës.

Shembull

Programi kryesor i versionit **PointF8b** të programit **PointF8a**, përmes së cilit tregohet ekzekutimi i njërit nga **3** funksionet, pointerët e të cilëve përfshihen në një fushë të pointerëve, duke e zgjedhur indeksin përkatës.

```
// Programi pointF8b
#include <iostream>
using namespace std;

void jeta(double x);
void dita(double x);
void nata(double x);

int main()
{
    int i;

    void (*a[3])(double)={jeta,dita,nata};

Leximi:
    cout << "\nZgjidh 0, 1 ose 2 për i: ";
    cin >> i;

    if (i>=0 && i<=2)
        (*a[i])(5.6);
    else
        goto Leximi;

    cout << "\n";
    return 0;
}
```

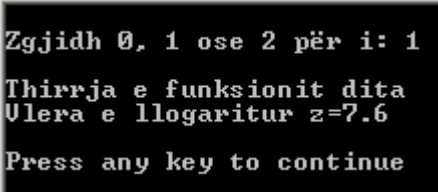
Këtu nuk është dhënë pjesa e nënprogrameve, meqë ato mbesin të njëjta si te versioni i programit **pointF8a**. Në program, përmes komandës për lexim **cin**, lexohet indeksi **i** i **3** vlerave të mundshme të indekseve të pointerëve. Nëse përmes tastierës kompjuterit i jepet ndonjë vlerë jashtë kufijve të vlerave të lejueshme, ekzekutohet komanda:

```
goto Leximi;
```

për kthim pas me qëllim të përsëritjes së leximit të vlerës së variablës **i**. Kështu, p.sh., nëse për variablën **i** përmes tastierës kompjuterit i jepet vlera **1**, në ekran do të shtypet rezultati që fitohet pas ekzekutimit të funksionit **dita**, ashtu siç shihet në Fig.5.38.

Fig.5.38

Pamja e ekranit pas ekzekutimit të programit **pointF8b**



```
Zgjidh 0, 1 ose 2 për i: 1
Thirrja e funksionit dita
Vlera e llogaritur z=7.6
Press any key to continue
```

Pointerët në struktura

Pointerët, përveç të dhënave të tipeve të zakonshme, p.sh., siç janë **int**, **double**, **char** etj., mund të shfrytëzohen edhe gjatë operimit me struktura. Në këtë rast, deklarimi i variablave të tipit pointer për strukturat dhe operimi me anëtarët e tyre nuk dallon aspak nga mënyra e shfrytëzimit të pointerëve të tipit të zakonshëm të të dhënave.

Shembull

Versione të programit **structP1**, në të cilët definohet struktura **brinjet** dhe pastaj inicializohen me vlera variablat **a** dhe **b**, të përfshira në strukturë, në rrugë direkte dhe me ndërmjetësimin e pointerit.

a. Pa pointer

```
// Programi structP1a
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};
```

```

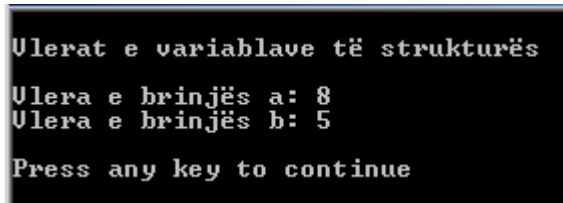
int main()
{
    brinjet kater;
    kater.a=8;
    kater.b=5;

    cout << "\nVlerat e variablave të strukturës"
          << "\n\nVlera e brinjës a: "
          << kater.a;
    cout << "\nVlera e brinjës b: "
          << kater.b
          << "\n\n";
    return 0;
}

```

Në program, fillimisht, është definuar struktura **brinjet** në të cilën përfshihen variablat **a** e **b**, dhe është deklaruar variabla e strukturës **kater**. Pastaj, pasi variablat në fjalë inicializohen direkt me vlerat **8** dhe **5**, përmes komandave përkatëse urdhërohet edhe shtypja e vlerave të tyre. Rezultati në ekran duket si në Fig.5.39.

Fig.5.39
Pamja e ekranit pas ekzekutimit të programit **structP1a**



```

Vlerat e variablave të strukturës
Vlera e brinjës a: 8
Vlera e brinjës b: 5
Press any key to continue

```

b. Me pointer

```

// Programi structP1b
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet D;
    brinjet *kater;
    kater=&D;
}

```

```
    (*kater).a=8;
    (*kater).b=5;

    cout << "\nVlerat e variablave të strukturës"
          << "\n\nVlera e brinjës a: "
          << (*kater).a;
    cout << "\nVlera e brinjës b: "
          << (*kater).b
          << "\n\n";
    return 0;
}
```

Në këtë version të programit, para se të inicializohen variablat, në rrugë të zakonshme deklarohet pointeri:

```
brinjet *kater;
```

Pastaj, për ta shfrytëzuar pointerin gjatë inicializimit dhe shtypjes së vlerave të variablave, adresa e variablës së objektit **D** përcillet te pointeri:

```
kater=&D;
```

Gjatë inicializimit të variablave me vlera përdoren shprehjet:

```
(*kater).a=8;
(*kater).b=5;
```

Këtu, shfrytëzimi i kllapave është i domosdoshëm, sepse pika (.) ka përparësi më të madhe se operatori *. Nëse pointeri shfrytëzohet pa i përdorur kllapat, interpretimi i shprehjeve do të jetë i gabueshëm.

Gjatë shtypjes së vlerave të variablave të përfshira në strukturë, në komandat për shtypje, ato përsëri shkruhen në formën:

```
(*kater).a
(*kater).b
```

Nëse ekzekutohet ky version i programit, rezultati do të jetë i njëjtë me atë që shihet në *Fig.5.39*.

Me qëllim të thjeshtimit të qasjes te komponentet e strukturës, në gjuhën C++ mund të shfrytëzohet *operatori shigjetë -> për qasje tek anëtarët*, i cili realizohet përmes simboleve minus (-) dhe më i madh (>).

Shembull

Versioni **structP1c** i programit **structP1b**, në të cilin për qasje te komponentet e strukturës shfrytëzohet operatori **->**.

```
// Programi structP1c
#include <iostream>
using namespace std;

struct brinjet
{
    double a,b;
};

int main()
{
    brinjet D;
    brinjet *kater;
    kater=&D;
    kater->a=8;
    kater->b=5;

    cout << "\nVlerat e variablave të strukturës"
         << "\n\nVlera e brinjës a: "
         << kater->a;
    cout << "\nVlera e brinjës b: "
         << kater->b
         << "\n\n";
    return 0;
}
```

Brenda programit njëkohësisht mund të shfrytëzohen operatorët ***** dhe **->** pa ndonjë kufizim.

Pointerët në objekte

Plotësisht njëllor sikurse te strukturat, pointerët mund të shfrytëzohen edhe gjatë operimit me objekte dhe me anëtarët e përfshirë në klasat përkatëse.

Qasja te variablat e klasës

Shfrytëzimi i pointerëve në objekte, gjatë qasjes te variablat që përfshihen në klasat përkatëse, nuk dallon nga shfrytëzimi i pointerëve për qasje te variablat që përfshihen në komponentet e strukturave.

Shembull

Versione të programit **classP1**, në të cilët definohet klasa **brinjet**, në komponentet e së cilës përfshihen variablat **a** dhe **b**, si dhe objekti përkatës **D**.

a. Pa pointer

```
// Programi classP1a
#include <iostream>
using namespace std;

struct brinjet
{
public:
    double a,b;
};

int main()
{
    brinjet D;
    D.a=8;
    D.b=5;

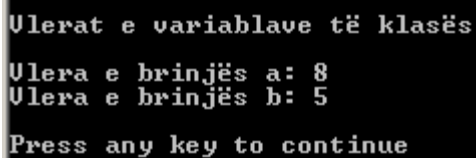
    cout << "\nVlerat e variablave të klasës"
          << "\n\nVlera e brinjës a: "
          << D.a;
    cout << "\nVlera e brinjës b: "
          << D.b
          << "\n\n";
    return 0;
}
```

Për qasje te variablat e përfshira në komponentet e klasës këtu shfrytëzohet objekti **D** dhe pika, p.sh., duke i shfrytëzuar shprehjet:

```
D.a=8;
D.b=5;
```

Rezultati që fitohet pas ekzekutimit të programit në ekran do të duket ashtu siç është dhënë në *Fig.5.40*.

Fig.5.40
Pamja e ekranit pas ekzekutimit të
programit classP1a



```
Vlerat e variablave të klasës
Vlera e brinjës a: 8
Vlera e brinjës b: 5
Press any key to continue
```

b. Me pointer

```
// Programi classP1b
#include <iostream>
using namespace std;
```

```

struct brinjet
{
public:
    double a,b;
};

int main()
{
    brinjet D;
    brinjet *kater;
    kater=&D;
    (*kater).a=8;
    (*kater).b=5;

    cout << "\nVlerat e variablave të klasës"
          << "\n\nVlera e brinjës a: "
          << (*kater).a;
    cout << "\nVlera e brinjës b: "
          << (*kater).b
          << "\n\n";
    return 0;
}

```

Këtu, për qasje te variablat e përfshira në komponentet e klasës përdoret pointeri i cili tregon tek objekti **D**. Deklarimi i pointerit dhe shfrytëzimi i tij gjatë inicializimit të variablave **a** dhe **b** me vlera, si dhe gjatë shtypjes së vlerave të tyre, nuk dallon aspak nga ajo që u dha te pjesa mbi pointerët në variablat e strukturave. Kështu, p.sh., gjatë inicializimit të variablave, shprehjet që shfrytëzohen shkruhen në këtë mënyrë:

```

(*kater).a=8;
(*kater).b=5;

```

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si ai që shihet në Fig. 5.39.

Edhe gjatë operimit me anëtarët e klasës mund të shfrytëzohet *operatori shigjetë ->*, plotësisht njëloj si edhe te strukturat. Kështu, dy shprehjet për inicializim të variablave do të shkruhen:

```

kater->a=8;
kater->b=5;

```

Qasja te funksionet e klasave

Për thirrje të funksioneve që përfshihen brenda komponenteve të klasave, shfrytëzimi i pointerëve në objektet përkatëse nuk dallon nga shfrytëzimi i tyre për qasje te funksionet në komponentet e klasave.

Shembull

Versione të programit **classP2**, në të cilët definohet klasa **Jeta**, që i përmban variablat **m** e **a**, si dhe funksionet **vlera** dhe **shtypja**.

a. Pa pointer

```
// Programi classP2a
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;

    void vlera(int k)
    {
        m=k;
    }

    void shtypja()
    {
        cout << "\nVlera e variablës private m="
              << m
              << "\n";
    }
};

int main()
{
    Jeta Dita;

    Dita.vlera(77);
    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> Dita.a;

    Dita.shtypja();
    cout << "\nVlera e variablës publike a="
```



```

        << Dita.a
        << "\n\n";
return 0;
}

```

Siç shihet në program, për qasje te funksionet që përfshihen në komponentet e klasës, para funksioneve shkruhet objekti **Dita**:

```

Dita.vlera(77);
Dita.shtypja();

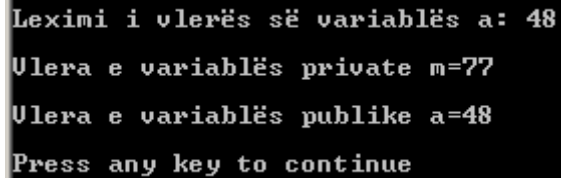
```

gjë që është njëloj si edhe gjatë qasjes te variabla **a**, kur shfrytëzohet shprehja **Dita.a**.

Pas ekzekutimit të programit të dhënë, rezultati në ekran do të duket si në Fig.5.41.

Fig.5.41

Pamja e ekranit pas ekzekutimit të programit classP2a



```

Leximi i vlerës së variablës a: 48
Vlera e variablës private m=77
Vlera e variablës publike a=48
Press any key to continue

```

b. Me pointer

```

// Programi classP2b
#include <iostream>
using namespace std;

class Jeta
{
private:
    int m;
public:
    double a;
    void vlera(int k)
    {
        m=k;
    }

    void shtypja()
    {
        cout << "\nVlera e variablës private m="
              << m
              << "\n";
    }
};

```

```
int main()
{
    Jeta Dita;
    Jeta *p;
    p=&Dita;

    (*p).vlera(77);

    cout << "\nLeximi i vlerës së variablës a: ";
    cin >> (*p).a;

    (*p).shtypja();
    cout << "\nVlera e variablës publike a="
         << (*p).a
         << "\n\n";
    return 0;
}
```

Këtu, gjatë thirrjes së funksioneve, pointeri **p**, i cili tregon te objekti **Dita**, shkruhet brenda kllapave, para funksioneve që thirren, në këtë mënyrë:

```
    (*p).vlera(77);
    (*p).shtypja();
```

Kjo formë e shfrytëzimit të pointerit nuk dallon aspak nga ajo që përdoret për qasje te variabla në komponenten publike të klasës:

```
    (*p).a
```

Pas ekzekutimit të këtij versioni të programit, rezultati do të duket njëjloj me atë që është dhënë në *Fig.5.42*.

Edhe gjatë thirrjes së funksioneve mund të shfrytëzohet operatori **->**.

Shembull

Versioni **classP2c** i programit **classP2b**, në të cilin për qasje te variablat dhe te funksionet që përfshihen në komponentet publike të klasës shfrytëzohet operatori **->**.

Në vijim është dhënë vetëm pjesa e programit kryesor të versionit **classP2c**, sepse pjesa tjetër e tij nuk dallon aspak nga versioni i programit **classP2b**, i cili u dha më sipër.

```
int main()
{
    Jeta Dita;
```

```
Jeta *p;  
p=&Dita;  
  
p->vlera(77);  
  
cout << "\nLeximi i vlerës së variablës a: ";  
cin >> p->a;  
  
p->shtypja();  
cout << "\nVlera e variablës publike a=" << p->a << "\n\n";  
return 0;  
}
```

Shfrytëzimi i pointerit në objekte gjatë qasjes te komponentet publike të klasave nuk dallon aspak, nëse funksionet që përfshihen në klasa definoohen jashtë strukturës së tyre.

	5	
Referencat		
Referencat e zakonshme 296		
Parametrat formalë referentë 302		
Fushat referente 306		
Parametrat referentë brenda strukturave 309		
Parametrat referentë brenda klasave 311		
Objektet referente 312		

Në gjuhën C++, përmes referencave mund të deklarohen dy ose më shumë variabla ose objekte, të cilat ruhen të dhëna të njëjta, si dhe përcillen ndryshimet që ndodhin në secilën prej tyre.

Për deklarimin e referencave shfrytëzohet *operatori për referim &*, i cili nuk dallohet nga simboli që shfrytëzohet për marrjen e adresave të pointerët.

Referencat kryesisht shfrytëzohen gjatë operimit me parametrat e funksioneve, gjë që është shpjeguar më parë edhe në kapitullin ku flitet mbi klasat.

Referencat e zakonshme

Brenda programeve mund të shfrytëzohen variabla referente të zakonshme, për të cilat thuhet se janë me *referencë të pavarur* (ang. independent reference). Para se të deklarohet një variabël referente, duhet të jetë deklaruar variabla së cilës ajo i referohet.

Variabla referente deklarohet:

```
t &r=v;
```

ku janë:

r - variabla referente.

t - tipi i variablës referente.

v - variabla së cilës i referohet variabla referente.

Pas këtij deklarimi, kompjuteri variablën **r** e llogarit si *sinonim* (ang. synonym, alias) të variablës **v**. Me këtë nënkuptohet se çdo ndryshim që ndodh te variabla **v** do të përcillet edhe te variabla **r** dhe anasjelltas. Siç u theksua edhe më sipër, deklarimi i variablës referente **r** lejohet nga kompjuteri pasi paraprakisht të jetë deklaruar variabla **v** të cilës ajo i referohet.

Shembull

Programi **ref1**, përmes së cilit tregohet deklarimi i variablës **z**, si referente të variablës **d**.

```
// Programi ref1
#include <iostream>
using namespace std;
int main()
{
    int d;

    d=793;

    cout << "\nVlera e variablës d: "
         << d
         << "\n";

    int &z=d;

    cout << "\nVlera e variablës z: "
         << z
         << "\n\n";
    return 0;
}
```

Në program, fillimisht, është deklaruar variabla **d** e tipit **int**, dhe pasi i shoqërohet vlera **793** ajo edhe shtypet. Pastaj, përmes komandës:

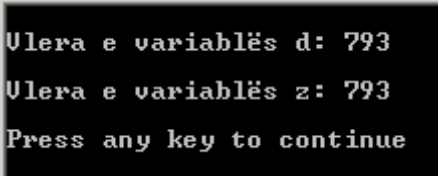
```
int &z=d;
```

variabla **z** deklarohet si variabël referente. Pas kësaj, variabla në fjalë inicializohet automatikisht me vlerën e variablës **d**, meqë bëhet sinonim i saj.

Nëse ekzekutohet programi i dhënë, rezultati që shtypet në ekran do të duket si në Fig.5.1.

Fig.5.1

Pamja e ekranit pas ekzekutimit të programit ref1



```
Vlera e variablës d: 793
Vlera e variablës z: 793
Press any key to continue
```

Variablës referente mund t'i shoqërohen vlera edhe në rrugë direkte, ngjashëm siç u shoqërohen vlera variablave të zakonshme. Si rezultat, ndryshimi i vlerës që ruhet te variabla referente përcillet edhe te variabla së cilës ajo i referohet.

Shembull

Versioni **ref2** i programit **ref1**, përmes së cilit tregohet ndikimi i ndryshimit të vlerës së variablës referente **z** edhe te variabla **d**.

```
// Programi ref2
#include <iostream>
using namespace std;
int main()
{
    int d;

    d=793;

    cout << "\nVlera e variablës d: "
         << d
         << "\n";

    int &z=d;

    cout << "\nVlera e variablës z: "
         << z
         << "\n\n";

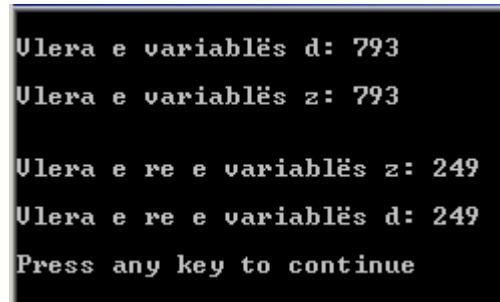
    z=249;

    cout << "\nVlera e re e variablës z: "
         << z
         << "\n\nVlera e re e variablës d: "
         << d
         << "\n\n";

    return 0;
}
```

Në pjesën e fundit të programit, variablës referente **z** i është shoqëruar vlera **249**. Si rezultat, ndryshimi i vlerës të variabla në fjalë përcillet edhe te variabla **d**, së cilës ajo i referohet, gjë që shihet edhe nga rezultati që shtypet në ekran (shih Fig.5.2).

Fig.5.2
Pamja e ekranit pas ekzekutimit të
programit *ref2*



```
Vlera e variablës d: 793
Vlera e variablës z: 793

Vlera e re e variablës z: 249
Vlera e re e variablës d: 249
Press any key to continue
```

Disa variabla referente njëkohësisht mund t'i referohen një variable të zakonshme.

Shembull

Versioni **ref3** i programit **ref1**, përmes së cilit tregohet deklarimi i dy variablave referente **z** e **v**, të cilat i referohen variablës **d**.

```
// Programi ref3
#include <iostream>
using namespace std;
int main()
{
    int d;

    d=793;

    cout << "\nPas referencës së parë"
         << "\n\nVariabla d: "
         << d;

    int &z=d;

    cout << "\nVariabla z: "
         << z
         << "\n\n";

    int &v=d;

    cout << "Pas referencës së dytë"
         << "\n\nVariabla d: "
         << d
         << "\nVariabla z: "
         << z
         << "\nVariabla v: "
         << v
         << "\n\n";
    return 0;
}
```

Këtu, si variabla referente janë deklaruar variablat **z** dhe **v**, të cilat njëkohësisht i referohen variablës **d**. Pas kësaj, te të tri variablat ruhet vlera **793**, e cila i është shoqëruar variablës **d** në fillim të programit, gjë që shihet edhe nga rezultati që shtypet në ekran (shih *Fig. 5.3*), pas ekzekutimit të programit të dhënë.

Fig.5.3

Pamja e ekranit pas ekzekutimit të programit *ref3*

```
Pas referencës së parë
Variabla d: 793
Variabla z: 793

Pas referencës së dytë
Variabla d: 793
Variabla z: 793
Variabla v: 793

Press any key to continue
```

Në programin e dhënë, pavarësisht se cila nga vlerat e **3** variablave ndryshohet, vlerat e tyre gjithnjë do të jenë të barabarta.

Brenda një programi, referimi i deklaruar ndaj një variable nuk mund të ndryshohet më.

Konstantet referente

Nëse para variablës referente shënohet fjala **const**, do të deklarohet një *konstante referente*. Në formë të përgjithshme deklarimi i konstantes referente duket:

```
const t &r=v;
```

ku janë:

- r** - konstanta referente.
- t** - tipi i konstantes referente.
- v** - vlera e cila i shoqërohet konstantes referente.

Pas këtij deklarimi, konstantes referente **r** i shoqërohet vlera **v**.

Shembull

Programi **ref4a**, përmes së cilit tregohet deklarimi i konstantes referente **z**, së cilës i shoqërohet vlera **538**.

```
// Programi ref4a
#include <iostream>
using namespace std;
int main()
{
    const int &z=538;
    cout << "\nVlera e konstantes z: "
```

```

        << z
        << "\n\n";
return 0;
}

```

Pasi të deklarohet konstantja referente **z** dhe t'i shoqërohet vlera **538**, ndryshimi i saj nuk do të lejohet nga kompjuteri, nëse p.sh., në vijim të programit shkruhet shprehja:

```
z=649;
```

Shprehjet për deklarimin e konstanteve referente nuk mund të shkruhen pa fjalën **const** në rastet kur atyre u shoqërohet ndonjë vlerë konstante. P.sh., komanda për deklarimin e variablës referente **z** te programi i dhënë më sipër nuk mund të shkruhet në formën:

```
int &z=538;
```

Konstanteve referente mund t'u shoqërohen edhe vlera të variablave.

Shembull

Versioni **ref4b** i programit **ref4a**, përmes së cilit tregohet shoqërimi i vlerës së variablës **x** gjatë deklarimit të konstantes referente **z**.

```

// Programi ref4b
#include <iostream>
using namespace std;
int main()
{
    int x=137;
    const int &z=x;

    cout << "\nVlera e konstantes z: "
         << z
         << "\n\n";
return 0;
}

```

Edhe në këtë rast, vlera e shoqëruar nuk mund të ndryshohet përmes ndonjë shoqërimi tjetër.

Parametrat formalë referentë

Nëse pas tipit të parametrut formal të një funksioni shënohet simboli **&**, variabla përkatëse paraqet një *parametër formal referent*. Pastaj, kur thirret funksioni duke e shënuar si parametër aktual një variabël të zakonshme në vend të parametrut formal referent, adresa e parametrut aktual i shoqërohet parametrut formal referent. Në këtë mënyrë, gjatë ekzekutimit të programit, *parametri formal referent e shfrytëzon vlerën që është vendosur në adresën e parametrut aktual përkatës*.

Shembull

Programi **funkPa**, përmes së cilit tregohet llogaritja e sipërfaqes së drejtkëndëshit me brinjët **a** dhe **b**, përmes funksionit **Jeta**, duke i shfrytëzuar edhe parametrat formalë referentë.

a. Me parametra të zakonshëm

```
// Programi funkPa
#include <iostream>
using namespace std;

double Jeta(double x,double y);

int main()
{
    double a,b,s;
    cout << "\nBrinjët e drejtkëndëshit"
          << "\n\nBrinja a: ";
    cin >> a;
    cout << "\nBrinja b: ";
    cin >> b;

    s=Jeta(a,b);

    cout << "\nSipërfaqja e llogaritur s: "
          << s
          << "\n\n";
    return 0;
}

double Jeta(double x,double y)
{
    return x*y;
}
```

Pas ekzekutimit të programit të dhënë, për vlerat e brinjëve të drejtkëndëshit, të cilat kompjuterit i jepen përmes tastierës, rezultati në ekran do të duket si në *Fig.5.4*.

Fig.5.4
 Pamja e ekranit pas ekzekutimit të
 programit **funkPa**

```
Brinjët e drejtkëndëshit
Brinja a: 6
Brinja b: 5
Sipërfaqja e llogaritur s: 30
Press any key to continue
```

b. Me parametra formalë referentë

```
// Programi funkPb
#include <iostream>
using namespace std;

double Jeta(double &x,double &y);

int main()
{
    double a,b,s;
    cout << "\nBrinjët e drejtkëndëshit"
          << "\n\nBrinja a: ";
    cin >> a;
    cout << "\nBrinja b: ";
    cin >> b;

    s=Jeta(a,b);

    cout << "\nSipërfaqja e llogaritur s: "
          << s
          << "\n\n";
    return 0;
}

double Jeta(double &x,double &y)
{
    return x*y;
}
```

Te ky version i programit, parametat formalë **x** dhe **y** të funksionit **jeta** janë marrë si parametra formalë referentë. Kurse, gjatë shfrytëzimit të funksionit për llogaritjen e vlersë së sipërfaqes **s**:

```
s=Jeta(a,b);
```

parametart formalë referentë janë zëvendësuar me variablat e zakonshme **a** dhe **b**.

Nëse ekzekutohet ky version i programit, për vlera të njëjta të brinjëve të drejtkëndëshit, rezultati do të duket si ai që është dhënë në *Fig.5.4*.

Siç është thënë edhe më parë, për parametrat formalë referentë dhe për parametrat aktualë mund të shfrytëzohen identifikatorë të njëjtë.

Shembull

Versioni **funkPc** i programit **funkPb**, tek i cili identifikatorët e parametrave formalë referentë dhe i atyre aktualë nuk dallojnë mes vete.

```
// Programi funkPc
#include <iostream>
using namespace std;

double Jeta(double &a,double &b);

int main()
{
    double a,b,s;
    cout << "\nBrinjët e drejtkëndëshit"
          << "\n\nBrinja a: ";
    cin >> a;
    cout << "\nBrinja b: ";
    cin >> b;

    s=Jeta(a,b);

    cout << "\nSipërfaqja e llogaritur s: "
          << s
          << "\n\n";
    return 0;
}

double Jeta(double &a,double &b)
{
    return a*b;
}
```

Në versionin e dhënë të programit, gjatë shfrytëzimit të funksionit **Jeta**, parametrat formalë referentë **a** dhe **b** janë zëvendësuar me parametra aktualë, të cilët paraqesin identifikatorë të njëjtë.

Parametrat referentë si rezultate

Parametrat referentë mund të shfrytëzohen edhe për marrjen e rezultateve, të cilat fitohen gjatë llogaritjeve të ndryshme brenda funksioneve.

Shembull

Versioni **funkPd** i programit për llogaritjen e sipërfaqes së drejtkëndëshit me brinjët **a** dhe **b**, tek i cili edhe sipërfaqja **s**, e cila llogaritet brenda funksionit **jeta**, deklarohet si parametër referent.

```
// Programi funkPd
#include <iostream>
using namespace std;

void Jeta(double &a,double &b,double &s);

int main()
{
    double a,b,s;
    cout << "\nBrinjët e drejtkëndëshit"
          << "\n\nBrinja a: ";
    cin >> a;
    cout << "\nBrinja b: ";
    cin >> b;

    Jeta(a,b,s);

    cout << "\nSipërfaqja e llogaritur s: "
          << s
          << "\n\n";
    return 0;
}

void Jeta(double &a,double &b,double &s)
{
    s=a*b;
}
```

Këtu, para emrit të nënprogramit është shënuar fjala **void**, përkatësisht në fund të komandës **return** te nënprogrami nuk është shënuar rezultati **s**, sepse vlera e sipërfaqes merret përmes parametrit aktual përkatës gjatë thirrjes së funksionit. Rezultati që fitohet në ekran pas ekzekutimit të programit do të duket ashtu siç është dhënë më parë në *Fig. 5.4*, nëse përmes tastierës kompjuterit i jepen vlera të njëjta hyrëse.

Fushat referente

Gjatë operimit me anëtarët e fushave, si referente mund të deklarohen edhe fushat. Si rezultat, vlerat e anëtarëve të fushës referente barazohen me vlerat e anëtarëve të një fushe tjetër.

Vektorët referentë

Në formë të përgjithshme, deklarimi i vektorit referent duket:

t (&R)[n]=V;

ku janë:

R - vektori referent.

t - tipi i vektorit referent.

n - numri i anëtarëve të vektorit referent.

V - vektori të cilit i referohet vektori referent.

Pas këtij deklarimi, vektorin **R** kompjuteri e llogarit si sinonim të vektorit **V**. Si rezultat, ndryshimet që ndodhin te vlerat e anëtarëve të njërit vektor do përcillet edhe tek anëtarët me indekse përkatëse të vektorit tjetër.

Shembull

Programi **refV**, përmes së cilit tregohet deklarimi i vektorit **B** si vektor referent i vektorit **A**.

```
// Programi refV
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int n=5;
    int i;
    double A[n]={7,3,9,2,4};

    cout << "\nVektori A\n\n";
    for (i=0;i<n;i++)
        cout << setw(3)
            << A[i];

    double (&B)[n]=A;

    cout << "\n\nVektori B\n\n";
    for (i=0;i<n;i++)
```

```

        cout << setw(3)
              << B[i];
    cout << "\n\n";
return 0;
}

```

Rezultati që shtypet në ekran pas ekzekutimit të programit të dhënë do të duket si në Fig.5.5.

```

Vektori A
  7  3  9  2  4
Vektori B
  7  3  9  2  4
Press any key to continue

```

Fig.5.5

Pamja e ekranit pas ekzekutimit të programit *refV1*

Nga rezultati i shtypur shihet se vlerat e anëtarëve të vektorëve **A** dhe **B** janë të barabarta. Kjo është rezultat i barazimit të tyre gjatë deklarimit të vektorit **B** si vektor referent i vektorit **A**. Por, vlerat e ndryshuara tek anëtarët e veçantë të cilit do nga vektorët automatikisht përcillen e dhe tek anëtarët e vektorit tjetër. Kështu, p.sh., nëse pas deklarimit të vektorit referent të programi i mësipërm shkruhen shprehjet:

```

A[2]=-5;
B[4]=-8;

```

rezultati që shtypet në ekran do të duket si në Fig.5.6.

```

Vektori A
  7  3  9  2  4
Vektori B
  7  3 -5  2 -8
Press any key to continue

```

Fig.5.5

Pamja e ekranit pas ekzekutimit të programit *refV1* pas ndryshimit të vlerave të dy anëtarëve të vektorëve

Matricat referente

Deklarimi i matricës referente në formë të përgjithshme duket kështu:

```
t (&R)[m][n]=M;
```


ku janë:

R - matrica referente.

t - tipi i matricës referente.

m, n - numri i rreshtave dhe i kolonave të matricës referente.

M - matrica së cilës i referohet matrica referente.

Pas këtij deklarimi, ndryshimet që ndodhin te vlerat e anëtarëve të njëjës matricë do të përcillen edhe tek anëtarët me indekse të njëjta të matricës tjetër, meqë kompjuteri i llogarit si matrica sinonime.

Shembull

Programi **refM**, përmes së cilit tregohet deklarimi i matricës **B** si matricë referente të matricës **A**.

```
// Programi refM
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    const int m=4,n=5;
    int i,j;

    double A[m][n]={{-4,6,9,2,1},
                    {6,-3,8,5,9},
                    {-1,7,3,8,5},
                    {4,3,2,-7,2}};

    cout << "\nMatrica A\n\n";
    for (i=0;i<m;i++)
    {
        for (j=0;j<n;j++)
            cout << setw(5)
                << A[i][j];
        cout << "\n";
    }

    double (&B)[m][n]=A;

    cout << "\nMatrica B\n\n";
    for (i=0;i<m;i++)
    {
        for (j=0;j<n;j++)
            cout << setw(5)
                << B[i][j];
        cout << "\n";
    }
}
```

```

    }

    cout << "\n";
return 0;
}

```

Nëse ekzekutohet programi i dhënë, do të shtypen vlerat e anëtarëve të dy matricave. Meqë matricat janë rferente mes vete, anëatrët e tyre do të kenë vlera të njëjta, ashtu siç shihet në *Fig. 5.6*.

Fig. 5.6
Pamja e ekranit pas ekzekutimit të programit
refM

```

Matrica A
-4  6  9  2  1
 6 -3  8  5  9
-1  7  3  8  5
 4  3  2 -7  2

Matrica B
-4  6  9  2  1
 6 -3  8  5  9
-1  7  3  8  5
 4  3  2 -7  2

Press any key to continue

```

Edhe këtu mund të provohet se si ndryshimet e vlerave të anëtarëve të njëra matricë automatikisht do të përcillen te matrica sinonime e saj.

Kufizime për variablat referente

Nuk lejohet deklarimi më shumë se një herë i variablës referente të njëjtë. Kështu, p.sh., kompjuteri nuk i lejon njëkohësisht deklaratimet:

```

int &x=a;
int &x=b;

```

sepse këtu tipi i variablës së njëjtë **x** deklarohet dy herë.

Nuk mund të deklarohet pointer në variablën referente, si dhe variablës referente nuk mund t'i referohemi.

Parametrat referentë brenda strukturave

Parametrat e funksioneve që shfrytëzohen brenda komponenteve të strukturave mund të merren si parametra referentë. Kjo ka rëndësi të veçantë kur prej strukturës merren rezultatet e llogaritjeve të ndryshme.

Shembull

Programi **refs**, në të cilin shfrytëzohet struktura **kater** me variablat e brinjëve **a** e **b**, si dhe e sipërfaqes **s** dhe e perimetrit **p** të drejtkëndëshit në komponentet e saj. Për t'i marrë nga struktura vlerat e variablave **s** dhe **p**, shfrytëzohet funksioni **merri** me parametrat referentë **x** dhe **y**.

```
// Programi refs
#include <iostream>
using namespace std;

struct kater
{
    double a,b,s,p;
    void lexo();
    void llogarit();
    void merri(double &x,double &y);
};

int main()
{
    kater alfa;
    double s,p;
    alfa.lexo();
    alfa.llogarit();
    alfa.merri(s,p);
    cout << "\nVlerat e llogaritura"
          << "\n\nSipërfaqja s="
          << s
          << "\nPerimetri p="
          << p
          << "\n\n";
    return 0;
}

void kater::lexo()
{
    cout << "\nVlerat e lexuara"
          << "\n\nBrinja a: ";
    cin >> a;
    cout << "Brinja b: ";
    cin >> b;
}

void kater::llogarit()
{
    s=a*b;
    p=2*(a+b);
}
```

```

return;
}
void kater::merri(double &x,double &y)
{
    x=s;
    y=p;
}

```

Nëse gjatë ekzekutimit të programit, për brinjët e drejtkëndëshit kompjuterit i jepen vlerat hyrëse **5** dhe **4**, rezultati në ekran do të duket ashtu siç është dhënë në Fig.5.7.

Fig.5.7
Pamja e ekranit pas ekzekutimit të programit
refs

```

Ulerat e lexuara
Brinja a: 5
Brinja b: 4

Ulerat e llogaritura
Sipërfaqja s=20
Perimetri p=18

Press any key to continue

```

Shfrytëzimi i parametrave referentë veçanërisht ka rëndësi, nëse duhet të merren të dhënat e përfshira në komponentet private të strukturave.

Parametrat referentë brenda klasave

Ngjashëm si te strukturat, edhe te klasat mund të shfrytëzohen parametra referentë si parametra të funksioneve brenda komponenteve të tyre, gjë që është shpjeguar edhe më parë te kapitulli mbi klasat.

Shembull

Versioni **refc** i programit **refs**, në të cilin në vend të strukturës shfrytëzohet klasa **kater**.

```

// Programi refC
#include <iostream>
using namespace std;

class kater
{
private:
    double a,b,s,p;
public:
    void lexo();
    void llogarit();
}

```

```
        void merri(double &x,double &y);  
};  
.....
```

Këtu është dhënë vetëm pjesa e parë e programit, ku deklarohet klasa **kater**. Kurse, pjesa tjetër e tij nuk dallon aspak nga programi **refs**, i cili u dha më sipër. Në këtë rast, vlerat që merren përmes funksionit **merri** përfshihen në komponenten private të klasës. Nëse ekzekutohet programi i dhënë, për vlera hyrëse të njëjta rezultati do të duket ashtu siç është dhënë në *Fig.5.7*.

Objektet referente

Objektet e strukturave dhe të klasave mund të deklarohen si objekte referente plotësisht njëloj, siç deklarohen edhe variablat e zakonshme. Si rezultat, të dhënat që u shoqërohen variablave brenda komponenteve të strukturave dhe të klasave të objekteve referente do të barazohen me vlerat e variablave përkatëse në objektet të cilave u referohen.

Shembull

Programi **ref0**, përmes së cilit tregohet deklarimi i objektit referent **nata** i strukturës **koha** të cilit i referohet objekti **dita** i strukturës së njëjtë.

```
// Programi ref0  
#include <iostream>  
using namespace std;  
struct koha  
{  
    int a;  
    float b;  
    void shtypja();  
};  
  
int main()  
{  
    koha dita;  
    cout << "\nLeximi i vlerave"  
        << "\n\n  Variabla a: ";  
    cin >> dita.a;  
    cout << "  Variabla b: ";  
    cin >> dita.b;  
    cout << "\nObjekti dita\n";  
    dita.shtypja();  
    koha &nata=dita;  
    cout << "\nObjekti nata\n";  
}
```

```

        nata.shtypja();
        cout << endl;
    return 0;
}

void koha::shtypja()
{
    cout << "\n  Variabla a="
          << a;
    cout << "\n  Variabla b="
          << b
          << "\n";
}

```

Në programin e dhënë, fillimisht, deklarohet objekti **dita** i strukturës **koha**, në komponentet e së cilës, përveç variablave **a** dhe **b**, përfshihet edhe funksioni **shtypja** përmes së cilit shtypen vlerat përkatëse të variablave në fjalë. Pastaj, deklarohet objekti **dita** dhe përmes komandave përkatëse lexohen vlerat e variablave të tij. Në fund, përmes komandës:

```
koha &nata=dita;
```

deklarohet objekti referent **nata**, i cili i referohet objektit **dita**. Në këtë mënyrë, objektet **dita** dhe **nata** bëhen objekte sinonime, gjë që rezulton në përcjelljen e vlerave të variablat brenda dy objekteve në fjalë, sa herë që ato ndryshohen të njëri prej objekteve.

Nëse ekzekutohet programi i dhënë, për vlerat hyrëse të cilat kompjuterit i jepen përmes tastierës, rezultati në ekran do të duket ashtu siç është dhënë në Fig.5.8.

```

Leximi i vlerave
  Variabla a: 9
  Variabla b: 5

Objekti dita
  Variabla a=9
  Variabla b=5

Objekti nata
  Variabla a=9
  Variabla b=5

Press any key to continue

```

Fig.5.8
Pamja e ekranit pas ekzekutimit të programit
ref0

Plotësisht njëllor mund të deklarohen objekte referente edhe kur kemi të bëjmë me klasat.

	6	
Fajllat		
Fajllat me qasje sekuenciale 317		
Qasja te fajllat në unazë 328		
Tekstet në fajlla 341		
Tekstet dhe numrat në fajlla 343		
Shkruarja dhe leximi i karaktereve 347		
Leximi i rreshtave 352		
Mode të hapjes së fajllave 356		
Pozita në fajll 360		
Fajllat me qasje direkte 372		
Objektet në fajlla 391		
Disa fajlla të hapur njëkohësisht 404		

Kur flitet për zhvendosjen e të dhënave, qoftë gjatë shtypjes në ekran, ose leximin e tyre duke ia dhënë kompjuterit përmes tastierës, si dhe gjatë shkruarjes në fajlla dhe leximit të tyre prej fajllave, shfrytëzohen objekte që njihen si *rrjedha* (ang. stream). Përmes objekteve të tilla dhe funksioneve (metodave) brenda komponenteve të klasave të ndryshme mundësohet *rrjedhja e të dhënave si vargje simbolesh*.

Shfrytëzimi i fajllave fillon me hapjen e tyre, gjatë së cilës emrave fizikë të tyre u shoqërohen *rrjedhat* përkatëse. Për operim me të dhënat që shkruhen në fajlla dhe lexohen prej tyre, ose siç thuhet - për operim me *rrjedhat e fajllave* (ang. file stream), në gjuhën C++ shfrytëzohen klasat për hyrje/dalje (ang. Input/Output, **I/O**), të nxjerra nga klasa bazë **ios**, organizimi hierarkik i të cilave shihet në *Fig.7.1*.

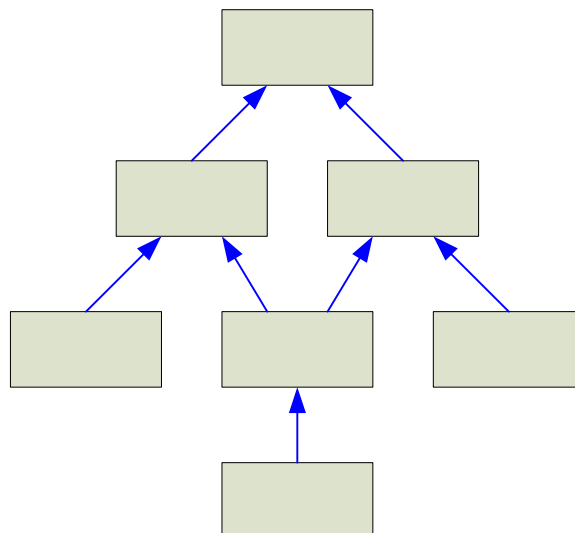


Fig. 7.1
Klasat për hyrje/dalje

Gjatë punës me fajlla, në fillim të çdo programi patjetër vendosen komandat paraprocesorike me fajllat e ballinës **iostream** dhe **fstream**.

Fajllat me qasje sekuenciale

Fajllat të cilët të dhënat shkruhen ose lexohen si sekuenca (vargje) bajtësh, paraqesin *fajlla me qasje sekuenciale* (ang. sequential-access files). Te fajllat e tillë, si njësi elementare që mund të shkruhet në fajll, ose të lexohet nga fajlli është *bajti*. Në një bajt mund të ruhet një karakter, ku me karakter nënkuptohet një *shifër e numrave dhjetorë* (**0**, **1**, ..., **9**), një *shkronjë* (**a**, **b**, ..., **z**, **A**, **B**, ..., **Z**) ose një *simbol special* (**+**, **-**, *****, **/**, **!**, **@**, **#**, **\$**, **&**, **%** etj.).

Zakonisht, te fajllat me qasje sekuenciale të dhënat shkruhen si tekste, prandaj këta fajlla paraqesin *fajlla tekstuale* (ang. text file). Për çdo karakter që shkruhet në fajllat e tillë shfrytëzohet një bajt. Kështu, p.sh., për shkruarje në fajll të numrit **254781** shfrytëzohen **6** bajtë, aq sa ka shifra numri i dhënë.

Gjatë shfrytëzimit të të dhënave te fajllat me qasje sekuenciale, është rëndë të pozicionohemi direkt në një të dhënë të dëshiruar. Gjithashtu, azhurimi i të dhënave në fajllat e tillë ka kufizime, meqë gjatë ndryshimit të të dhënave brenda tyre mund të shfrytëzohet vetëm numri ekzistues i bajtëve. Kështu, p.sh., nëse në vend të numrit **3** shifror shkruhet një numër **5** shifror, të dhënat në fajll do të dëmtohen, sepse mbulohen **2** bajtë të të dhënës pas numrit **3** shifror. Për azhurimin e numrit në fjalë, në këtë rast duhet të krijohet një fajll i ri, në të cilin fillimisht do të përshkruhet komplet përmbajtja e fajllit para numrit që azhurohet, pastaj të insertohet numri **5** shifror dhe në fund të fajllit të përshkruhet edhe pjesa e mbetur e tij.

Shkruarja në fajll

Të dhënat mund të shkruhen në fajll pasi paraprakisht të hapet fajlli, duke krijuar një objekt, përkatësisht rrjedhë të klasës **ofstream**. Për krijimin e objektit dhe hapjen e fajllit, komanda përkatëse në formën e saj themelore shkruhet:

```
ofstream r("f",ios::out);
```

ku janë:

r - rrjedha e deklaruar si objekt i klasave për punë me fajlla.

f - emri i fajllit që hapet.

ios::out - modi i hapjes së fajllit për shkruarje.

Me emrin **f** të fajllit që hapet këtu nënkuptohet edhe folderi (direktoriumi), përkatësisht komplet shtegu në të cilin vendoset fajlli.

Shembull

Programi **file1**, përmes së cilit në fajllin **Koha.txt** regjistrohen vlerat **77** dhe **58.94** të variablave **k** dhe **x**.

```
// Programi file1
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    int k=77;
    double x=58.94;

    ofstream Shkruaj("D:/Libra/Koha.txt",ios::out);

    Shkruaj << k
            << ' '
            << x;

    cout << "\nShkruarja në fajll përfundoi"
         << "\n\n";
    return 0;
}
```

Në program, përmes komandës:

```
ofstream Shkruaj("D:/Libra/Koha.txt",ios::out);
```

këtu krijohet rrjedha **Shkruaj** si objekt i klasës **ofstream**, përmes së cilës hapet fajlli **Koha.txt**, në folderin **Libra** të diskut **D:**, gjë që përcaktohet me shprehjen e shkruar nën thonjëza:

```
"D:/Libra/Koha.txt"
```

Më sakt, me shprehjen:

```
D:/Libra/
```

këtu nënkuptohet shtegu ku vendoset fajlli.

Prapashtesa **.txt** e fajllit të shembulli i dhënë më sipër është zgjedhur lirisht, meqë kemi të bëjmë me fajll tekstual. Por, kjo prapashtesë mund të merret si **.doc**, ose edhe e çfarëdoshme.

Përmes pjesës **ios::out**, jepet modi i hapjes së fajllit dhe kompjuteri njoftohet se të fajlli që hapet do të *shkruhet në dhënë*, përkatësisht fajlli do të *përdoret për dalje*.

Shkruarja në fajll e vlerave të variablave **k** dhe **x** realizohet përmes komandës:

```
Shkruaj << k
        << ' '
        << x;
```

e cila është e ngjashme me komandën e zakonshme **cout**, që shfrytëzohet për shtypje në ekran. Këtu, me zbrazësinë e përfshirë nën thonjëza është paraparë që mes vlerave të variablave **k** dhe **x** në disk të lihet një vend i zbrazët.

Në fund të programit, me komandën:

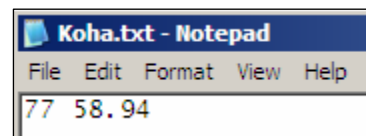
```
cout << "\nShkruarja në fajll përfundoi"
      << "\n\n";
```

në ekran shtypet mesazhi i shënuar nën thonjëza, për të na njoftuar se shkruarja në fajll ka përfunduar.

Nëse fajlli **Koha.txt** hapet me ndonjë program për përpunim të tekstit, p.sh., siç është programi *Microsoft Notepad*, në të do të shihen vlerat e variablave **k** dhe **x**, të shkruara me një zbrazësinë mes tyre, ashtu siç është dhënë në Fig.7.2.

Fig.7.2

Përmbajtja e fajllit **Koha.txt** pas ekzekutimit të programit **file1**



Zbrazësira mes dy numrave lihet me qëllim që ato të dallohen mes vete, përkatësisht të mos duken si një numër, gjë që ka rëndësi gjatë leximit të tyre nga fajlli.

Fajlli mund të hapet për shkruarje edhe duke e krijuar objektin e rrjedhës **r** përkatëse përmes komandës:

```
ofstream r("f");
```

përkatësisht pa e deklaruar modin e hapjes së tij **ios::out**, sepse kur shfrytëzohet klasa **ofstream**, ky mod është *i nënkuptuar* (ang. default). Në shembullin e programit të dhënë më sipër, kjo komandë do të shkruhet:

```
ofstream Shkruaj("D:/Libra/Koha.txt");
```

Gjatë hapjes së fajllit përmes komandës në fjalë, nëse fajlli që hapet ka ekzistuar më parë, kompjuteri *së pari e fshin dhe pastaj e rihap atë*.

Pasi të përfundojë shfrytëzimi i fajllit, ai duhet të mbyllet përmes funksionit **close**, i cili në formë të përgjithshme shkruhet:

```
r.close();
```

ku me **r** është shënuar rrjedha e deklaruar si objekt i klasës së shfrytëzuar për hapje të fajllit. Kështu, p.sh., te programi **file1**, që u dha më sipër, komanda për mbyllje të fajllit të shfrytëzuar do të duket:

```
Shkruaj.close();
```

Kjo komandë duhet të vendoset në pjesën e fundit të programit, pasi të përfundojë shkruarja e të dhënave dhe qasja e mëtejshme në fajll të jetë e panevojshme. Nëse nuk shfrytëzohet kjo komandë, kompjuteri automatikisht do ta mbyllë fajllin, kur përfundon ekzekutimi i tij.

Memoria ndërmjetësuese

Gjatë shkruarjes së të dhënave në fajll, ato fillimisht regjistrohen në një *memorie ndihmëse*, ose në *memorie ndërmjetësuese*, përkatësisht në *bafer* (ang. buffer). Meqë baferi ka një madhësi të kufizuar, pas mbushjes së tij, të dhënat prej baferit përcillen në fajll. Por, duke e përdorur manipulatorin **endl**, përveç që shkruhet në fajll karakteri manipulorit për kalim në rresht të ri, kompjuteri e zbraz edhe baferin, duke i shkruar të dhënat në fajll, pavarësisht se a është mbushur ai me të dhëna.

Baferi mund të zbrazet edhe duke e shfrytëzuar funksionin **flush**. Por, për dallim nga manipulatori **endl**, gjatë shfrytëzimit të këtij funksioni në fajll nuk shkruhet karakteri për kalim në rresht të ri.

Përdorimi i funksionit **flush**, ose i manipulatorit **endl** për zbrazje të baferit lidhet me rritjen e sigurisë së shkruarjes së të dhënave në fajll. Kjo ka rëndësi veçanërisht në kushtet e ndërprerjes së pakontrolluar të punës së kompjuterit, p.sh., kur ndërpritet furnizimi me energji elektrike. Në ato raste, të dhënat që ruhen në bafer nuk do të shkruhen në fajll, sepse me ndërprerjen e punës së kompjuterit, përmbajtja e baferit fshihet.

Në programin **file1** të dhënë më sipër, përdorimi i manipulatorit **endl** dhe i funksionit **flush** do të dukej ashtu siç është dhënë në **3** versionet vijuese të komandës për shkruarje në fajll.

- ```
Shkruaj << k
 << ' '
 << x
 << flush;
```
- ```
Shkruaj << k
        << ' '
        << x;
      Shkruaj.flush();
```
- ```
Shkruaj << k
 << ' '
 << x
 << endl;
```

Baferi ndërmjetësues zbrazet dhe komplet përmbajtja e tij përcillet në disk edhe kur mbyllet fajlli, qoftë përmes komandës **close**, ose kur fajlli mbyllet automatikisht në fund të programit.

## Leximi nga fajlli

Të dhënat e regjistruara në fajll mund të lexohen pasi paraprakisht të hapet fajlli për lexim. Për krijimin e objektit dhe hapjen e fajllit, komanda përkatëse në formën e saj themelore shkruhet:

```
ifstream r("f",ios::in);
```

ku janë:

**r** - rrjedha e deklaruar si objekt i klasës **ifstream**.

**f** - emri i fajllit që hapet.

**ios::in** - modi i hapjes së fajllit për lexim.

Si edhe gjatë hapjes së fajllit për shkruarje, me emrin e fajllit **f** nënkuptohet edhe komplet shtegu në të cilin vendoset fajlli.

**Shembull**

Programi **file2**, përmes së cilit nga fajlli **Koha.txt**, lexohen vlerat **77** dhe **58.94** të regjistruara paraprakisht me ekzekutimin e programit **file1** të dhënë më sipër.

```
// Programi file2
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
 int k;
 double x;

 ifstream Lexo("D:/Libra/Koha.txt",ios::in);

 Lexo >> k
 >> x;

 cout << "\nVlerat e lexuara nga fajlli"
 << "\n\n k="
 << k
 << "\n x="
 << x
 << "\n\n";
 return 0;
}
```

Këtu, për hapje të fajllit **Koha.txt** në modin për lexim, i cili është krijuar gjatë ekzekutimit të programit **file1**, është shfrytëzuar komanda:

```
ifstream Lexo("D:/Libra/Koha.txt",ios::in);
```

përmes së cilës deklarohet rrjedha **Lexo** si objekt i klasës **ifstream**.

Rezultati që shtypet në ekran pas ekzekutimit të programit të dhënë do të duket si në *Fig.7.3*.

Fig.7.3

Pamja e ekranit pas ekzekutimit të programit **file2**

```
Ulerat e lexuara nga fajlli
k=??
x=8.94
Press any key to continue
```

Fajlli mund të hapet për lexim edhe duke e krijuar objektin përkatës përmes komandës:

```
ifstream r("f");
```

pa e deklaruar edhe modin e hapjes së tij **ios::in**, sepse, kur shfrytëzohet klasa **ifstream**, ky mod është i nënkuptueshëm. Në shembullin e programit të dhënë më sipër, komanda në fjalë shkruhet:

```
ifstream Lexo("D:/Libra/Koha.txt");
```

Nuk është e thënë që variablat e shfrytëzuara gjatë leximit të të dhënave nga fajlli të quhen njëloj me variablat që janë shfrytëzuar për shkruarjen e tyre. Por, nënkuptohet se tipet e tyre duhet të zgjedhen ashtu që t'u përkasin vlerave që lexohen.

Edhe fajllat e hapur për lexim, në fund duhet të mbyllen duke e shfrytëzuar funksionin **close**, ashtu siç u shpjegua më sipër. Kështu, p.sh., për mbylljen e fajllit të shfrytëzuar për lexim në programin **file2**, funksioni duket:

```
Lexo.close();
```

## Kontrollimi i hapjes së fajllave

Tentimi për hapje të fajllave mund të mos jetë i suksesshëm. Gjatë kësaj, me qëllim të kontrollimit të gabimeve eventuale në hapje të tyre, shfrytëzohet komanda:

```
if(!r)
```

ku me **r** nënkuptohet rrjedha që i shoqërohet fajllit, përkatësisht objekti që krijohet gjatë hapjes së tij.

### Shembull

Versioni **file2g** i programit **file2**, përmes së cilit tregohet kontrollimi i hapjes së suksesshme të fajllit **Koha.txt**.



```
// Programi file2g
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
 int k;
 double x;

 ifstream Lexo("D:/Libra/Koha.txt", ios::in);

 if(!Lexo)
 {
 cout << "\nGabim gjatë hapjes së fajllit";
 goto Fundi;
 }

 Lexo >> k;
 Lexo >> x;

 cout << "\nVlerat e lexuara nga fajlli"
 << "\n\n k="
 << k
 << "\n x="
 << x;

Fundi:
 cout << "\n\n";
 return 0;
}
```

Siç shihet nga pjesa e theksuar e programit, nëse vlera e shprehjes brenda kllapave të komandës **if** është **true**, në ekran do të shtypet mesazhi:

**Gabim gjatë hapjes së fajllit**

dhe përmes komandës **goto Fundi**, ekzekutimi i programit përfundon. Që ta vërejmë gjenerimin e mesazhit në fjalë, p.sh., mund të marrim se në komandën për hapje të fajllit, në vend të diskut **D:** gabimisht është shënuar disku **C:** në të cilin nuk ekziston folderi **Libra**, kështu:

```
ifstream Lexo("C:/Libra/Koha.txt", ios::in);
```

Nënkuptohet se me efekt të njëjtë, shprehja brenda kllapave të komandës **if** mund të shkruhet edhe në këto forma:

```
if(!Lexo==true)
```

dhe

```
if(Lexo==false)
```

Gjatë shtypjes në ekran të mesazhit për gabim, mund të shfrytëzohet edhe komanda speciale **cerr**, në këtë mënyrë:

```
cerr << "\nGabim gjatë hapjes së fajllit";
```

Kjo komandë është e ngjashme me komandën **cout**, e cila është më e përgjithshme, sepse përveç për shtypje në ekran, shfrytëzohet edhe për shkruarje në pajisje të tjera (p.sh. në disk, në printer etj.).

Në komponentet e klasave që përdoren për operim me fajlla ekziston edhe funksioni **is\_open** përmes së cilit mund të kontrollohet hapja e suksesshme e fajllave. Kështu, p.sh., te programi **file2g**, i dhënë më sipër, përdorimi i funksionit në fjalë do të duket:

```
if (!Lexo.is_open())
{
 cout << "\nGabim gjatë hapjes së fajllit";
 goto Fundi;
}
```

## Deklarimi i objektit para hapjes

Fajlli mund të hapet për shkruarje pasi paraprakisht të jetë deklaruar objekti përkatës i klasës **ofstream**, në këtë mënyrë:

```
ofstream r;
r.open("f", ios::out);
```

ku janë:

**r** - rrjedha e deklaruar si objekt i klasave për punë me fajlla.

**f** - emri i fajllit që hapet.

**ios::out** - modi i hapjes së fajllit për shkruarje.

### Shembull

Versioni **file1a** i programit **file1**, të dhënë më sipër, tek i cili para se të hapet fajlli **Koha.txt** deklarohet rrjedha **Shkruaj** si objekt i klasës **ofstream**.

```
// Programi file1a
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
 ofstream Shkruaj;
 int k=77;
 double x=58.94;

 Shkruaj.open("D:/Libra/Koha.txt",ios::out);

 Shkruaj << k
 << ' '
 << x;

 cout << "\nShkruarja në fajll përfundoi"
 << "\n\n";
 return 0;
}
```

Këtu, mund të shfrytëzohet versioni i hapjes së fajllit pa e shënuar modin **ios::out**, kështu:

```
ofstream r;
r.open("f");
```

sepse, siç u tha edhe më sipër, ai nënkuptohet, meqë shfrytëzohet klasa **ofstream**.

Edhe gjatë leximit, fajlli mund të hapet duke e krijuar paraprakisht rrjedhën përkatëse si objekt të klasës **ifstream**:

```
ifstream r;
r.open("f",ios::in);
```

**Shembull**

Versioni **file2a** i programit **file2** të dhënë më sipër, tek i cili para se të hapet fajlli **Koha.txt** deklarohet objekti **Lexo** i klasës **ifstream**.

```
// Programi file2a
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
 ifstream Lexo;
 int k;
 double x;

 Lexo.open("D:/Libra/Koha.txt", ios::in);

 Lexo >> k
 >> x;

 cout << "\nVlerat e lexuara nga fajlli"
 << "\n\n k="
 << k
 << "\n x="
 << x
 << "\n\n";
 return 0;
}
```

Nënkuptohet se para ekzekutimit të këtij programi, te fajlli **Koha.txt** duhet të regjistrohen të dhënat që lexohen, p.sh., duke e ekzekutuar paraprakisht programin **file1a**. Rezultati në ekran, pas ekzekutimit të këtij versioni të programit do të duket si në *Fig.7.3*.

Edhe këtu mund të shfrytëzohet versioni pa e shënuar modin e hapjes **ios::in**, kështu:

```
ifstream r;
r.open("f");
```

sepse ai nënkuptohet, meqë është shfrytëzuar klasa **ifstream**.

Në praktikë, zakonisht, përdoren format e dhëna këtu për hapje të fajllave pasi të jenë krijuar rrjedhat si objekte të klasave përkatëse. Këto forma janë të domosdoshme, kur brenda një programi fajllat rihapen pasi paraprakisht të jenë mbyllur, sepse mënyra e krijimit të rrjedhave gjatë hapjes së fajllave, për shkak të dyfishimit të objekteve, e imponon nevojën e shfrytëzimit të rrjedhave me emra të tjerë.

## Qasja te fajllat në unazë

Komandat për shkruarje në fajlla dhe në lexim prej tyre mund të përfshihen edhe brenda unazave, pasi paraprakisht deklarohen rrjedhat si objekte të klasave përkatëse dhe të jenë hapur fajllat.

## Shkruarja në unazë

Me qëllim të shkruarjes, p.sh., në fajlla të të dhënave që fitohen përmes llogaritjeve të ndryshme brenda unazave, komandat për shkruarje në fajlla mund të përfshihen në strukturat e tyre.

### Shembull

Programi **file3**, përmes së cilit në fajllin **Rrethi.txt** regjistrohen vlerat e llogaritura të sipërfaqes **s** dhe të perimetrit **p** të rrethit, për vlera të ndryshme të rrezes **r**, mes vlerës fillestare **a=3** dhe vlerës përfundimtare **b=7**, duke e ndryshuar me hapin **h=0.5**.

```
// Programi file3
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
 double a=3,b=7,h=0.5,pi=3.1415926,r,s,p;

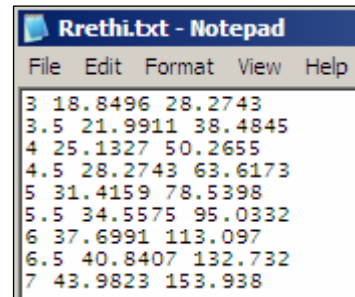
 ofstream Shkruaj("D:/Libra/Rrethi.txt",ios::out);

 for(r=a;r<=b;r=r+h)
 {
 s=2*pi*r;
 p=pi*r*r;
 Shkruaj << r
 << ' '
 << s
 << ' '
 << p
 << endl;
 }

 cout << "\nShkruarja në fajll përfundoi"
 << "\n\n";
```

```
return 0;
}
```

Nëse pas ekzekutimit të programit të dhënë hapet fajlli përmes programit *Microsoft Notepad*, në të do të shihen vlerat e shkruara, ashtu siç është dhënë në *Fig.7.4*.



*Fig.7.4*

*Përmbajtja e fajllit Rrethi.txt pas ekzekutimit të programit file3*

Këtu, në çdo rresht të fajllit janë shkruar **3** vlera: rrezja **r**, sipërfaqja **s** dhe perimetri përkatës **p**, të ndara mes vete me nga një zbrazësi. Por, nëse në vend të manipulorit **endl**, në fund të komandës për shkruarje në fajll, nën thonjëza shënohet një zbrazësi:

```
Shkruaj << r
 << ' '
 << s
 << ' '
 << p
 << ' ';
```

të dhënat në fajll do të vendosen në një rresht të vetëm. Në fakt, kur hapet fajlli **Rrethi.txt** përmes tekstprocesorit, paraqitja në rreshta të veçantë e të dhënave do të varet nga prania e simbolit për kalim në rreshta të rinj.

## Shfrytëzimi i manipulatoreve

Me qëllim të shkruarjes në fajll në numër të caktuar vendesh, mund të shfrytëzohet manipulatori **setw**. Gjatë kësaj, në fillim të programit duhet të vendoset komanda paraprocesorike me fajllin **io manip**.

### Shembull

Versioni **file3a** i programit **file3** të dhënë më sipër, tek i cili gjatë shkruarjes së të dhënave në fajllin **Rrethi.txt** shfrytëzohet edhe manipulatori **setw**.

```

// Programi file3a
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

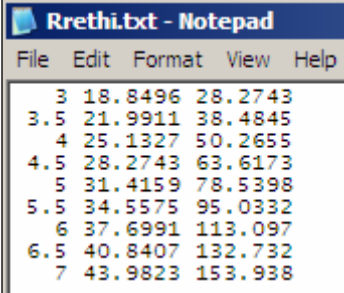
int main()
{
 double a=3,b=7,pi=3.1415926,r,s,p;

 ofstream Shkruaj("D:/Libra/Rrethi.txt",ios::out);

 for(r=a;r<=b;r=r+0.5)
 {
 s=2*pi*r;
 p=pi*r*r;
 Shkruaj << setw(4)
 << r
 << " "
 << setw(7)
 << s
 << " "
 << setw(7)
 << p
 << endl;
 }
 cout << "\nShkruarja në fajll përfundoi"
 << "\n\n";
 return 0;
}

```

Nëse në këtë rast, pas ekzekutimit të programit të dhënë hapet fajlli përmes programit *Microsoft Notepad*, përmbajtja e fajllit do të duket si në Fig.7.5.



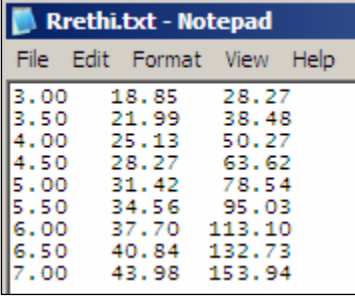
|     |         |         |
|-----|---------|---------|
| 3   | 18.8496 | 28.2743 |
| 3.5 | 21.9911 | 38.4845 |
| 4   | 25.1327 | 50.2655 |
| 4.5 | 28.2743 | 63.6173 |
| 5   | 31.4159 | 78.5398 |
| 5.5 | 34.5575 | 95.0332 |
| 6   | 37.6991 | 113.097 |
| 6.5 | 40.8407 | 132.732 |
| 7   | 43.9823 | 153.938 |

Fig.7.5  
Përmbajtja e fajllit *Rrethi.txt* pas ekzekutimit të programit *file3a*

Vlerat në fajllin **Rrethi.txt** mund të shkruhen edhe me një precizitet të caktuar, duke e shfrytëzuar versionin **file3b** të programit **file3a**, në të cilin te komanda për shkruarje në fajll përdoren edhe manipulatorët **fixed** dhe **setprecision**, p.sh., ashtu siç është dhënë në vijim.

```
Shkruaj << fixed
 << setprecision(2)
 << setw(4)
 << r
 << ' '
 << setw(7)
 << s
 << ' '
 << setw(7)
 << p
 << endl;
```

Pas kësaj, nëse ekzekutohet programi **file3b**, të dhënat brenda fajllit do të shkruhen me dy vende pas pikës dhjetore (shih *Fig.7.6*), të rrumbullakësura në bazë të rregullave të njohura për rrumbullakësim.



|      |       |        |
|------|-------|--------|
| 3.00 | 18.85 | 28.27  |
| 3.50 | 21.99 | 38.48  |
| 4.00 | 25.13 | 50.27  |
| 4.50 | 28.27 | 63.62  |
| 5.00 | 31.42 | 78.54  |
| 5.50 | 34.56 | 95.03  |
| 6.00 | 37.70 | 113.10 |
| 6.50 | 40.84 | 132.73 |
| 7.00 | 43.98 | 153.94 |

*Fig.7.6*

*Përmbajtja e fajllit Rrethi.txt pas ekzekutimit të programit file3b*

## Leximi në unazë

Për leximin e të dhënave që janë shkruar në më shumë rreshta të një fajlli duhet të shfrytëzohet unaza përkatëse, e ngjashme me unazën që është shfrytëzuar gjatë shkruarjes së tyre në fajll.

### Shembull

Programi **file4** për leximin e të dhënave të regjistruara në fajllin **Rrethi.txt**, të cilat janë shkruar paraprakisht në fajll përmes programit **file3b**.



```
// Programi file4
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main()
{
 double r,s,p;
 int i;

 ifstream Leximi("D:/Libra/Rrethi.txt",ios::in);

 cout << "\nTë dhënat e lexuara nga fajlli"
 << "\n\n r s p\n\n"
 << fixed
 << setprecision(2);

 for(i=1;i<=9;i++)
 {
 Leximi >> r
 >> s
 >> p;

 cout << setw(8)
 << r
 << setw(7)
 << s
 << setw(7)
 << p
 << "\n";
 }
 cout << endl;
 return 0;
}
```

Këtu, për leximin e **9** rreshtave me të dhëna në fajllin **Rrethi.txt** është shfrytëzuar unaza me variablën **i**, vlerat e së cilës ndryshojnë mes vlerës **1** dhe **9**, me hapin **1**. Pas ekzekutimit të programit, rezultati që shtypet në ekran përmes komandës **cout** do të duket si në *Fig.7.7*.

Fig.7.7  
 Pamja e ekranit pas ekzekutimit të  
 programit **file4**

| Të dhënat e lexuara nga fajlli |       |        |
|--------------------------------|-------|--------|
| r                              | s     | p      |
| 3.00                           | 18.85 | 28.27  |
| 3.50                           | 21.99 | 38.48  |
| 4.00                           | 25.13 | 50.27  |
| 4.50                           | 28.27 | 63.62  |
| 5.00                           | 31.42 | 78.54  |
| 5.50                           | 34.56 | 95.03  |
| 6.00                           | 37.70 | 113.10 |
| 6.50                           | 40.84 | 132.73 |
| 7.00                           | 43.98 | 153.94 |
| Press any key to continue      |       |        |

Por, zakonisht, nuk dihet se sa të dhëna janë shkruar në fajll. Prandaj, për leximin e të dhënave nga fajlli, zakonisht shfrytëzohet unaza **while**, ekzekutimi i së cilës përsëritet derisa nuk arrihet në fund të fajllit.

#### Shembull

Versioni **file4a** i programit **file4**, tek i cili për leximin e të dhënave të cilat janë regjistruara paraprakisht në fajllin **Rrethi.txt** shfrytëzohet unaza **while**.

```
// Programi file4a
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main()
{
 double r,s,p;

 ifstream Lexo("D:/Libra/Rrethi.txt",ios::in);

 cout << "\nTë dhënat e lexuara nga fajlli"
 << "\n\n r s p\n\n"
 << fixed
 << setprecision(2);

 while (!Lexo.eof())
```

```
{
 Lexo >> r
 >> s
 >> p;
 if (!Lexo.eof())
 cout << setw(8)
 << r
 << setw(7)
 << s
 << setw(7)
 << p
 << "\n";
 }
 cout << endl;
return 0;
}
```

Në programin e dhënë, procedura e leximit të të dhënave nga fajlli do të vazhdojë derisa është **false** vlera brenda kllapave të komandës:

```
while (!Lexo.eof())
```

përkatësisht, derisa *pointeri i pozitës në fajll* nuk tregon në fundin e tij, kur vlera e funksionit **eof** bëhet **true**. Kjo do të thotë se komanda në fjalë mund të shkruhet edhe kështu:

```
while (!Lexo.eof()==true)
```

gjë që nënkuptohet nga kompjuteri.

Kur mbërrihet në fund të fajllit, funksionit **eof** i shoqërohet vlera **-1**, e cila vlerë nuk është e shkruar në fajll.

Përmes komandës:

```
if (!Lexo.eof())
```

para shtypjes në ekran të vlerave të lexuara, shqyrtohet se mos ndoshta gjatë ekzekutimit të komandës për lexim është arritur në fund të fajllit. Nëse nuk shfrytëzohet kjo komandë, rreshti i fundit në tabelën me vlera do të shtypet dy herë.

## Flamujt e statusit

Gjatë punës me fajlla mund të shfrytëzohen *flamujt e statusit* (ang. status flags), të dhënë në *Fig. 7.8*, vlerat e të cilëve lidhen me objektet e rrjedhave të fajllave.

| Emri i flamurit | Tregon                                              |
|-----------------|-----------------------------------------------------|
| <b>eofbit</b>   | arritjen në fund të fajllit.                        |
| <b>failbit</b>  | dështim gjatë operimit.                             |
| <b>badbit</b>   | operim të gabueshëm.                                |
| <b>hardfail</b> | gabim të pakorrigjueshëm.                           |
| <b>goodbit</b>  | operim të suksesshëm (asnjë flamur nuk ndryshohet). |

Fig.7.8 Flamujt e statusit të objekteve të rrjedhave të fajllave

Vlerat e flamujve të statusit merren përmes funksioneve të dhëna në kolonën e parë të tabelës që shihet në Fig.7.9.

| Funksioni      | Përdorimi                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>eof()</b>   | E jep vlerën <b>true</b> , nëse <b>eofbit</b> është inicializuar.                                                     |
| <b>fail()</b>  | E jep vlerën <b>true</b> , nëse inicializohet njëri nga flamujt: <b>failbit</b> , <b>badbit</b> ose <b>hardfail</b> . |
| <b>bad()</b>   | E jep vlerën <b>true</b> , nëse inicializohet njëri nga flamujt: <b>badbit</b> ose <b>hardfail</b> .                  |
| <b>good()</b>  | E jep vlerën <b>true</b> , nëse nuk inicializohet asnjë flamur, përkatësisht nëse çdo gjë është në rregull.           |
| <b>clear()</b> | I fshin vlerat në të gjithë flamujt e statusit të rrjedhës.                                                           |

Fig.7.9 Funksionet për marrje të vlerave të flamujve të statusit

#### Shembull

Versioni **file4b** i programit **file4a**, në të cilin brenda unazës urdhërohet shtypja e vlerave të funksioneve **eof**, **fail** dhe **good**.

```
// Programi file4b
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main()
{
 double r,s,p;

 ifstream Lexo("D:/Libra/Rrethi.txt",ios::in);
```

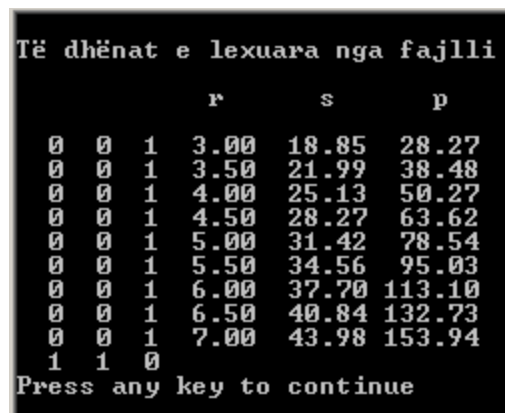
```

cout << "\nTë dhënat e lexuara nga fajlli"
 << "\n\n r s p\n\n"
 << fixed
 << setprecision(2);

while (!Lexo.eof())
{
 Lexo >> r
 >> s
 >> p;
 cout << " "
 << Lexo.eof()
 << " "
 << Lexo.fail()
 << " "
 << Lexo.good();
 if (!Lexo.eof())
 cout << setw(6)
 << r
 << setw(7)
 << s
 << setw(7)
 << p
 << "\n";
}
cout << endl;
return 0;
}

```

Nëse ekzekutohet programi i dhënë, në tri kolonat e para do të shtypen vlerat e funksioneve **eof**, **fail** dhe **good**, ashtu siç shihet në *Fig.7.10*.



```

Të dhënat e lexuara nga fajlli
 r s p
0 0 1 3.00 18.85 28.27
0 0 1 3.50 21.99 38.48
0 0 1 4.00 25.13 50.27
0 0 1 4.50 28.27 63.62
0 0 1 5.00 31.42 78.54
0 0 1 5.50 34.56 95.03
0 0 1 6.00 37.70 113.10
0 0 1 6.50 40.84 132.73
0 0 1 7.00 43.98 153.94
1 1 0
Press any key to continue

```

*Fig.7.10*  
Pamja e ekranit pas ekzekutimit të  
programit *file4b*

Nga rezultatet e shtypura në tri kolonat e para shihet se gjatë gjithë kohës, kur nga fajlli lexohen të dhënat, në ekran shtypen vlerat **0**, **0** dhe **1**. Kjo do të

thotë se funksionet **eof** dhe **fail** i japin vlerat **false**, sepse nuk është mbërri në fund të fajllit dhe leximi nuk ka dështuar. Kurse vlera e funksionit **good** është **true**, sepse gjatë kësaj kohe leximi i të dhënave është në rregull. Në fund, kur lexohet komplet përmbajtja e fajllit, vlerat e funksioneve në fjalë ndryshojnë në **1, 1** dhe **0**, gjë që është rezultat i ndryshimit të vlerave të flamujve përkatës, mbështetur në logjikën e shpjeguar më sipër.

Komanda e unazës **while** mund të shkruhet edhe në këtë mënyrë:

```
while(Lexo && !Lexo.eof())
```

Me pjesën e parë të kësaj komande përcaktohet që ekzekutimi i unazës të vazhdojë derisa rrjedha **Lexo** e fajllit është **true**, përndryshe duhet të dilet nga unaza.

## Shkruarja dhe leximi në një program

Brenda një programi, përveç që mund të shkruhen të dhënat në fajll, njëkohësisht ato edhe mund të lexohen.

## Shfrytëzimi i dy objekteve të veçanta

Për shkruarjen e të dhënave në fajll dhe leximin e tyre prej fajlli mund të shfrytëzohen dy objekte të veçanta - njëri për shkruarje dhe tjetri për lexim.

### Shembull

Programi **file5**, përmes së cilit në fajllin **Rrethi.txt**, së pari shkruhen të dhënat e rrezes **r**, të sipërfaqes **s** dhe të perimetrit **p** të rrethit. Pastaj, në pjesën e dytë të programit lexohen të dhënat e shkruara në fajllin në fjalë dhe të njëjtat shtypen në ekran.

```
// Programi file5
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main()
{
 double a=3,b=7,h=0.5,pi=3.1415926,r,s,p;

 // ----- Shkruarja në fajll -----

 ofstream Shkruaj("D:/Libra/Rrethi.txt",ios::out);
```

```
for(r=a;r<=b;r=r+h)
{
 s=2*pi*r;
 p=pi*r*r;
 Shkruaj << fixed
 << setprecision(2)
 << setw(4)
 << r
 << ' '
 << setw(7)
 << s
 << ' '
 << setw(7)
 << p
 << endl;
}
cout << "\nShkruarja në fajll përfundoi"
 << "\n";

Shkruaj.close();

// ----- Leximi prej fajllit -----

ifstream Lexo("D:/Libra/Rrethi.txt",ios::in);

cout << "\nTë dhënat e lexuara nga fajlli"
 << "\n\n r s p\n\n"
 << fixed
 << setprecision(2);

while (!Lexo.eof())
{
 Lexo >> r
 >> s
 >> p;

 if (!Lexo.eof())
 cout << setw(8)
 << r
 << setw(7)
 << s
 << setw(7)
 << p
 << "\n";
}
cout << endl;
return 0;
}
```

Këtu, në të vërtetë, janë bashkuar në një program dy versionet e programeve **file3b** dhe **file4a** të dhëna më sipër. Por, para hapjes së fajllit për lexim, ai është mbyllur, duke e shfrytëzuar komandën:

```
Shkruaj.close();
```

gjë që nuk është e domosdoshme.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket ashtu siç është dhënë në *Fig.7.7*.

## Shfrytëzimi i një objekti të vetëm

Gjatë hapjes së fajllit ekzistues, objekti përkatës mund të deklarohet duke e shfrytëzuar klasën **fstream**, në këtë formë:

```
fstream r("f",ios::out|ios::in);
```

ku janë:

**r** - rrjedha e deklaruar si objekt i klasave për punë me fajlla.

**f** - emri i fajllit që hapet.

**ios::out** - modi i hapjes së fajllit për shkruarje.

**ios::in** - modi i hapjes së fajllit për lexim.

Pas kësaj, rrjedha **r** që krijohet mund të shfrytëzohet si për shkruarje ashtu edhe për lexim.

Radha e shkruarjes së modeve të hapjes së fajllit, në pjesën e fundit të komandës së dhënë më sipër, nuk ka rëndësi.

### Shembull

Programi **file6**, përmes së cilit në fajllin **Koha.txt**, së pari regjistrohen vlerat **77** dhe **58.94** të variablave **k** dhe **x** dhe pastaj të njëjtat lexohen nga fajlli dhe shtypen në ekran.

```
// Programi file6
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main()
{
```

```
 int k=77;
 double x=58.94;
```

```
 fstream Dita("D:/Libra/Koha.txt",ios::in|ios::out);
```



```
// ----- Shkruarja në fajll -----

 Dita << k
 << ' '
 << x;

 cout << "\nShkruarja në fajll përfundoi"
 << "\n";

// ----- Leximi nga fajlli -----
 Dita.seekg(0);
 Dita >> k
 >> x;

 cout << "\nVlerat e lexuara nga fajlli"
 << "\n\n"
 << " k=" << k
 << "\n x=" << x
 << "\n\n";

return 0;
}
```

Në program, në fakt janë bashkuar pjesët e programeve **file1** dhe **file2** të dhëna më parë. Por, këtu, duke e shfrytëzuar klasën **fstream**, krijohet objekti **Dita** dhe hapet fajlli **Koha.txt** në dy mode - për shkruarje dhe për lexim.

Nëse ekzekutohet programi i dhënë, të dhënat që regjistrohen në fajll do të duken ashtu siç është dhënë në *Fig.7.2*. Kurse, rezultati i leximit nga fajlli në ekran do të shtypet ashtu siç është dhënë në *Fig.7.3*.

Nënkuptohet se mund të shfrytëzohet edhe versioni i hapjes së fajllit për shkruarje dhe lexim, tek i cili do të shfrytëzohet rrjedha **r** e klasës **fstream** e deklaruar më parë, kështu:

```
fstream r;
r.open("f",ios::out|ios::in);
```

Praktikisht, në shembullin e programit të dhënë më sipër, deklarimi i objektit **Dita** dhe shfrytëzimi i tij në hapjen e fajllit **Koha.txt** do të duket:

```
fstream Dita;
Dita.open("D:/Libra/Koha.txt",ios::out|ios::in);
```

## Tekstet në fajlla

Në fajlla mund të shkruhen dhe të lexohen edhe tekste, përkatësisht stringje.

### Shembull

Programi **file7** përmes së cilit tregohet shkruarja në fajllin **Dita.txt** e një fjalie dhe leximi i saj përmes variablës **g** të tipit **string**.

```
// Programi file7
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main()
{
// ----- Shkruarja në fajll -----
 ofstream Shkruaj("D:/Libra/Dita.txt",ios::out);

 Shkruaj << "Programimi i orientuar në objekte";

 cout << "\nShkruarja në fajll përfundoi"
 << "\n";

 Shkruaj.close();
// ----- Leximi nga fajlli -----
 string g;

 fstream Lexo("D:/Libra/Dita.txt",ios::in);

 cout << "\nTeksti që lexohet nga fajlli\n\n";

 while (!Lexo.eof())
 {
 Lexo >> g;
 cout << g
 << endl;
 }
 cout << "\nLeximi nga fajlli përfundoi"
 << "\n\n";
 return 0;
}
```

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.7.11.

Fig.7.11

Pamja e ekranit pas ekzekutimit të programit **file7**

```
Shkruarja në fajll përfundoi
Teksti që lexohet nga fajlli
Programimi
i
orientuar
në
objekte
Leximi nga fajlli përfundoi
Press any key to continue
```

Gjatë leximit nga fajlli, te stringu **g** lexohen me radhë fjalët e përfshira në fjali, gjë që shihet edhe nga shtypja e tyre në ekran. Në fjalët që lexohen nuk përfshihen edhe zbrazësit, sepse ato kompjuteri i interpreton si hapësira ndarëse mes të dhënave të veçanta, e që në këtë rast janë fjalët e përfshira në fjali. Kjo më së miri shihet nga rezultati që do të shtypet (shih Fig.7.12), nëse ekzekutohet versioni **file7a** i programit **file7**, tek i cili në fund të komandës për shtypje në ekran nuk figuron manipulatori **endl**:

```
// Programi file7a
.....
while (!Lexo.eof())
{
 Lexo >> g;
 cout << g;
}
cout << "\n\nLeximi nga fajlli përfundoi"
 << "\n\n";
return 0;
}
```

Fig.7.12

Pamja e ekranit pas ekzekutimit të versionit të programit **file7a**

```
Shkruarja në fajll përfundoi
Teksti që lexohet nga fajlli
Programimiiorientuarnëobjekte
Leximi nga fajlli përfundoi
Press any key to continue
```

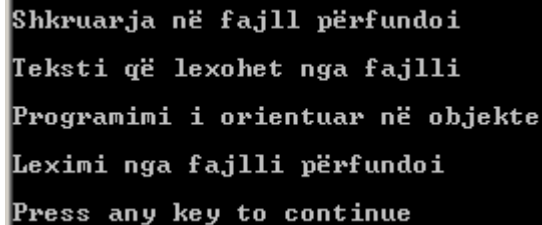
Që fjalia të shtypet në ekran në një rresht dhe me zbrazësira mes fjalëve të veçanta, gjatë shtypjes së çdo pjese të tekstit të lexuar duhet të shtypet edhe një zbrazësi.

#### Shembull

Pjesa e fundit e versionit **file7b** të programit **file7**, përmes së cilit gjatë shtypjes së fjalisë së lexuar nga fajlli parashihet edhe shtypja e zbrazësisë mes fjalëve të veçanta.

```
// Programi file7b
.....
while (!Lexo.eof())
{
 Lexo >> g;
 cout << g
 << ' ';
}
cout << "\n\nLeximi nga fajlli përfundoi"
 << "\n\n";
return 0;
}
```

Nëse ekzekutohet versioni i dhënë i programit, rezultati në ekran do të duket ashtu siç është dhënë në Fig.7.13.



```
Shkruarja në fajll përfundoi
Teksti që lexohet nga fajlli
Programimi i orientuar në objekte
Leximi nga fajlli përfundoi
Press any key to continue
```

Fig.7.13

Pamja e ekranit pas ekzekutimit të versionit të programit **file7b**

## Tekstet dhe numrat në fajlla

Vlerat numerike që shkruhen në fajll mund të shoqërohen edhe me tekste, plotësisht njëloj siç shoqërohen edhe gjatë shtypjes në ekran. Por, këtu duhet pasur kujdes që mes të dhënave të veçanta të lihet së paku një zbrazësi, ashtu që, siç u tha edhe më parë, kompjuteri t'i dallojë të dhënat e veçanta.

#### Shembull

Programi **file8**, përmes së cilit në fajllin **Koha.txt** regjistrohen vlerat **863** dhe **58.94** të variablave **k** dhe **x**, të shoqëruara me tekste.

```
// Programi file8
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
 int k=863;
 double x=58.94;

 ofstream Shkruaj("D:/Libra/Koha.txt",ios::out);

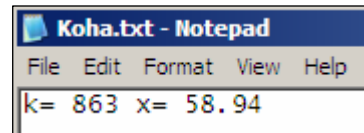
 Shkruaj << "k= "
 << k
 << " x= "
 << x;

 cout << "\nShkruarja në fajll përfundoi"
 << "\n\n";
 return 0;
}
```

Pas ekzekutimit të programit të dhënë, përmbajtja e fajllit **Koha.txt** do të duket si në Fig.7.14.

Fig.7.14

*Përmbajtja e fajllit **Koha.txt** pas ekzekutimit të programit **file8***



Prej këtu shihet se tekstet e shënuara nën thonjëza si dhe vlerat numerike të variablave shkruhen në fajll, duke i llogaritur si tekste edhe zbrazësitat e përfshira nën thonjëza.

Për leximin e të dhënave tekstuale dhe numerike, të shkruara në fajll, duhet të shfrytëzohen variabla numerike të tipeve përkatëse dhe variablave të tipit **string**.

#### Shembull

Versioni **file8a** i programit **file8**, përmes së cilit nga fajlli **Koha.txt** lexohen të dhënat e shkruara përmes programit **file8** dhe të njëjtat shtypen në ekran në rreshta të veçantë.

```
// Programi file8a
#include <iostream>
```

```

#include <fstream>
#include <string>
using namespace std;

int main()
{
 int k;
 double x;
 string s1,s2;

 ifstream Lexo("D:/Libra/Koha.txt",ios::in);

 Lexo >> s1
 >> k
 >> s2
 >> x;

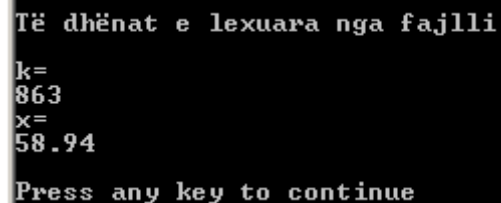
 cout << "\nTë dhënat e lexuara nga fajlli\n\n"
 << s1
 << "\n"
 << k
 << "\n"
 << s2
 << "\n"
 << x
 << "\n\n";

 return 0;
}

```

Në program, për leximin e vlerave numerike janë shfrytëzuar variablat **k** dhe **x**, të njëjta me ato që u shfrytëzuan te programi **file8** gjatë shkruarjes së tyre në fajll. Por, ato mund të quhen edhe ndryshe, gjë që nuk ka rëndësi. Kurse për leximin e dy teksteve janë shfrytëzuar variablat **s1** dhe **s2** të tipit **string**. Informatat e lexuara shtypen në ekran përmes komandave të zakonshme për shtypje, ashtu që rezultati në ekran do të duket si në Fig.7.15.

Fig.7.15  
Pamja e ekranit pas ekzekutimit të  
versionit të programit **file8a**



```

Të dhënat e lexuara nga fajlli
k=
863
x=
58.94
Press any key to continue

```

Gjatë leximit të vlerave numerike të cilat në fajll ruhen si sekuenca simbolesh tekstuale, kompjuteri i konverton ato në numra, duke e pasur parasysh tipin e variablave ku ruhen vlerat e lexuara.

Rezultati në ekran mund të shtypet edhe ndryshe, ashtu që të ketë një pamje të zakonshme.

**Shembull**

Pjesa e fundit e versionit **file8b** të programit **file8a**, tek i cili gjatë shtypjes së të dhënave në ekran është shfrytëzuar një formë më e përshtatshme e komandës për shtypje.

```
// Programi file8b
.....
cout << "\nTë dhënat e lexuara nga fajlli\n\n"
 << s1
 << k
 << " "
 << s2
 << x
 << "\n\n";
return 0;
}
```

Nëse ekzekutohet versioni në fjalë i programit, në një rresht të ekranit si rezultat do të shtypen **k=863** dhe **x=58.94**.

Për leximin nga fajlli mund të shfrytëzohet një string, pavarësisht nga ajo se të dhënat në fajll janë shkruar si stringje dhe vlera numerike.

**Shembull**

Versioni **file8b** i programit **file8** për leximin e të dhënave të shkruara në fajllin **Koha.txt** përmes programit **file7**, në stringun **g** dhe të njëjtat shtypen në ekran.

```
// Programi file8b
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main()
{
 string g;

 fstream Lexo("D:/Libra/Koha.txt", ios::in);

 cout << "\nTë dhënat e lexuara nga fajlli\n\n";
```

```

while (!Lexo.eof())
{
 Lexo >> g;
 cout << g
 << ' ';
}
cout << "\n\nLeximi nga fajlli përfundoi"
 << "\n\n";
return 0;
}

```

Rezultati që do të shihet në ekran pas ekzekutimit të programit **file8b** do të duket si në Fig.7.16.

Fig.7.16  
Pamja e ekranit pas ekzekutimit të  
versionit të programit **file8b**

```

Të dhënat e lexuara nga fajlli
k= 863 x= 58.94
Leximi nga fajlli përfundoi
Press any key to continue

```

Këtu, është lexuar dhe është shtypur në ekran komplet përmbajtja e fajllit, përfshirë edhe zbrazësitat që janë lënë mes të dhënave të veçanta.

## Shkruarja dhe leximi i karaktereve

Për shkruarjen e të dhënave në fajll dhe leximin e tyre nga fajlli si karaktere, mund të shfrytëzohen funksionet **put** dhe **get**. Në formë të përgjithshme, versionet themelore të këtyre dy funksioneve duken:

```

r.put(c);
r.get(c);

```

ku janë:

- r** - rrjedha e deklaruar si objekt i klasave për punë me fajlla.
- c** - karakteri që shkruhet në fajll, ose lexohet nga fajlli.

### Shembull

Programi **file9**, përmes së cilit në fajllin **Koha.txt** shkruhet teksti, i cili ruhet në stringun **A**, duke i shkruar një nga një karakteret e përfshirë në string.

```

// Programi file9
#include <iostream>

```



```
#include <fstream>
#include <string>
using namespace std;

int main()
{
 int i;
 double k;
 string A="Programimi i orientuar në objekte";

 ofstream Shkruaj("D:/Libra/Koha.txt",ios::out);

 k=A.size();

 for (i=0;i<k;i++)
 Shkruaj.put(A[i]);

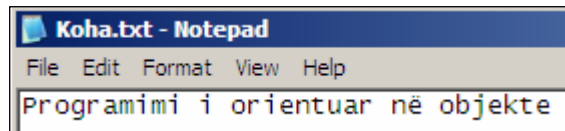
 cout << "\nShkruarja në fajll përfundoi"
 << "\n\n";
 return 0;
}
```

Në program, brenda unazës **for** është përfshirë komanda:

```
Shkruaj.put(A[i]);
```

përmes së cilës shkruhen në fajll shronjat e veçanta të përfshira në stringun **A**. Nëse pas ekzekutimit të programit të dhënë hapet fajlli **Koha.txt** duke e shfrytëzuar programin *Microsoft Notepad*, përmbajtja e tij do të duket ashtu siç është dhënë në Fig.7.17.

Fig.7.17  
Përmbajtja e fajllit *Koha.txt* pas  
ekzekutimit të programit *file9*



Unaza për shkruarje në fajll mund të realizohet edhe pa e shfrytëzuar variablën **k** në të cilën ruhet gjatësia e stringut **A**:

```
for (i=0;A[i]!='\0';i++)
 Shkruaj.put(A[i]);
```

Gjatë kësaj, përfundimi i ekzekutimit të unazës është përcaktuar me karakterin zero **'\0'**, i cili gjendet në fund të stringut **A**.

Përmbajtja e fajllit mund të lexohet duke i lexuar të dhënat si karaktere të veçanta.

**Shembull**

Programi **file10**, përmes së cilit lexohet përmbajtja e fajllit **Koha.txt** pasi në fajll të jetë shkruar fjalja e përmendur më sipër, duke e ekzekutuar programin **file9**.

```
// Programi file10
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
 char z;

 ifstream Lexo("D:/Libra/Koha.txt",ios::in);

 cout << "\nFjalja që lexohet nga fajlli\n\n";

 while(!Lexo.eof())
 {
 Lexo.get(z);
 if(!Lexo.eof())
 cout << z;
 }
 cout << "\n\nLeximi nga fajlli përfundoi"
 << "\n\n";
 return 0;
}
```

Këtu, brenda unazës për lexim të përmbajtjes së fajllit është përfshirë komanda:

```
Lexo.get(z);
```

me të cilën lexohen një nga një shkronjat e fjalisë së shkruar në fajll përmes variablës **z**. Gjatë kësaj, rezultati që shtypet në ekran do të duket si në *Fig.7.18*.

*Fig.7.18*

```
Fjalja që lexohet nga fajlli
Programimi i orientuar në objekte
Leximi nga fajlli përfundoi
Press any key to continue
```

*Pamja e ekranit pas ekzekutimit të versionit të programit file10*

Nëse duam që shkronjat e gjuhës shqipe **ë** të shkruhen saktë në ekran, pasi të lexohen nga disku, përkatësisht para se të shtypen në ekran, duhet të zëvendësohen me simbolin **%**, i cili fitohet përmes kombinimit të tasteve **Alt+0137**. Për këtë qëllim, një version i pjesës së programit brenda unazës **while** mund të duket si në vijim.

```
while(!Lexo.eof())
{
 Lexo.get(z);
 if(!Lexo.eof())
 {
 if(z=='ë')
 z='%';
 cout << z;
 }
}
```

Njëlloj duhet të veprohet edhe me shkronjat e tjera të alfabetit të gjuhës shqipe, të cilat nuk përfshihen në alfabetin ndërkombëtar.

Problemi i shkronjave të gjuhës shqipe mund të eliminohet edhe nëse në fajll shkruhen simbolet të cilat në ekran shtypen saktë. Me këtë nëkuptohet se në shembullin e marrë më sipër stringu që regjistrohet në disk duhet të shkruhet:

```
string A="Programimi i orientuar n% objekte";
```

Plotësisht njëlloj, siç u shpjegua më sipër, do të lexohet përmbajtja e çfarëdo fajlli në të cilin janë shkruar tekste, vlera numerike, programe etj.

## Forma të tjera të unazave për lexim

Unazat për lexim të karaktereve nga disku dhe shtypje të tyre në ekran mund të shkruhen edhe ndryshe. Dy versione të tyre janë dhënë në vijim.

- ```
while (Lexo)
{
    Lexo.get(z);
    if(Lexo)
        cout << z;
}
```

Këtu, në momentin kur mbërrihet në fund të fajllit, vlera logjike e rrjedhës **Lexo** brenda kllapave të komandës **while** është **false** dhe prandaj ekzekutimi i unazës përfundon.

-

```
while (Lexo.get(z))
{
    if(Lexo)
        cout << z;
}
```

Edhe në këtë rast, leximi ndërpritet duke e pasur parasysh logjikën e shpjeguar në versionin paraprak të unazës për lexim.

Shfrytëzimi i pointerëve

Gjatë shkruarjes në fajlla përmes funksionit **put**, ose gjatë leximit nga fajllat përmes funksionit **get**, mund të shfrytëzohen edhe pointerët.

Shembull

Versioni **file11** i programit **file9**, tek i cili gjatë shkruarjes së fjalisë në fajllin **Koha.txt** shfrytëzohet pointeri **a**.

```
// Programi file11
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    char *a="Programimi i orientuar në objekte";

    ofstream Shkruaj("D:/Libra/Koha.txt",ios::out);

    while (*a)
        Shkruaj.put(*a++);

    cout << "\nShkruarja në fajll përfundoi"
         << "\n\n";
    return 0;
}
```

Nëse pas ekzekutimit të programit të dhënë hapet fajlli **Koha.txt** përmes programit *Microsoft Notepad*, përmbajtja e tij do të jetë e njëjtë me atë që shihet në Fig.7.17.

Leximi i rreshtave

Nga fajllat mund të lexohen rreshtat komplet, përfshirë edhe zbrazësitat që mund të paraqiten brenda tyre. Gjithashtu, të dhënat në këto rreshta mund të lexohen deri në paraqitjen e një simboli të caktuar. Në të dy këto raste, për leximin e të dhënave nga fajlli, shfrytëzohet funksioni **getline**.

Leximi i komplet rreshtave

Për leximin e të dhënave të cilat në fajlla shkruhen në rreshta të veçantë, mund të shfrytëzohet funksioni **getline**, i cili në formë të përgjithshme duket:

```
r.getline(g,m);
```

ku janë:

r - rrjedha e deklaruar si objekt i klasave për punë me fajlla.

g - variabla, përkatësisht baferi në të cilin ruhet teksti i lexuar.

m - gjatësia e stringut që lexohet.

Gjatë shfrytëzimit të këtij funksioni, prej fajllave lexohen stringjet, përfshirë edhe zbrazësitat, derisa nuk haset karakteri '**\n**' për kalim në rresht të ri. Teksti i lexuar vendoset në *baferin g*, dhe gjatësia maksimale e tij mund të jetë **m**-karaktere.

Shembull

Programi **file12**, përmes së cilit në fajllin **Koha.txt** fillimisht shkruhen **10** rreshta dhe pastaj të njëjtat lexohen përmes funksionit **getline** dhe shtypen në ekran.

```
// Programi file12
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main()
{
```

```

    int i;
// ----- Shkruarja në fajll -----
    ofstream Shkruaj("D:/Libra/Koha.txt",ios::out);

    for (i=1;i<=10;i++)
        Shkruaj << "      Rreshti i "
                << i
                << endl;

    cout << "\nShkruarja në fajll përfundoi"
         << "\n";

// ----- Leximi nga fajlli -----
    const int m=20;
    char A[m];

    ifstream Lexo("D:/Libra/Koha.txt",ios::in);

    cout << "\nTeksti që lexohet nga fajlli\n\n";

    while (!Lexo.eof())
    {
        Lexo.getline(A,m);
        cout << A
              << endl;
    }

    cout << "Leximi nga fajlli përfundoi"
         << "\n\n";
return 0;
}

```

Këtu, përmes funksionit:

```
Lexo.getline(A,m);
```

prej fajllit lexohen komplet rreshtat me më së shumti **m**-karaktere. Gjatë kësaj, të dhënat e lexuara ruhen te fusha **A**, e cila shërben si bafer me madhësi prej **m**-karakteresh. Rezultati që shtypet në ekran do të duket ashtu siç shihet në

Fig.7.19.

```

Shkruarja në fajll përfundoi
Teksti që lexohet nga fajlli

      Rreshti i 1
      Rreshti i 2
      Rreshti i 3
      Rreshti i 4
      Rreshti i 5
      Rreshti i 6
      Rreshti i 7
      Rreshti i 8
      Rreshti i 9
      Rreshti i 10

Leximi nga fajlli përfundoi
Press any key to continue

```

*Fig.7.19**Pamja e ekranit pas ekzekutimit të versionit të programit **file12***

Në programin e dhënë, unaza për leximin e rreshtave të veçantë mund të shkruhet edhe duke e përfshirë funksionin **getline** brenda kllapave të komandës **while**. Kështu, pjesa e fundit e versionit **file12a** të programit **file12**, pas modifikimit në fjalë, do të duket si në vijim.

```
// Programi file12a
.....
while (Lexo.getline(A,m))
    cout << A
        << endl;

    cout << "Leximi nga fajlli përfundoi"
        << "\n\n";
return 0;
}
```

Nëse ekzekutohet versioni i dhënë i programit në fjalë, rezultati në ekran do të jetë i njëjtë me atë që është dhënë në *Fig.7.19*.

Leximi deri në simbolin e caktuar

Rreshtat brenda fajllave mund të lexohen deri në paraqitjen e një simboli të caktuar brenda tyre. Për këtë qëllim funksioni **getline** në formën e tij të përgjithshme shkruhet:

```
r.getline(g,m,'s');
```

ku janë:

- r** - rrjedha e deklaruar si objekt i klasave për punë me fajlla.
- g** - stringu që lexohet nga fajlli.
- m** - gjatësia e stringut që lexohet.
- s** - simboli kufitar deri tek i cili lexohet fjalia.

Shembull

Programi **file13**, përmes së cilit në fajllin **Koha.txt** fillimisht shkruhet një fjali dhe pastaj, përmes funksionit **getline** lexohet vetëm pjesa e saj deri te shkronja **ë**.

```
// Programi file13
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
// ----- Shkruarja në fajll -----
    ofstream Alfa("D:/Libra/Koha.txt",ios::out);

    Alfa << "Programimi i orientuar në objekte"
        << endl;
    Alfa.close();

// ----- Leximi nga fajlli -----
    const int m=30;
    char A[m];

    ifstream Beta("D:/Libra/Koha.txt",ios::in);
    cout << "\nTeksti që lexohet nga fajlli\n\n";
        Beta.getline(A,m,'ë');
        cout << A
            << endl;
    cout << "\nLeximi nga fajlli përfundoi"
        << "\n\n";
    return 0;
}
```

Nëse ekzekutohet programi i dhënë, përmes komandës:

```
Alfa << "Programimi i orientuar në objekte"
<< endl;
```

fillimisht në fajllin **Koha.txt** do të shkruhet fjalia e shënuar nën thonjëza. Pastaj, pas mbylljes dhe rihapjes së fajllit, përmes funksionit:

```
Beta.getline(A,m,'ë');
```

te baferi **A**, me madhësi prej **m**-simbolesh, do të lexohet rreshti deri te shkronja e parë **ë**. Kjo do të thotë se nga fajlli do të lexohet një pjesë e fjalisë që është regjistruar në të, ashtu siç shihet në *Fig.7.20*.

Fig.7.20

```
Teksti që lexohet nga fajlli
Programimi i orientuar n
Leximi nga fajlli përfundoi
Press any key to continue
```


Pamja e ekranit pas ekzekutimit të versionit të programit file13

Mode të hapjes së fajllave

Gjatë hapjes së fajllave, pas emrit të tyre, përveç dy modeve për hapje, të cilët u përmendën më sipër, mund të shënohen edhe mode të tjera, të cilat shihen në kolonën e parë të tabelës së dhënë në *Fig.7.21*, ose edhe kombinime të tyre.

Modi i hapjes	Përshkrimi
ios::in	Hapet fajlli për hyrje.
ios::out	Hapet fajlli për dalje.
ios::trunc	Nëse fajlli ekziston që më parë, përmbajtja e tij do të fshihet.
ios::app	Nëse fajlli ekziston që më parë, të dhënat e reja shtohen në fund të fajllit. Përndryshe, nëse fajlli nuk ka ekzistuar, krijohet një fajll i zbrazët.
ios::nocreate	Nëse fajlli nuk ekziston, hapja e tij dështon.
ios::noreplace	Nëse fajlli ekziston, hapja e tij dështon.
ios::ate	Hapet fajlli dhe pointeri i pozicionit në fajll vendoset në fund të tij. Por, shkruarja ose leximi i të dhënave mund të bëhet kudo në fajll.
ios::binary	Hapet fajlli si fajll binar.

Fig.7.21 Mode të hapjes së fajllave

Modi **trunc**, pavarësisht se nuk shënohet, nga kompjuteri merret si mod i nënkuptuar gjatë hapjes së fajllave nën modin **out**.

Modi **ate** ka kuptim vetëm nëse përmbajtja e fajllit që hapet nuk fshihet. Kjo, p.sh., ndodh nëse fajlli hapet që në të njëkohësisht të mund të shkruhet dhe të lexohet, sepse në atë rast fajlli i hapur nuk fshihet.

Kombinimet e modeve të hapjes

Fajllat, përveç në modet e dhëna në tabelën e *Fig.7.21*, mund të hapen edhe duke shfrytëzuar kombinime të këtyre modeve. Më parë u dhanë shembuj të hapjes së fajllave, duke i shfrytëzuar njëkohësisht modet **out** dhe **in**. Kurse

në vijim përmes shembujve do të shpjegohen hapjet e fajllave, me kombinime të modeve të hapjes së tyre.

Shembull

Programi **file14**, përmes së cilit tregohet shfrytëzimi i njëkohshëm i modeve të hapjes së fajllit **out** dhe **trunc**.

```
// Programi file14
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream Shkruaj;
    int k=77;
    double x=58.94;

    Shkruaj.open("D:/Libra/Koha.txt", ios::out);

    Shkruaj << k
              << ' '
              << x;

    cout << "\nShkruarja e parë përfundoi"
          << "\n";

    Shkruaj.close();

    k=865;
    x=-643.47;

    Shkruaj.open("D:/Libra/Koha.txt", ios::out
                                                         |ios::trunc);
    Shkruaj << k
              << ' '
              << x;

    cout << "\nShkruarja e dytë përfundoi"
          << "\n\n";
    return 0;
}
```

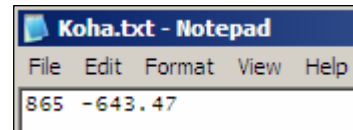
Në program, pas hapjes së parë të fajllit, shkruhen vlerat **77** dhe **58.94**, ashtu siç është treguar në *Fig.7.2*. Kurse, pas mbylljes dhe rihapjes së fajllit përmes komandës:

```
Shkruaj.open("D:/Libra/Koha.txt",ios::out
              |ios::trunc);
```

së pari fshihet përmbajtja e tij dhe pastaj shkruhen vlerat e reja të variablave **k** dhe **x**, ashtu siç shihet në Fig.7.22.

Fig.7.22

Përmbajtja e fajllit **Koha.txt** pas ekzekutimit të programit **file14** dhe rishkruarjes së vlerave



Nënkuptohej se komanda e dytë për hapje të fajllit me efekt të njëjtë mund të shkruhet edhe më shkurt:

```
Shkruaj.open("D:/Libra/Koha.txt",ios::trunc);
```

Shembull

Versioni **file15** i programit **file14**, përmes së cilit tregohet shfrytëzimi i njëkohshëm i modeve të hapjes së fajllit **out** dhe **app**.

```
// Programi file15
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream Shkruaj;
    int k=77;
    double x=58.94;

    Shkruaj.open("D:/Libra/Koha.txt",ios::out);
    Shkruaj << k
              << ' '
              << x
              << endl;

    cout << "\nShkruarja e parë përfundoi"
          << "\n";

    Shkruaj.close();

    k=865;
```

```

x=-643.47;

Shkruaj.open("D:/Libra/Koha.txt",ios::out
|ios::app);

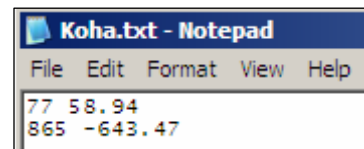
Shkruaj << k
<< ' ',
<< x;

cout << "\nShkruarja e dytë përfundoi"
<< "\n\n";
return 0;
}

```

Për dallim nga shembulli i programit **file14**, këtu gjatë hapjes së dytë të fajllit është shfrytëzuar komanda ku paraqitet edhe modi i hapjes **ios::app**, me të cilin nënkuptohet shtimi i të dhënave që shkruhen në fajll pa i fshirë ato ekzistueset. Si rezultat, përmbajtja e fajllit **Koha.txt**, pas ekzekutimit të programit të dhënë, do të duket ashtu siç është dhënë në Fig.7.23.

Fig.7.23
Përmbajtja e fajllit **Koha.txt** pas ekzekutimit të programit **file15**



Këtu, me qëllim të kalimit në rreshtin vijues, në fund të shkruarjes së vlerave të para është shfrytëzuar komanda **endl**.

Hapja e fajllave në modin binar

Në përgjithsi, përveç në *modin tekstual*, i cili është modi i nënkuptuar, fajllat mund të hapen edhe në *modin binar*, duke e shfrytëzuar opcionin **ios::binary**. Kur fajllat hapen në modin binar, të dhënat kompjuteri i sheh si vargje bajtësh, pa i interpretuar simbolet që përfshihen brenda tyre. Kështu, p.sh., karakteri që shfrytëzohet për të urdhëruar kalimin në rresht të ri nuk shihet si komandë, por vetëm si një simbol i zakonshëm. Për këtë arsye, shkruarja dhe leximi të fajllat e hapur si binarë është më e shpejtë, krahasuar me fajllat tekstualë.

Shembull

Programi **file16**, përmes së cilit në fajllin **Koha.bin** shkruhen **3** rreshta, duke e hapur fajllin në modin binar.

```

// Programi file16
#include <iostream>
#include <fstream>

```

```

using namespace std;

int main()
{
    int i;

    ofstream Shkruaj("D:/Libra/Koha.bin", ios::out
                    | ios::binary);

    for (i=1; i<=3; i++)
        Shkruaj << " Rreshti i "
                << i
                << endl;

    cout << "\nShkruarja në fajll përfundoi"
         << "\n\n";

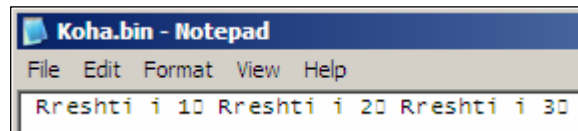
    return 0;
}

```

Nëse pas ekzekutimit të programit të dhënë hapet fajlli **Koha.bin**, përmbajtja e tij do të duket ashtu siç është dhënë në Fig.7.24.

Fig.7.24

Përmbajtja e fajllit **Koha.bin** pas ekzekutimit të programit **file16**



Prej këtu shihet se manipulatori **endl** në fajll është shkruar si një simbol i cili nga kompjuteri nuk interpretohet si komandë për kalim në rresht të ri.

Në program, prapashtesa e fajllit është marrë si **.bin**, gjë që praktikohet kur kemi të bëjmë me fajlla binarë. Por, lirisht mund të zgjedhen edhe prapashtesa të tjera, përfshirë edhe prapashtesën **.txt**.

Nëse gjatë hapjes së fajllit nuk shënohet modi **ios::binary**, ai hapet si fajll tekstual, meqë ky mod është i nënkuptuar.

Pozita në fajll

Pozita në të cilën shkruhen të dhënat në fajll, ose pozita nga e cila lexohen të dhënat nga fajlli, përcaktohet me *pozitën e pointerit në fajll* (ang. file-position pointer). Sa herë që shkruhet në fajll, ose lexohet nga fajlli, pointeri i pozitës në fajll rritet automatikisht.

Pozita e pointerit në fajll mund të përcaktohet edhe direkt, duke i shfrytëzuar funksionet:

seekp - te fajllat e hapur për shkruarje (shkronja **p** lidhet me fjalën *put*).

seekg - te fajllat e hapur për lexim (shkronja **g** lidhet me fjalën *get*).

Në versionet themelore këto dy funksione shkruhen kështu:

```
r.seekp(n);
r.seekg(n);
```

ku janë:

- r** - rrjedha e deklaruar si objekt i klasave për punë me fajll.
- n** - numri rendor i bajtit në fajll.

Shembull

Programi **file17** përmes së cilit, në variablën **z**, nga fajlli **Koha.txt** lexohet përmbajtja e bajtit me numër rendor **k**, nëse vlera e variablës **k** kompjuterit i jepet përmes tastierës.

```
// Programi file17
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    int k;
    char z;
    fstream Gama("D:/Libra/Koha.txt", ios::out
                | ios::in
                | ios::trunc);

    Gama << "Programimi në C++"
          << endl;

    cout << "\nPozita e simbolit që lexohet: ";
    cin >> k;

    Gama.seekg(k);
    Gama.get(z);

    cout << "\nKarakterit në bajtin me numër rendor "
          << k
          << " është: "
          << z
          << "\n\n";

    return 0;
}
```

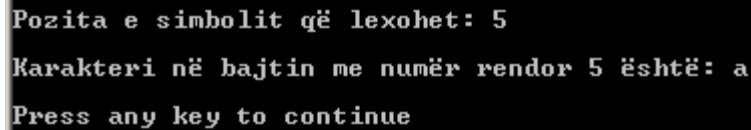
Nëse në fajll janë shkruar **m**-bajtë, numrat rendorë të tyre janë: **0, 1, 2, ..., m-1**. Në programin e dhënë, pozita e pointerit në fajll është përcaktuar përmes komandës:

```
Gama.seekg(k);
```

Pas kësaj, duke e shfrytëzuar komandën:

```
Gama.get(z);
```

nga fajlli lexohet përmbajtja e bajtit ku është pozicionuar pointeri. Kështu, p.sh., nëse përmes tastierës, për variablën **k** kompjuterit i jepet vlera **5**, rezultati në ekran do të duket ashtu siç është dhënë në Fig.7.25.



```
Pozita e simbolit që lexohet: 5
Karakter i në bajtin me numër rendor 5 është: a
Press any key to continue
```

Fig.7.25 Pamja e ekranit pas ekzekutimit të versionit të programit **file17**

Funksionet **seekp** dhe **seekg** mund të shfrytëzohen edhe për pozicionimin e pointerit në fajll gjatë shkruarjes ose leximit prej pozitës së caktuar brenda tij.

Shembull

Programi **file18**, përmes së cilit fjalja e shkruar në fajllin **Koha.txt** lexohet dhe shtypet në ekran prej bajtit me numrin rendor **k**, nëse vlera e variablës **k** kompjuterit i jepet si vlerë hyrëse përmes tastierës.

```
// Programi file18
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    int k;
    char z;
    fstream Gama("D:/Libra/Koha.txt", ios::out
                | ios::in
                | ios::trunc);
```

```

Gama << "Programimi i orientuar në objekte"
    << endl;

cout << "\nPozita prej ku lexohet: ";
cin >> k;

Gama.seekg(k);

cout << "\nFjalja e lexuar nga fajlli:\n\n";

while (!Gama.eof())
{
    Gama.get(z);
    if (!Gama.eof())
        cout << z;
}

cout << "\nLeximi nga fajlli përfundoi"
    << "\n\n";
return 0;
}

```

Në program, pasi kompjuterit t'i jepet vlera hyrëse për variablën **k**, përmes komandës:

```
Gama.seekg(k);
```

pointeri i pozitës në fajll pozicionohet në bajtin me numër rendor **k**. Pastaj, duke e shfrytëzuar unazën **while**, lexohet dhe shtypet në ekran pjesa e fjalisë, duke filluar nga bajti në fjalë, ashtu siç shihet në Fig.7.26.

Fig.7.26

Pamja e ekranit pas ekzekutimit të versionit të programit file18

```

Pozita prej ku lexohet: 11
Fjalja e lexuar nga fajlli:
i orientuar në objekte
Leximi nga fajlli përfundoi
Press any key to continue

```

Leximi i pozitës aktuale në fajll

Për leximin e numrit rendor të bajtit të pozitës të pointerit në fajll shfrytëzohen funksionet:

tellp - te fajllat e hapur për shkruarje.

tellg - te fajllat e hapur për lexim.

Në formë të përgjithshme, funksionet në fjalë shkruhen:

```
n=r.tellp();  
n=r.tellg();
```

ku janë:

r - rrjedha e deklaruar si objekt i klasave për punë me fajlla.

n - pozita aktuale e pointerit të fajllit.

Shembull

Versioni **file18a** i programit **file18**, përmes së cilit para shkronjave të pjesës së fjalisë që lexohet prej fajllit **Koha.txt** shtypen edhe numrat rendorë **n** të bajtëve përkatës.

```
// Programi file18a  
#include <iostream>  
#include <iomanip>  
#include <fstream>  
using namespace std;  
  
int main()  
{  
    int k,n;  
    char z;  
    fstream Gama("D:/Libra/Koha.txt",ios::out  
                |ios::in  
                |ios::trunc);  
  
    Gama << "Programimi i orientuar në objekte"  
    << endl;  
  
    cout << "\nPozita ku fillon leximi: ";  
    cin >> k;  
  
    Gama.seekg(k);  
  
    cout << "\n Pozita Shkronja\n\n";  
    while (!Gama.eof())  
    {  
        n=Gama.tellg();  
        Gama.get(z);  
        if (!Gama.eof())  
            cout << setw(5)  
                << n  
                << setw(8)
```

```

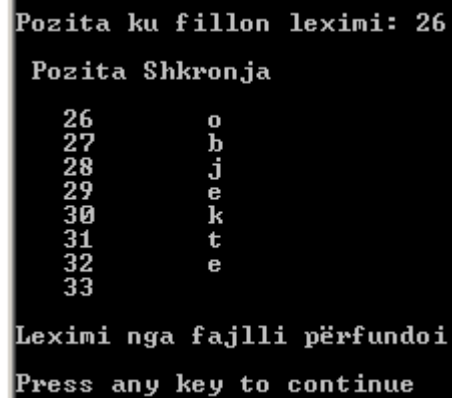
        << z
        << endl;
    }
    cout << "Leximi nga fajlli përfundoi"
        << "\n\n";
return 0;
}

```

Këtu, brenda unazës në të cilën lexohen nga fajlli dhe shtypen në ekran shkronjat e fjalisë, përmes komandës:

```
n=Gama.tellg();
```

lexohen dhe shtypen edhe numrat rendorë (pozitat) të bajtëve përkatës, ashtu siç shihet në Fig.7.27.



```

Pozita ku fillon leximi: 26
Pozita Shkronja
26      o
27      b
28      j
29      e
30      k
31      t
32      e
33
Leximi nga fajlli përfundoi
Press any key to continue

```

Fig.7.27

Pamja e ekranit pas ekzekutimit të programit *file18a*

Qasja dhe lëvizja e lirë brenda fajllave

Funksionet **seekp**, **seekg**, **tellp** dhe **tellg** mund të shfrytëzohen për qasje dhe lëvizje të lirë brenda fajllave.

Shembull

Programi **file19**, përmes së cilit gjenden pozitat e shkronjës së caktuar **x** brenda fjalisë së shkruar në fajllin **Koha.txt**, nëse shkronja **x** kompjuterit i jepet si vlerë hyrëse përmes tastierës.

```

// Programi file19
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

```

```

int main()
{
    int m;
    char x,z;
    fstream Gama("D:/Libra/Koha.txt",ios::out
                |ios::in
                |ios::trunc);

    Gama << "Programimi i orientuar në objekte"
          << endl;

    cout << "\nShkronja që kërkohet: ";
    cin >> x;

    Gama.seekg(0);

    cout << "\nPozitat e shkronjës "
          << x
          << "\n\n";

    while (!Gama.eof())
    {
        m=Gama.tellg();
        Gama.get(z);
        if (!Gama.eof() && z==x)
            cout << setw(8)
                  << m
                  << endl;
    }
    cout << "\nLeximi nga fajlli përfundoi"
          << "\n\n";
    return 0;
}

```

Nëse, p.sh., përmes tastierës për variablën **x** e cila lexohet, kompjuterit i jepet shkronja **i**, rezultati në ekran do të duket ashtu siç është dhënë në Fig.7.28.

```

Shkronja që kërkohet: i
Pozitat e shkronjës i
      7
      9
     11
     15
Leximi nga fajlli përfundoi
Press any key to continue

```

Fig.7.28
Pamja e ekranit pas ekzekutimit të versionit të
programit *file19*

Qasja relative në fajll

Funksionet **seekp** dhe **seekg** mund të shkruhen edhe duke ua shtuar brenda kllapave parametrin e dytë, përmes së cilit përcaktohet *drejtimi i qasjes* (ang. seek direction), në këtë mënyrë:

```
r.seekg(n,d);
r.seekp(n,d);
```

ku janë:

- r** - rrjedha e deklaruar si objekt i klasave për punë me fajlla.
- n** - numri i bajtëve për sa zhvendoset pointeri në drejtimin e zgjedhur.
- d** - drejtimi në të cilin zhvendoset pointeri.

Për parametrin **d** mund të zgjedhet njëri nga tri opcionet:

- ios::beg** - zhvendosje prej fillimit të fajllit kah fundi i tij.
- ios::end** - zhvendosje prej fundit të fajllit kah fillimi i tij.
- ios::cur** - zhvendosje prej pozitës aktuale kah fundi i fajllit.

Këto forma të shkruarjes së funksioneve **seekg** dhe **seekp** në fakt shfrytëzohen për zhvendosje të pointerit në fajll prej pozitës aktuale, për **n**-bajtë në drejtimin e zgjedhur.

Shembull

Programi **file20**, përmes së cilit gjendet madhësia në bajtë e fajllit **Koha.txt**, duke i shfrytëzuar funksionet **seekg** dhe **tellg**.

```
// Programi file20
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int m;
    ofstream Alfa("D:/Libra/Koha.txt",ios::out);
    Alfa << "Programimi i orientuar në objekte"
        << endl;
    cout << "\nShkruarja në fajll përfundoi"
        << "\n";
    Alfa.close();
    ifstream Beta("D:/Libra/Koha.txt",ios::in);
```

```

    Beta.seekg(0, ios::end);
    m=Beta.tellg();

    cout << "\nMadhësia e fajllit është "
          << m
          << " bajtë"
          << "\n\n";
return 0;
}

```

Në program, përmes komandës:

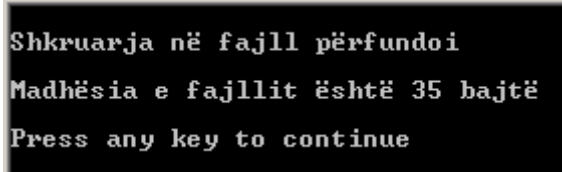
```
Beta.seekg(0, ios::end);
```

pointeri i pozitës në fajll zhvendoset për lexim për **0** bajt prej fundit të tij, përkatësisht *pointeri zhvendoset në fund të fajllit*. Pastaj, duke e shfrytëzuar komandën:

```
m=Gama.tellg();
```

lexohet vlera e pointerit të pozitës në fajll, e cila vlerë në këtë rast *është e barabartë me madhësinë e fajllit*. Rezultati në ekran do të duket ashtu siç është dhënë në Fig.7.29.

Fig.7.29
Pamja e ekranit pas ekzekutimit të versionit të programit **file20**



```

Shkruarja në fajll përfundoi
Madhësia e fajllit është 35 bajtë
Press any key to continue

```

Nga pozita aktuale e pointerëve në fajlla mund të lëvizet për një numër të caktuar bajtësh, gjë që kryesisht shfrytëzohet gjatë leximit të të dhënave prej tyre.

Shembull

Programi **file20a**, përmes së cilit nga fajllit **Koha.txt**, lexohet shkronja që gjendet në pozitën e fundit të tij.

```

// Programi file20a
#include <iostream>
#include <fstream>
using namespace std;
int main()
{

```

```

int m;
ofstream Alfa("D:/Libra/Koha.txt",ios::out);
Alfa << "Programimi i orientuar në objekte"
    << endl;
cout << "\nShkruarja në fajll përfundoi"
    << "\n";
Alfa.close();

ifstream Beta("D:/Libra/Koha.txt",ios::in);

Beta.seekg(0,ios::end);
m=Beta.tellg();

char a;
Beta.seekg(m-3,ios::beg);
Beta.get(a);

cout << "\n\nNë fund të fajllit gjendet shkronja "
    << a
    << "\n\n";
return 0;
}

```

Meqë numrat rendor të bajtëve, përkatësisht shkronjave të veçanta në fjali, shkojnë prej **0** deri në **32**, për ta lexuar shkronjën e fundit **e** në të pointeri pozicionohet përmes komandës:

```
Beta.seekg(m-3,ios::beg);
```

Gjatë ekzekutimit të kësaj komande, prej pozitës aktuale në fillim të fajllit, pointeri zhvendoset për **m-3=35-3=32** bajtë në fund të tij.

Shembull

Programi **file21**, tek i cili gjatë leximit nga fajlli **Koha.txt**, tregohet rritja e pointerit të pozitës në fajll për nga **3** bajtë, duke e shfrytëzuar funksionin **seekg**.

```

// Programi file21
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;
int main()
{
    char z;
    fstream Gama("D:/Libra/Koha.txt",ios::out

```

```

                                                    |ios::in
                                                    |ios::trunc);
Gama << "Programimi i orientuar në objekte"
    << endl;

Gama.seekg(0);

cout << "\nShkronjat e lexuara nga fajlli:\n\n";

while (!Gama.eof())
{
    Gama.seekg(2, ios::cur);
    Gama.get(z);
    if (!Gama.eof())
        cout << setw(5)
              << Gama.tellg()
              << setw(5)
              << z
              << endl;
    }
    cout << "\nLeximi nga fajlli përfundoi"
         << "\n\n";
return 0;
}

```

Këtu, përmes komandës:

```
Gama.seekg(2, ios::cur);
```

urdhërohet që pointeri i pozitës në fajll të rritet për **2** bajtë nga pozita aktuale e tij (drejtimi i lëvizjes përcaktohet me **ios::cur**). Por, meqë gjatë leximit nga fajlli, pas çdo bajti të lexuar, vlera e pointerit rritet automatikisht për **1**, në shembullin e programit të dhënë ai do të rritet për **3** bajtë.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket ashtu siç shihet në *Fig.7.30*.

```

Shkronjat e lexuara nga fajlli:

 3      o
 6      a
 9      m
12      i
15      r
18      n
21      a
24      n
27      o
30      e
33      e

Leximi nga fajlli përfundoi
Press any key to continue

```

Fig.7.30

*Pamja e ekranit pas ekzekutimit të programit **file21***

Nëse në programin **file21** funksioni **seekg** shkruhet:

```
Gama.seekg(0, ios::cur);
```

nga fajlli **Koha.txt** do të lexohet komplet fjalia, sepse me të urdhërohet zhvendosja e pointerit për **0** bajt, përkatësisht mbetet vetëm rritja e tij automatike për **1** bajt pas çdo leximi.

Pozita e pointerit në fajll mund të përcaktohet edhe duke u zhvendosur prej fillimit të tij për një numër të caktuar bajtësh.

Shembull

Programi **file22**, përmes së cilit nga fajlli **Koha.txt** lexohet çdo i katërti bajt, duke u zhvendosur prej fillimit të tij për **k** bajtë përmes funksionit **seekg**.

```
// Programi file22
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int k,m;
    char z;
    fstream Gama("D:/Libra/Koha.txt",ios::out
                                     |ios::in
                                     |ios::trunc);
    Gama << "Programimi i orientuar në objekte"
          << endl;

    cout << "\nShkronjat e lexuara nga fajlli:\n\n";

    Gama.seekg(0,ios::end);
    m=Gama.tellg();

    k=0;
    while (k<=m)
    {
        Gama.seekg(k,ios::beg);
```



```

        Gama.get(z);
        cout << z
              << "...";
        k=k+4;
    }

    cout << "\n\nLeximi nga fajlli përfundoi"

        << "\n\n";
return 0;
}

```

Në program, përmes komandës:

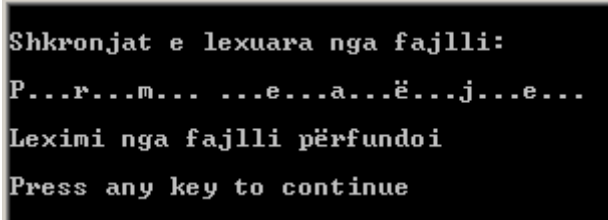
```
Gama.seekg(k, ios::beg);
```

pointeri i pozicionit në fajll vendoset direkt në bajtin me numër rendor **k**. Por, meqë variabla **k** rritet për **4**, ajo mund të marrë vlera, të cilat i tejkalojnë numrat rendorë të bajtëve në fajll. Për këtë qëllim, përsëritja e unazës **while** është kufizuar me kushtin **k<=m**, ku **m** e paraqet madhësinë e fajllit në bajtë.

Nëse ekzekutohet programi i dhënë, rezultati që shtypet në ekran do të duket ashtu siç është dhënë në Fig.7.31, ku me tri pikat e shtypura tregohen shkronjat e pa lexuara të fjalisë, me përjashtim të atyre në fund të fjalisë.

Fig.7.31

Pamja e ekranit pas ekzekutimit të programit *file22*



```

Shkronjat e lexuara nga fajlli:
P...r...m... ..e...a...ë...j...e...
Leximi nga fajlli përfundoi
Press any key to continue

```

Fajllat me qasje direkte

Për qasje direkte të të dhënave të përfshira në fajlla, si dhe për azhurimin e tyre, shfrytëzohen *fajlla me qasje direkte* (ang. random-access files). Në praktikë, fajllat e tillë deklarohen si fajlla binarë dhe formohen duke shfrytëzuar *regjistrime me gjatësi fikse*. Në këtë mënyrë, për shkak të gjatësisë fikse, direkt mund të gjenden adresat e regjistrimeve të veçanta brenda fajllave, përkatësisht numrat rendorë të bajtëve brenda tyre.

Të fajllat me qasje direkte, të dhënat shkruhen ose lexohen si blloqe të dhënash, duke i shfrytëzuar funksionet **write** dhe **read**. Gjatë kësaj, hapësira

memoruese, e cila shfrytëzohet në fajll për ruajtjen e të dhënave të veçanta, është fikse dhe varet nga tipi i variablave përkatëse. Kështu, p.sh., për ruajtjen e vlerës **32458372** të një variable të tipit **int**, në fajll do të shfrytëzohet hapësirë **4**-bajtëshe, gjë që dallon nga fajllat me qasje sekuenciale, te të cilët numri i njëjtë vendoset në **8**-bajtë (për çdo shifër shfrytëzohet **1** bajt).

Shkruarja në fajlla

Funksioni **write** për shkruarje në fajlla, në formën e tij bazike, duket:

```
r.write((char*) &v, sizeof(v));
```

ku janë:

r - rrjedha e deklaruar si objekt i klasave për punë me fajlla.

&v - adresa e variablës **v**, vlera e së cilës shkruhet në fajll.

sizeof(v) - numri i bajtëve të të cilët shkruhet vlera e variablës **v**.

Përmes funksionit **write**, vlera e variablës **v** shkruhet në hapësirën e fajllit e cila ka madhësi prej **sizeof(v)** bajtë. Gjatë kësaj, para shkruarjes në fajll, adresa e variablës **&v** konvertohet në pointerin e tipit karakter (**char***).

Shembull

Programi **fileD1**, përmes së cilit vlera e variablës **a** shkruhet në fajllin me qasje direkte **Delta.bin**.

```
// Programi fileD1
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int a;
    a=348576312;
    ofstream Alfa ("C:/Delta.bin", ios::out|ios::binary);
    Alfa.write((char*) &a, sizeof(a));

    cout << "\nShkruarja në fajll përfundoi"
         << "\n\n";
    return 0;
}
```

Nëse kontrollohet fajlli **Delta.bin**, pas shkruarjes së vlerës në të do të shihet se madhësia e tij është **4** bajtë, sepse variabla **k**, vlera e së cilës vendoset

në fajll, është e tipit **int**. Meqë vlera në fajll nuk regjistrohet si tekst, hapja e tij përmes ndonjë tekstprocesori është e pamundur, ose, edhe nëse hapet, në të nuk do të shihet vlera e regjistruar. Para shkruarjes së vlerës në fajll, kompjuteri atë e paketon në një format të veçantë, ashtu që edhe pse numri ka **9** shifra ai të regjistrohet në **4** bajtë (numri i njëjtë në një fajll tekstual regjistrohet në **9** bajtë).

Për shkruarje në fajll mund të shfrytëzohet edhe versioni vijues i funksionit **write**:

```
r.write((const char*) &v, sizeof(v));
```

Gjatë kësaj, para shkruarjes në fajll, adresa e variablës **&v** konvertohet në pointer si konstante e tipit karakter (**const char***).

Leximi nga fajllat

Për leximin e të dhënave të shkruara në fajlla me qasje direkte shfrytëzohet funksioni **read**, i cili në formë të përgjithshme shkruhet:

```
r.read((char*) &v, sizeof(v));
```

ku janë:

r - rrjedha e deklaruar si objekt i klasave për punë me fajlla.

&v - adresa e variablës **v**, vlera e së cilës lexohet nga fajlli.

sizeof(v) - numri i bajtëve nga të cilët lexohet vlera e variablës **v**.

Përmes funksionit **read**, vlera e variablës **v** lexohet nga hapësira e fajllit, e cila ka madhësi prej **sizeof(v)** bajtësh, me ndërmjetësimin e pointerit të tipit karakter **char***.

Shembull

Programi **fileD2**, përmes së cilit te variabla **b** lexohet vlera e cila është shkruar në fajllin me qasje direkte **Delta.bin** përmes programit **fileD1**.

```
// Programi fileD2
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int b;

    ifstream Beta("C:/Delta.bin", ios::in|ios::binary);
    Beta.read((char*) &b, sizeof(b));
```

```

        cout << "\nVlera e lexuar nga fajlli b="
              << b
              << "\n\n";
return 0;
}

```

Këtu, për leximin e vlerës nga fajlli është shfrytëzuar variabla **b**, e tipit të njëjtë me variablën **a**, e cila është shfrytëzuar te programi **fileD1** gjatë shkruarjes së saj në fajll. Variabla **a** mund të merret edhe gjatë leximit, pa qenë nevoja që të dallohet variabla që shfrytëzohet gjatë shkruarjes nga ajo që shfrytëzohet gjatë leximit.

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket si në Fig.7.32.

Fig.7.32

*Pamja e ekranit pas
ekzekutimit të programit
fileD2*

```

Vlera e lexuar nga fajlli b=348576312
Press any key to continue

```

Gjatë shkruarjes në fajll dhe leximit prej tij, funksioni **sizeof**, i shfrytëzuar te dy programet e mësipërme, mund të shkruhet edhe duke e shënuar vetëm tipin **t** të variablës **v**, si **sizeof(t)**. Por, këto dy forma të funksionit në fjalë mund të shkruhen edhe pa i shfrytëzuar kllapat:

```

sizeof v
sizeof t

```

Shembull

Programi **fileD3**, përmes së cilit te fajlli me qasje direkte **Delta.bin** së pari shkruhet vlera e variablës **z**, e cila është deklaruar si variabël e tipit **double** dhe pastaj vlera e njëjtë edhe lexohet.

```

// Programi fileD3
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    double z;
    z=-543719.8532;

```

```

ofstream Alfa ("C:/Delta.bin", ios::out | ios::binary);
Alfa.write((char*) &z, sizeof(double));

cout << "\nShkruarja në fajll përfundoi"
      << "\n";

Alfa.close();

ifstream Beta("C:/Delta.bin", ios::in | ios::binary);
Beta.read((char*) &z, sizeof(double));

cout << fixed
      << "\nVlera e lexuar nga fajlli z="
      << z
      << "\n\n";
return 0;
}

```

Në këtë rast, vlera e variablës **z** shkruhet në **8** bajtë të fajllit, aq sa nevojiten për vlerat e tipit **double**. Gjatë kësaj, numri i bajtëve në të cilët shkruhet dhe pastaj lexohet vlera në fjalë përcaktohet përmes funksionit **sizeof(double)**.

Rezultati që shtypet në ekran pas ekzekutimit të programit të dhënë do të duket ashtu siç është dhënë në Fig.7.33.

Fig.7.33
Pamja e ekranit pas
ekzekutimit të
programit fileD3

```

Shkruarja në fajll përfundoi
Vlera e lexuar nga fajlli z=-543719.853200
Press any key to continue

```

Vlerat e disa variablave në fajlla

Te fajllat me qasje direkte zakonisht shkruhen më shumë të dhëna, duke shfrytëzuar edhe variabla të tipeve të ndryshme. Gjatë kësaj, për shkruarjen ose edhe leximin e secilës prej tyre shfrytëzohen komanda të veçanta **write** dhe **read**.

Shembull

Programi **fileD4**, përmes së cilit te fajlli me qasje direkte **Delta.bin** së pari shkruhen dhe pastaj lexohen vlerat e variablave **a** e **z**, njëra e tipit **int** dhe tjetra e tipit **double**.

```

// Programi fileD4
#include <iostream>

```

```

#include <fstream>
using namespace std;
int main()
{
    int a;
    double z;

    a=348576312;
    z=-543719.8532;

    ofstream Alfa ("C:/Delta.bin",ios::out|ios::binary);
    Alfa.write((char*) &a,sizeof(a));
    Alfa.write((char*) &z,sizeof(z));

    cout << "\nShkruarja në fajll përfundoi"
         << "\n";

    Alfa.close();

    ifstream Beta("C:/Delta.bin",ios::in|ios::binary);
    Beta.read((char*) &a,sizeof(a));
    Beta.read((char*) &z,sizeof(z));

    cout << fixed
         << "\nVlerat e lexuara nga fajlli:"
         << "\n\na="
         << a
         << "\n\nz="
         << z
         << "\n\n";
    return 0;
}

```

Pas ekzekutimit të programit të dhënë, rezultati që shtypet në ekran do të duket ashtu siç është dhënë në Fig.7.34.

Fig.7.34
Pamja e ekranit pas ekzekutimit të programit
fileD4

```

Shkruarja në fajll përfundoi
Vlerat e lexuara nga fajlli:
a=348576312
z=-543719.853200
Press any key to continue

```

Ngjashëm veprohet edhe nëse në fajlla me qasje direkte shkruhen ose lexohen vlerat e më shumë variabla.

Vlerat e fushave në fajlla

Në fajllat me qasje direkte mund të shkruhen vlerat e fushave njëdimensionale (vektorëve), fushave dydimensionale (matricave), ose edhe të atyre shumëdimensionale.

Vlerat e vektorëve

Komandat për shkruarje të vektorëve në fajlla me qasje direkte dhe me leximin e tyre prej fajllave nuk dallojnë nga ato të shfrytëzuara gjatë shkruarjes dhe leximit të vlerave të variablave të zakonshme.

Shembull

Programi **fileD5**, përmes së cilit tregohet shkruarja e vektorit **A(n)** te fajlli me qasje direkte **Delta.bin**, pastaj leximi i tij nga fajlli dhe shtypja në ekran.

```
// Programi fileD5
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int const n=6;
    int i,A[n]={7,3,9,2,4,1};

    // ----- Shkruarja në fajll e vektorit -----

    ofstream Alfa ("C:/Delta.bin",ios::out|ios::binary);
    Alfa.write((char*) &A,sizeof(A));

    cout << "\nShkruarja në fajll përfundoi"
         << "\n";

    Alfa.close();

    // --- Shoqërimi i vlerave zero anëtarëve të vektorit --

    for (i=0;i<n;i++)
        A[i]=0;

    // ----- Leximi i vektorit nga fajlli -----
    ifstream Beta("C:/Delta.bin",ios::in|ios::binary);
```

```

    Beta.read((char*) &A, sizeof(A));

// ----- Shtypja e vektorit në ekran -----
    cout << "\nVlerat e lexuara nga fajlli:\n";
    for (i=0; i<n; i++)
        cout << "\n      A["
            << i
            << "]= "
            << A[i];
    cout << "\n\n";
return 0;
}

```

Në program, me qëllim që të shihet se vlerat e anëtarëve të vektorit lexohen prej fajllit, para se të lexohen atyre u shoqërohen vlera zero. Në pjesën e fundit të programit, vlerat e lexuara nga fajlli shtypen në ekran, ashtu siç është dhënë në Fig.7.35.

Fig.7.35
Pamja e ekranit pas ekzekutimit të
programit **fileD5**

```

Shkruarja në fajll përfundoi
Ulerat e lexuara nga fajlli:

A[0]=7
A[1]=3
A[2]=9
A[3]=2
A[4]=4
A[5]=1

Press any key to continue

```

Madhësia e hapësirës memoruese që shfrytëzohet nga fajlli **Delta.bin**, në të cilin shkruhen vlerat e anëtarëve të vektorit **A(n)**, do të jetë **24** bajtë, sepse vektori ka **6** anëtarë të tipit **int**, me madhësi **4** bajtë secili.

Në programin e mësipërm, komandat për shkruarje në fajll dhe leximit prej tij mund të shkruhen edhe në format:

```

Alfa.write((char*) &A, n*sizeof(int));
Beta.read((char*) &A, n*sizeof(int));

```

Në fakt, këtu hapësira në të cilën shkruhet vektori në fajll, ose lexohet prej tij, përcaktohet duke e shumëzuar numrin e anëtarëve të vektorit **n** dhe numrin e bajtëve që shfrytëzohen prej tyre **sizeof(int)**.

Vektori mund të shkruhet edhe duke i shkruar anëtarët një nga një, përmes një unaze.

Shembull

Versioni **fileD6** i programit **fileD5**, përmes së cilit tregohet shkruarja një nga një e anëtarëve të vektorit **A(n)** te fajlli me qasje direkte **Delta.bin**.



Nga programi **fileD5**, versioni **fileD6** i tij do të dallohet vetëm në pjesën për shkruarje në fajll, e cila do të duket:

```
.....  
ofstream Alfa ("C:/Delta.bin",ios::out|ios::binary);  
for (i=0;i<n;i++)  
    Alfa.write((char*) &A[i],sizeof(A[i]));  
.....
```

Edhe leximi i anëtarëve të vektorit mund të realizohet, duke i lexuar një nga një anëtarët e tij nga fajlli.

Shembull

Versioni **fileD7** i programit **fileD5**, përmes së cilit tregohet leximi një nga një i anëtarëve të vektorit **A(n)** nga fajlli me qasje direkte **Delta.bin**.

Nga programi **fileD5**, versioni **fileD7** i tij do të dallohet vetëm në pjesën për lexim nga fajlli dhe shtypje në ekran, e cila do të duket:

```
.....  
cout << "\nVlerat e lexuara nga fajlli:\n";  
  
for (i=0;i<n;i++)  
{  
    Beta.read((char*) &A[i],sizeof(A[i]));  
    cout << "\n    A["  
        << i  
        << "]="  
        << A[i];  
}  
cout << "\n\n";  
return 0;  
}
```

Këtu, për lexim mund të shfrytëzohet një variabël ndihmëse, e cila është fushë, ose edhe variabël e zakonshme e tipit të njëjtë. Kështu, p.sh., nëse për lexim shfrytëzohet variabla e zakonshme **b**, pjesa e fundit e programit do të duket si në vijim.

```
.....  
cout << "\nVlerat e lexuara nga fajlli:\n";  
int b;  
for (i=0;i<n;i++)  
{
```

```

        Beta.read((char*) &b,sizeof(b));
        cout << "\n      A["
              << i
              << "]= "
              << b;
    }
    cout << "\n\n";
    return 0;
}

```

Në këtë rast, vlerat që lexohen ruhen përkohësisht te variabla ndihmëse **b**.

Vlerat e matricave

Në fajllat me qasje direkte mund të shkruhen dhe të lexohen të dhënat e përfshira në matrica, plotësisht njëloj siç shkruhen dhe lexohen vektorët.

Shembull

Programi **fileD8**, përmes së cilit tregohet shkruarja në fajllin **Delta.bin** e vlerave të matricës **A(m,n)** dhe pastaj edhe leximi i tyre prej fajllit.

```

// Programi fileD8
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;
int main()
{
    int const m=4,n=5;
    int A[m][n]={12,4,7,-2,24},
                {3,-8,25,64,1},
                {28,69,85,33,6},
                {-9,18,2,5,-17}};

    int i,j;

    // ----- Shkruarja në fajll e matricës -----

    ofstream Alfa ("C:/Delta.bin",ios::out|ios::binary);
    Alfa.write((char*) &A,sizeof(A));

    cout << "\nShkruarja në fajll përfundoi"
          << "\n";

    Alfa.close();

    // --- Shoqërimi i vlerave zero anëtarëve të matricës --

```

```

        for (i=0;i<m;i++)
            for (j=0;j<n;j++)
                A[i][j]=0;

// ----- Leximi i matricës nga fajlli -----

        ifstream Beta("C:/Delta.bin",ios::in|ios::binary);
        Beta.read((char*) &A,sizeof(A));
// ----- Shtypja e matricës në ekran -----

        cout << "\nMatrica e lexuar nga fajlli:\n\n";
        for (i=0;i<m;i++)
        {
            for (j=0;j<n;j++)
                cout << setw(5)
                    << A[i][j];
            cout << endl;
        }
        cout << "\n";
    return 0;
}

```

Nëse ekzekutohet programi i dhënë, rezultati në ekran do të duket ashtu siç shihet në Fig.7.36.

Fig.7.36
Pamja e ekranit pas ekzekutimit të programit
fileD8

```

Shkruarja në fajll përfundoi
Matrica e lexuar nga fajlli:

 12    4    7   -2   24
  3   -8   25   64    1
 28   69   85   33    6
 -9   18    2    5  -17

Press any key to continue

```

Anëtarët e matricave në fajll mund të shkruhen ose të lexohen edhe një nga një, ashtu siç u tregua në pjesën paraprake për vektorët. Gjithashtu, gjatë leximit të anëtarëve të matricave mund të shfrytëzohen variabla ndihmëse të zakonshme, ose edhe fusha të tjera.

Gjatë shkruarjes në fajlla dhe leximit prej tyre, plotësisht njëloj mund të veprohet edhe nëse kemi të bëjmë me fusha shumëdimensionale, siç u tregua më sipër për vektorët dhe matricat.

Vlerat e llogaritura në fajlla

Në fajllat me qasje direkte mund të shkruhen edhe vlerat që llogariten brenda programit.

Shembull

Programi **fileL**, përmes së cilit tregohet shkruarja në fajllin me qasje direkte **Drejt.bin** të vlerave të brinjëve **a** e **b** të drejtkëndëshit, si dhe sipërfaqes **s** dhe perimetrit **p** përkatës të tij. Vlera e brinjës **b** merret fikse, kurse vlerat e brinjës **a** ndryshohen me hapin **0.5**, mes vlerave **1** dhe **5**. Në fund, vlerat e shkruara lexohen nga fajlli dhe shtypen në ekran.

```
// Programi fileL
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main()
{
    double a,b,s,p;
    ofstream Alfa ("C:/Drejt.bin",ios::out|ios::binary);

    // --- Llogaritja dhe shkruarja në fajll -----

    b=6;
    for(a=1;a<=5;a=a+0.5)
    {
        s=a*b;
        p=2*(a+b);

        Alfa.write((char*) &a,sizeof(a));
        Alfa.write((char*) &b,sizeof(b));
        Alfa.write((char*) &s,sizeof(s));
        Alfa.write((char*) &p,sizeof(p));
    }

    cout << "\nShkruarja në fajll përfundoi"
         << "\n\n";

    Alfa.close();

    // --- Leximi nga fajlli dhe shtypja në ekran -----

    ifstream Beta("C:/Drejt.bin",ios::in|ios::binary);

    cout << "Të dhënat e lexuara nga fajlli\n\n"
         << "   a       b       s       p\n\n";

    while(!Beta.eof())
    {
```

```

        Beta.read((char*) &a,sizeof(a));
        Beta.read((char*) &b,sizeof(b));
        Beta.read((char*) &s,sizeof(s));
        Beta.read((char*) &p,sizeof(p));

    if (!Beta.eof())
    cout << fixed
        << setprecision(2)
        << setw(6)
        << a
        << setw(6)
        << b
        << setw(8)
        << s
        << setw(8)
        << p
        << endl;
    }
    cout << endl;
return 0;
}

```

Përmes unazës **for** në fillim të programit shkruhen në fajll vlerat e brinjëve të drejtkëndëshit si dhe vlerat përkatëse të llogaritura të sipërfaqes **s** dhe të perimetrit **p**. Pastaj, pas mbylljes, fajlli rihapet dhe nga ai lexohen vlerat dhe shtypen në ekran ashtu siç shihet në Fig.7.37.

Fig.7.37
Pamja e ekranit pas ekzekutimit të
programit *fileL*

```

Shkruarja në fajll përfundoi
Të dhënat e lexuara nga fajlli

```

a	b	s	p
1.00	6.00	6.00	14.00
1.50	6.00	9.00	15.00
2.00	6.00	12.00	16.00
2.50	6.00	15.00	17.00
3.00	6.00	18.00	18.00
3.50	6.00	21.00	19.00
4.00	6.00	24.00	20.00
4.50	6.00	27.00	21.00
5.00	6.00	30.00	22.00

```

Press any key to continue

```

Tekstet në fajlla

Te fajllat me qasje direkte mund të shkruhen edhe tekste, plotësisht njëloj siç shkruhen vlerat numerike. Por, gjatë kësaj, për çdo simbol të përfshirë në

tekst kompjuteri shfrytëzon një bajt dhe tekstet shkruhen si edhe te fajllat me qasje sekuenciale.

Shembull

Programi **fileT1**, përmes së cilit tregohet shkruarja në fajllin me qasje direkte **Omega.txt** e tekstit **T**. Pastaj, për t'u shtypur në ekran, teksti lexohet nga fajlli te vektori **G**.

```
// Programi fileT1
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    char T[]="  Koha e bukur",G[15];
    ofstream Alfa("C:/Omega.txt",ios::out|ios::binary);

    Alfa.write((char *) &T,sizeof(T));

    cout << "\nShkruarja në fajll përfundoi"
         << "\n\n";

    Alfa.close();

    cout << "Teksti i lexuar nga fajlli\n\n";

    ifstream Beta("C:/Omega.txt",ios::in|ios::binary);

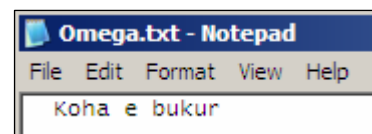
    Beta.read((char *) &G,sizeof(T));

    Beta.close();
    cout << G
         << "\n\n";
    return 0;
}
```

Nëse pas ekzekutimit të programit, hapet fajlli **Omega.txt** përmes programit *Microsoft Notepad*, përmbajtja e tij do të shihet direkt, ashtu siç është dhënë në Fig.7.38.

Fig.7.38

Përmbajtja e fajllit **Omega.txt** pas ekzekutimit të programit **fileT1**



Nëse ekzekutohet programi i dhënë, rezultati që shtypet në ekran do të duket ashtu siç shihet në *Fig.7.39*.

Fig.7.39

Pamja e ekranit pas ekzekutimit të programit fileT1

```
Shkruarja në fajll përfundoi
Teksti i lexuar nga fajlli
    Koha e bukur
Press any key to continue
```

Tekstet dhe vlerat numerike në fajlla

Te fajllat me qasje direkte, si edhe tek ata me qasje sekuenciale, njëkohësisht mund të shkruhen tekste dhe vlera numerike. Gjatë kësaj, komandat që shfrytëzohen për shkruarje nuk dallohen nga ato që u shpjeguan në pjesën paraprake.

Shembull

Programi **fileT2**, përmes së cilit tregohet shkruarja në fajllin me qasje direkte **Omega.txt** i tekstit **T** dhe vlerës numerike të variablës **x**. Pastaj, për t'u shtypur në ekran, teksti lexohet nga fajlli te vektori **G**, kurse vlera numerike lexohet me ndërmjetësimin e variablës **x**.

```
// Programi fileT2
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    double x=768352.3489;
    char T[]="    Koha e bukur",G[15];
    ofstream Alfa("C:/Omega.txt",ios::out|ios::binary);

    Alfa.write((char *) &x,sizeof(x));
    Alfa.write((char *) &T,sizeof(T));

    cout << "\nShkruarja në fajll përfundoi"
         << "\n\n";

    Alfa.close();

    cout << "Të dhënat e lexuara nga fajlli\n\n";

    ifstream Beta("C:/Omega.txt",ios::in|ios::binary);
```

```

Beta.read((char *) &x, sizeof(x));
Beta.read((char *) &G, sizeof(T));

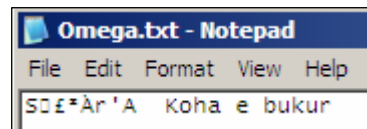
Beta.close();
cout << "x="
      << x
      << G
      << "\n\n";
return 0;
}

```

Nëse pas ekzekutimit të programimit hapet fajlli duke e shfrytëzuar programin *Microsoft Notepad*, përmbajtja e tij do të duket ashtu siç është dhënë në Fig.7.40.

Fig.7.40

Përmbajtja e fajllit *Omega.txt* pas ekzekutimit të programit *fileT2*

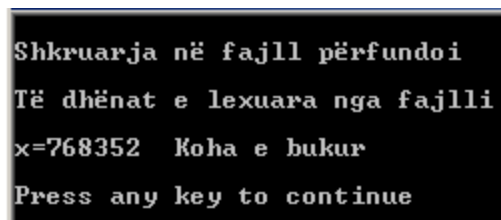


Prej këtu shihet se vlera e variablës **x**, e cila është shkruar në **8** bajtët e parë të fajllit, nuk mund të lexohet direkt. Kurse, teksti i cili është shkruar në pjesën e dytë të fajllit, ashtu siç u pa edhe në shembullin e programit paraprak, është i lexueshëm direkt.

Pas ekzekutimit të programit të dhënë, rezultati që shtypet në ekran do të duket ashtu siç është dhënë në Fig.7.41.

Fig.7.41

Pamja e ekranit pas ekzekutimit të programit *fileT2*



Qasja direkte në të dhënat e fajllave

Meqë te fajllat me qasje direkte regjistrimet e veçanta kanë gjatësi fikse, përkatësisht të dhënat shkruhen në një numër fiks bajtësh, atyre mund t'u qasemi direkt. Gjatë kësaj, duhet të llogaritet numri rendor i bajtit, ku fillon regjistrimi i të dhënës të cilës duhet t'i qasemi. Pastaj, duke i shfrytëzuar funksionet përkatëse, pointeri i pozitës në fajll vendoset aty ku fillon leximi.

Shembull

Programi **fileQ1**, përmes së cilit tregohet leximi i anëtarit të

 caktuar të vektorit **A(n)** nga fajlli me qasje direkte **Delta.bin**.

```
// Programi fileQ1
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int const n=6;
    int A[n]={7,3,9,2,14,1};
    int k,x;

// --- Shkruarja në fajll -----

    ofstream Alfa ("C:/Delta.bin",ios::out|ios::binary);
    Alfa.write((char*) &A,sizeof(A));

    cout << "\nShkruarja në fajll përfundoi"
          << "\n";

    Alfa.close();

    ifstream Beta("C:/Delta.bin",ios::in|ios::binary);
// --- Pozicionimi te bajti ku fillon leximi -----

    cout << "\nNumri i anëtarit që lexohet, mes 0 dhe "
          << n-1
          << ": ";
    cin >> k;

    Beta.seekg(k*sizeof(int),ios::beg);

    cout << "\nVlera që duhet lexuar fillon te bajti i "
          << Beta.tellg();

    Beta.read((char*) &x,sizeof(int));

    cout << "\n\nNga fajlli u lexua vlera "
          << x
          << "\n\n";
    return 0;
}
```

Këtu, fillimisht vlerat e anëtarëve të vektorit **A(n)** shkruhen në fajll. Pastaj, përmes funksionit **Beta.seekg** pointeri i pozitës në fajll pozicionohet te bajti me numër rendor **k*sizeof(int)**, ku **k** është numri rendor (indeksi) i anëtarit të vektorit, vlera e të cilit lexohet nga fajlli. Kjo, do të thotë se për **k=3**,

pointeri në fjalë do të pozicionohet te bajti i **12**-të, meqë prej tij fillon shkruarja në fajll e vlerës së anëtarit të **3**-të. Përmes funksionit **Beta.tellg** merret numri rendor i bajtit ku lexohet vlera dhe i njëjti shtypet në ekran. Në fund, përmes komandës për lexim, vlera e zgjedhur lexohet te variabla **x** dhe shtypet në ekran.

Nëse ekzekutohet programi i dhënë, për vlerën hyrëse **3** të variablës **k**, rezultati do të duket ashtu siç është dhënë në Fig.7.42.

Fig.7.42
Pamja e ekranit pas
ekzekutimit të
programit *fileQ1*

```
Shkruarja në fajll përfundoi
Numri i anëtarit që lexohet, mes 0 dhe 5: 3
Vlera që duhet lexuar fillon te bajti i 12
Nga fajlli u lexua vlera 2
Press any key to continue
```

Shfrytëzimi i të dhënave nga fajllat

Gjatë llogaritjeve të ndryshme, mund të shfrytëzohen edhe të dhënat që ruhen në fajlla me qasje direkte.

Shembull

Programi **fileQ2**, përmes së cilit tregohet llogaritja e shumës **s** të anëtarëve me indeks çift të vektorit **A(n)**, të cilët fillimisht shkruhen në fajllin **Delta.bin** dhe pastaj, përmes qasjes direkte, lexohen nga fajlli dhe i shtohen shumës.

```
// Programi fileQ2
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;
int main()
{
    int const n=7;
    int A[n]={7,3,9,2,4,1,5},i,x;
    unsigned int k,m;

    ofstream Alfa ("C:/Delta.bin",ios::out|ios::binary);
    Alfa.write((char*) &A,sizeof(A));
```

```
cout << "\nShkruarja në fajll përfundoi"
      << "\n";

Alfa.close();

ifstream Beta("C:/Delta.bin",ios::in|ios::binary);

Beta.seekg(0,ios::end);
m=Beta.tellg();

cout << "\nMadhësia e fajllit është "
      << m
      << " bajtë\n"
      << "\n      i      k      A[i]\n\n";

double s=0;
i=0;
do
{
    Beta.seekg(i*sizeof(int),ios::beg);
    k=Beta.tellg();
    Beta.read((char*) &x,sizeof(int));
    s=s+x;
    cout << setw(5)
          << i
          << setw(7)
          << k
          << setw(7)
          << x
          << endl;
    i=i+2;
}
while (i<n);
cout << "\nShuma e llogaritur është s="
      << s
      << "\n\n";
return 0;
}
```

Vlerat që shfrytëzohen gjatë llogaritjes së shumës këtu lexohen direkt nga fajlli. Pozicionimi te bajtët në të cilët fillon vendosja e këtyre vlerave, bëhet përmes funksionit **seekg**, duke e shkruar atë në formën:

```
Beta.seekg(i*sizeof(int),ios::beg);
```

ku me shprehjen **i*sizeof(int)** llogaritet numri rendor i bajtit në të cilin fillon leximi i vlerës **x**, të anëtarit të **i**-të të vektorit. Ky numër pastaj merret përmes shprehjes:

```
k=Beta.tellg();
```

Rezultati, i cili fitohet në ekran, pas ekzekutimit të programit të dhënë, do të duket si në Fig.7.43.

```
Shkruarja në fajll përfundoi
Madhësia e fajllit është 28 bajtë

  i      k      A[i]
  0      0      7
  2      8      9
  4     16      4
  6     24      5

Shuma e llogaritur është s=25
Press any key to continue
```

Fig.7.43

Pamja e ekranit pas ekzekutimit të programit *fileQ2*

Objektet në fajlla

Të dhënat e përfshira në komponentet e objekteve të strukturave ose të klasave mund të shkruhen në fajlla me qasje sekuenciale, ose në fajlla me qasje direkte. Gjithashtu, gjatë leximit të të dhënave prej fajllave ato mund të ruhen në komponentet e objekteve të strukturave ose të klasave.

Objektet te fajllat me qasje sekuenciale

Te fajllat me qasje sekuenciale, objektet shkruhen ose lexohen prej tyre, duke i shkruar ose lexuar vlerat e komponenteve përkatëse, ashtu siç shkruhen ose siç lexohen vlerat e variablave të zakonshme.

Shembull

Programi **file01**, përmes së cilit tregohet shkruarja në fajllin me qasje sekuenciale **Drejt.txt** të objektit **Dita** të strukturës **Drejt**, në komponentet e të cilave përfshihen brinjët **a** e **b** të drejtkëndëshit, si dhe sipërfaqja **s** dhe perimetri **p** përkatës i tij. Vlera e brinjës **b** merret fikse, kurse vlerat e brinjës **a** ndryshohen me hapin **0.5**, mes vlerave **1** dhe **5**.

```
// Programi file01
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

struct Drejt
{
    double a,b,s,p;
};
int main()
{
    Drejt Dita;

    // --- Shkruarja e të dhënave në fajll -----
    ofstream Alfa ("C:/Drejt.txt",ios::out);

    Dita.b=6;
    for(Dita.a=1;Dita.a<=5;Dita.a=Dita.a+0.5)
    {
        Dita.s=Dita.a*Dita.b;
        Dita.p=2*(Dita.a+Dita.b);
        Alfa << fixed
            << setprecision(2)
            << setw(6)
            << Dita.a
            << setw(6)
            << Dita.b
            << setw(7)
            << Dita.s
            << setw(7)
            << Dita.p
            << endl;
    }

    cout << "\nShkruarja në fajll përfundoi"
         << "\n\n";

    Alfa.close();

    // --- Leximi i të dhënave nga fajlli -----
    ifstream Beta("C:/Drejt.txt",ios::in);

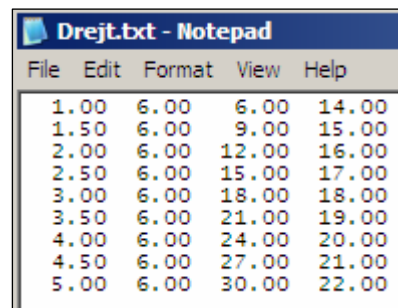
    cout << "Të dhënat e lexuara nga fajlli\n\n"
         << "    a        b        s        p\n\n";
```

```

while(!Beta.eof())
{
    Beta >> Dita.a
        >> Dita.b
        >> Dita.s
        >> Dita.p;
    if (!Beta.eof())
        cout << fixed
            << setprecision(2)
            << setw(6)
            << Dita.a
            << setw(6)
            << Dita.b
            << setw(8)
            << Dita.s
            << setw(8)
            << Dita.p
            << endl;
}
cout << endl;
return 0;
}

```

Në program, fillimisht, llogariten vlerat e sipërfaqes **s** dhe të perimetrit **p** të drejtkëndëshit për vlera të ndryshme të brinjës **a**, të cilat përfshihen në komponentet e strukturës **Drejt**. Të njëjtat shkruhen në fajllin **Drejt.txt**, ashtu siç shkruhen vlerat e variablave të zakonshme. Nëse pas ekzekutimit të programit hapet fajli me të dhëna, do ta kemi pamjen e dhënë Fig.7.44.



Drejt.txt - Notepad			
File	Edit	Format	View Help
1.00	6.00	6.00	14.00
1.50	6.00	9.00	15.00
2.00	6.00	12.00	16.00
2.50	6.00	15.00	17.00
3.00	6.00	18.00	18.00
3.50	6.00	21.00	19.00
4.00	6.00	24.00	20.00
4.50	6.00	27.00	21.00
5.00	6.00	30.00	22.00

Fig.7.44

Përmbajtja e fajllit **Drejt.txt** pas ekzekutimit të programit **file01**

Rezultati që shtypet në ekran pas ekzekutimit të programit do të duket ashtu siç është dhënë më herët te Fig.7.37.

Plotësisht njëloj mund të veprohet edhe nëse në fajll shkruhen të dhënat e përfshira në komponentet me të dhëna të objekteve të klasave.

Shembull

Versioni **file01a** i programit **file01**, tek i cili vlerat e brinjëve **a** dhe **b** të drejtkëndëshit, si dhe sipërfaqja **s** dhe

 perimetri **p** i tij përfshihen në klasën **Drejt**.

Në versionin **file01a** të programit **file01**, deklarimi i strukturës **Drejt** duhet të zëvendësohet me deklarimin e klasës përkatëse:

```
class Drejt
{
public:
    double a,b,s,p;
};
```

Pjesët e tjera të programit mbeten të pandryshuara.

Objektet te fajllat me qasje direkte

Vlerat e variablove brenda objekteve të strukturave ose të klasave mund të shkruhen në fajllat me qasje direkte, ose të lexohen nga ato, *përmes variablove të komponenteve me të dhëna* brenda tyre, ose *përmes blloqeve me të dhëna* në të cilat përfshihen komplet komponentet në fjalë.

Përmes variablove të komponenteve me të dhëna

Të dhënat brenda objekteve të strukturave ose të klasave mund të shkruhen në fajllat me qasje direkte, duke i shkruar ose duke i lexuar vlerat e variablove të cilat përfshihen në komponentet me të dhëna të tyre.

Shembull

Versioni **file02** i programit **file01**, përmes së cilit tregohet shkruarja në fajllin me qasje direkte **Drejt.bin** të objektit **Dita** të strukturës **Drejt**.

```
// Programi file02
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

void shtyp(double a,double b,double s,double p);

struct Drejt
{
    double a,b,s,p;
};

int main()
```

```

{
    Drejt Dita;
    ofstream Alfa ("C:/Drejt.bin",ios::out|ios::binary);

    Dita.b=6;
    for(Dita.a=1;Dita.a<=5;Dita.a=Dita.a+0.5)
    {
        Dita.s=Dita.a*Dita.b;
        Dita.p=2*(Dita.a+Dita.b);
        Alfa.write((char*) &(Dita.a),sizeof(Dita.a));
        Alfa.write((char*) &(Dita.b),sizeof(Dita.b));
        Alfa.write((char*) &(Dita.s),sizeof(Dita.s));
        Alfa.write((char*) &(Dita.p),sizeof(Dita.p));
    }

    cout << "\nShkruarja në fajll përfundoi"
          << "\n\n";

    Alfa.close();

    ifstream Beta("C:/Drejt.bin",ios::in|ios::binary);

    cout << "Të dhënat e lexuara nga fajlli\n\n"
          << "    a        b        s        p\n\n";
    while(!Beta.eof())
    {
        Beta.read((char*) &(Dita.a),sizeof(Dita.a));
        Beta.read((char*) &(Dita.b),sizeof(Dita.b));
        Beta.read((char*) &(Dita.s),sizeof(Dita.s));
        Beta.read((char*) &(Dita.p),sizeof(Dita.p));
        if (!Beta.eof())
            shtyp(Dita.a,Dita.b,Dita.s,Dita.p);
    }
    cout << endl;
    return 0;
}

void shtyp(double a,double b,double s,double p)
{
    cout << fixed
          << setprecision(2)
          << setw(6)
          << a
          << setw(6)
          << b
          << setw(8)
          << s
          << setw(8)
          << p

```



```

        << endl;
return;
}

```

Këtu, brenda unazës së parë, pas llogaritjes së vlerave të sipërfaqes **s** dhe të perimetrit **p** të drejkëndëshit për kombinimet e veçanta të vlerave të brinjëve **a** dhe **b**, ato shkruhen në fajll duke shfrytëzuar për secilën variabël një komandë të veçantë **write**. Ngjashëm veprohet në unazën e dytë, gjatë leximit të të dhënave nga fajlli, duke shfrytëzuar katër komanda **read**. Për shtypje të vlerave të lexuara shfrytëzohet funksioni **shtyp**, i cili është definuar jashtë strukturës si funksion i pavarur.

Rezultati që shtypet në ekran pas ekzekutimit të programit të dhënë do të duket ashtu siç është dhënë më herët te *Fig.7.37*.

Rezultati i leximit nga fajlli nuk ndryshon nëse gjatë hapjes së tij nuk shfrytëzohet edhe opcioni **ios::binary**, përkatësisht nëse për hapje shfrytëzohet komanda:

```
ifstream Beta("C:/Drejt.bin",ios::in);
```

Nëse në fajll shkruhen të dhënat që përfshihen në komponentet me të dhëna të objekteve të klasave, brenda programeve do të ndryshojë vetëm deklarimi i klasave përkatëse.

Përmes blloqeve me të dhëna

Në fajlla mund të shkruhen, ose prej fajllave mund të lexohen komplet pjesët me të dhëna të objekteve, duke i llogaritur si blloqe të dhënash. Për këtë qëllim shfrytëzohen funksionet **write** dhe **read**, të cilat në formë të përgjithshme shkruhen:

```

r.write(reinterpret_cast<char*>(&o),sizeof(o));
r.read(reinterpret_cast<char*>(&o),sizeof(o));

```

ku janë:

r - rrjedha e deklaruar si objekt i klasave për punë me fajlla.

&o - adresa e objektit **o** që shkruhet në fajll.

sizeof(o) - numri i bajtëve në fajll, të cilët shkruhen vlerat e përfshira në komponentet me të dhëna të objektit **o**.

Këtu, shkruarja ose leximi i të dhënave bëhet me ndërmjetësimin e pointerit të tipit karakter **char***. Por, gjatë kësaj, për konvertimin e adresës së objektit (**&o**) në pointerin e tipit karakter (**char***), shfrytëzohet operatori **reinterpret_cast**.

Shembull

Versioni **file03** i programit **file02**, përmes së cilit tregohet shkruarja në fajllin me qasje direkte **Drejt.bin** të objektit **Dita** të strukturës **Drejt**, duke e llogaritur pjesën me të dhëna të objektit si një bllok me të dhëna.

```
// Programi file03
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

void shtyp(double a,double b,double s,double p);

struct Drejt
{
    double a,b,s,p;
};

int main()
{
    Drejt Dita;
    ofstream Alfa ("C:/Drejt.bin",ios::out|ios::binary);

    Dita.b=6;
    for(Dita.a=1;Dita.a<=5;Dita.a=Dita.a+0.5)
    {
        Dita.s=Dita.a*Dita.b;
        Dita.p=2*(Dita.a+Dita.b);
        Alfa.write(reinterpret_cast<char*>(&Dita),
                    sizeof(Dita));
    }

    cout << "\nShkruarja në fajll përfundoi"
          << "\n\n";

    Alfa.close();

    ifstream Beta("C:/Drejt.bin",ios::in|ios::binary);

    cout << "Të dhënat e lexuara nga fajlli\n\n"
          << "    a        b        s        p\n\n";
```

```

while(!Beta.eof())
{
    Beta.read(reinterpret_cast<char*>(&Dita),
              sizeof(Dita));
    if (!Beta.eof())
        shtyp(Dita.a,Dita.b,Dita.s,Dita.p);
}
cout << endl;
return 0;
}

```

Pas ekzekutimit të programit të dhënë, rezultati në ekran do të jetë i njëjtë me atë që është dhënë në *Fig.7.37*. Në programi **shtyp**, i cili shfrytëzohet gjatë shtypjes në ekran të të dhënave që lexohen prej fajllit, është i njëjtë me atë që shfrytëzohet te programi **file02**.

Këtu, gjatë shkruarjes së të dhënave në fajll, për çdo regjistrim kompjuteri shfrytëzon **32** bajtë, sepse brenda tyre përfshihen vlerat e **4** variablave të komponenteve të strukturës, të cilat janë të tipit double (për shkruarjen e vlerave të secilës prej tyre shfrytëzohen nga **8** bajtë).

Shembull

Pjesa e dytë e versionit **file04** të programit **file03**, tek i cili urdhërohet shtypja e numrave rendorë **k** të bajtëve ku fillojnë regjistrimet e veçanta.

```

// Programi file04
.....
.....
ifstream Beta("C:/Vlerat.bin",ios::in|ios::binary);
int k;

cout << "Të dhënat e lexuara nga fajlli\n\n"
      << "    k    a    b    s    p\n\n";

while(!Beta.eof())
{
    k=Beta.tellg();
    cout << setw(5)
          << k;
    Beta.read(reinterpret_cast<char*>(&Dita),
              sizeof(Dita));
    if (!Beta.eof())
        shtyp(Dita.a,Dita.b,Dita.s,Dita.p);
}
cout << endl;

```

```
return 0;
}
```

Rezultati që shtypet në ekran pas ekzekutimit të programit të dhënë do të duket si në *Fig.7.45*.

Fig.7.45
Pamja e ekranit pas ekzekutimit të
programit **file04**

Shkruarja në fajll përfundoi				
Të dhënat e lexuara nga fajlli				
k	a	b	s	p
0	1.00	6.00	6.00	14.00
32	1.50	6.00	9.00	15.00
64	2.00	6.00	12.00	16.00
96	2.50	6.00	15.00	17.00
128	3.00	6.00	18.00	18.00
160	3.50	6.00	21.00	19.00
192	4.00	6.00	24.00	20.00
224	4.50	6.00	27.00	21.00
256	5.00	6.00	30.00	22.00
288				
Press any key to continue				

Në kolonën e parë të kësaj table janë shtypur numrat rendorë **k** të bajtëve ku fillojnë regjistrimet e veçanta.

Për shkruarje në fajll mund të shfrytëzohet edhe versioni vijues i funksionit **write**:

```
r.write(reinterpret_cast<const char*>(&o), sizeof(o));
```

Gjatë kësaj, para shkruarjes në fajll, adresa e objektit **&o** konvertohet në pointer si konstante e tipit karakter **const char***.

Nëse në fajll shkruhen të dhënat që përfshihen në komponentet e objekteve të klasave, përveç deklartimit të klasave, procedura e shkruarjes së të dhënave në fajlla dhe të leximit të tyre nga fajllat nuk do të ndryshojë aspak nga ajo që shfrytëzohet për objektet e strukturave.

Format e dhëna më sipër për shkruarjen e të dhënave të fajllat me qasje direkte, ose për leximin prej tyre, plotësisht njëloj mund të shfrytëzohen edhe për variabla të zakonshme, ose edhe për fushat.

Objektet me funksione

Nëse në komponentet e strukturave ose të klasave, përveç variablave me të dhëna paraqiten edhe funksione, gjatë shkruarjes në fajll të pjesëve me të dhëna të objekteve përkatëse, funksionet nuk shkruhen në fajlla. Prandaj, edhe në këto raste, komandat për shkruarje në fajlla të objekteve, ose për lexim prej tyre, nuk do të dallohen aspak nga format që janë dhënë në pjesën paraprake.

Shembull

Versioni **file05** i programeve të dhëna më sipër, tek i cili objekti i klasës **Drejt**, që shkruhet në fajllin **Drejt.bin** me qasje direkte, e përmban edhe funksionin **shtyp**, i cili shfrytëzohet për shtypje të rezultateve.

```
// Programi file05
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

class Drejt
{
public:
    double a,b,s,p;
    void shtyp()
    {
        cout << fixed
              << setprecision(2)
              << setw(6)
              << a
              << setw(6)
              << b
              << setw(8)
              << s
              << setw(8)
              << p
              << endl;
        return;
    }
};

int main()
```

```

{
    Drejt Dita;
    ofstream Alfa ("C:/Vlerat.bin",ios::out|ios::binary);

    Dita.b=6;
    for(Dita.a=1;Dita.a<=5;Dita.a=Dita.a+0.5)
    {
        Dita.s=Dita.a*Dita.b;
        Dita.p=2*(Dita.a+Dita.b);
        Alfa.write(reinterpret_cast<char*>(&Dita),
                    sizeof(Dita));
    }

    cout << "\nShkruarja në fajll përfundoi"
          << "\n";

    Alfa.close();

    ifstream Beta("C:/Vlerat.bin",ios::in|ios::binary);

    cout << "\nTë dhënat e lexuara nga fajlli\n\n"
          << "    a        b        s        p\n\n";
    while(!Beta.eof())
    {
        Beta.read(reinterpret_cast<char*>(&Dita),
                  sizeof(Dita));
        if (!Beta.eof())
            Dita.shtyp();
    }
    cout << endl;
    return 0;
}

```

Pas ekzekutimit të programit të dhënë, rezultati në ekran do të jetë i njëjtë me atë që është dhënë më herët në *Fig.7.37*.

Në praktikë, objektet që shkruhen në fajlla mund të përmbajnë të dhëna të tipeve të përziera.

Shembull

Programi **file06**, përmes së cilit tregohet shkruarja në fajllin **Jeta.bin** e të dhënave të përfshira në komponentet e objektit **studenti** të klasës **person**, si dhe leximin nga fajlli dhe shtypjen e tyre në ekran.

```

// Programi file06
#include <iostream>

```

```
#include <fstream>
#include <iomanip>
using namespace std;

class person
{
private:
    char emri[8], qyteti[10];
    int viti;
public:
    void lexo();
    void shtyp();
};

int main()
{
    char a;
    person studenti;
    ofstream Alfa("C:/Jeta.bin", ios::out | ios::binary);

    cout << "\nTë dhënat nga tastiera\n";

    do
    {
        studenti.lexo();
        Alfa.write(reinterpret_cast<char*>(&studenti),
                    sizeof(studenti));

        cout << "Person tjetër, P-për Po, J-për Jo: ";
        cin >> a;
    }
    while(a=='P');

    cout << "\nTë dhënat e lexuara nga fajlli\n"
         << "\n    Emri    Qyteti    Viti\n\n";

    Alfa.close();

    ifstream Beta("C:/Jeta.bin", ios::in | ios::binary);

    while(!Beta.eof())
    {
        Beta.read(reinterpret_cast<char*>(&studenti),
                  sizeof(studenti));

        if(!Beta.eof())
            studenti.shtyp();
    }
    cout << "\n";
}
```

```

return 0;
}

void person::lexo()
{
    cout << "\nEmri .....: ";
    cin >> emri;
    cout << "Qyteti ....: ";
    cin >> qyteti;
    cout << "Viti .....: ";
    cin >> viti;
    cout << "\n";
}

void person::shtyp()
{
    cout << setw(8)
         << emri
         << setw(10)
         << qyteti
         << setw(7)
         << viti
         << endl;
}

```

Në program, për leximin e të dhënave, të cilat kompjuterit i jepen përmes tastierës, shfrytëzohet funksioni **lexo**. Procesi i leximit përsëritet në unazën **do**, derisa për variablën **a** të tipit karakter përmes tastierës kompjuterit i jepet shkronja **P**. Përndryshe, cilado shkronjë tjetër që të shtypet, procesi i leximit të të dhënave ndërpritet. Pas kësaj, përmes unazës **while** lexohen të dhënat e shkruara në fajll, të cilat njëkohësisht shtypen në ekran, duke e shfrytëzuar funksionin **shtyp**.

Nëse ekzekutohet programi i dhënë dhe kompjuterit përmes tastierës, p.sh. i jepen të dhënat për **2** persona, rezultati që shtypet në ekran do të duket ashtu siç është dhënë në *Fig.7.46*.

```

Të dhënat nga tastiera
Emri .....: Arta
Qyteti ....: Shkupi
Viti .....: 1985

Person tjetër, P-për Po, J-për Jo: P

Emri .....: Driton
Qyteti ....: Prizreni
Viti .....: 1984

Person tjetër, P-për Po, J-për Jo: J

Të dhënat e lexuara nga fajlli
      Emri      Qyteti      Viti
      Arta      Shkupi      1985
      Driton    Prizreni     1984

Press any key to continue

```


*Fig.7.46**Pamja e ekranit pas ekzekutimit të programit **file06***

Disa fajlla të hapur njëkohësisht

Në një program mund të hapen njëkohësisht disa fajlla, qofshin për shkruarje ose për lexim. Gjatë kësaj, logjika e shfrytëzimit të tyre nuk dallon nga ajo kur në program kemi të bëjmë vetëm me një fajll të hapur.

Shembull

Programi **fileW** përmes së cilit të dhënat që shkruhen te fajlli **Rrethi.txt** lexohen dhe rishkruhen te fajlli **RrethiK.txt**. Gjatë kësaj, janë të hapur njëkohësisht dy fajlla sekuencial.

```
// Programi fileW
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;

int main()
{
    double a=3,b=7,pi=3.1415926,r,s,p;

    ofstream Alfa("D:/Libra/Rrethi.txt",ios::out);

    for(r=a;r<=b;r=r+0.5)
    {
        s=2*pi*r;
        p=pi*r*r;
        Alfa << fixed
              << setprecision(2)
              << setw(4)
              << r
```

```

        << ' '
        << setw(7)
        << s
        << ' '
        << setw(7)
        << p
        << endl;
    }
    cout << "\nPërrundoi shkruarja te Rrethi.txt"
        << "\n";
    Alfa.close();
    ifstream Lexo("D:/Libra/Rrethi.txt",ios::in);
    ofstream Shkruaj("D:/Libra/RrethiK.txt",ios::out);
    while(!Lexo.eof())
    {
        Lexo >> r
            >> s
            >> p;
        if (!Lexo.eof())
            Shkruaj << fixed
                << setprecision(2)
                << setw(4)
                << r
                << ' '
                << setw(7)
                << s
                << ' '
                << setw(7)
                << p
                << endl;
    }
    cout << "\nPërfundoi shkruarja te RrethiK.txt"
        << "\n\n";
    return 0;
}

```

Në program, fillimisht për vlera të ndryshme të rrezes **r** janë llogaritur sipërfaqet **s** dhe perimetrat **p** të rrethit me rreze **r**. Njëkohësisht, vlerat e llogaritura janë shkruar te fajlli **Rrethi.txt**. Pastaj, pasi mbyllet fajlli në fjalë, ai rihapet për lexim. Por, njëkohësisht hapet edhe fajlli tjetër **RrethiK.txt**, në të cilin do të shkruhen vlerat e lexuara nga fajlli **Rrethi.txt**.

Nëse pas përfundimit të ekzekutimit të programit hapen fajllat **Rrethi.txt** dhe **RrethiK.txt**, duke e shfrytëzuar programin *Microsoft Notepad*, do të shihet se të dhënat që përfshihen brenda tyre janë të njëjta, përkatësisht fajlli i dytë paraqet një kopje të fajllit të parë.

Bibliografia

1. **Chris H. Pappas, William H. Murray**
The Complete Reference, Visual C++.Net
McGraw-Hill/Osborne, New York 2002
2. **Ulka Kirch-Prinz, Peter Prinz**
Programming in C++, A Complete Guide
Jones and Bartlett Publishers, Sudbury, USA 2002
3. **Julian Templeman, Andy Olsen**
Microsoft Visual C++.NET, Step by Step
Microsoft Corporation, Redmond, Washington 2002
4. **Victor Shtern**
Core C++, A Software Engineering Approach
Prentice Hall PTR, New Jersey 2000
5. **Robert Lafore**
Object-Oriented Programming in C++
SAMS, Indianapolis, Indiana 1999
6. **Bjarne Stroustrup**
C++ Programming Language
Addison-Wesley Publishing Company, Massachusetts 1997
7. **H.M. Deitel, P. J. Deitel**
How to Program C
Prentice Hall, Englewood Cliffs, New Jersey 1994
8. **Jesse Libery**
Teach Yourself C++ in 21 Days
Sams Publishing, Indianapolis, Indiana

9. S. B. Lippman, J. Lajoie

C++ Primer

Addison-Wesley Publishing Company, Massachusetts

10. Kris Jamsa, Lars Klander

C/C++ Programmer's Bible

Gulf Publishing Company, Houston, Texas

11. Rob McGregor

Practical C++

QUE, Indianapolis, Indiana 1999

12. H.M. Deitel, P. J. Deitel

How to Program C++

Prentice Hall, Upper Saddle River, New Jersey 2003

13. H.M. Deitel, P. J. Deitel

How to Program C

Prentice Hall, Upper Saddle River, New Jersey 2004

14. Jim Keogh, John Shapley Gray

C++ Programmer's Notebook

Prentice Hall PTR, Upper Saddle River, New Jersey 2002

15. Agni H. Dika

Algoritmet, njohuri themelore, me programe në C++

Fakulteti Elektroteknik, Prishtinë 2002, 2004

16. James P. Cohoon, Jack W. Davidson

C++ Program Design

Irwin/McGraw-Hill, USA 1997

17. Agni Dika

Bazat e programimit në C++

Fakulteti i Inxhinierisë Elektrike dhe Kompjuterike, Prishtinë 2004

18. D. S. Malik

C++ Programming: From Problem Analysis to Program Design

Course Technology, Massachusetts 2002

19. Frank L. Friedman, Elliot B. Koffman

Problem Solving, Abstarction, and Design Using C++

Pearson Addison Wesley, USA 2004

20. Stanley B. Lippman, Josée Lajoie, Barbara E. Moo

C++ Primer, Fourth Edition
Addison-Wesley, USA 2005

21. Herbert Schildt

C++ from the Ground Up
McGraw-Hill/Osborne, Berkeley, California 2003

22. Julian Templeman, Andy Olsen

Microsoft VISUAL C++ .NET, Step by Step
Microsoft Press, 2002

23. Chris H. Pappas, William H. Murray, III

Visual C++ .NET, The Complete Reference
McGraw-Hill/Osborne, Berkeley, California 2002

Universiteti i Europës Juglindore
Fakulteti i Shkencave dhe i Teknologjive Bashkëkohore

Agni Dika
Programimi i Orientuar në Objekte
në C++

Lektor
Dr. Ilaz Metaj

Kopertina
AfiDesign
Prishtinë

CIP - Katalogizimi në publikim
Biblioteka Popullore dhe Universitare
"Sv. Kliment Ohridski", Shkup
004.432

DIKA, Agni, Programimi i Orientuar në Objekte në C++
/ Agni Dika. - Tetovë: ArbëriaDesign, 2005. 409 faqe.; 24 cm

ISBN 9989-866-25-2

a) C++ (Gjuhë Programuese) - Libër për Arsim të Lartë
COBISS.MK-ID 63119626